Politecnico di Torino

Master of Science in Computer Engineering

Department of Control and Computer Engineering (DAUIN)

Database and Data Mining Group (DBDMG)

# Large Language Models for Software Engineering

## Theoretical foundations, methodologies, and practical architectures

**Comprehensive master's course study guide**

Authored for graduate students, researchers, and practitioners

*Course Instructors:*
**Prof. Flavio Giobergia**
**Prof. Riccardo Coppola**
Politecnico di Torino

*Academic Year:*
**2025 / 2026**

*Target Program:*
**M.Sc. Computer Engineering**

Compiled and Typeset with LaTeX and `tcolorbox`
Torino, Italy — Academic Year 2025/2026

# Contents

**Preface** iii

**Notation and conventions** v

**I Foundations of large language models** 1

**1 Foundations of deep learning** 2
- 1.1 Historical context and theoretical motivation . . . 3
- 1.2 The perceptron and the linear inseparability barrier . . . 6
- 1.3 Non-linearity and activation functions . . . 10
- 1.4 Loss functions and optimization dynamics . . . 17
- 1.5 Computational graphs, backpropagation, and the feed-forward network . . . 23

**2 Recurrent neural networks** 30
- 2.1 Limitations of feed-forward architectures on sequential data . . . 31
- 2.2 Recurrent neural networks: intuition, unrolling, and cell mechanics . . . 34
- 2.3 Weight sharing and backpropagation through time . . . 38
- 2.4 Pathologies of vanilla RNNs and gated architectures . . . 42
- 2.5 Sequence-to-sequence modeling, encoder-decoder architectures, and the information bottleneck . . . 47

**3 Language models** 52
- 3.1 Introduction to language models and the modeling task . . . 53
- 3.2 The Markov assumption, $n$-gram models, and operational inference . . . 56
- 3.3 Limitations of statistical language models and the path to neural representations 61

**4 Distributed word representations and Word2Vec** 69
- 4.1 The representation problem: why continuous embeddings are mandatory . . . 70
- 4.2 The baseline approach: local one-hot encodings and why they fail . . . 71
- 4.3 Distributed representations and the distributional hypothesis . . . 76
- 4.4 The embedding training loop and semantic geometry evolution . . . 77
- 4.5 Word2Vec continuous bag-of-words (CBOW) . . . 80
- 4.6 Word2Vec continuous skip-gram and the softmax bottleneck . . . 83
- 4.7 Scaling softmax: hierarchical softmax and negative sampling . . . 85
- 4.8 Limitations of Word2Vec and the out-of-vocabulary (OOV) barrier . . . 90
- 4.9 FastText: subword character $n$-gram embeddings . . . 92
- 4.10 Emergent vector space geometry: clustering, metrics and relational analogies . . . 94
- 4.11 Modern subword tokenization and input pipelines for foundation models . . . 97

**5 The Transformer architecture and attention mechanisms 114**
5.1 The Self-Attention Primitive . . . 115
5.2 Transformer Architecture Overview . . . 116
5.3 Attention Principles . . . 118
5.4 Scaled Dot-Product Attention . . . 121
5.5 Attention Mechanism Types . . . 125
5.6 Multi-Head Attention . . . 132
5.7 Residual Connections and Normalization . . . 134
5.8 Positional Representations . . . 138
5.9 Computational Complexity . . . 139

**6 Transformer Architectures 141**
6.1 Encoder-Decoder Models . . . 141
6.2 Encoder-Only Models . . . 143
6.3 Decoding Strategies . . . 153
6.4 Architectural Paradigms Synthesis . . . 165
6.5 Architectural Paradigms . . . 166
6.6 GPT-1 . . . 169
6.7 GPT-2 . . . 172

**7 Pre-Training and Scaling Laws 178**
7.1 Evolution of Language Models . . . 178
7.2 Kaplan Scaling Laws . . . 183
7.3 GPT-3 and In-Context Learning . . . 186
7.4 The Undertraining Crisis . . . 190
7.5 Chinchilla Compute-Optimal Scaling . . . 194
7.6 Hardware Realities and Open Models . . . 200

**8 Evaluation and Benchmarks 205**
8.1 Principles of Evaluation . . . 206
8.2 Evaluation Tasks and Problem Formulations . . . 225
8.3 Standardized Benchmark Suites . . . 231
8.4 Evaluation Synthesis . . . 235

# Preface

## Pedagogical vision

Large language models (LLMs) are increasingly used in software engineering tasks that involve code synthesis and repair, requirements analysis, and coordination among software agents. These applications rely on probabilistic models trained to represent and generate sequences of text and code. Understanding their use therefore requires both the mathematical foundations of language modeling and an account of how model behavior interacts with software development processes.

This guide accompanies the Master of Science course *Large Language Models for Software Engineering* at Politecnico di Torino, taught by Prof. Flavio Giobergia and Prof. Riccardo Coppola. The course is organized around two related areas:

1. **Part I: Foundations of large language models** — Develops the mathematical and architectural foundations of language modeling, from $n$-gram Markov chains and feedforward networks to distributed word embeddings, recurrent networks, and Transformers. The broader course syllabus also includes pre-training scaling laws, instruction tuning (RLHF and DPO), and parameter-efficient fine-tuning (LoRA and QLoRA).

2. **Part II: Large language models for software engineering** — Examines how model capabilities relate to the software development life cycle (SDLC), including prompt engineering, agent frameworks such as ReAct, automated test generation, software maintenance and refactoring, evaluation measures such as Pass@$k$ and CodeBLEU, and security, intellectual property, and ethical considerations.

## Visual callout structure

Conceptual interpretations, including attention as a differentiable lookup operation and the geometry of high-dimensional vector spaces, accompany the relevant mathematical development. Explanations also address common mathematical errors, such as using inconsistent logarithm bases when computing perplexity. Research context is cited in the text, including foundational work on Transformers and language-model scaling [1–3].

## Examination framework and course organization

Assessment in the course consists of a written examination and a practical project. The stated examination format and course schedule are:

1. **Written Examination (18 Points)**:

    - The examination lasts 75 minutes and typically contains about 12 questions, combining multiple-choice items with open-ended technical or mathematical derivations.

- Its scope covers foundational and applied software-engineering topics; a score of at least 10/18 is required for a passing result.

2. **Practical Course Project (14 Points)**:
   - The project involves implementing an applied LLM-for-software-engineering system in Python and PyTorch, individually or in a small team.
   - The examination and project contribute 18 and 14 points, respectively, for a total of 32; a score above 30 earns *30 e Lode* (honors).
3. **Lecture and Lab Schedule**:
   - Lectures take place on Monday from 08:30 to 11:30 in Room 8 and cover theoretical foundations.
   - Laboratories take place on Wednesday from 11:30 to 13:00 in Room R1 and focus on practical workshops, coding, and PyTorch.

## How to use this study guide

Each chapter develops its subject through explanatory prose, mathematical derivations, algorithmic procedures, and software-engineering applications. Readers can use the notation chapter as a reference and follow citations to the research that informs the discussion. Checking derivations and implementing core algorithms provide complementary ways to examine the behavior of the models.

# Notation and conventions

Throughout this study guide, we adopt consistent mathematical, algorithmic, and typographic conventions aligned with standard deep learning and software engineering literature.

## Linear algebra and tensor operations

| Symbol | Type | Description |
|---|---|---|
| $x, y, z$ | Scalar | Real-valued scalars or scalar components ($x \in \mathbb{R}$). |
| $\mathbf{x}, \mathbf{y}, \mathbf{h}$ | Vector | Column vectors ($\mathbf{x} \in \mathbb{R}^d$). Lowercase bold font. |
| $\mathbf{x}^\top$ | Transpose | Transpose of vector $\mathbf{x}$ (row vector $\mathbf{x}^\top \in \mathbb{R}^{1\times d}$). |
| $\mathbf{A}, \mathbf{W}, \mathbf{X}$ | Matrix | Real matrices ($\mathbf{A} \in \mathbb{R}^{m\times n}$). Uppercase bold font. |
| $\mathbf{A}_{i,j}$ or $a_{ij}$ | Scalar | Entry at row $i$ and column $j$ of matrix $\mathbf{A}$. |
| $\mathbf{I}$ or $\mathbf{I}_d$ | Matrix | Identity matrix of dimension $d \times d$. |
| $\|\mathbf{x}\|_2$ or $\|\mathbf{x}\|$ | Scalar | Euclidean ($L_2$) norm of vector $\mathbf{x}$, $\sqrt{\sum_i x_i^2}$. |
| $\|\mathbf{A}\|_F$ | Scalar | Frobenius norm of matrix $\mathbf{A}$, $\sqrt{\sum_{i,j} A_{i,j}^2}$. |
| $\langle \mathbf{u}, \mathbf{v} \rangle$ or $\mathbf{u}^\top \mathbf{v}$ | Scalar | Standard dot (inner) product between vectors $\mathbf{u}$ and $\mathbf{v}$. |
| $\odot$ | Operator | Element-wise (Hadamard) product between tensors of identical shape. |
| $\otimes$ | Operator | Outer product or Kronecker tensor product. |

Table 1: Linear algebra and tensor notation summary.

## Probability and information theory

## Neural networks, transformers, and software engineering

| Symbol | Type | Description |
|---|---|---|
| $\mathbb{P}(A)$ | Scalar | Probability of event $A$ ($\mathbb{P} : \mathcal{F} \to [0, 1]$). |
| $\mathbb{P}(X = x)$ | Scalar | Probability mass function for discrete random variable $X$. |
| $\mathbb{P}(Y \mid X)$ | Distribution | Conditional probability distribution of $Y$ given $X$. |
| $\mathbb{E}_{x\sim P}[f(x)]$ | Scalar | Expected value of function $f(x)$ under distribution $P$. |
| $\mathbb{V}\text{ar}(X)$ | Scalar | Variance of random variable $X$, $\mathbb{E}[(X - \mathbb{E}[X])^2]$. |
| $\mathcal{H}(P)$ | Scalar | Shannon entropy of distribution $P$, $-\sum_x P(x) \log P(x)$. |
| $\mathcal{H}(P, Q)$ | Scalar | Cross-entropy between target distribution $P$ and model $Q$. |
| $D_{\mathrm{KL}}(P \parallel Q)$ | Scalar | Kullback–Leibler (KL) divergence from $Q$ to $P$. |
| $\mathrm{PPL}(W)$ | Scalar | Perplexity of word sequence $W$ under a language model. |

Table 2: Probability and information theoretic notation.

| Symbol | Type | Description |
|---|---|---|
| $\mathcal{V}$ | Set | Discrete vocabulary of tokens; $\lvert\mathcal{V}\rvert = V$ denotes vocabulary size. |
| $T$ or $T$ | Integer | Sequence length (number of input or output tokens). |
| $d_{\text{model}}$ | Integer | Hidden embedding dimensionality of the Transformer model. |
| $d_{\text{ff}}$ | Integer | Dimensionality of the intermediate Feed-Forward Network layer. |
| $d_k, d_v$ | Integer | Dimensionality of query/key and value projections per head. |
| $h$ | Integer | Number of parallel attention heads ($h \cdot d_k = d_{\text{model}}$). |
| $\sigma(z)$ | Function | Standard logistic sigmoid function: $\sigma(z) = \frac{1}{1+e^{-z}}$. |
| $\mathrm{softmax}(\mathbf{z})$ | Function | Softmax normalized exponential vector operator. |
| $\mathrm{ReLU}(z)$ | Function | Rectified Linear Unit activation: $\max(0, z)$. |
| $\mathrm{GELU}(z)$ | Function | Gaussian Error Linear Unit activation. |
| $\mathcal{L}$ or $\mathcal{L}$ | Scalar | Scalar objective / loss function minimized during optimization. |
| Pass@$k$ | Metric | Probability that at least one of $k$ generated code samples passes unit tests. |
| CodeBLEU | Metric | Multi-aspect code evaluation metric combining BLEU, AST, and data flow. |

Table 3: Model architecture parameters and SE metrics.

# Part I

# Foundations of large language models

CHAPTER 1

# Foundations of deep learning

The fundamental challenge of artificial intelligence resides in discovering computational mechanisms capable of mapping high-dimensional, unstructured sensory and symbolic observations into structured semantic representations. Classical machine learning systems approached this objective by decoupling feature engineering from statistical inference: domain experts manually designed deterministic transformation pipelines to extract handcrafted numerical descriptors, which were subsequently supplied to shallow linear classifiers or regression models. While computationally tractable in controlled environments, this paradigm proved fragile when confronted with the continuous variability, high dimensionality, and compositional complexity of natural language and perceptual signals.

Deep learning fundamentally alters this architectural philosophy by adopting a connectionist, data-driven formulation. Rather than relying on human engineering to prescribe fixed representational primitives, deep neural networks learn hierarchical feature representations directly from raw data through the end-to-end composition of parameterized, differentiable mathematical functions. Each successive layer computes an abstract geometric transformation of its input, constructing intermediate latent representations that disentangle the underlying factors of variation.

This chapter establishes the theoretical, geometric, and algorithmic foundations of deep feed-forward architectures. We trace the lineage of neural computation from early biological abstractions and the single-layer perceptron, formalize the geometric barriers imposed by linear decision boundaries, derive the space-folding mechanics of non-linear activation functions, establish the statistical grounding of task-specific loss functions under maximum likelihood estimation, and formulate reverse-mode automatic differentiation on computational graphs as the universal optimization engine for deep learning.

## 1.1 Historical context and theoretical motivation

Understanding the structural necessity of deep neural networks requires examining why classical algorithmic paradigms and shallow statistical models proved insufficient for complex perceptual and linguistic tasks. The progression from symbolic reasoning to modern representation learning illustrates how computational and geometric constraints shaped modern neural architectures.

### 1.1.1 From symbolic artificial intelligence to connectionism

For several decades following the Dartmouth workshop of 1956, artificial intelligence research was dominated by symbolic systems, colloquially termed Good Old-Fashioned Artificial Intelligence (GOFAI) [4]. This paradigm operated on the physical symbol system hypothesis, positing that intelligent behavior could be instantiated through the formal manipulation of discrete, explicit symbols according to hand-engineered logical rules. Knowledge was encoded using production systems, semantic networks, and first-order predicate calculus, where inference engines evaluated deterministic truth values over closed ontologies.

While symbolic systems demonstrated notable competence in closed, deterministic micro-worlds such as chess or theorem proving, they encountered severe operational bottlenecks when deployed in real-world environments characterized by noise, perceptual ambiguity, and continuous variations. Hand-crafted rule bases were inherently brittle: any scenario outside the codified axioms triggered catastrophic failure, and resolving contradictory evidence in large rule graphs became computationally intractable. More critically, symbolic systems lacked an intrinsic metric of semantic similarity; the discrete symbols `cat` and `feline` shared no mathematical relationship in coordinate space, precluding continuous generalization.

In response to these limitations, the connectionist paradigm emerged, proposing that cognitive processing is realized through distributed computation over vast networks of simple, interconnected processing units [5–7]. Rather than storing knowledge in localized, explicit symbolic rules, connectionist networks encode knowledge in the continuous, distributed pattern of synaptic connection strengths (weights) across the entire system. Representation and computation are inherently continuous and differentiable, enabling models to interpolate smoothly across input spaces, tolerate corrupt or incomplete inputs through graceful degradation, and acquire representations directly from empirical data through statistical learning.

### 1.1.2 The curse of dimensionality and high-dimensional geometry

Early statistical approaches to non-linear pattern recognition attempted to circumvent the limitations of linear models by projecting input vectors $\mathbf{x} \in \mathbb{R}^D$ into higher-dimensional feature spaces via fixed basis functions $\phi(\mathbf{x}) : \mathbb{R}^D \to \mathbb{R}^M$. A standard linear model was then evaluated on the transformed representations:

$$y(\mathbf{x}; \mathbf{w}) = \mathbf{w}^\top \phi(\mathbf{x}) = \sum_{j=1}^{M} w_j \phi_j(\mathbf{x}) + w_0, \tag{1.1}$$

where $\mathbf{w} \in \mathbb{R}^M$ denotes the parameter vector and $w_0$ represents the scalar bias [8]. Although this formulation preserves linear parameter optimization, it suffers directly from the curse of dimensionality when scaling to high-dimensional input spaces.

Consider constructing a complete polynomial basis of order $P$ over an input dimension $D$. The total number of independent polynomial basis functions required scales asymptotically as:

$$M = \binom{D + P}{P} = \mathcal{O}\left(D^P\right). \tag{1.2}$$

For an input space of moderate dimensionality, such as $D = 100$, a third-order polynomial expansion ($P = 3$) demands $M \approx 1.7 \times 10^5$ parameters, while a fifth-order polynomial ($P = 5$) requires $M \approx 7.5 \times 10^7$ parameters. When processing natural images or tokenized textual contexts where $D \geq 10^3$, the parameter count exceeds any feasible computational or sample budget.

The curse of dimensionality manifests with equal severity in non-parametric and partitioning methods. Suppose the input domain $[0, 1]^D$ is partitioned into regular grid cells by dividing each dimension into $K$ intervals. The total number of hypercubical cells is:

$$N_{\text{cells}} = K^D. \tag{1.3}$$

If $D = 50$ and each axis is divided into only $K = 10$ bins, the volume is partitioned into $10^{50}$ distinct cells. Because any realistic dataset contains $N \ll 10^{50}$ observations, the overwhelming majority of cells will remain completely unpopulated, causing localized neighborhood estimators to fail through severe sample sparsity.

Furthermore, the geometry of high-dimensional Euclidean space exhibits counter-intuitive properties that undermine distance-based algorithms [8]. Consider a hypersphere of radius $r = 1$ in $D$ dimensions. The volume of the hypersphere is proportional to $r^D$. The fraction of this volume lying within a thin spherical shell of thickness $\epsilon \in (0, 1)$ near the outer surface is given by:

$$\frac{V_D(1) - V_D(1 - \epsilon)}{V_D(1)} = 1 - (1 - \epsilon)^D. \tag{1.4}$$

As the dimensionality $D \to \infty$, the term $(1 - \epsilon)^D \to 0$, which causes the volume fraction to converge asymptotically to 1 for any non-zero $\epsilon$:

$$\lim_{D \to \infty} \left[1 - (1 - \epsilon)^D\right] = 1. \tag{1.5}$$

Consequently, in high-dimensional spaces, nearly all the volume of a hypersphere is concentrated in an infinitesimally thin boundary shell. A complementary phenomenon occurs with multivariate Gaussian distributions $\mathcal{N}(\mathbf{0}, \sigma^2\mathbf{I})$: transforming to spherical polar coordinates and integrating over directional variables reveals that the radial probability density $p(r) \propto r^{D-1}e^{-r^2/(2\sigma^2)}$ concentrates entirely within a narrow spherical shell of radius $r \approx \sigma\sqrt{D}$. Data points in high dimensions do not cluster around the mean; they reside along an equatorial hypersphere, causing pairwise Euclidean distances between random observations to concentrate tightly around a single constant value.

### 1.1.3 The manifold hypothesis and the expressive power of depth

The realization that high-dimensional data points become mutually equidistant under isotropic assumptions led to the formulation of the manifold hypothesis [8, 9]. The hypothesis states that natural high-dimensional datasets (such as natural language sentences, audio waveforms, or natural images) do not populate the ambient Euclidean space $\mathbb{R}^D$ uniformly; rather, they concentrate in the immediate vicinity of a low-dimensional, non-linear embedded sub-manifold $\mathcal{M} \subset \mathbb{R}^D$ having intrinsic dimension $d \ll D$.

Fixed basis function models cannot exploit this intrinsic low-dimensional structure because their basis functions $\phi_j(\mathbf{x})$ are chosen prior to observing the data distribution. To model arbitrary functions over the ambient space, fixed basis methods must tile the entire $D$-dimensional volume uniformly, falling victim to the exponential scaling of $K^D$. Conversely, artificial neural networks resolve this impasse by replacing fixed basis functions with parameterized, data-dependent basis functions that are optimized jointly to conform to the geometry of the data manifold.

The capacity of neural networks to parameterize complex manifolds depends critically on network depth. Consider a feed-forward network utilizing Rectified Linear Unit (ReLU)

activation functions. Because the ReLU function is piecewise linear, any network constructed from ReLU activations computes a continuous, piecewise linear function that partitions the input domain $\mathbb{R}^{d_{\text{in}}}$ into a collection of convex polyhedral regions, within each of which the model evaluates a distinct affine mapping [10, 11].

For a shallow network comprising a single hidden layer with $K$ hidden units and a scalar input ($d_{\text{in}} = 1$), the model can construct at most $K + 1$ linear regions, parameterized by $3K + 1$ weights and biases:

$$y = \phi_0 + \sum_{j=1}^{K} \phi_j \max\left(0, \theta_{j0} + \theta_{j1} x\right). \tag{1.6}$$

To increase the number of linear regions in a shallow architecture, one must expand the hidden width linearly: achieving $O(K)$ linear regions requires $O(K)$ parameters.

In sharp contrast, stacking multiple non-linear layers allows deeper layers to compute piecewise linear compositions of the piecewise linear functions evaluated by preceding layers. Each successive layer folds the input space, replicating the linear partitions constructed by earlier stages. Formalized by Montúfar et al. [11], a deep neural network with $L$ hidden layers, each possessing $K$ hidden units, can partition the input space $\mathbb{R}^{d_{\text{in}}}$ into a number of distinct linear regions that scales exponentially with depth:

$$N_{\text{regions}} \geq \left( \left\lfloor \frac{K}{d_{\text{in}}} \right\rfloor \right)^{(L-1)d_{\text{in}}} K^{d_{\text{in}}} = O\left(K^{L \cdot d_{\text{in}}}\right), \tag{1.7}$$

while the total parameter count grows only linearly with depth and quadratically with width:

$$N_{\text{parameters}} = O\left(L \cdot K^2\right). \tag{1.8}$$

This exponential disparity in expressive efficiency provides the fundamental mathematical justification for deep architectures: deep composition generates exponentially complex, multi-scale decision boundaries using a polynomial parameter budget, enabling neural networks to model intricate manifold topologies that remain inaccessible to shallow models.

## 1.2 The perceptron and the linear inseparability barrier

The conceptual building block of modern neural architectures is the artificial perceptron. To understand why modern deep architectures are structured as hierarchical non-linear networks, one must first analyze the computational properties and geometric limitations of this foundational linear operator.

### 1.2.1 The artificial perceptron

The mathematical formulation of the artificial neuron originated in the simplified physiological model of biological neurons proposed by Warren McCulloch and Walter Pitts in 1943 [5], which Frank Rosenblatt subsequently formalized and operationalized as the perceptron learning model in 1958 [6]. The biological abstraction captures three primary mechanisms: dendrites receive incoming electrochemical impulses, the soma integrates these inputs, and the axon fires an action potential if the accumulated potential surpasses an excitation threshold.

Formally, consider an input vector $\mathbf{x} = [x_1, x_2, \dots, x_n]^\top \in \mathbb{R}^n$. Each input component $x_i$ is scaled by an associated scalar weight $w_i \in \mathbb{R}$, which quantifies the synaptic efficacy of that specific connection. The neuron aggregates these weighted contributions and adds an intrinsic scalar parameter termed the bias, $b \in \mathbb{R}$, yielding a scalar pre-activation value $z \in \mathbb{R}$:

$$z = \sum_{i=1}^{n} w_i x_i + b = \mathbf{w}^\top \mathbf{x} + b, \tag{1.9}$$

where $\mathbf{w} = [w_1, w_2, \dots, w_n]^\top \in \mathbb{R}^n$ represents the weight vector. The pre-activation value is subsequently processed by an activation function $f : \mathbb{R} \to \mathcal{Y}$ to produce the final scalar output $y$:

$$y = f(z) = f\left(\mathbf{w}^\top \mathbf{x} + b\right). \tag{1.10}$$

In Rosenblatt's classical formulation, the activation function is instantiated as the discontinuous Heaviside step function:

$$f(z) = \begin{cases} 1 & \text{if } z \geq 0, \\ 0 & \text{if } z < 0, \end{cases} \tag{1.11}$$

producing a binary decision variable indicating class membership.

To streamline the algebraic notation, the formulation can be expressed homogeneously by augmenting the input vector with a dummy feature $x_0 = 1$ and defining an augmented weight vector $\tilde{\mathbf{w}} = [w_0, w_1, \dots, w_n]^\top \in \mathbb{R}^{n+1}$ where $w_0 = b$. Under this homogeneous convention, Equation (1.9) simplifies to a single inner product:

$$z = \sum_{i=0}^{n} w_i x_i = \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}, \tag{1.12}$$

where $\tilde{\mathbf{x}} = [1, x_1, \dots, x_n]^\top \in \mathbb{R}^{n+1}$.

### 1.2.2 The geometry of decision hyperplanes

The operational behavior of the artificial perceptron is governed by the geometric properties of its decision boundary in the Euclidean input space $\mathbb{R}^n$. The decision boundary corresponds to the locus of points where the pre-activation vanishes identically:

$$\mathcal{H} = \left\{ \mathbf{x} \in \mathbb{R}^n \;\middle|\; \mathbf{w}^\top \mathbf{x} + b = 0 \right\}. \tag{1.13}$$

In an $n$-dimensional space, Equation (1.13) defines an $(n-1)$-dimensional affine subspace termed a hyperplane. In a two-dimensional input space ($n = 2$), the hyperplane reduces to a standard straight line:

$$w_1 x_1 + w_2 x_2 + w_0 = 0 \iff x_2 = -\frac{w_1}{w_2} x_1 - \frac{w_0}{w_2}, \tag{1.14}$$

assuming $w_2 \neq 0$.

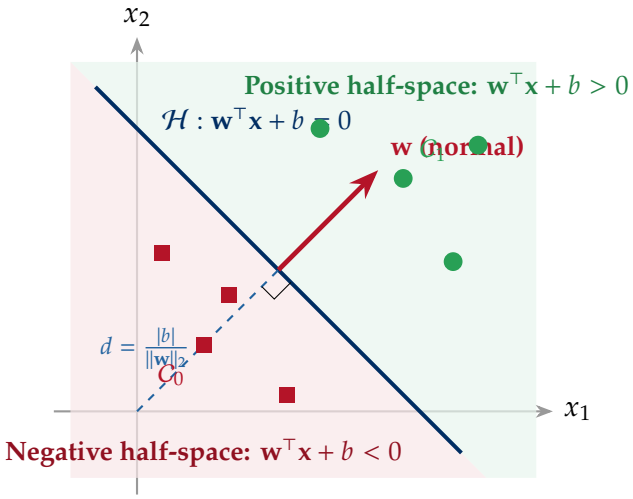



Figure 1.1: Geometric partitioning of a two-dimensional input space by an artificial perceptron decision hyperplane $\mathcal{H}$. The parameter vector $\mathbf{w}$ is strictly orthogonal to the decision boundary, while the scalar bias $b$ determines the orthogonal translation of the hyperplane relative to the origin.

The geometric orientation and placement of the hyperplane are governed by two mathematical properties:

(i) **Orthogonality of the weight vector**: The parameter vector $\mathbf{w}$ is strictly orthogonal to every vector lying within the hyperplane $\mathcal{H}$. To verify this, consider two arbitrary points $\mathbf{x}_A, \mathbf{x}_B \in \mathcal{H}$. By definition of the hyperplane:

$$\mathbf{w}^\top \mathbf{x}_A + b = 0 \quad \text{and} \quad \mathbf{w}^\top \mathbf{x}_B + b = 0. \tag{1.15}$$

Subtracting the two equalities yields:

$$\mathbf{w}^\top (\mathbf{x}_A - \mathbf{x}_B) = 0. \tag{1.16}$$

Because the difference vector $\mathbf{v} = \mathbf{x}_A - \mathbf{x}_B$ is parallel to the hyperplane and its inner product with $\mathbf{w}$ vanishes, $\mathbf{w}$ is normal to the decision boundary.

(ii) **Orthogonal distance to the boundary**: The signed orthogonal distance $d(\mathbf{x}, \mathcal{H})$ from an arbitrary point $\mathbf{x} \in \mathbb{R}^n$ to the hyperplane $\mathcal{H}$ is obtained by projecting the displacement vector $(\mathbf{x} - \mathbf{x}_p)$, where $\mathbf{x}_p \in \mathcal{H}$, onto the unit normal vector $\frac{\mathbf{w}}{\|\mathbf{w}\|_2}$:

$$d(\mathbf{x}, \mathcal{H}) = \frac{\mathbf{w}^\top (\mathbf{x} - \mathbf{x}_p)}{\|\mathbf{w}\|_2} = \frac{\mathbf{w}^\top \mathbf{x} - \mathbf{w}^\top \mathbf{x}_p}{\|\mathbf{w}\|_2} = \frac{\mathbf{w}^\top \mathbf{x} + b}{\|\mathbf{w}\|_2}. \tag{1.17}$$

Evaluating this expression at the origin ($\mathbf{x} = \mathbf{0}$) yields the perpendicular distance from the coordinate origin to the hyperplane:

$$d_{\text{origin}} = \frac{|b|}{\|\mathbf{w}\|_2}. \tag{1.18}$$

The perceptron partitions the input space into two half-spaces: a positive half-space where $\mathbf{w}^\top \mathbf{x} + b > 0$ (yielding $y = 1$) and a negative half-space where $\mathbf{w}^\top \mathbf{x} + b < 0$ (yielding $y = 0$).

### 1.2.3 The linear stacking collapse proof

Given the limitations of a single linear decision boundary, an intuitive proposal involves cascading multiple perceptrons into a multi-layer hierarchical network, where intermediate units supply inputs to subsequent stages. However, if the activation functions between successive layers remain purely linear (that is, $f(z) = z$), the multi-layer architecture collapses algebraically into an ordinary single-layer linear model.

*Proof of linear stacking collapse.* Consider a multi-layer feed-forward network comprising $L$ fully connected linear layers. Let $\mathbf{x} \in \mathbb{R}^{d_0}$ denote the input vector. The first hidden layer computes an affine transformation parameterized by weight matrix $\mathbf{W}_1 \in \mathbb{R}^{d_0 \times d_1}$ and bias vector $\mathbf{b}_1 \in \mathbb{R}^{d_1}$:

$$\mathbf{h}_1 = \mathbf{W}_1^\top \mathbf{x} + \mathbf{b}_1. \tag{1.19}$$

The second hidden layer processes $\mathbf{h}_1$ linearly via $\mathbf{W}_2 \in \mathbb{R}^{d_1 \times d_2}$ and $\mathbf{b}_2 \in \mathbb{R}^{d_2}$:

$$\mathbf{h}_2 = \mathbf{W}_2^\top \mathbf{h}_1 + \mathbf{b}_2 = \mathbf{W}_2^\top \left( \mathbf{W}_1^\top \mathbf{x} + \mathbf{b}_1 \right) + \mathbf{b}_2. \tag{1.20}$$

Expanding this expression by distributing the matrix multiplication yields:

$$\mathbf{h}_2 = (\mathbf{W}_1 \mathbf{W}_2)^\top \mathbf{x} + \left( \mathbf{W}_2^\top \mathbf{b}_1 + \mathbf{b}_2 \right). \tag{1.21}$$

By defining an effective composite weight matrix $\mathbf{W}' \in \mathbb{R}^{d_0 \times d_2}$ and an effective composite bias vector $\mathbf{b}' \in \mathbb{R}^{d_2}$:

$$\mathbf{W}' = \mathbf{W}_1 \mathbf{W}_2 \quad \text{and} \quad \mathbf{b}' = \mathbf{W}_2^\top \mathbf{b}_1 + \mathbf{b}_2, \tag{1.22}$$

Equation (1.21) simplifies to:

$$\mathbf{h}_2 = \mathbf{W}'^\top \mathbf{x} + \mathbf{b}'. \tag{1.23}$$

By mathematical induction, compounding $L$ successive linear layers yields:

$$\mathbf{y} = \mathbf{W}_{\text{eff}}^\top \mathbf{x} + \mathbf{b}_{\text{eff}}, \quad \text{where} \quad \mathbf{W}_{\text{eff}} = \prod_{l=1}^{L} \mathbf{W}_l. \tag{1.24}$$

Because the product of $L$ linear operators is itself a linear operator, stacking arbitrary numbers of purely linear layers provides zero additional expressive capacity over a single affine layer. Consequently, depth without non-linearity is functionally vacuous.

### 1.2.4 The XOR linear inseparability barrier

The catastrophic consequence of the linear restriction was formally exposed by Marvin Minsky and Seymour Papert in their seminal 1969 monograph [12], which demonstrated that a single-layer perceptron is incapable of computing the basic exclusive-OR (XOR) logic function.

The XOR truth table maps two binary inputs $x_1, x_2 \in \{0, 1\}$ to a scalar output $y \in \{0, 1\}$ according to:

$$\text{XOR}(0,0) = 0, \quad \text{XOR}(0,1) = 1, \quad \text{XOR}(1,0) = 1, \quad \text{XOR}(1,1) = 0. \tag{1.25}$$

For a perceptron with threshold activation $f(z) = \mathbf{1}_{z>0}$ to compute this function, there must exist real parameters $w_1, w_2, b \in \mathbb{R}$ satisfying four simultaneous linear inequalities:

$$(x_1 = 0, x_2 = 0) \implies w_1(0) + w_2(0) + b \leq 0 \iff b \leq 0, \tag{1.26}$$
$$(x_1 = 0, x_2 = 1) \implies w_1(0) + w_2(1) + b > 0 \iff w_2 + b > 0, \tag{1.27}$$
$$(x_1 = 1, x_2 = 0) \implies w_1(1) + w_2(0) + b > 0 \iff w_1 + b > 0, \tag{1.28}$$
$$(x_1 = 1, x_2 = 1) \implies w_1(1) + w_2(1) + b \leq 0 \iff w_1 + w_2 + b \leq 0. \tag{1.29}$$

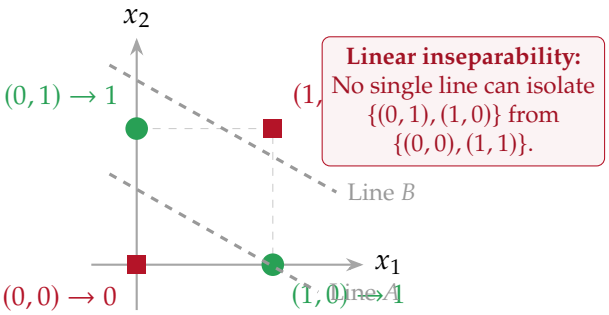



Figure 1.2: The XOR linear inseparability barrier. The two input classes alternate diagonally across the vertices of the unit square, preventing any single linear hyperplane from isolating Class 1 from Class 0.

*Proof of algebraic contradiction.* Summing the strict inequalities (1.27) and (1.28) yields:

$$(w_2 + b) + (w_1 + b) > 0 \iff w_1 + w_2 + 2b > 0. \tag{1.30}$$

Conversely, summing inequalities (1.26) and (1.29) produces:

$$(w_1 + w_2 + b) + b \leq 0 + 0 \iff w_1 + w_2 + 2b \leq 0. \tag{1.31}$$

Inequality (1.30) and Inequality (1.31) are mutually contradictory: the scalar quantity $w_1 + w_2 + 2b$ cannot simultaneously be strictly positive and non-positive ($0 < w_1 + w_2 + 2b \leq 0$). Hence, no parameters $\{w_1, w_2, b\}$ exist that solve the system. Because the input points alternate diagonally across the vertices of the unit square, the dataset is linearly inseparable, establishing an insurmountable barrier for purely linear units.

## 1.3 Non-linearity and activation functions

To overcome the linear stacking collapse and solve non-linearly separable problems such as XOR, neural networks must interleave affine transformations with point-wise non-linear mapping functions. These mappings, termed activation functions, provide the mathematical mechanism that enables multi-layer networks to approximate arbitrary functional relationships.

### 1.3.1 Coordinate space folding and the resolution of XOR

Non-linear activation functions resolve the linear inseparability barrier by non-linearly warping and folding the input coordinate space. When input points are projected through an intermediate non-linear layer, the underlying geometric distances are altered such that points belonging to different classes, initially entangled in the original space, become linearly separable in the transformed hidden space [9].

To illustrate this mechanism concretely, we examine the analytical solution to the XOR problem using a two-layer Multi-Layer Perceptron (MLP) equipped with Rectified Linear Unit (ReLU) activations [9]:

$$\mathbf{h} = g\left(\mathbf{W}^\top \mathbf{x} + \mathbf{c}\right), \quad y = \mathbf{w}^\top \mathbf{h} + b, \tag{1.32}$$

where $g(z) = \max(0, z)$ applies element-wise. Let the network parameters be configured as:

$$\mathbf{W} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \quad \mathbf{c} = \begin{bmatrix} 0 \\ -1 \end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix} 1 \\ -2 \end{bmatrix}, \quad b = 0. \tag{1.33}$$

The hidden layer pre-activation vector is given by:

$$\mathbf{z} = \mathbf{W}^\top \mathbf{x} + \mathbf{c} = \begin{bmatrix} x_1 + x_2 \\ x_1 + x_2 - 1 \end{bmatrix}. \tag{1.34}$$

Applying the non-linear ReLU activation $g(\mathbf{z})$ evaluates the hidden representations $\mathbf{h} = [h_1, h_2]^\top$ for all four input patterns:

$$\mathbf{x} = [0,0]^\top \implies \mathbf{z} = [0,-1]^\top \implies \mathbf{h} = [\max(0,0), \max(0,-1)]^\top = [0,0]^\top, \tag{1.35}$$
$$\mathbf{x} = [0,1]^\top \implies \mathbf{z} = [1,0]^\top \implies \mathbf{h} = [\max(0,1), \max(0,0)]^\top = [1,0]^\top, \tag{1.36}$$
$$\mathbf{x} = [1,0]^\top \implies \mathbf{z} = [1,0]^\top \implies \mathbf{h} = [\max(0,1), \max(0,0)]^\top = [1,0]^\top, \tag{1.37}$$
$$\mathbf{x} = [1,1]^\top \implies \mathbf{z} = [2,1]^\top \implies \mathbf{h} = [\max(0,2), \max(0,1)]^\top = [2,1]^\top. \tag{1.38}$$

The geometric effect of the non-linearity is readily visible: the inputs $(0,1)$ and $(1,0)$, which occupied disjoint coordinates in the original input space, are mapped to the identical latent coordinate $\mathbf{h} = [1,0]^\top$.

The output layer computes a linear combination of the hidden coordinates:

$$y = \mathbf{w}^\top \mathbf{h} + b = 1 \cdot h_1 - 2 \cdot h_2 + 0. \tag{1.39}$$

Evaluating the output on each transformed latent point yields:

$$\mathbf{x} = [0,0]^\top \implies y = 1(0) - 2(0) = 0, \tag{1.40}$$
$$\mathbf{x} = [0,1]^\top \implies y = 1(1) - 2(0) = 1, \tag{1.41}$$
$$\mathbf{x} = [1,0]^\top \implies y = 1(1) - 2(0) = 1, \tag{1.42}$$
$$\mathbf{x} = [1,1]^\top \implies y = 1(2) - 2(1) = 0. \tag{1.43}$$

The model reproduces the XOR truth table with exact precision. By collapsing $(0,1)$ and $(1,0)$ onto a shared latent location while projecting $(1,1)$ away along the second dimension ($h_2 = 1$), the non-linear activation folded the coordinate space, transforming a linearly inseparable problem into a linearly separable representation.

### 1.3.2 The universal approximation theorem

The theoretical expressive power conferred by non-linear hidden units is formalized by the Universal Approximation Theorem (UAT), proved independently by George Cybenko (1989) for sigmoidal activations [13] and Kurt Hornik, Maxwell Stinchcombe, and Halbert White (1989) for general non-linearities [14].

**Theorem 1.1:** *Universal approximation theorem*

Let $\sigma : \mathbb{R} \to \mathbb{R}$ be a continuous, bounded, non-constant activation function. Let $I_n = [0,1]^n$ denote the compact $n$-dimensional unit hypercube, and let $C(I_n)$ denote the space of continuous real-valued functions on $I_n$. Then, for any target function $f \in C(I_n)$ and any arbitrary error tolerance $\epsilon > 0$, there exists an integer $M \in \mathbb{N}$, a set of real coefficients $v_j \in \mathbb{R}$, weight vectors $\mathbf{w}_j \in \mathbb{R}^n$, and biases $b_j \in \mathbb{R}$ such that the finite sum:

$$F(\mathbf{x}) = \sum_{j=1}^{M} v_j \sigma\left(\mathbf{w}_j^\top \mathbf{x} + b_j\right) \tag{1.44}$$

satisfies:

$$\sup_{\mathbf{x} \in I_n} \left| F(\mathbf{x}) - f(\mathbf{x}) \right| < \epsilon. \tag{1.45}$$

In functional analysis terms, the family of single-hidden-layer feed-forward networks is dense in $C(I_n)$ with respect to the uniform supremum metric.

*Constructive intuition via localized bump functions.* The mathematical mechanism underlying Cybenko's proof can be understood through the constructive superposition of localized rectangular "bump" functions. Consider a one-dimensional input $x \in \mathbb{R}$ and a standard logistic sigmoid activation $\sigma(z) = \frac{1}{1+e^{-z}}$. By taking the difference between two sigmoids displaced by a distance $\Delta = b - a$ and scaled by an arbitrary steepness parameter $w \gg 0$, one constructs a localized pulse or bump function:

$$B(x; a, b, w) = \sigma(w(x - a)) - \sigma(w(x - b)). \tag{1.46}$$

As the scaling factor $w \to \infty$, the sigmoid transitions approximate ideal Heaviside steps, causing $B(x)$ to converge to a rectangular gate function that equals 1 on the interval $(a, b)$ and vanishes outside:

$$\lim_{w \to \infty} B(x; a, b, w) = \begin{cases} 1 & \text{if } x \in (a, b), \\ 0 & \text{otherwise.} \end{cases} \tag{1.47}$$

In an $n$-dimensional domain, intersecting $2n$ such sigmoidal half-spaces constructs an $n$-dimensional hypercubical localized bump function that isolates a localized neighborhood of volume $\epsilon^n$. By scaling the amplitude of each bump to match the local value of the target function $f(\mathbf{x})$ and tiling the compact domain $I_n$ with a finite collection of such bumps, the network constructs a piecewise-constant Riemann approximation of $f(\mathbf{x})$. Because continuous functions on compact domains are uniformly continuous, taking sufficiently fine partitions guarantees that the approximation error remains strictly bounded below $\epsilon$.

While the Universal Approximation Theorem provides a foundational mathematical guarantee, two critical caveats govern its practical interpretation:

(i) **Existence versus learnability**: The theorem is purely an existence proof. It guarantees that a configuration of parameters exists that approximates $f$, but provides no assurance that gradient-based optimization algorithms (such as gradient descent) will successfully discover those parameters from finite, noisy training samples.

(ii) **Width versus depth efficiency**: The theorem guarantees approximation using a single hidden layer, but the required hidden width $M$ may scale exponentially with the input dimension $n$. As established in Section 1.1.3, deep hierarchical composition constructs multi-scale piecewise linear partitions using polynomial parameter budgets, rendering deep architectures exponentially more parameter-efficient than shallow networks of equivalent capacity.

### 1.3.3 Taxonomy of activation functions

The choice of non-linear activation function directly influences gradient flow, convergence velocity, and representational capacity across deep networks. Table 1.1 summarizes the mathematical formulations, analytical derivatives, output ranges, and primary failure modes of the canonical activation functions.

Table 1.1: Comparative taxonomy of modern neural activation functions.

| Function | Formulation $\sigma(z)$ | Derivative $\frac{d\sigma}{dz}$ | Range | Zero-Centered | Primary Failure Mode |
|---|---|---|---|---|---|
| **Sigmoid** | $\frac{1}{1+e^{-z}}$ | $\sigma(z)(1-\sigma(z))$ | $(0,1)$ | No | Severe gradient saturation; zig-zag updates |
| **Tanh** | $\frac{e^z-e^{-z}}{e^z+e^{-z}}$ | $1-\tanh^2(z)$ | $(-1,1)$ | Yes | Gradient saturation at extreme inputs |
| **ReLU** | $\max(0,z)$ | $\mathbf{1}_{z>0}$ | $[0,\infty)$ | No | "Dying ReLU" (inactive negative half-plane) |
| **Leaky ReLU** | $\max(\alpha z, z)$ | $\mathbf{1}_{z>0}+\alpha\mathbf{1}_{z\leq 0}$ | $(-\infty,\infty)$ | Partial | Hyperparameter sensitivity of slope $\alpha$ |
| **GELU** | $z\Phi(z)$ | $\Phi(z)+z\phi(z)$ | $(-0.17,\infty)$ | Partial | Higher transcendental computational cost |
| **SwiGLU** | $\text{Swish}(\mathbf{xW})\otimes(\mathbf{xV})$ | Gated Jacobian | $(-\infty,\infty)$ | Partial | Increases parameter overhead by 1.5× |
| **Softmax** | $\frac{e^{z_i}}{\sum_j e^{z_j}}$ | $S_i(\delta_{ij}-S_j)$ | $(0,1)$ | No | Sensitive to logit scale; $O(\lvert\mathcal{V}\rvert)$ cost |

#### Logistic sigmoid

The logistic sigmoid function squashes the entire real line into a bounded interval:

$$\sigma(z)=\frac{1}{1+e^{-z}}=\frac{e^z}{e^z+1}. \tag{1.48}$$

Its first derivative can be written directly in terms of its function values:

$$\frac{d\sigma}{dz}=\frac{e^{-z}}{(1+e^{-z})^2}=\left(\frac{1}{1+e^{-z}}\right)\left(\frac{e^{-z}}{1+e^{-z}}\right)=\sigma(z)\left(1-\sigma(z)\right). \tag{1.49}$$

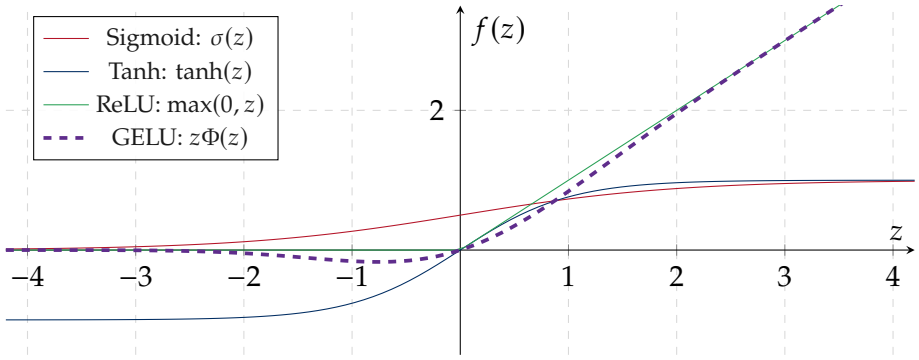



Figure 1.3: Comparison of primary activation functions across the operational domain $z \in [-4, 4]$.

Because the output is bounded in $(0, 1)$, the sigmoid function maps real logits to valid Bernoulli probabilities, making it the canonical choice for binary classification output layers.

However, when employed within deep hidden layers, the sigmoid function exhibits two severe pathologies:

(i) **Gradient saturation**: The maximum value of the derivative occurs at the origin, where $\sigma'(0) = 0.5(1 - 0.5) = 0.25$. When the magnitude of the pre-activation becomes moderately large ($|z| > 4$), the derivative rapidly decays to zero ($\sigma'(z) \approx 0$). During backpropagation, error gradients are scaled by this factor; compounding multiple values bounded above by 0.25 causes the backpropagated gradient to vanish exponentially with depth.

(ii) **Non-zero-centered outputs**: Because $\sigma(z) \in (0, 1)$, the activations entering subsequent layers are strictly positive ($a_i > 0$). In a linear transformation $z = \sum_i w_i a_i + b$, the gradient of the loss with respect to weight $w_i$ is $\frac{\partial \mathcal{L}}{\partial w_i} = \frac{\partial \mathcal{L}}{\partial z} a_i$. Because all $a_i > 0$, the partial derivatives for all incoming weights to a unit share the exact sign of $\frac{\partial \mathcal{L}}{\partial z}$. Consequently, parameter updates can only move simultaneously in all-positive or all-negative directions, inducing inefficient zig-zag optimization trajectories.

### Hyperbolic tangent

The hyperbolic tangent (tanh) function resolves the non-zero-centered limitation of the sigmoid by mapping inputs into the symmetric, zero-centered interval $(-1, 1)$:

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} = 2\sigma(2z) - 1. \tag{1.50}$$

Its first derivative evaluates to:

$$\frac{d \tanh(z)}{dz} = 1 - \tanh^2(z). \tag{1.51}$$

At the origin, the derivative evaluates to $\tanh'(0) = 1.0$, which is four times larger than that of the sigmoid function, facilitating stronger initial gradient propagation. Furthermore, because outputs have an expected value near zero under symmetric weight initializations, the covariance shifts that produce zig-zag gradient dynamics are mitigated. Nonetheless, the function remains susceptible to saturation plateaus: when $|z| \geq 3$, $|\tanh(z)| \approx 1$ and $\tanh'(z) \to 0$, causing gradients to vanish in deep architectures.

### Rectified linear unit

The Rectified Linear Unit (ReLU), popularized in deep learning by Glorot and Bengio [15], introduces a non-saturating threshold operator:

$$\mathrm{ReLU}(z) = \max(0, z) = \begin{cases} z & \text{if } z > 0, \\ 0 & \text{if } z \leq 0. \end{cases} \tag{1.52}$$

Its derivative is piecewise constant:

$$\frac{d\mathrm{ReLU}(z)}{dz} = \begin{cases} 1 & \text{if } z > 0, \\ 0 & \text{if } z < 0, \end{cases} \tag{1.53}$$

with the sub-gradient convention setting the derivative to 0 (or 0.5) at the non-differentiable point $z = 0$.

The advantages of ReLU over sigmoidal activations are structural:

(i) **Non-saturating gradient flow**: For positive pre-activations ($z > 0$), the derivative is identically equal to 1.0. The gradient passes through the unit without attenuation, mitigating the vanishing gradient problem across deep feed-forward and convolutional networks.

(ii) **Computational efficiency**: Evaluating $\max(0, z)$ requires only a comparison and conditional assignment, avoiding expensive floating-point exponentials.

(iii) **Induced sparsity**: When $z \leq 0$, the unit outputs exactly zero. In large networks, a significant proportion of units are inactive for any given input, yielding sparse representations that improve interpretability and computational economy.

*The dying ReLU pathology.* Despite these advantages, ReLU networks suffer from the "dying ReLU" problem [10]. If a large gradient update shifts a neuron's incoming weights and bias such that its pre-activation $z = \mathbf{w}^\top \mathbf{x} + b$ is negative for all training examples, the unit outputs zero and its derivative evaluates to zero across the entire dataset:

$$\forall \mathbf{x} \in \mathcal{D}_{\text{train}}, \quad z(\mathbf{x}) < 0 \implies \frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \frac{\partial \mathcal{L}}{\partial z}\mathbf{x} = \mathbf{0}. \tag{1.54}$$

Once a unit enters this regime, gradient descent cannot provide non-zero updates to alter its incoming weights, permanently deactivating the neuron.

### Leaky rectified linear unit and variants

To eliminate the dead state of standard ReLUs, the Leaky ReLU assigns a small, fixed positive slope $\alpha > 0$ to the negative half-plane:

$$\mathrm{LReLU}(z) = \max(\alpha z, z) = \begin{cases} z & \text{if } z > 0, \\ \alpha z & \text{if } z \leq 0, \end{cases} \tag{1.55}$$

typically parameterized with $\alpha = 0.01$. Its derivative evaluates to:

$$\frac{d\mathrm{LReLU}(z)}{dz} = \begin{cases} 1 & \text{if } z > 0, \\ \alpha & \text{if } z \leq 0. \end{cases} \tag{1.56}$$

Because the gradient remains non-zero ($\alpha > 0$) for negative inputs, neurons receive corrective gradients even when inactive, preventing permanent deactivation. In Parametric ReLU (PReLU), the slope $\alpha$ is treated as a learnable parameter optimized jointly via backpropagation.

### Gaussian error linear unit

The Gaussian Error Linear Unit (GELU), formulated by Hendrycks and Gimpel in 2016 [16], serves as the standard activation function across modern foundation models, including BERT, GPT-2, and GPT-3. GELU integrates stochastic regularization directly into a deterministic activation function by weighting inputs by their probability under a standard Gaussian cumulative distribution:

$$\text{GELU}(z) = z\Phi(z) = zP(X \leq z), \quad \text{where} \quad X \sim \mathcal{N}(0,1). \tag{1.57}$$

The cumulative normal distribution $\Phi(z)$ is defined by the error function erf:

$$\Phi(z) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{z} e^{-\frac{t^2}{2}} dt = \frac{1}{2}\left[1 + \text{erf}\left(\frac{z}{\sqrt{2}}\right)\right]. \tag{1.58}$$

For efficient hardware execution, GELU is frequently evaluated using a fast cubic approximation:

$$\text{GELU}(z) \approx 0.5z\left(1 + \tanh\left(\sqrt{\frac{2}{\pi}}\left(z + 0.044715z^3\right)\right)\right). \tag{1.59}$$

Unlike ReLU, GELU is non-monotonic, smooth, and continuously differentiable everywhere. For negative inputs, GELU exhibits a small curvature dip with a local minimum at $z \approx -0.7517$ where $f(z) \approx -0.170$, before smoothly approaching zero as $z \to -\infty$. This smooth transition allows small negative activations to convey gradient information while suppressing large negative outliers.

### SwiGLU and gated linear units

Modern Large Language Models (such as LLaMA, Mistral, and PaLM) employ Gated Linear Unit (GLU) variants within their feed-forward blocks, specifically SwiGLU [17]. A standard GLU computes the element-wise product of two linear transformations, one of which is modulated by a gating activation:

$$\text{GLU}(\mathbf{x}, \mathbf{W}, \mathbf{V}, \mathbf{b}, \mathbf{c}) = (\mathbf{x}\mathbf{W} + \mathbf{b}) \otimes \sigma\left(\mathbf{x}\mathbf{V} + \mathbf{c}\right), \tag{1.60}$$

where $\otimes$ denotes the element-wise Hadamard product. In SwiGLU, the logistic sigmoid gate is replaced by the Swish activation ($\text{Swish}_\beta(z) = z\sigma(\beta z)$ with $\beta = 1$):

$$\text{SwiGLU}(\mathbf{x}) = (\mathbf{x}\mathbf{W}) \otimes \text{Swish}\left(\mathbf{x}\mathbf{V}\right). \tag{1.61}$$

This bilinear multiplicative interaction enables hidden representations to dynamically modulate individual feature channels, yielding superior convergence stability and empirical language modeling performance, offset by a 1.5× parameter expansion within the intermediate MLP projection.

### The softmax operator

When neural network outputs represent categorical probability distributions over $C$ mutually exclusive classes, the scalar unconstrained pre-activation logits $\mathbf{z} = [z_1, z_2, \ldots, z_C]^\top \in \mathbb{R}^C$ must be mapped onto the probability simplex:

$$\Delta^{C-1} = \left\{ \mathbf{p} \in \mathbb{R}^C \;\middle|\; p_i \geq 0, \; \sum_{i=1}^{C} p_i = 1 \right\}. \tag{1.62}$$

This transformation is performed by the Softmax operator:

$$\hat{y}_i = \text{softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{C} e^{z_j}}. \tag{1.63}$$

Because the exponential function is strictly positive ($e^{z_i} > 0$), all output probabilities are strictly positive, and normalizing by the partition function $\sum_j e^{z_j}$ guarantees unit sum.

To compute the Jacobian matrix of the Softmax operator, we evaluate the partial derivative of output $\hat{y}_i$ with respect to logit $z_k$:

$$\frac{\partial \hat{y}_i}{\partial z_k} = \frac{\partial}{\partial z_k}\left(\frac{e^{z_i}}{\sum_{j=1}^{C} e^{z_j}}\right). \tag{1.64}$$

Applying the quotient rule, we distinguish two cases. When $i = k$:

$$\frac{\partial \hat{y}_i}{\partial z_i} = \frac{e^{z_i}\sum_j e^{z_j} - e^{z_i}e^{z_i}}{\left(\sum_j e^{z_j}\right)^2} = \hat{y}_i - \hat{y}_i^2 = \hat{y}_i(1 - \hat{y}_i). \tag{1.65}$$

When $i \neq k$:

$$\frac{\partial \hat{y}_i}{\partial z_k} = \frac{0 \cdot \sum_j e^{z_j} - e^{z_i}e^{z_k}}{\left(\sum_j e^{z_j}\right)^2} = -\hat{y}_i\hat{y}_k. \tag{1.66}$$

Combining both cases using the Kronecker delta $\delta_{ik}$ yields the compact Jacobian formulation:

$$\frac{\partial \hat{y}_i}{\partial z_k} = \hat{y}_i\left(\delta_{ik} - \hat{y}_k\right). \tag{1.67}$$

*Numerical stability via shift invariance.* Direct floating-point evaluation of Equation (1.63) causes numerical overflow if $z_i \gg 0$ (e.g., $e^{1000} \to \infty$ yielding `NaN`) or underflow if all $z_i \ll 0$. The Softmax operator satisfies the shift-invariance property: for any arbitrary scalar constant $c \in \mathbb{R}$:

$$\text{softmax}(\mathbf{z} + c)_i = \frac{e^{z_i + c}}{\sum_j e^{z_j + c}} = \frac{e^c e^{z_i}}{e^c \sum_j e^{z_j}} = \frac{e^{z_i}}{\sum_j e^{z_j}} = \text{softmax}(\mathbf{z})_i. \tag{1.68}$$

In practical implementations, numerical stability is guaranteed by choosing $c = -\max_k z_k$. The largest exponentiated term evaluates to $e^0 = 1.0$, which prevents overflow while ensuring the denominator contains at least one non-vanishing term.

## 1.4 Loss functions and optimization dynamics

To train a parameterized neural network, one must define a scalar criterion that quantifies the discrepancy between model predictions and empirical ground truth. This objective, termed the loss function, provides the supervisory signal that guides parameter optimization.

### 1.4.1 The operational role of loss functions

Formally, let $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$ denote a training dataset sampled from an underlying data-generating distribution $P_{\text{data}}(\mathbf{x}, y)$. A neural network parameterized by $\theta \in \mathbb{R}^P$ evaluates a forward prediction mapping $\hat{y}_i = f(\mathbf{x}_i; \theta)$. The loss function $\mathcal{L}(\hat{y}_i, y_i) \in \mathbb{R}_{\geq 0}$ assigns a non-negative scalar penalty to prediction errors.

Under the framework of Empirical Risk Minimization (ERM), the training objective is to discover a parameter configuration $\theta^*$ that minimizes the empirical risk across the observed sample:

$$\theta^* = \arg\min_\theta \mathcal{R}_{\text{emp}}(\theta) = \arg\min_\theta \left[ \frac{1}{N} \sum_{i=1}^N \mathcal{L}\left(f(\mathbf{x}_i; \theta), y_i\right) \right]. \tag{1.69}$$

The loss function serves as the sole mathematical feedback channel through which parameter errors are communicated to optimization algorithms. Without a well-defined scalar loss, the system possesses no objective measure of performance and cannot formulate directional gradient vectors for parameter improvement.

### 1.4.2 Task-conditioned output topologies

A fundamental principle of statistical machine learning is that a single universal loss function cannot exist. Different tasks operate across distinct mathematical domains:

(i) **Continuous regression**: Target variables span the unconstrained real line $y \in \mathbb{R}$ or vector spaces $\mathbb{R}^K$.

(ii) **Binary classification**: Targets represent dichotomous categorical indicators $y \in \{0, 1\}$.

(iii) **Multi-class classification**: Targets represent discrete class memberships $y \in \{1, \ldots, C\}$, geometrically residing on the probability simplex $\Delta^{C-1}$.

To design mathematically principled loss functions across these diverse regimes, Simon Prince [10] formulates the Maximum Likelihood Estimation (MLE) recipe:

Step 1: Define an appropriate conditional probability distribution $P(y \mid \mathbf{x}; \phi)$ over the target space, parameterized by vector $\phi$.

Step 2: Configure the neural network to output the parameters $\phi = f(\mathbf{x}; \theta)$ of that chosen distribution (e.g., predicting mean $\mu$ for a Gaussian, or probability $p$ for a Bernoulli).

Step 3: Train the network to maximize the log-likelihood of the observed dataset, or equivalently, minimize the negative log-likelihood (NLL):

$$\mathcal{L}(\theta) = -\sum_{i=1}^N \log P\left(y_i \,\middle|\, \mathbf{x}_i; \theta\right). \tag{1.70}$$

This recipe ensures that the loss function is mathematically consistent with the output geometry and noise distribution of the underlying task.

### 1.4.3 Task-specific loss taxonomy and statistical derivations

Applying the maximum likelihood recipe across canonical distributions produces the standard loss taxonomy employed in modern neural networks.

#### Regression via Gaussian noise models

In continuous regression tasks, we assume the true target $y \in \mathbb{R}$ differs from the model's deterministic prediction $\hat{y} = f(\mathbf{x}; \theta)$ by additive zero-mean Gaussian noise $\epsilon \sim \mathcal{N}(0, \sigma^2)$:

$$y = f(\mathbf{x}; \theta) + \epsilon \implies P(y \mid \mathbf{x}; \theta, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y - f(\mathbf{x}; \theta))^2}{2\sigma^2}\right). \tag{1.71}$$

Evaluating the negative log-likelihood over $N$ independently and identically distributed observations yields:

$$\begin{aligned} \mathcal{L}_{\text{NLL}}(\theta) &= -\sum_{i=1}^{N} \log\left[\frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - f(\mathbf{x}_i; \theta))^2}{2\sigma^2}\right)\right] \\ &= \frac{N}{2} \log\left(2\pi\sigma^2\right) + \frac{1}{2\sigma^2} \sum_{i=1}^{N} \left(y_i - f(\mathbf{x}_i; \theta)\right)^2. \end{aligned} \tag{1.72}$$

Assuming homoscedastic noise ($\sigma^2$ constant), discarding constant terms, and scaling by $\frac{1}{N}$ recovers the standard Mean Squared Error (MSE) objective:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^{N} \left(y_i - \hat{y}_i\right)^2. \tag{1.73}$$

If the noise distribution is modeled instead as Laplacian noise $\epsilon \sim \text{Laplace}(0, b)$ with density $p(\epsilon) = \frac{1}{2b} \exp(-|\epsilon|/b)$, minimizing the negative log-likelihood recovers the Mean Absolute Error (MAE) or $L_1$ loss:

$$\mathcal{L}_{\text{MAE}} = \frac{1}{N} \sum_{i=1}^{N} \left|y_i - \hat{y}_i\right|, \tag{1.74}$$

which exhibits enhanced robustness to dataset outliers because penalties scale linearly rather than quadratically.

#### Binary classification via Bernoulli distributions

For binary classification ($y \in \{0, 1\}$), the target is modeled as a Bernoulli random variable conditioned on the input:

$$P(y \mid \mathbf{x}) = \hat{y}^y \left(1 - \hat{y}\right)^{1-y}, \quad \text{where} \quad \hat{y} = \sigma(z) \in (0, 1). \tag{1.75}$$

Evaluating the negative log-likelihood of this distribution yields the Binary Cross-Entropy (BCE) loss:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N} \sum_{i=1}^{N} \left[y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)\right]. \tag{1.76}$$

The expression functions as a ground-truth selector:

(i) When ground-truth $y_i = 1$, the second term $(1 - y_i)$ vanishes identically, reducing the penalty to $-\log \hat{y}_i$. The loss approaches 0 as $\hat{y}_i \to 1$, but penalizes incorrect predictions with an asymptote approaching $+\infty$ as $\hat{y}_i \to 0$.

(ii) When ground-truth $y_i = 0$, the first term vanishes, reducing the penalty to $-\log(1 - \hat{y}_i)$, which penalizes predictions approaching $\hat{y}_i \to 1$ with infinite cost.

**Multi-class classification via categorical distributions**

In multi-class classification over $C$ mutually exclusive categories ($y \in \{1, \dots, C\}$), targets are represented as one-hot vectors $\mathbf{y} \in \{0,1\}^C$ where $\sum_{k=1}^{C} y_k = 1$. The network predicts a categorical distribution $\hat{\mathbf{y}} = \text{softmax}(\mathbf{z}) \in \Delta^{C-1}$. The conditional likelihood is given by the multinomial density:

$$P(\mathbf{y} \mid \mathbf{x}) = \prod_{k=1}^{C} \hat{y}_k^{y_k}. \tag{1.77}$$

Minimizing the negative log-likelihood yields the Categorical Cross-Entropy (CCE) loss:

$$\mathcal{L}_{\text{CCE}} = -\sum_{k=1}^{C} y_k \log \hat{y}_k = -\log \hat{y}_c, \tag{1.78}$$

where $c \in \{1, \dots, C\}$ denotes the index of the true positive class ($y_c = 1$).

**Analytical proof of MSE classification failure**

A common pedagogical error involves applying Mean Squared Error to classification tasks equipped with sigmoidal outputs. To demonstrate why this combination fails, we evaluate the gradient of the loss with respect to the pre-activation logit $z$ for a binary classifier ($\hat{y} = \sigma(z)$) under both MSE and BCE.

*Case 1: MSE logit gradient.* The MSE loss for a single observation is $\mathcal{L}_{\text{MSE}} = (y - \hat{y})^2$. Applying the chain rule:

$$\frac{\partial \mathcal{L}_{\text{MSE}}}{\partial z} = \frac{\partial \mathcal{L}_{\text{MSE}}}{\partial \hat{y}} \frac{d\hat{y}}{dz} = -2\left(y - \hat{y}\right)\sigma'(z) = -2\left(y - \hat{y}\right)\hat{y}\left(1 - \hat{y}\right). \tag{1.79}$$

Now, evaluate the gradient when the model is confident and wrong: suppose the true label is $y = 1$, but the network currently produces a strongly negative logit $z = -5$, such that $\hat{y} = \sigma(-5) \approx 0.0067$. The residual error $(y - \hat{y}) = 1 - 0.0067 = 0.9933$ is maximal. However, the derivative evaluates to:

$$\frac{\partial \mathcal{L}_{\text{MSE}}}{\partial z} = -2(0.9933) \cdot (0.0067)(0.9933) \approx -0.0132 \approx 0. \tag{1.80}$$

Because the sigmoid derivative saturates ($\sigma'(z) \to 0$ as $|z| \gg 0$), the gradient vanishes despite maximal prediction error. The parameter updates stall, preventing gradient descent from correcting catastrophic misclassifications.

*Case 2: BCE logit gradient.* Evaluating the logit gradient under Binary Cross-Entropy:

$$\begin{aligned}
\frac{\partial \mathcal{L}_{\text{BCE}}}{\partial z} &= \frac{\partial \mathcal{L}_{\text{BCE}}}{\partial \hat{y}} \frac{d\hat{y}}{dz} \\
&= \left[-\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}}\right]\hat{y}\left(1-\hat{y}\right) \\
&= \frac{-y(1-\hat{y}) + (1-y)\hat{y}}{\hat{y}(1-\hat{y})}\hat{y}\left(1-\hat{y}\right) \\
&= -y + y\hat{y} + \hat{y} - y\hat{y} \\
&= \hat{y} - y.
\end{aligned} \tag{1.81}$$

Under cross-entropy, the saturating term $\hat{y}(1-\hat{y})$ in the denominator of $\frac{\partial \mathcal{L}}{\partial \hat{y}}$ cancels the identical term in the sigmoid derivative $\frac{d\hat{y}}{dz}$. The resulting gradient evaluates to the linear residual error

$\hat{y} - y$. If $y = 1$ and $\hat{y} \approx 0.0067$, the gradient evaluates to $\hat{y} - y = 0.0067 - 1 = -0.9933$, providing a strong, unattenuated corrective signal.

Furthermore, analyzing the second derivative demonstrates that $\frac{\partial^2 \mathcal{L}_{\text{MSE}}}{\partial z^2}$ flips signs across logit space, creating non-convex regions and saddle plateaus [9]. Conversely, for BCE:

$$\frac{\partial^2 \mathcal{L}_{\text{BCE}}}{\partial z^2} = \frac{d}{dz}\left(\hat{y} - y\right) = \sigma'(z) = \hat{y}\left(1 - \hat{y}\right) > 0, \quad \forall z \in \mathbb{R}. \tag{1.82}$$

The strictly positive second derivative proves that Binary Cross-Entropy is strictly convex with respect to the output logits, guaranteeing that no non-convex plateaus impede convergence.

### 1.4.4 The analytical intractability of direct root finding

In classical linear regression with quadratic loss, the parameter optimization problem admits an exact, closed-form analytical solution. Consider an input data matrix $\mathbf{X} \in \mathbb{R}^{N \times D}$ and target vector $\mathbf{y} \in \mathbb{R}^N$. The quadratic loss function is:

$$\mathcal{L}(\mathbf{w}) = \frac{1}{2N} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 . \tag{1.83}$$

To minimize this loss, one computes the gradient with respect to $\mathbf{w}$ and sets it equal to the zero vector:

$$\nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}) = \frac{1}{N} \mathbf{X}^\top (\mathbf{X}\mathbf{w} - \mathbf{y}) = \mathbf{0}. \tag{1.84}$$

This condition reduces to the system of linear equations:

$$\mathbf{X}^\top \mathbf{X}\mathbf{w} = \mathbf{X}^\top \mathbf{y}, \tag{1.85}$$

which, assuming $\mathbf{X}^\top \mathbf{X}$ is non-singular, yields the Ordinary Least Squares (OLS) normal equations:

$$\mathbf{w}^* = \left(\mathbf{X}^\top \mathbf{X}\right)^{-1} \mathbf{X}^\top \mathbf{y}. \tag{1.86}$$

In sharp contrast, setting $\nabla_\theta \mathcal{L}(\theta) = \mathbf{0}$ fails completely for deep neural networks. In multi-layer architectures, the model output is constructed through nested compositions of affine transformations and non-linear activation functions:

$$\hat{y} = f(\mathbf{x}; \theta) = \mathbf{W}_L^\top \sigma\left(\mathbf{W}_{L-1}^\top \dots \sigma\left(\mathbf{W}_1^\top \mathbf{x} + \mathbf{b}_1\right) + \mathbf{b}_{L-1}\right) + \mathbf{b}_L. \tag{1.87}$$

When differentiated, the parameters $\theta = \{\mathbf{W}_l, \mathbf{b}_l\}_{l=1}^L$ appear inside complex transcendental functions (exponentials, piecewise steps, products). Setting the resulting gradient equations to zero yields a coupled system of non-linear transcendental algebraic equations with millions of unknowns:

$$\nabla_\theta \mathcal{L}(\theta) = \sum_{i=1}^{N} \frac{\partial \mathcal{L}_i}{\partial f_i} \nabla_\theta f(\mathbf{x}_i; \theta) = \mathbf{0}. \tag{1.88}$$

This system possesses no closed-form analytical root. Moreover, the loss surface $\mathcal{L}(\theta)$ is highly non-convex, characterized by complex landscapes with multiple local minima, degenerate saddle points, and ill-conditioned ravines. Consequently, deep neural networks cannot be solved analytically; their optimization requires iterative, numerical gradient-based methods.

### 1.4.5 Gradient descent and modern optimization heuristics

Iterative numerical optimization computes successive parameter updates by evaluating local first-order gradient vectors.

### Vanilla gradient descent

The foundational iterative algorithm is Gradient Descent (GD). At each iteration $t$, the algorithm updates parameter vector $\theta_t$ by taking a step proportional to the negative gradient of the empirical risk:

$$\theta_{t+1} = \theta_t - \alpha \nabla_\theta \mathcal{L}(\theta_t), \tag{1.89}$$

where $\alpha > 0$ denotes the learning rate hyperparameter.

While intuitive, vanilla gradient descent encounters three severe operational pathologies:

(i) **Sensitivity to learning rate**: If $\alpha$ is chosen too small, parameter updates crawl inefficiently, requiring prohibitive computational time. Conversely, if $\alpha$ exceeds the reciprocal of the maximum eigenvalue of the Hessian ($\alpha > \frac{2}{\lambda_{\max}}$), the updates overshoot valleys and diverge.

(ii) **Oscillations in ill-conditioned ravines**: When the loss surface exhibits high condition numbers (where curvature along one axis is drastically steeper than along another, $\lambda_{\max} \gg \lambda_{\min}$), gradient descent oscillates violently across the steep walls while making negligible progress along the flat base.

(iii) **Stagnation at saddle points**: In high-dimensional optimization landscapes, local minima are rare; the dominant topological features are saddle points where the gradient vanishes ($\nabla_\theta \mathcal{L} \approx \mathbf{0}$) but some directions exhibit negative curvature. Vanilla GD decelerates severely in the vicinity of saddle plateaus.

### Momentum

To dampen high-frequency oscillations across steep ravine walls and accelerate descent along flat valleys, Polyak (1964) introduced the momentum method [9]. Momentum simulates physical inertia by accumulating an exponentially decaying moving average of past gradients into a velocity vector $\mathbf{v}_t$:

$$\mathbf{v}_t = \beta \mathbf{v}_{t-1} + (1-\beta)\nabla_\theta \mathcal{L}(\theta_t), \tag{1.90}$$

$$\theta_{t+1} = \theta_t - \alpha \mathbf{v}_t, \tag{1.91}$$

where $\beta \in [0, 1)$ represents the momentum decay coefficient (typically $\beta = 0.9$). Along directions where the gradient repeatedly reverses sign, the oscillating terms cancel out in the running average, dampening instability. Along directions where the gradient consistently points in the same direction, the velocity accumulates to an effective magnitude $\frac{1}{1-\beta}$ times larger than the instantaneous gradient, accelerating convergence through plateaus.

### Adam: Adaptive Moment Estimation

In deep architectures and large language models, different parameters operate at widely disparate scales and receive gradients of vastly differing frequencies (e.g., embeddings of rare tokens receive sparse, large updates). Adaptive learning rate algorithms assign individual learning rates to each parameter.

The universal optimization standard across foundation models is Adam (Adaptive Moment Estimation), introduced by Kingma and Ba in 2014 [18]. Adam tracks both the exponentially decaying average of past gradients (first raw moment) and the exponentially decaying average of squared gradients (second uncentered moment):

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1-\beta_1)\mathbf{g}_t, \tag{1.92}$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1-\beta_2)\mathbf{g}_t^2, \tag{1.93}$$

where $\mathbf{g}_t = \nabla_\theta \mathcal{L}(\theta_t)$, and $\mathbf{g}_t^2$ denotes the element-wise Hadamard square $\mathbf{g}_t \odot \mathbf{g}_t$. The decay hyperparameters are standardly initialized to $\beta_1 = 0.9$ and $\beta_2 = 0.999$.

Because $\mathbf{m}_0$ and $\mathbf{v}_0$ are initialized to the zero vector $\mathbf{0}$, the moment vectors are biased toward zero, particularly during initial training steps when $\beta_1$ and $\beta_2$ are close to 1. To correct for this initialization artifact, Adam applies analytical bias corrections:

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t} \quad \text{and} \quad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t}. \tag{1.94}$$

The parameter update is then evaluated coordinate-wise:

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} \odot \hat{\mathbf{m}}_t, \tag{1.95}$$

where $\epsilon \approx 10^{-8}$ prevents division by zero. Adam dynamically rescales updates: parameters with large historical variance receive smaller, dampened steps, while parameters with sparse, small gradients receive amplified steps, ensuring robust optimization dynamics across diverse neural layers.

## 1.5 Computational graphs, backpropagation, and the feed-forward network

To execute gradient-based optimization algorithms such as Adam, the learning system must compute the partial derivatives of the scalar loss function with respect to every parameter in the network ($\nabla_\theta \mathcal{L}$). This section establishes the graph-theoretic formalism of computational graphs, derives the reverse-mode automatic differentiation algorithm, and synthesizes these mechanisms into a complete multi-layer feed-forward network.

### 1.5.1 Methods of gradient evaluation

As formalized by Christopher Bishop and Hugh Bishop (2024, Ch. 8) [8], there exist four distinct methodological approaches to computing derivatives in numerical systems:

(i) **Analytical differentiation**: The manual derivation of derivative expressions by hand using pencil and paper calculus. While mathematically exact, this approach is rigid, labor-intensive, and prone to human algebraic errors. Furthermore, modifying network architecture requires completely re-deriving the analytical formulas from scratch.

(ii) **Numerical differentiation (finite differences)**: The numerical approximation of partial derivatives via perturbation formulas:

$$\frac{\partial \mathcal{L}}{\partial \theta_i} \approx \frac{\mathcal{L}(\theta + \epsilon \mathbf{e}_i) - \mathcal{L}(\theta)}{\epsilon}, \tag{1.96}$$

where $\mathbf{e}_i$ denotes the $i$-th standard basis vector, and $\epsilon \approx 10^{-5}$. While universally applicable to black-box functions, numerical differentiation is computationally prohibitive: evaluating the gradient across a model with $D$ parameters requires $D + 1$ complete forward passes, yielding an asymptotic computational complexity of $O(D)$. For modern foundation models where $D \geq 10^9$, computing a single gradient step would require over one billion forward passes. Furthermore, finite differences suffer from numerical roundoff and truncation errors.

(iii) **Symbolic differentiation**: Computer algebra systems (such as Mathematica or SymPy) construct symbolic mathematical expressions for derivatives using calculus rules. While exact, symbolic differentiation treats mathematical expressions as abstract trees and suffers from exponential *expression swell*: redundant subexpressions are duplicated repeatedly across the expression tree, resulting in massive computational overhead. Moreover, symbolic engines cannot easily differentiate algorithms containing procedural control flow (loops, branches, and conditional recursion).

(iv) **Automatic differentiation**: The algorithmic evaluation of exact numerical derivatives by decomposing complex functions into sequences of elementary mathematical primitives whose derivatives are known. Rather than manipulating symbolic expressions, automatic differentiation evaluates numerical derivatives directly during program execution, providing exact gradients at machine precision with computational cost proportional to the forward function evaluation.

### 1.5.2 Computational graphs and reverse-mode automatic differentiation

Modern deep learning frameworks implement automatic differentiation by representing neural network execution as a Directed Acyclic Graph (DAG) $G = (\mathcal{V}, \mathcal{E})$ [8, 9].

(i) The set of vertices $\mathcal{V}$ represents computational operations (such as $+$, $\times$, exp, tanh) or intermediate tensor variables.

(ii) The set of directed edges $\mathcal{E}$ represents the directional flow of tensor values, where a directed edge $(u, v) \in \mathcal{E}$ indicates that node $v$ directly depends on the value computed by node $u$.

Forward evaluation corresponds to traversing the graph according to a topological sort: any node $v$ can be evaluated once all its incoming parent nodes parents($v$) have completed execution.

*Reverse-mode automatic differentiation (Backpropagation).* To compute gradients, reverse-mode automatic differentiation traverses the computational graph in reverse topological order, starting from the final scalar loss $\mathcal{L}$ and propagating gradient signals backward toward the input and parameter leaves.

Following the adjoint notation introduced by Linnainmaa (1970) and Rumelhart et al. (1986) [7], we define the adjoint $\bar{u}$ of any intermediate node $u$ as the partial derivative of the final scalar loss with respect to $u$:

$$\bar{u} \equiv \frac{\partial \mathcal{L}}{\partial u}. \tag{1.97}$$

The multivariable chain rule of calculus states that if a variable $u$ influences the loss $\mathcal{L}$ through multiple downstream dependent nodes $v \in \text{children}(u)$, the total sensitivity $\bar{u}$ is obtained by summing the contributions transmitted through all outgoing edges:

$$\bar{u} = \sum_{v \in \text{children}(u)} \bar{v} \frac{\partial v}{\partial u} = \sum_{v \in \text{children}(u)} \frac{\partial \mathcal{L}}{\partial v} \frac{\partial v}{\partial u}. \tag{1.98}$$

The computational efficiency of reverse-mode automatic differentiation is profound. Consider a function mapping $D$ parameters to a single scalar loss: $f : \mathbb{R}^D \to \mathbb{R}$.

(i) **Forward-mode automatic differentiation** computes directional derivatives (tangents $\dot{v} = \frac{\partial v}{\partial \theta_i}$) forward from the inputs. Computing the gradient with respect to all $D$ parameters requires $D$ distinct forward sweeps, scaling as $O(D)$.

(ii) **Reverse-mode automatic differentiation** initiates a single adjoint evaluation at the scalar loss ($\bar{\mathcal{L}} = \frac{\partial \mathcal{L}}{\partial \mathcal{L}} = 1.0$) and sweeps backward across the graph in reverse topological order. All $D$ partial derivatives are evaluated simultaneously in a single backward pass.

The time complexity of the reverse sweep is bounded by $O(|\mathcal{V}| + |\mathcal{E}|)$, which is approximately equal to twice the computational cost of the forward pass ($T_{\text{backward}} \approx 2 \times T_{\text{forward}}$), completely independent of the parameter dimension $D$. This fundamental efficiency renders backpropagation the engine of deep learning.

### 1.5.3 Worked arithmetic calculation of backpropagation

To demystify the abstract multivariable calculus of reverse-mode autodiff, we reconstruct the exact step-by-step arithmetic trace demonstrated in the course lecture slides.

Consider a single training observation $(x, y) \in \mathbb{R} \times \mathbb{R}$. The model computes a prediction $\hat{y}$ through the product of two scalar parameters $\theta_1, \theta_2 \in \mathbb{R}$:

$$\hat{y} = \theta_1 \theta_2 x. \tag{1.99}$$

The optimization objective is the scalar squared error loss:

$$\mathcal{L} = (\hat{y} - y)^2 = (\theta_1 \theta_2 x - y)^2. \tag{1.100}$$

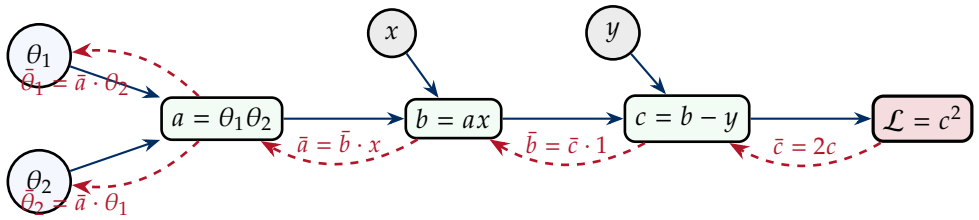



Figure 1.4: Computational graph representation for the lecture worked example $\mathcal{L} = (\theta_1\theta_2 x - y)^2$. Solid blue arrows indicate forward topological execution; dashed red arrows trace the reverse-mode adjoint propagation.

*Forward evaluation trace.* The computational graph decomposes this loss into a sequence of four elementary intermediate operations:

$$\text{Node } a = \theta_1\theta_2, \tag{1.101}$$
$$\text{Node } b = a \cdot x = \theta_1\theta_2 x, \tag{1.102}$$
$$\text{Node } c = b - y = \theta_1\theta_2 x - y, \tag{1.103}$$
$$\text{Node } \mathcal{L} = c^2 = \left(\theta_1\theta_2 x - y\right)^2. \tag{1.104}$$

*Backward adjoint propagation trace.* In the reverse pass, we evaluate the adjoints $\bar{u} = \frac{\partial \mathcal{L}}{\partial u}$ in reverse topological order:

Step 1: **Initialize root adjoint**: At the root node, the derivative of the loss with respect to itself is identically unity:

$$\bar{\mathcal{L}} = \frac{\partial \mathcal{L}}{\partial \mathcal{L}} = 1. \tag{1.105}$$

Step 2: **Adjoint of node** $c$: Differentiating $\mathcal{L} = c^2$ with respect to $c$:

$$\bar{c} = \frac{\partial \mathcal{L}}{\partial c} = \bar{\mathcal{L}}\frac{\partial \mathcal{L}}{\partial c} = 1 \cdot (2c) = 2\left(\theta_1\theta_2 x - y\right). \tag{1.106}$$

Step 3: **Adjoint of node** $b$: Differentiating $c = b - y$ with respect to $b$:

$$\bar{b} = \frac{\partial \mathcal{L}}{\partial b} = \bar{c}\frac{\partial c}{\partial b} = \bar{c} \cdot 1 = 2\left(\theta_1\theta_2 x - y\right). \tag{1.107}$$

Step 4: **Adjoint of node** $a$: Differentiating $b = a \cdot x$ with respect to $a$:

$$\bar{a} = \frac{\partial \mathcal{L}}{\partial a} = \bar{b}\frac{\partial b}{\partial a} = \bar{b} \cdot x = \left[2\left(\theta_1\theta_2 x - y\right)\right] x = 2x\left(\theta_1\theta_2 x - y\right). \tag{1.108}$$

Step 5: **Adjoint of parameter** $\theta_1$: Differentiating $a = \theta_1\theta_2$ with respect to $\theta_1$:

$$\bar{\theta}_1 = \frac{\partial \mathcal{L}}{\partial \theta_1} = \bar{a}\frac{\partial a}{\partial \theta_1} = \bar{a} \cdot \theta_2 = \left[2x\left(\theta_1\theta_2 x - y\right)\right]\theta_2 = 2\theta_2 x\left(\theta_1\theta_2 x - y\right). \tag{1.109}$$

Step 6: **Adjoint of parameter** $\theta_2$: Differentiating $a = \theta_1\theta_2$ with respect to $\theta_2$:

$$\bar{\theta}_2 = \frac{\partial \mathcal{L}}{\partial \theta_2} = \bar{a}\frac{\partial a}{\partial \theta_2} = \bar{a} \cdot \theta_1 = \left[2x\left(\theta_1\theta_2 x - y\right)\right]\theta_1 = 2\theta_1 x\left(\theta_1\theta_2 x - y\right). \tag{1.110}$$

With the exact partial derivatives computed, parameter updates under gradient descent evaluate to:

$$\theta_1 \leftarrow \theta_1 - \alpha\bar{\theta}_1 = \theta_1 - 2\alpha\theta_2 x\left(\theta_1\theta_2 x - y\right), \tag{1.111}$$

$$\theta_2 \leftarrow \theta_2 - \alpha\bar{\theta}_2 = \theta_2 - 2\alpha\theta_1 x\left(\theta_1\theta_2 x - y\right). \tag{1.112}$$

This minimal worked example illustrates the operational universality of reverse-mode automatic differentiation: regardless of model depth or topological branching, the global gradient is resolved into localized, modular partial derivative multiplications evaluated backward along computational edges.

### 1.5.4 The complete multi-layer feed-forward network

We now synthesize all established primitives—linear projections, non-linear activation functions, task-specific loss criteria, and reverse-mode automatic differentiation—into a unified, fully vectorized multi-layer Feed-Forward Network (FFN).

#### Vectorized layer equations

Consider an $L$-layer network. Let $\mathbf{x} \in \mathbb{R}^{d_0}$ denote the input vector, and define $\mathbf{a}^{(0)} \equiv \mathbf{x}$. For each layer $l \in \{1, 2, \ldots, L\}$:

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)\top}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \quad \mathbf{a}^{(l)} = f^{(l)}\left(\mathbf{z}^{(l)}\right), \tag{1.113}$$

where:

$$\mathbf{W}^{(l)} \in \mathbb{R}^{d_{l-1}\times d_l} \quad \text{denotes the weight transformation matrix for layer } l, \tag{1.114}$$

$$\mathbf{b}^{(l)} \in \mathbb{R}^{d_l} \quad \text{denotes the scalar bias vector for layer } l, \tag{1.115}$$

$$\mathbf{z}^{(l)} \in \mathbb{R}^{d_l} \quad \text{denotes the pre-activation logit vector,} \tag{1.116}$$

$$\mathbf{a}^{(l)} \in \mathbb{R}^{d_l} \quad \text{denotes the post-activation latent feature representation,} \tag{1.117}$$

$$f^{(l)}(\cdot) \quad \text{denotes the element-wise non-linear activation function.} \tag{1.118}$$

The final output vector $\hat{\mathbf{y}} = \mathbf{a}^{(L)} \in \mathbb{R}^{d_L}$ produces predictions tailored to task topology (e.g., continuous predictions via linear activations for regression, or probability distributions via Softmax for classification).

#### Vectorized backpropagation equations

Let $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}^*)$ denote the scalar loss evaluated on ground truth $\mathbf{y}^*$. In vectorized matrix calculus, we define the layer error vector $\boldsymbol{\delta}^{(l)} \in \mathbb{R}^{d_l}$ as the gradient of the loss with respect to the pre-activation vector $\mathbf{z}^{(l)}$:

$$\boldsymbol{\delta}^{(l)} \equiv \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(l)}}. \tag{1.119}$$

*Output layer error $\boldsymbol{\delta}^{(L)}$.* Applying the chain rule to the output layer:

$$\boldsymbol{\delta}^{(L)} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(L)}} = \nabla_{\mathbf{a}^{(L)}}\mathcal{L} \odot f'^{(L)}\left(\mathbf{z}^{(L)}\right), \tag{1.120}$$

where $\odot$ denotes the element-wise Hadamard product. As proven in Equation (1.81), when pairing cross-entropy loss with Softmax or Sigmoid activations, this expression simplifies directly to the residual difference:

$$\boldsymbol{\delta}^{(L)} = \hat{\mathbf{y}} - \mathbf{y}^*. \tag{1.121}$$

*Hidden layer error recursion* $\boldsymbol{\delta}^{(l)}$. For any internal hidden layer $l \in \{1, \dots, L-1\}$, the error vector $\boldsymbol{\delta}^{(l)}$ is evaluated recursively backward from layer $l+1$:

$$\boldsymbol{\delta}^{(l)} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(l)}} = \left( \frac{\partial \mathbf{z}^{(l+1)}}{\partial \mathbf{z}^{(l)}} \right)^{\top} \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(l+1)}} = \left( \mathbf{W}^{(l+1)} \boldsymbol{\delta}^{(l+1)} \right) \odot f'^{(l)} \left( \mathbf{z}^{(l)} \right). \tag{1.122}$$

Equation (1.122) demonstrates the structural symmetry of backpropagation: the error signal $\boldsymbol{\delta}^{(l+1)}$ is projected backward through the transpose of the forward weight matrix $\mathbf{W}^{(l+1)}$ and subsequently modulated by the local derivative of the activation function $f'^{(l)}(\mathbf{z}^{(l)})$.

*Parameter gradient evaluation.* Once the error vectors $\boldsymbol{\delta}^{(l)}$ are evaluated, the gradients of the loss with respect to the weight matrices and bias vectors are obtained via outer products:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \mathbf{a}^{(l-1)} \left( \boldsymbol{\delta}^{(l)} \right)^{\top} \in \mathbb{R}^{d_{l-1} \times d_l}, \tag{1.123}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}} = \boldsymbol{\delta}^{(l)} \in \mathbb{R}^{d_l}. \tag{1.124}$$

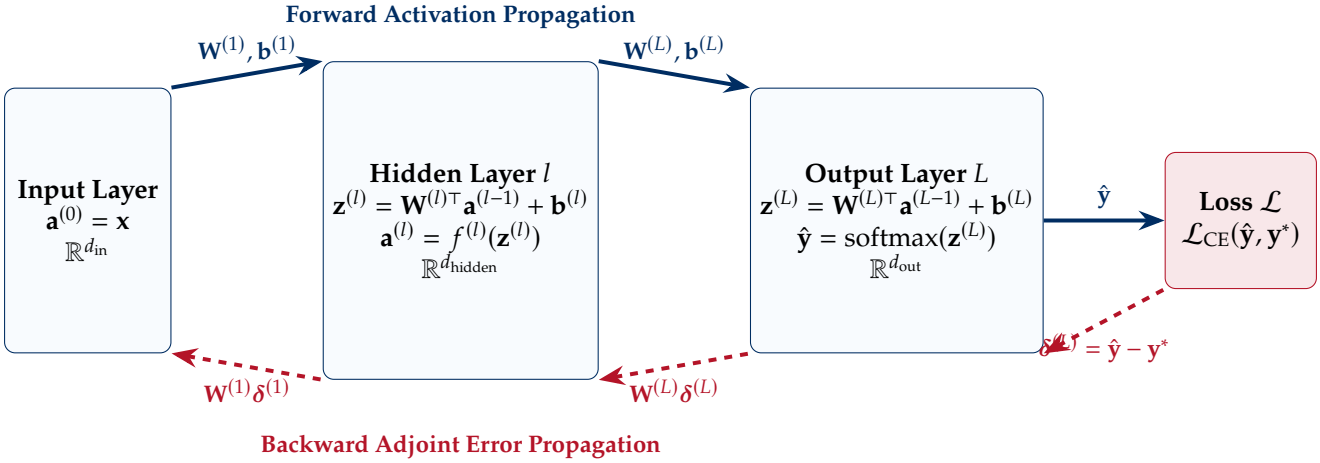



Figure 1.5: Vectorized dataflow schematic across a multi-layer Feed-Forward Network. Solid blue lines trace forward activations; dashed red lines trace the reverse-mode propagation of error vectors $\boldsymbol{\delta}^{(l)}$.

### Structural limitations of feed-forward networks for sequence processing

While the multi-layer feed-forward network provides a universal framework for static pattern recognition, its fundamental architectural assumptions break down when deployed on sequential data, such as natural language text, source code, and time series [8, 9]. Specifically, standard feed-forward networks exhibit three structural failure modes:

(i) **Fixed-dimensional input constraints**: An FFN requires an input vector of strictly fixed dimension $d_{\text{in}}$. However, natural language utterances and software engineering artifacts vary arbitrarily in length. Forcing variable-length sentences into a fixed vector through zero-padding or truncation either wastes computation on empty tokens or discards critical semantic context.

(ii) **Absence of temporal memory**: Feed-forward layers operate statelessly: each input vector $\mathbf{x}$ is processed in total isolation from preceding observations. The network possesses no persistent internal hidden state to store linguistic dependencies, resolve pronouns, or maintain scope across long programmatic contexts.

(iii) **Inability to share parameters across sequence positions**: In a standard linear layer $\mathbf{z} = \mathbf{W}^\top \mathbf{x} + \mathbf{b}$, each input feature is assigned a unique, position-dependent weight. If an informative linguistic motif (such as the keyword `function` or the phrase *"not found"*) appears at token position $t = 1$, the network cannot recognize that identical motif at token position $t = 15$ without separately learning an independent set of weights for those coordinates.

Overcoming these limitations requires an architecture equipped with persistent internal states and position-invariant parameter sharing. This fundamental representational shift directly motivates the transition to Recurrent Neural Networks, explored in Chapter 2.

Chapter 2

# Recurrent neural networks

The foundational feed-forward architectures established in the preceding chapter operate as static spatial mappings between fixed-dimensional Euclidean vector spaces. In a conventional multi-layer perceptron, an input vector $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$ is mapped deterministically to an output vector $\mathbf{y} \in \mathbb{R}^{d_{\text{out}}}$ through a sequence of parameterized affine transformations interleaved with coordinate-wise non-linearities. While this formulation provides universal function approximation under compact domains, its topological contract assumes that every input observation arrives as a self-contained, isolated point of fixed dimensionality, devoid of internal temporal ordering or dynamic duration.

Real-world perceptual and symbolic processes—most notably natural language text, acoustic speech signals, software execution traces, and biological macromolecules—fundamentally violate this static spatial premise. Natural sequences exhibit variable lengths across observations, temporal dependencies where the interpretation of a token depends intrinsically on antecedent context, and translation-invariant motifs that retain identical semantic meaning regardless of the temporal index at which they appear. Applying static feed-forward networks to such sequential signals requires ad-hoc zero-padding, rigid context window truncation, or the redundant allocation of localized parameters across spatial coordinates, inducing severe sample inefficiencies and representational brittleness.

This chapter establishes the theoretical, algorithmic, and architectural foundations of Recurrent Neural Networks (RNNs). We begin by formalizing the structural failure modes of feed-forward networks on sequential data, motivating the necessity of persistent internal memory and position-invariant parameter sharing. We derive the mathematical mechanics of recurrent state transitions, unroll the network across time into an equivalent directed acyclic graph, and formulate the exact gradient accumulation dynamics of Backpropagation Through Time (BPTT). We then analyze the analytical pathologies of vanilla recurrence—specifically the exponential decay and explosion of temporal Jacobians—and examine how gated recurrent architectures, such as the Long Short-Term Memory network and the Gated Recurrent Unit, enforce constant error propagation. Finally, we explore the sequence-to-sequence encoder-decoder paradigm and demonstrate how the fundamental information bottleneck of compressing variable-length sequences into a single static context vector directly motivates the Attention mechanism.

## 2.1 Limitations of feed-forward architectures on sequential data

Understanding the emergence of recurrent architectures requires analyzing why standard feed-forward networks, despite their universal approximation capabilities, are structurally ill-suited for sequence processing. The limitations of feed-forward networks stem from their rigid dimensional contracts, their localized parameter binding, and their inability to model temporal dynamics without severe compromises.

### 2.1.1 Fixed dimensionality constraints on inputs and outputs

A standard fully connected neural layer computes an affine transformation followed by an activation function:

$$\mathbf{h} = \sigma\left(\mathbf{W}\mathbf{x} + \mathbf{b}\right), \tag{2.1}$$

where $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$ denotes the input vector, $\mathbf{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ represents the weight parameter matrix, and $\mathbf{b} \in \mathbb{R}^{d_{\text{out}}}$ is the bias vector [9]. The algebraic definition of matrix multiplication requires that the column dimension of $\mathbf{W}$ strictly matches the dimensionality of the input vector $\mathbf{x}$. Consequently, any neural network constructed from feed-forward layers imposes an unyielding contract: every input vector must possess exactly $d_{\text{in}}$ coordinates, and every output prediction must produce exactly $d_{\text{out}}$ values.

In structured machine learning domains, such as tabular databases, this fixed dimensional contract is entirely appropriate because each coordinate of $\mathbf{x}$ corresponds to a specific, predefined feature column. However, unstructured perceptual and symbolic signals are intrinsically variable in extent. In natural language processing, a text sample may span a single-word query, a five-token sentence, or a document containing thousands of words. Similarly, raw acoustic recordings vary in duration from fractions of a second to multiple hours, and computer vision images vary across capture devices in height and width before standardization.

When practitioners attempt to process variable-length sequences using static feed-forward networks, they are forced to define an arbitrary upper bound on sequence length, denoted $T_{\max}$, and enforce dimensional uniformity via artificial zero-padding or heuristic truncation. If an observation sequence has length $T < T_{\max}$, the remaining $T_{\max} - T$ slots are populated with synthetic null tokens; conversely, if $T > T_{\max}$, the sequence is truncated, discarding all information residing beyond the arbitrary horizon. This operational compromise introduces substantial structural deficiencies:

1. **Computational waste**: Zero-padded dimensions must undergo full matrix multiplications throughout all layers, consuming memory bandwidth and floating-point operations on non-informative null values.

2. **Information loss**: Rigid truncation destroys long-range context and truncates syntactic clauses that are essential for accurate semantic inference.

3. **Spurious gradient signals**: Unless complex masking logic is manually applied at every layer, artificial padding tokens contribute synthetic activations that skew batch statistics and destabilize optimization.

The dimensional rigidity of the output layer is equally problematic. In sequential tasks such as syntactic parsing, semantic segmentation, or machine translation, the target output sequence is not a static scalar or fixed-size vector; it is itself an ordered sequence whose length $T_{\text{out}}$ varies dynamically and cannot be determined prior to observing the input signal. A feed-forward network with a fixed output dimension $d_{\text{out}}$ is mathematically incapable of generating variable-length sequences without enforcing artificial output horizons.

### 2.1.2 Lack of position invariance and weight localization

The second fundamental pathology of feed-forward networks is their spatial and temporal parameter localization. In a dense layer, each scalar weight $W_{ji}$ is dedicated exclusively to the connection between input coordinate $i$ and hidden neuron $j$. This architecture binds each parameter to an absolute coordinate index in the input vector space [8].

To conceptualize the consequences of parameter localization, consider an image classification task evaluated on flattened pixel vectors. Suppose an image contains the visual motif of a dog located in the upper-left quadrant of the frame. The feed-forward network activates the subset of weights associated with those specific coordinate indices to detect the object. If an identical dog appears in the lower-right quadrant of another image, the corresponding visual features activate an entirely disjoint set of pixel coordinates, corresponding to completely different columns of $\mathbf{W}$. Because the network lacks parameter sharing across spatial positions, it cannot transfer what it learned about dogs in the upper-left corner to dogs appearing in the lower-right corner. The model is forced to independently rediscover and parameterize the visual concept of a dog at every discrete spatial coordinate.

When applied to sequential text processing, this parameter localization leads to severe sample inefficiency. Consider a character-level capitalization task where an input character sequence must be mapped to its correctly cased variant. In natural language, an opening quotation mark " serves as a structural syntactic cue indicating that the immediately following character must be capitalized as the start of a dialogue utterance. In a feed-forward network where a sequence of $T$ characters is concatenated into a single input vector:

$$\mathbf{x} = \left[\mathbf{x}_1^\top, \mathbf{x}_2^\top, \ldots, \mathbf{x}_T^\top\right]^\top \in \mathbb{R}^{T \cdot d_c}, \tag{2.2}$$

the network must learn separate, independent weight parameters to detect quotation marks at position $t = 1$, position $t = 2$, position $t = 3$, and every subsequent temporal offset up to $T$. If the training corpus contains numerous examples of dialogue starting at index 1 but none starting at index 14, the model will fail to capitalize a dialogue quote appearing at index 14, despite possessing the relevant knowledge at index 1.

Because identical linguistic and acoustic motifs occur at arbitrary temporal offsets, a sequence processing model must satisfy *temporal translation equivariance*: shifting an input pattern in time should produce an identical, equivalently shifted representation without requiring separate parameters. A feed-forward network completely lacks this inductive bias, demanding a parameter budget that scales quadratically with sequence length:

$$N_{\text{parameters}} = O\left(T \cdot d_{\text{in}} \cdot d_{\text{hidden}}\right), \tag{2.3}$$

which becomes computationally intractable for long textual contexts and renders generalization to unseen sequence positions virtually impossible.

### 2.1.3 The structural requirement for dynamic temporal processing

The failure of feed-forward architectures establishes the formal requirements for an authentic sequential learning paradigm. Mathematically, an ordered sequence is defined as a tuple:

$$\mathbf{X} = (\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_T), \quad \mathbf{x}_t \in \mathbb{R}^{d_x}, \quad T \in \mathbb{N}^+, \tag{2.4}$$

where the temporal index $t$ specifies chronological or symbolic ordering, and the total length $T$ varies across observations.

To process such sequences effectively, an architecture must satisfy three primary design imperatives:

1. **Length-agnostic ingestion**: The model must operate on sequences of arbitrary length $T$ through an iterative transition operator whose parametric specification is strictly independent of $T$.

2. **Position-invariant parameter sharing**: The transformation applied at time step $t$ must utilize the exact same parameter tensors as the transformation applied at time step $t'$, enabling structural motifs learned at one temporal position to generalize across all positions.

3. **Contextual persistence and state propagation**: The representation computed at step $t$ must not depend solely on the isolated token $\mathbf{x}_t$, but must integrate a synthesized historical record of all prior observations $(\mathbf{x}_1, \ldots, \mathbf{x}_{t-1})$.

These three requirements directly motivate the architecture of Recurrent Neural Networks, which replace the static, one-shot feed-forward mapping with a dynamical system governed by an internal recurrent state.

## 2.2 Recurrent neural networks: intuition, unrolling, and cell mechanics

Recurrent Neural Networks address the structural limitations of feed-forward networks by introducing an internal state that persists across temporal steps. Instead of consuming an entire sequence simultaneously, an RNN ingests one token at a time, continuously updating its internal memory to synthesize past observations with current sensory input.

### 2.2.1 The recurrent intuition and persistent hidden state

The fundamental conceptual mechanism of a recurrent neural network can be understood through the pedagogical analogy of a human reader taking running notes while reading a document. A human reader does not attempt to ingest an entire book in a single visual fixation; rather, the reader scans the text sequentially, word by word. As each new word is read, the reader does not discard previous knowledge, nor do they carry the verbatim text of every preceding page in immediate working memory. Instead, the reader maintains a continuous mental summary—a running piece of paper—recording essential context, active subjects, unresolved clauses, and grammatical state. When a new word is encountered, the reader combines their current mental notes with the incoming word, updates the notes, and optionally generates an interpretation or expectation of upcoming text.

In a Recurrent Neural Network, this mental notepad is formalized as the *hidden state vector*, denoted $\mathbf{h}_t \in \mathbb{R}^{d_h}$. The hidden state serves as an internal, distributed latent representation that encodes the historical trajectory of the sequence up to time step $t$ [19]. At each discrete time step $t \in \{1, \dots, T\}$, the recurrent unit operates as an operator characterized by a dual-input, dual-output interface:

$$\text{Inputs at step } t: \quad \mathbf{x}_t \in \mathbb{R}^{d_x} \text{ (current sequence element)}, \quad \mathbf{h}_{t-1} \in \mathbb{R}^{d_h} \text{ (previous state)}, \tag{2.5}$$

$$\text{Outputs at step } t: \quad \hat{\mathbf{y}}_t \in \mathbb{R}^{d_y} \text{ (step prediction)}, \quad \mathbf{h}_t \in \mathbb{R}^{d_h} \text{ (updated state)}. \tag{2.6}$$

Before any sequence elements are observed (at step $t = 0$), the network must initialize its internal memory. In the absence of prior context, the initial state is typically defined as a vector of zeros:

$$\mathbf{h}_0 = \mathbf{0} \in \mathbb{R}^{d_h}, \tag{2.7}$$

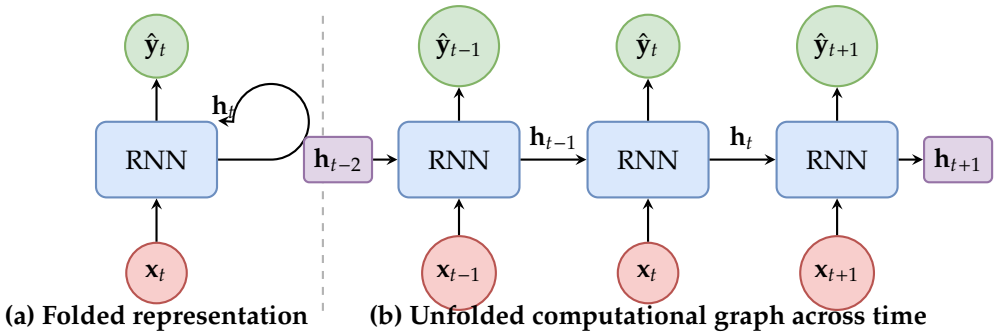
representing a clean, unwritten slate. As sequence tokens $\mathbf{x}_1, \mathbf{x}_2, \dots$ arrive, the recurrent transition function modifies $\mathbf{h}_t$ at every step, allowing downstream predictions $\hat{\mathbf{y}}_t$ to be conditioned on the complete historical context $\mathbf{x}_{\leq t}$.

### 2.2.2 Time unfolding and unrolled computational graphs

The recurrent relationship is traditionally depicted in two equivalent representational forms: the compact *folded* diagram and the explicit *unfolded* computational graph.

In the folded representation (Figure 2.1a), the network is illustrated as a single processing unit possessing a directed cyclic feedback loop: the hidden state output $\mathbf{h}_t$ is fed directly back into the cell to serve as the state input for the subsequent step. While algebraically concise, cyclic graphs cannot be directly evaluated by standard automatic differentiation engines, which require acyclic computation paths to establish reverse topological orderings.

To resolve this cyclic dependency, the recurrent execution is *unfolded in time* (Figure 2.1b). Unfolding expands the recurrent unit across the temporal sequence of length $T$, replicating the cell into a chain of $T$ interconnected feed-forward units. In this unfolded representation, the cyclic feedback loop is mapped onto a sequence of forward-directed horizontal edges connecting unit $t-1$ to unit $t$. The resulting structure forms a classical Directed Acyclic Graph (DAG), enabling standard forward inference and reverse-mode automatic differentiation across the temporal chain.




**(a) Folded representation** **(b) Unfolded computational graph across time**

Figure 2.1: Comparison between the compact folded representation of a recurrent neural network with a self-referential temporal feedback loop and its unfolded computational Directed Acyclic Graph (DAG) across consecutive time steps.

To observe how time unfolding coordinates contextual reasoning, consider tracking the character-level casing problem introduced earlier. Suppose the model receives the sequential lowercase stream `['"', 'y', 'o', 'u']`:

1. **Step 1 ($t = 1$)**: The model ingests $\mathbf{x}_1 =$ `'"'` and initial state $\mathbf{h}_0 = \mathbf{0}$. The network emits $\hat{\mathbf{y}}_1 =$ `'"'` and transitions to state $\mathbf{h}_1$. Internally, the parameter transformations configure $\mathbf{h}_1$ to encode the semantic feature: *"an opening quotation mark has occurred; dialogue has begun; the subsequent token must be capitalized"*.

2. **Step 2 ($t = 2$)**: The model ingests $\mathbf{x}_2 =$ `'y'` alongside $\mathbf{h}_1$. Driven by the contextual indicator stored in $\mathbf{h}_1$, the output projection produces an uppercase token $\hat{\mathbf{y}}_2 =$ `'Y'`. Concurrently, the state updates to $\mathbf{h}_2$, encoding: *"the initial dialogue letter has been capitalized; subsequent letters in the current token should revert to lowercase"*.

3. **Step 3 ($t = 3$)**: The model ingests $\mathbf{x}_3 =$ `'o'` alongside $\mathbf{h}_2$. Consulting $\mathbf{h}_2$, the model emits lowercase $\hat{\mathbf{y}}_3 =$ `'o'` and maintains a state reflecting active word continuation.

Through this sequential unfolding, the network resolves local contextual ambiguities dynamically, utilizing the persistent state vector to propagate structural syntactic memory across successive steps.

### 2.2.3 Internal cell mechanics and state-space transformations

Having examined the external input-output behavior of the recurrent unit, we now formulate the internal algebraic operations governing the Elman recurrent cell [19]. The interior of an Elman cell consists of three parameterized affine projections and two non-linear mapping stages.

The mathematical transformation unfolds in four explicit stages:

1. **Input projection**: The input vector $\mathbf{x}_t \in \mathbb{R}^{d_x}$ is projected into the hidden state space via the input-to-hidden parameter matrix $\mathbf{W}_{xh} \in \mathbb{R}^{d_h \times d_x}$:

$$\mathbf{u}_t = \mathbf{W}_{xh}\mathbf{x}_t \in \mathbb{R}^{d_h}. \tag{2.8}$$

   This operation aligns the dimensionality of the incoming observation with the hidden representation space, mapping disparate input modalities (such as one-hot character vectors or continuous embeddings) into a standardized coordinate frame.

2. **Recurrent state transformation**: The previous hidden state $\mathbf{h}_{t-1} \in \mathbb{R}^{d_h}$ is transformed through the hidden-to-hidden recurrent transition matrix $\mathbf{W}_{hh} \in \mathbb{R}^{d_h \times d_h}$:

$$\mathbf{v}_t = \mathbf{W}_{hh}\mathbf{h}_{t-1} \in \mathbb{R}^{d_h}. \tag{2.9}$$

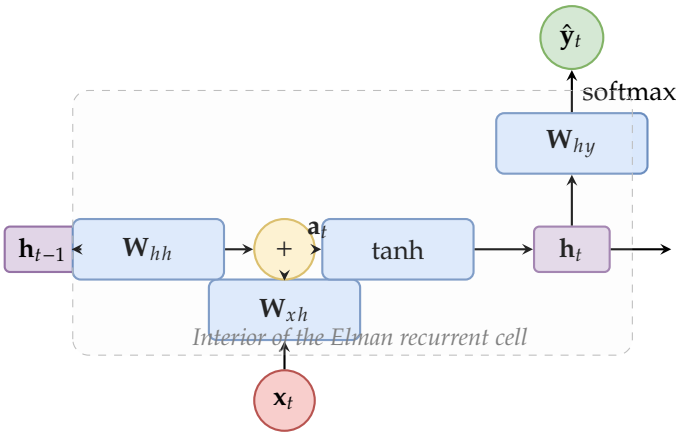



Figure 2.2: Detailed architectural anatomy of the Elman recurrent cell, illustrating the affine projections of the incoming token $\mathbf{x}_t$ and antecedent state $\mathbf{h}_{t-1}$, the linear state aggregation node, the hyperbolic tangent activation function, and the output classification head.

This transformation rotates and scales the historical memory representation, filtering and projecting antecedent information into coordinates compatible with the incoming input.

3. **Linear aggregation and activation**: The projected input $\mathbf{u}_t$ and projected antecedent state $\mathbf{v}_t$ are summed coordinate-wise alongside a learnable hidden bias vector $\mathbf{b}_h \in \mathbb{R}^{d_h}$, yielding the pre-activation state vector $\mathbf{a}_t \in \mathbb{R}^{d_h}$:

$$\mathbf{a}_t = \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h. \tag{2.10}$$

The pre-activation vector is then mapped through an element-wise non-linear activation function, canonically chosen as the hyperbolic tangent (tanh):

$$\mathbf{h}_t = \tanh(\mathbf{a}_t) = \frac{\exp(\mathbf{a}_t) - \exp(-\mathbf{a}_t)}{\exp(\mathbf{a}_t) + \exp(-\mathbf{a}_t)} \in (-1, 1)^{d_h}. \tag{2.11}$$

The hyperbolic tangent squashes state activations into the symmetric bounded interval $(-1, 1)$, preventing unbounded numerical growth across successive temporal steps while centering activations around zero.

4. **Output projection and distribution**: If the task demands a token prediction at time step $t$, the updated state $\mathbf{h}_t$ is projected into the output target space via the hidden-to-output matrix $\mathbf{W}_{hy} \in \mathbb{R}^{d_y \times d_h}$ and bias $\mathbf{b}_y \in \mathbb{R}^{d_y}$:

$$\mathbf{z}_t = \mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y \in \mathbb{R}^{d_y}, \tag{2.12}$$

where $\mathbf{z}_t$ represents the unnormalized logit vector. For categorical prediction over a discrete vocabulary $\mathcal{V}$ with dimension $d_y = |\mathcal{V}|$, normalized class probabilities are computed via the softmax operator:

$$\hat{\mathbf{y}}_t = \operatorname{softmax}(\mathbf{z}_t) = \frac{\exp(\mathbf{z}_t)}{\sum_{k=1}^{d_y} \exp(z_{t,k})} \in \Delta^{d_y - 1}. \tag{2.13}$$

To maintain rigorous mathematical clarity, Table 2.1 summarizes the explicit shapes and functional roles of all parameters and representations comprising the Elman recurrent architecture.

| Mathematical Symbol | Tensor Shape | Space / Domain | Functional Role |
|---|---|---|---|
| $\mathbf{x}_t$ | $d_x \times 1$ | $\mathbb{R}^{d_x}$ | Input sequence token at step $t$ |
| $\mathbf{h}_t$ | $d_h \times 1$ | $(-1, 1)^{d_h}$ | Recurrent latent state vector at step $t$ |
| $\hat{\mathbf{y}}_t$ | $d_y \times 1$ | $\Delta^{d_y - 1}$ | Predicted probability distribution at step $t$ |
| $\mathbf{W}_{xh}$ | $d_h \times d_x$ | $\mathbb{R}^{d_h \times d_x}$ | Input-to-hidden projection matrix |
| $\mathbf{W}_{hh}$ | $d_h \times d_h$ | $\mathbb{R}^{d_h \times d_h}$ | Hidden-to-hidden recurrent transition matrix |
| $\mathbf{W}_{hy}$ | $d_y \times d_h$ | $\mathbb{R}^{d_y \times d_h}$ | Hidden-to-output projection matrix |
| $\mathbf{b}_h$ | $d_h \times 1$ | $\mathbb{R}^{d_h}$ | Hidden pre-activation bias vector |
| $\mathbf{b}_y$ | $d_y \times 1$ | $\mathbb{R}^{d_y}$ | Output logit bias vector |

Table 2.1: Explicit tensor dimensions, mathematical domains, and functional roles of representations and learnable parameters in the standard Elman recurrent cell.

## 2.3 Weight sharing and backpropagation through time

The computational elegance and representational power of Recurrent Neural Networks arise from two structural principles: the strict sharing of parameter tensors across all sequence coordinates, and the recursive backpropagation of error signals through the unfolded computational graph.

### 2.3.1 The weight-sharing paradigm and position invariance

In standard feed-forward networks, processing an input sequence of length $T$ requires distinct weight matrices for each temporal offset, causing parameter count to explode linearly with sequence duration. In sharp contrast, a recurrent neural network enforces *parameter tying*: the exact same weight matrices and bias vectors are reused at every time step:

$$\mathbf{W}_{xh}^{(1)} = \mathbf{W}_{xh}^{(2)} = \cdots = \mathbf{W}_{xh}^{(T)} = \mathbf{W}_{xh}, \quad \mathbf{W}_{hh}^{(1)} = \mathbf{W}_{hh}^{(2)} = \cdots = \mathbf{W}_{hh}^{(T)} = \mathbf{W}_{hh}. \tag{2.14}$$

This architectural invariance yields two decisive advantages:

1. **Position-invariant inference**: Because the transition function is parameterized identically across time, the network applies uniform operational logic to input tokens regardless of their absolute temporal offset. An opening quotation mark encountered at $t = 1$, $t = 25$, or $t = 100$ activates the exact same input projection $\mathbf{W}_{xh}$ and recurrent transition $\mathbf{W}_{hh}$, triggering identical state updates and eliminating the need to independently learn position-specific rules.

2. **Length-invariant parameter complexity**: The total number of learnable parameters in an Elman RNN is:
$$N_{\text{parameters}} = d_h \cdot d_x + d_h^2 + d_h + d_y \cdot d_h + d_y. \tag{2.15}$$
Crucially, Equation 2.15 contains no term depending on sequence length $T$. The model can process a sequence of 5 tokens or 500 tokens using the exact same parameter budget, enabling deployment on sequences of arbitrary and varying lengths.

### 2.3.2 Backpropagation through time (BPTT) formulation

Because the computational graph of an RNN unrolls across time into an interconnected chain of operations sharing identical parameters, computing the gradients of the training objective requires a specialized formulation of reverse-mode automatic differentiation termed *Backpropagation Through Time* (BPTT) [20].

Consider a training sequence of length $T$ paired with target ground-truth one-hot vectors $\mathbf{y}_1^*, \ldots, \mathbf{y}_T^*$. At each time step $t$, the network outputs a predicted probability distribution $\hat{\mathbf{y}}_t$ and evaluates a token-level categorical cross-entropy loss:

$$\mathcal{L}_t\left(\hat{\mathbf{y}}_t, \mathbf{y}_t^*\right) = -\sum_{k=1}^{d_y} y_{t,k}^* \log \hat{y}_{t,k}. \tag{2.16}$$

The total scalar sequence loss $\mathcal{L}$ is defined as the temporal summation over all token losses:

$$\mathcal{L} = \sum_{t=1}^{T} \mathcal{L}_t\left(\hat{\mathbf{y}}_t, \mathbf{y}_t^*\right). \tag{2.17}$$

To update the parameters via gradient descent, we must evaluate the partial derivatives of $\mathcal{L}$ with respect to the shared weight matrices: $\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{hy}}$, $\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{xh}}$, and $\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{hh}}$.

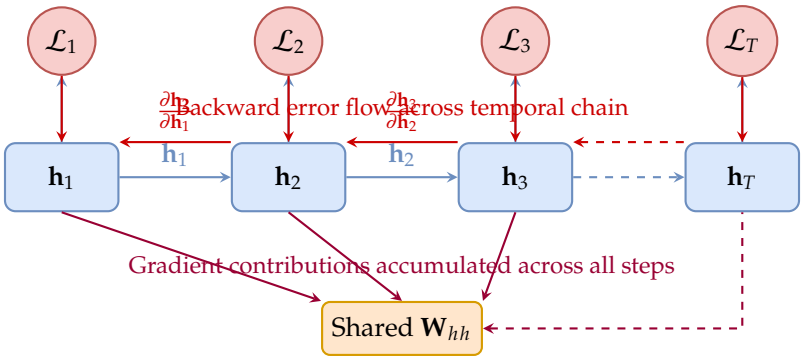



Figure 2.3: The computational flow of Backpropagation Through Time (BPTT). Forward activations propagate chronologically (blue arrows), generating per-step losses. Error signals propagate backward along the temporal chain (red arrows), and local gradient contributions are accumulated into the shared parameter tensor (purple arrows).

The output projection gradient $\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{hy}}$ depends only on current state activations. By applying the standard cross-entropy softmax gradient derived in Chapter 1:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{hy}} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}_t}{\partial \mathbf{W}_{hy}} = \sum_{t=1}^{T} \left(\hat{\mathbf{y}}_t - \mathbf{y}_t^*\right) \mathbf{h}_t^\top. \tag{2.18}$$

In contrast, evaluating the gradient with respect to the recurrent matrix $\mathbf{W}_{hh}$ requires accounting for the multivariable chain rule across all ancestral steps. Because $\mathbf{W}_{hh}$ was used to compute $\mathbf{h}_t$, which in turn depended on $\mathbf{h}_{t-1}$, which depended on $\mathbf{h}_{t-2}$, a perturbation in $\mathbf{W}_{hh}$ affects the loss $\mathcal{L}_t$ through multiple distinct pathways across time.

*Derivation of the recurrent gradient accumulation.* To formalize this temporal dependence, let $\frac{\partial^+ \mathbf{h}_k}{\partial \mathbf{W}_{hh}}$ denote the *direct* partial derivative of $\mathbf{h}_k$ with respect to $\mathbf{W}_{hh}$, treating antecedent state $\mathbf{h}_{k-1}$ as a constant:

$$\frac{\partial^+ \mathbf{h}_k}{\partial \mathbf{W}_{hh}} = \operatorname{diag}\left(1 - \mathbf{h}_k^2\right) \otimes \mathbf{h}_{k-1}^\top, \tag{2.19}$$

where $\otimes$ denotes the tensor outer product. The total derivative of step loss $\mathcal{L}_t$ with respect to $\mathbf{W}_{hh}$ sums over all temporal paths from step $k = 1$ up to $k = t$:

$$\frac{\partial \mathcal{L}_t}{\partial \mathbf{W}_{hh}} = \sum_{k=1}^{t} \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} \frac{\partial^+ \mathbf{h}_k}{\partial \mathbf{W}_{hh}}, \tag{2.20}$$

where $\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k}$ represents the temporal Jacobian matrix describing how hidden activations at step $k$ propagate forward to influence hidden activations at step $t$.

Summing over all sequence time steps $t \in \{1, \ldots, T\}$, the complete gradient with respect to $\mathbf{W}_{hh}$ evaluates to:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{hh}} = \sum_{t=1}^{T} \sum_{k=1}^{t} \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} \frac{\partial^+ \mathbf{h}_k}{\partial \mathbf{W}_{hh}}. \tag{2.21}$$

By identical reasoning, the gradient with respect to the input projection matrix $\mathbf{W}_{xh}$ accumulates across time as:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{xh}} = \sum_{t=1}^{T} \sum_{k=1}^{t} \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} \frac{\partial^+ \mathbf{h}_k}{\partial \mathbf{W}_{xh}}, \quad \text{where} \quad \frac{\partial^+ \mathbf{h}_k}{\partial \mathbf{W}_{xh}} = \operatorname{diag}\left(1 - \mathbf{h}_k^2\right) \otimes \mathbf{x}_k^\top. \tag{2.22}$$

Equation 2.21 embodies the core operational mechanism of BPTT: during training, the network first performs a forward pass from $t = 1$ to $T$, computing and caching all states $\mathbf{h}_t$ and

per-step losses $\mathcal{L}_t$. Subsequently, the backward pass propagates error vectors backward through time, accumulating gradient contributions from every historical step into the shared parameter tensors.

### 2.3.3 Empirical training dynamics: the Sherlock Holmes casing benchmark

To ground the mathematical formulation of BPTT in empirical practice, we analyze the training dynamics of a character-level Elman recurrent network tasked with sentence case correction. The experimental framework directly mirrors the diagnostic benchmark evaluated during course lectures.

*Experimental setup.* A vanilla Elman RNN is trained on textual excerpts drawn from Arthur Conan Doyle's *The Adventures of Sherlock Holmes*. The input sequence consists of raw sentences converted entirely to lowercase, represented as a stream of 256-dimensional one-hot vectors corresponding to standard ASCII characters ($d_x = 256$). The network utilizes a hidden state dimension $d_h = 32$, initialized with $\mathbf{h}_0 = \mathbf{0}$, and projects to a 256-dimensional categorical output distribution ($d_y = 256$). Optimization is conducted over 10 epochs using the Adam optimizer [18] minimizing token-level cross-entropy loss.

The global training loss trajectory exhibits a characteristic progression: starting from an initial loss $\mathcal{L} \approx 3.0$, the loss drops to 2.0 after 2 epochs, reaches 1.72 at epoch 4, drops sharply to 0.95 at epoch 6, and asymptotically plateaus around 0.85 by epoch 10. Inspecting the qualitative outputs generated for a held-out test utterance reveals five distinct developmental stages of sequential pattern acquisition:

$$\text{Input text:} \quad \texttt{"it may be so, mr.  holmes"} \tag{2.23}$$

| Epoch | Loss $\mathcal{L}$ | Predicted Character Sequence Output |
|---|---|---|
| 0 | 3.00 | `\x00 ...\x00` (repeated null padding bytes) |
| 1 | 2.45 | `eeee eeee eeee eeee eeee eeee` |
| 3 | 1.72 | `ie mae be so, mr.  holmes` |
| 5 | 0.95 | `"it may be so, mr.  holmes"` |
| 7 | 0.88 | `"It may be so, mr.  Holmes"` |
| 10 | 0.84 | `"It may be so, mr.  Holmes"` |
| **Target** | — | `"It may be so, Mr.  Holmes"` |

Table 2.2: Empirical evolution of character-level casing predictions produced by a 32-dimensional Elman RNN across 10 training epochs on an excerpt from *The Adventures of Sherlock Holmes*. The progression illustrates the acquisition of padding bias, unigram frequency priors, whitespace structure, identity copying, and local punctuation-driven capitalization.

Deconstructing the progression recorded in Table 2.2 reveals how gradient descent navigates the sequential hypothesis space:

1. **Epoch 0 (Null-byte padding bias)**: The uninitialized model produces repeated null bytes (`\x00`). In training batches where variable-length strings are aligned using zero-padding, the null character exhibits high empirical prevalence. The optimizer initially exploits this zero-order marginal distribution to achieve trivial loss reduction.

2. **Epoch 1 (Unigram letter frequency prior)**: As backpropagation penalizes null outputs on non-padding tokens, the network discovers the marginal unigram frequency distribution of the English language. It repeatedly predicts the character `'e'`—the most statistically frequent English character—interleaved with periodic whitespace tokens.

3. **Epoch 3 (Whitespace and lexical boundaries)**: The model masters word boundary segmentation. It reliably predicts space characters following character clusters, establishing basic lexical cadence even while specific character identities remain noisy.

4. **Epoch 5 (Identity copying and the steep loss drop)**: The loss experiences a dramatic decline from 1.72 to 0.95. The recurrent network discovers the global structural invariant of the task: *most output characters are identical to their input counterparts*. By configuring the composite transformation $\mathbf{W}_{hy} \tanh(\mathbf{W}_{xh}\mathbf{x}_t) \approx \mathbf{x}_t$, the network implements an identity bypass, echoing the input string verbatim.

5. **Epoch 7 (Local syntactic rule discovery)**: The network learns fine-grained sequential rules. Having observed that opening quotation marks precede dialogue, the state $\mathbf{h}_t$ transitions to capitalize the subsequent character, transforming `"it` into `"It`. Furthermore, it learns that the character sequence `holmes` constitutes a proper noun, correctly capitalizing the `'H'` in `Holmes`.

*The causal horizon failure.* Despite reaching training convergence by Epoch 10, the model exhibits a persistent failure: it fails to capitalize the title `mr.`, leaving it as lowercase `mr. Holmes`. This failure exposes the fundamental theoretical limitation of strictly causal, one-to-one recurrent processing. When the network ingests the character `'m'` at time step $t = 15$, its decision to output `'M'` or `'m'` must be made strictly on the basis of antecedent observations $\mathbf{x}_{\leq 15}$ = `'"it may be so, '`. At this point in time, the network has not yet observed the upcoming characters `'r'`, `'.'`, or `'holmes'`. In general English prose, words beginning with the letter `'m'` (such as *may, much, more, my, must*) are lowercase in over 95% of occurrences. Because the causal one-to-one RNN is prohibited from observing future tokens, Bayes' optimal decision rule under cross-entropy forces the network to predict lowercase `'m'`.

Resolving this structural ambiguity requires an architecture that can inspect the entire sentence—including upcoming future tokens—before committing to an output classification.

## 2.4 Pathologies of vanilla RNNs and gated architectures

While the Elman recurrent network resolves the dimensional and parameter-sharing limitations of feed-forward models, it suffers from severe analytical and computational pathologies that impede learning across long temporal spans.

### 2.4.1 The vanishing and exploding gradient problem

The primary mathematical pathology governing vanilla recurrent architectures is the vanishing and exploding gradient phenomenon [21, 22]. To understand its origin, we examine the temporal Jacobian matrix connecting hidden state $\mathbf{h}_k$ to hidden state $\mathbf{h}_t$ in Equation 2.21.

Applying the multivariate chain rule across $t-k$ intermediate recurrent steps yields a continuous product of Jacobian matrices:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} = \prod_{j=k+1}^{t} \frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}}. \tag{2.24}$$

Recalling the hidden state transition equation $\mathbf{h}_j = \tanh(\mathbf{W}_{xh}\mathbf{x}_j + \mathbf{W}_{hh}\mathbf{h}_{j-1} + \mathbf{b}_h)$, the single-step temporal Jacobian evaluates to:

$$\frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}} = \operatorname{diag}\left(1 - \mathbf{h}_j^2\right) \mathbf{W}_{hh}^\top, \tag{2.25}$$

where $\operatorname{diag}(1-\mathbf{h}_j^2) \in \mathbb{R}^{d_h \times d_h}$ is a diagonal matrix containing the first derivatives of the element-wise hyperbolic tangent function, $\tanh'(a) = 1 - \tanh^2(a)$.

Substituting Equation 2.25 back into the chain product of Equation 2.24 produces:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} = \prod_{j=k+1}^{t} \operatorname{diag}\left(1 - \mathbf{h}_j^2\right) \mathbf{W}_{hh}^\top. \tag{2.26}$$

The matrix product in Equation 2.26 contains $t-k$ successive matrix multiplications of the transposed recurrent weight matrix $\mathbf{W}_{hh}^\top$, modulated at each step by the activation derivatives.

Let $\lambda_{\max}$ denote the largest eigenvalue (spectral radius) of the recurrent weight matrix $\mathbf{W}_{hh}$. Because the derivative of hyperbolic tangent is strictly bounded such that $0 < 1 - h_{j,m}^2 \leq 1$, the asymptotic magnitude of the temporal Jacobian is bounded by:

$$\left\| \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} \right\| \leq \left(\gamma \left\| \mathbf{W}_{hh}^\top \right\|\right)^{t-k}, \quad \text{where } \gamma \leq 1. \tag{2.27}$$

This analytical bound establishes two opposing failure modes:

1. **Vanishing gradients**: If the spectral radius satisfies $\|\mathbf{W}_{hh}\| < 1$, or if hidden activations operate in saturating regimes where $1 - h_j^2 \ll 1$, the product decays exponentially toward zero as temporal distance $t-k$ increases:

$$\lim_{t-k\to\infty} \left\| \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} \right\| = 0. \tag{2.28}$$

   To appreciate the speed of this decay, consider a sequence of moderate length where $t-k = 50$, and suppose the effective scaling factor is 0.90. Over 50 time steps, the gradient scaling factor diminishes to:

$$0.90^{50} \approx 0.00515, \tag{2.29}$$

attenuating the error signal by more than two orders of magnitude. For $t - k = 100$, the scaling factor drops to $0.90^{100} \approx 2.65 \times 10^{-5}$. Consequently, error signals computed at time step $t$ exert virtually zero influence on parameter updates at distant antecedent steps $k \ll t$. The network becomes incapable of learning long-range temporal dependencies.

2. **Exploding gradients**: If the spectral radius satisfies $\|\mathbf{W}_{hh}\| > 1$ and activations remain in the linear regime near zero ($\gamma \approx 1$), the matrix product grows exponentially with temporal distance:
$$\lim_{t-k\to\infty} \left\| \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} \right\| = \infty. \tag{2.30}$$
Exploding gradients cause parameter updates $\Delta\mathbf{W} = -\eta\nabla_{\mathbf{W}}\mathcal{L}$ to become astronomically large, catapulting parameters across the loss surface into regions of numerical overflow (`NaN`) and destroying previously acquired representations.

While exploding gradients can be mitigated using the heuristic engineering technique of *gradient clipping by global norm* [22]:

$$\mathbf{g} \leftarrow \begin{cases} \mathbf{g} & \text{if } \|\mathbf{g}\|_2 \le v, \\ v\frac{\mathbf{g}}{\|\mathbf{g}\|_2} & \text{if } \|\mathbf{g}\|_2 > v, \end{cases} \tag{2.31}$$

where $v > 0$ defines the maximum allowable gradient norm, gradient clipping cannot resolve vanishing gradients. When error gradients vanish to machine precision zero, no simple rescaling can restore the lost directional signal.

### 2.4.2 Long-term dependency decay and computational sequentiality

The mathematical vanishing of gradients manifests behaviorally as the loss of long-term temporal memory. Because the hidden state $\mathbf{h}_t \in \mathbb{R}^{d_h}$ has a strictly finite coordinate dimensionality, it must compress everything observed from step 1 to step $t$ into a fixed-size vector. At each step, Equation 2.11 subjects the state to non-linear transformation and vector addition with incoming tokens:

$$\mathbf{h}_t = \tanh\left(\mathbf{W}_{xh}\mathbf{x}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h\right). \tag{2.32}$$

In practice, new observations continuously overwrite the coordinates of $\mathbf{h}_t$. Unless specific architectural constraints prevent overwriting, information concerning tokens observed dozens of steps in the past decays exponentially, leaving the hidden state dominated almost entirely by the most recent 5 to 10 tokens.

To illustrate this failure mode, consider the diagnostic sequence evaluated in lecture:

Prompt: `"The mouse was getting chased by the cat through the big house.` (2.33)

`After a long chase, the cat finally managed to catch the [ ]"` (2.34)

To correctly predict the target noun `"mouse"` at the sequence terminus, a language model must retain the identity of the original grammatical patient (`"mouse"`) across an intervening clause spanning more than 20 tokens. In a standard Elman RNN, the intervening tokens (*"through the big house. After a long chase, the cat finally managed to catch the"*) trigger repeated state transitions that overwrite the coordinates storing `"mouse"`. Consequently, when the model predicts the final token, its state reflects recent terms such as *"catch"*, prompting the generation of semantically plausible but contextually disconnected completions, such as `"ball"` or `"tail"`, while failing to recover the true referent.

Beyond memory degradation, vanilla RNNs suffer from a profound hardware inefficiency: *computational sequentiality*. By definition, evaluating state $\mathbf{h}_t$ requires antecedent state $\mathbf{h}_{t-1}$ as an

explicit mathematical input. Consequently, a recurrent neural network cannot compute step $t$ until step $t-1$ has fully terminated:

$$\mathbf{h}_1 \longrightarrow \mathbf{h}_2 \longrightarrow \mathbf{h}_3 \longrightarrow \cdots \longrightarrow \mathbf{h}_T. \tag{2.35}$$

This temporal serialization creates a strict dependency chain along the sequence dimension, precluding the parallel evaluation of sequence tokens on modern massively parallel hardware accelerators (GPUs and TPUs). While batch instances can be processed in parallel across sequences, training runtime along the sequence dimension scales strictly as $O(T)$, severely limiting the feasibility of scaling vanilla RNNs to long document contexts.

### 2.4.3 Gated recurrent units: LSTM and GRU

To overcome the vanishing gradient pathology and prevent the continuous overwriting of historical context, Hochreiter and Schmidhuber introduced the *Long Short-Term Memory* (LSTM) network in 1997 [23].

The foundational architectural innovation of the LSTM is the decoupling of memory into two distinct operational streams:

1. **Cell state ($\mathbf{c}_t \in \mathbb{R}^{d_h}$)**: Functions as a dedicated, linear long-term memory trunk (colloquially termed the "conveyor belt").
2. **Hidden state ($\mathbf{h}_t \in \mathbb{R}^{d_h}$)**: Serves as the short-term, task-facing output representation used for immediate predictions and gate conditioning.

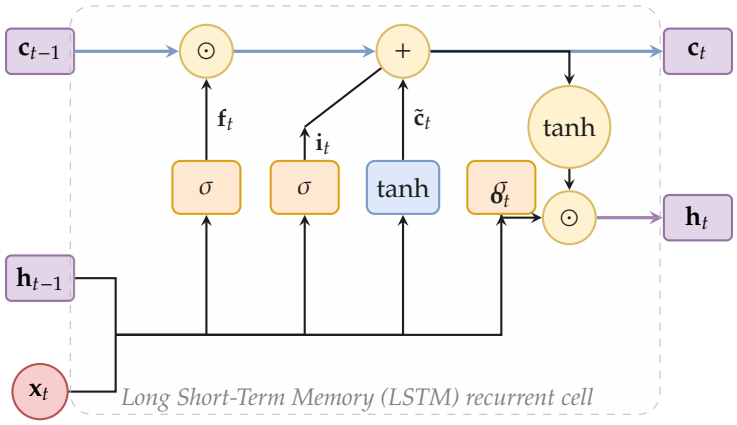



Figure 2.4: Detailed architectural schematic of the Long Short-Term Memory (LSTM) cell, illustrating the linear cell-state trunk ($\mathbf{c}_t$), the multiplicative forget gate ($\mathbf{f}_t$), input gate ($\mathbf{i}_t$), candidate state ($\tilde{\mathbf{c}}_t$), and output gate ($\mathbf{o}_t$).

The interaction between cell memory and incoming data is governed by three non-linear gates, parameterized using logistic sigmoid functions $\sigma(z) = (1 + e^{-z})^{-1} \in [0, 1]$. Because the sigmoid output ranges strictly between 0 and 1, gate activations act as coordinate-wise attenuation coefficients: a value of 0 blocks information transmission entirely, while a value of 1 permits complete transmission.

The algebraic execution of the LSTM unfolds in five sequential stages:

1. **Forget gate**: Determines what fraction of the antecedent cell state $\mathbf{c}_{t-1}$ should be discarded:

$$\mathbf{f}_t = \sigma\left(\mathbf{W}_f \left[\mathbf{h}_{t-1}, \mathbf{x}_t\right] + \mathbf{b}_f\right) \in [0, 1]^{d_h}, \tag{2.36}$$

where $[\mathbf{h}_{t-1}, \mathbf{x}_t] \in \mathbb{R}^{d_h + d_x}$ denotes vector concatenation, $\mathbf{W}_f \in \mathbb{R}^{d_h \times (d_h + d_x)}$ is the forget weight matrix, and $\mathbf{b}_f \in \mathbb{R}^{d_h}$ is the forget bias.

2. **Input gate and candidate memory**: Regulates what new information should be added to the persistent cell state:

$$\mathbf{i}_t = \sigma\left(\mathbf{W}_i\left[\mathbf{h}_{t-1}, \mathbf{x}_t\right] + \mathbf{b}_i\right) \in [0,1]^{d_h}, \tag{2.37}$$

$$\tilde{\mathbf{c}}_t = \tanh\left(\mathbf{W}_c\left[\mathbf{h}_{t-1}, \mathbf{x}_t\right] + \mathbf{b}_c\right) \in (-1,1)^{d_h}, \tag{2.38}$$

where $\mathbf{i}_t$ controls write access, and $\tilde{\mathbf{c}}_t$ creates a new candidate vector of bounded additions.

3. **Cell state update (The Constant Error Carousel)**: The new cell state $\mathbf{c}_t$ is computed via linear combination:

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, \tag{2.39}$$

where $\odot$ denotes element-wise (Hadamard) vector multiplication.

4. **Output gate and hidden state**: Computes the visible output state by filtering the updated cell state:

$$\mathbf{o}_t = \sigma\left(\mathbf{W}_o\left[\mathbf{h}_{t-1}, \mathbf{x}_t\right] + \mathbf{b}_o\right) \in [0,1]^{d_h}, \tag{2.40}$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh\left(\mathbf{c}_t\right) \in (-1,1)^{d_h}. \tag{2.41}$$

*Proof of the Constant Error Carousel (CEC).* To understand why the LSTM eliminates the vanishing gradient pathology, we differentiate the cell state update (Equation 2.39) with respect to the antecedent cell state $\mathbf{c}_{t-1}$:

$$\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-1}} = \operatorname{diag}\left(\mathbf{f}_t\right) + \frac{\partial \mathbf{f}_t}{\partial \mathbf{c}_{t-1}} \odot \mathbf{c}_{t-1} + \frac{\partial \mathbf{i}_t}{\partial \mathbf{c}_{t-1}} \odot \tilde{\mathbf{c}}_t + \mathbf{i}_t \odot \frac{\partial \tilde{\mathbf{c}}_t}{\partial \mathbf{c}_{t-1}}. \tag{2.42}$$

Along the primary diagonal path, the gradient is dominated by the direct linear transmission term:

$$\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-1}} \approx \operatorname{diag}\left(\mathbf{f}_t\right). \tag{2.43}$$

Across an extended temporal interval from step $k$ to step $t$, the accumulated error gradient evaluates to:

$$\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_k} = \prod_{j=k+1}^{t} \operatorname{diag}\left(\mathbf{f}_j\right). \tag{2.44}$$

Crucially, Equation 2.44 does not involve repeated multiplications of a weight matrix $\mathbf{W}_{hh}^{\top}$, nor does it involve saturating activation derivatives $\tanh'$. If the network learns to saturate the forget gates near one ($\mathbf{f}_j \approx \mathbf{1}$), the gradient product evaluates to the identity matrix:

$$\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_k} = \prod_{j=k+1}^{t} \mathbf{I} = \mathbf{I}. \tag{2.45}$$

Under this condition, error gradients propagate backward across arbitrary temporal distances without exponential attenuation, creating a constant error highway (the Constant Error Carousel) that enables the acquisition of long-range dependencies spanning hundreds of steps.

*The Gated Recurrent Unit (GRU).* In 2014, Cho et al. introduced the Gated Recurrent Unit [24], offering a streamlined alternative to the LSTM that merges cell state and hidden state into a single representation $\mathbf{h}_t$, controlled by two gates:

$$\text{Reset gate:} \quad \mathbf{r}_t = \sigma\left(\mathbf{W}_r\left[\mathbf{h}_{t-1}, \mathbf{x}_t\right] + \mathbf{b}_r\right) \in [0,1]^{d_h}, \tag{2.46}$$

$$\text{Update gate:} \quad \mathbf{z}_t = \sigma\left(\mathbf{W}_z\left[\mathbf{h}_{t-1}, \mathbf{x}_t\right] + \mathbf{b}_z\right) \in [0,1]^{d_h}, \tag{2.47}$$

$$\text{Candidate state:} \quad \tilde{\mathbf{h}}_t = \tanh\left(\mathbf{W}_h\left[\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t\right] + \mathbf{b}_h\right) \in (-1,1)^{d_h}, \tag{2.48}$$

$$\text{State interpolation:} \quad \mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t. \tag{2.49}$$

The update gate $\mathbf{z}_t$ acts as an adaptive convex interpolator between past memory $\mathbf{h}_{t-1}$ and candidate update $\tilde{\mathbf{h}}_t$, performing the combined roles of the LSTM's forget and input gates while reducing parameter count by approximately 25%.

| **Architectural Property** | **Vanilla Elman RNN** | **LSTM** | **GRU** |
|---|---|---|---|
| Separate cell memory ($\mathbf{c}_t$) | No | Yes | No |
| Number of gating sub-units | 0 | 3 ($\mathbf{f}_t, \mathbf{i}_t, \mathbf{o}_t$) | 2 ($\mathbf{r}_t, \mathbf{z}_t$) |
| Parameter count scaling | $O\left(d_h^2 + d_h d_x\right)$ | $O\left(4(d_h^2 + d_h d_x)\right)$ | $O\left(3(d_h^2 + d_h d_x)\right)$ |
| Gradient propagation highway | None (exponential decay) | Linear additive trunk | Linear convex interpolation |
| Computational parallelizability | Strictly serialized | Strictly serialized | Strictly serialized |

Table 2.3: Architectural comparison between the vanilla Elman RNN, the Long Short-Term Memory (LSTM) network, and the Gated Recurrent Unit (GRU).

As summarized in Table 2.3, while gated architectures resolve the analytical vanishing gradient problem through additive linear memory trunks, they remain fundamentally serialized along the sequence dimension, preserving the computational bottleneck that prevents efficient parallel scaling.

## 2.5 Sequence-to-sequence modeling, encoder-decoder architectures, and the information bottleneck

All recurrent configurations examined thus far operate under a *one-to-one mapping paradigm*, where each incoming token $\mathbf{x}_t$ immediately triggers the emission of a corresponding prediction $\hat{\mathbf{y}}_t$. While suitable for synchronous sequence tagging, this formulation fails when applied to general sequence-to-sequence transformations.

### 2.5.1 Limitations of one-to-one sequence mapping

Synchronous one-to-one sequence processing exhibits two fatal structural failure modes when applied to complex linguistic tasks such as machine translation, summarization, or dialogue:

1. **Sequence length disparity ($T_x \neq T_y$)**: In natural language translation, source and target sentences rarely contain the same number of words. Consider translating an English utterance into Italian:

$$\text{English source:} \quad \texttt{[It] [is] [indeed] [sunny] [today]} \quad (T_x = 5 \text{ tokens}), \tag{2.50}$$

$$\text{Italian target:} \quad \texttt{[Effettivamente] [oggi] [è] [soleggiato]} \quad (T_y = 4 \text{ tokens}). \tag{2.51}$$

   Under a one-to-one mapping architecture, emitting 4 Italian words requires ingesting exactly 4 English tokens, leaving the 5th token unaligned. If the target sequence is longer than the source ($T_y > T_x$), the recurrent unit exhausts its input tokens and has no valid data to ingest for subsequent steps.

2. **Non-monotonic alignment and lack of future context**: Even if sequence lengths happen to match, the semantic alignment between languages is non-monotonic and structurally inverted. In the example above, the initial Italian word `"Effettivamente"` translates the English adverb `"indeed"`, which occupies the *third* position in the source sentence. Under causal one-to-one processing, the network must generate the first output token after observing only `"It"`, long before the conditioning token `"indeed"` has entered the model.

Table 2.4 illustrates this structural misalignment, demonstrating why causal synchronous emission cannot resolve cross-lingual translation.

| Step $t$ | Source Token $\mathbf{x}_t$ | Target Token $y_t^*$ | Semantic Alignment | Causal Feasibility |
|---|---|---|---|---|
| 1 | `It` | `Effettivamente` | Maps to $\mathbf{x}_3$ (`indeed`) | **No** (future context required) |
| 2 | `is` | `oggi` | Maps to $\mathbf{x}_5$ (`today`) | **No** (future context required) |
| 3 | `indeed` | `è` | Maps to $\mathbf{x}_2$ (`is`) | **No** (inverted syntax) |
| 4 | `sunny` | `soleggiato` | Maps to $\mathbf{x}_4$ (`sunny`) | Partially (local match) |
| 5 | `today` | $\varnothing$ | No target token | **No** (length mismatch) |

Table 2.4: Cross-lingual sequence alignment between English source and Italian target sentences, illustrating the failure of causal one-to-one recurrent mapping due to non-monotonic word order and length disparity.

### 2.5.2 The encoder-decoder paradigm

To resolve the constraints of one-to-one mapping, Sutskever et al. [25] and Cho et al. [24] decoupled sequence processing into two specialized recurrent networks: an *Encoder* and a *Decoder*.

The operational workflow of the Seq2Seq architecture (Figure 2.5) decouples sequence encoding from generation through three distinct phases:

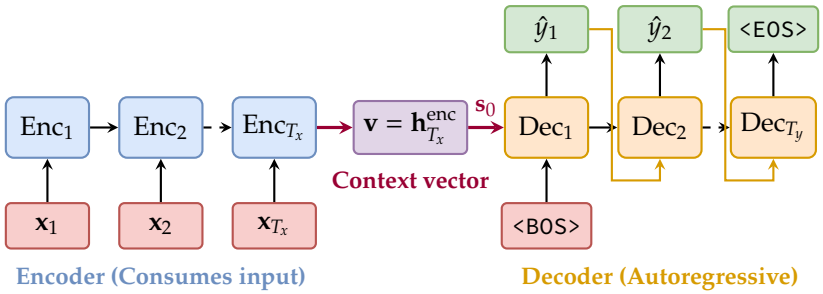



Figure 2.5: The Sequence-to-Sequence (Seq2Seq) Encoder-Decoder architecture. The Encoder ingests the full source sequence of length $T_x$, compressing it into a single context vector $\mathbf{v}$. The Decoder is initialized with $\mathbf{v}$ and generates target tokens autoregressively until emitting the end-of-sequence delimiter <EOS>.

1. **Sequence encoding**: The Encoder RNN sequentially ingests the entire source sequence $\mathbf{x}_1, \ldots, \mathbf{x}_{T_x}$ without generating any output tokens. At the final source token $T_x$, the encoder's terminal hidden state is extracted to serve as the *context vector*:
$$\mathbf{v} = \mathbf{h}^{\text{enc}}_{T_x} \in \mathbb{R}^{d_h}. \tag{2.52}$$
The vector $\mathbf{v}$ represents a compressed semantic summary of the entire source utterance.

2. **Decoder state initialization**: The Decoder RNN initializes its internal state using the context vector:
$$\mathbf{s}_0 = \mathbf{v}, \tag{2.53}$$
thereby conditioning all subsequent generation steps on the global context of the input.

3. **Autoregressive generation**: Generation commences by feeding a synthetic start-of-sequence token, denoted <BOS> (Beginning of Sequence), into the decoder:
$$\mathbf{s}_1 = \text{DecoderRNN}\left(\mathbf{s}_0, \mathbf{e}_{\texttt{<BOS>}}\right). \tag{2.54}$$
The decoder projects state $\mathbf{s}_1$ to emit a categorical probability distribution over target vocabulary $\mathcal{V}_y$, sampling or greedily selecting token $\hat{y}_1$. In the subsequent step, the previously generated token $\hat{y}_1$ is fed back as the input token to compute $\mathbf{s}_2$:
$$\mathbf{s}_t = \text{DecoderRNN}\left(\mathbf{s}_{t-1}, \mathbf{e}_{\hat{y}_{t-1}}\right). \tag{2.55}$$
Generation terminates when the decoder emits a dedicated end-of-sequence token, denoted <EOS> (End of Sequence), allowing target length $T_y$ to be determined dynamically by the model.

Mathematically, the encoder-decoder framework models the conditional probability of target sequence $Y = (y_1, \ldots, y_{T_y})$ given source sequence $X = (\mathbf{x}_1, \ldots, \mathbf{x}_{T_x})$ via the exact probability chain rule:
$$P(Y \mid X) = \prod_{t=1}^{T_y} P\left(y_t \mid y_1, \ldots, y_{t-1}, \mathbf{v}\right). \tag{2.56}$$

### 2.5.3 Empirical validation: revised Sherlock Holmes casing

To evaluate how the encoder-decoder paradigm resolves the causal horizon failure identified in Section 2.3.3, we revisit the character casing benchmark on the ambiguous test utterance:

Input text: `"indeed iam sherlock holmes,"` (2.57)

Target text: `"Indeed I am Sherlock Holmes,"` (2.58)

Under the one-to-one causal RNN evaluated in Section 2.3.3, the model committed four distinct capitalization errors:

1. **The initial token `'i'` in `indeed`**: Capitalized correctly only because it immediately followed an opening quote.
2. **The isolated pronoun `'i'` in `iam`**: Remained lowercase (`'i'`) because the causal model, seeing only antecedent tokens `"indeed "`, could not determine whether `'i'` represented the personal pronoun *I* or the initial letter of common words such as *in*, *is*, *it*, *if*.
3. **The initial `'s'` in `sherlock`**: Remained lowercase because `"iam "` is followed by lowercase words in over 90% of training contexts.
4. **The initial `'h'` in `holmes`**: Capitalized correctly only through local bigram memorization following `sherlock`.

When the exact same sequence is evaluated using a Sequence-to-Sequence Encoder-Decoder architecture, the model resolves all four tokens without ambiguity:

$$\text{Seq2Seq Output:} \quad \texttt{"Indeed I am Sherlock Holmes,"} \tag{2.59}$$

The conceptual basis for this success resides in the timing of representation formation. In the Seq2Seq paradigm, the Encoder ingests the entire sentence `"indeed iam sherlock holmes,"` before the Decoder generates a single character. When the Decoder initiates generation from $\mathbf{s}_0 = \mathbf{v}$, the context vector $\mathbf{v}$ already encodes the presence of the proper noun `sherlock holmes` and the pronoun `i`. With full bidirectional information synthesized into the context vector, the decoder generates correctly capitalized characters from the very first token, resolving ambiguities that are mathematically unsolvable under causal one-to-one processing.

### 2.5.4 The information bottleneck and the prelude to attention

Despite the empirical triumph of the Encoder-Decoder paradigm on short sentences, the architecture encounters an inescapable theoretical barrier: the *fixed-vector information bottleneck* [26].

In the standard Seq2Seq formulation, the Encoder must compress all semantic, syntactic, stylistic, and relational information present in a source sequence of arbitrary length $T_x$ into a single, fixed-dimensional Euclidean vector $\mathbf{v} \in \mathbb{R}^{d_h}$. Mathematically, this transformation represents a severe dimensionality reduction mapping:

$$f_{\text{enc}} : \mathbb{R}^{T_x \times d_x} \longrightarrow \mathbb{R}^{d_h}. \tag{2.60}$$

For a short sentence of 5 words, compressing the input into $d_h = 512$ dimensions preserves sufficient fidelity for accurate reconstruction. However, as sequence length increases to 20, 50, or 100 words, the amount of information in the source sequence grows linearly with $T_x$, while the capacity of the context vector $\mathbf{v}$ remains strictly constant at $d_h$.

Consequently, the context vector becomes saturated. As the encoder processes later words in a long document, earlier clauses are progressively overwritten and attenuated by the recurrent transition operator. When the decoder begins generating target tokens, the context vector $\mathbf{v}$ retains vivid representations of the final few source tokens, but retains only degraded, blurred traces of earlier tokens.

Empirical evaluations by Cho et al. [24] and Bahdanau et al. [26] demonstrated that the translation performance of standard RNN encoder-decoder models plummets catastrophically when sentence length exceeds 15 to 20 words. Even when employing multi-layer LSTMs with thousands of hidden units, compressing an arbitrary-length document into a single static vector remains an insurmountable information-theoretic bottleneck.

We are therefore confronted with the dual architectural crises of recurrent sequence modeling:

1. **The computational crisis**: The strict sequential recurrence $\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t)$ serializes training along the sequence dimension, precluding GPU hardware parallelization and enforcing an $O(T)$ training wall-clock runtime.

2. **The representational crisis**: Compressing variable-length context into a single static state vector $\mathbf{v} \in \mathbb{R}^{d_h}$ destroys long-range dependencies and forces decoders to reconstruct sentences from lossy, blurred summaries.

Resolving these dual crises requires a fundamental paradigm shift: replacing static vector compression with dynamic, query-conditioned associative retrieval. Rather than forcing a single vector to summarize an entire sequence, the model should maintain a collection of all intermediate encoder states and allow the decoder to dynamically "attend" to the most relevant source positions at each step of generation. This foundational breakthrough—the Attention Mechanism—forms the conceptual bridge that dismantles the recurrent paradigm and underpins the modern Transformer architecture.

Chapter 3

# Language models

The preceding chapters established the computational foundations of deep learning and analyzed recurrent neural networks as dynamical systems capable of processing variable-length sequences. However, before deploying neural architectures to process or generate human text, a foundational theoretical question must be resolved: what does it mean, in formal mathematical terms, to model a natural language? Rather than treating text as arbitrary strings of alphanumeric characters or hand-crafted symbolic parse trees, modern computational linguistics conceptualizes language as a discrete stochastic process governed by an underlying, non-uniform probability distribution over sequential utterances.

The goal of language modeling is to formulate computational mechanisms capable of distinguishing between probable, semantically coherent utterances and implausible, grammatically corrupted sequences. Historically, this objective originated in the pioneering work of Claude Shannon and the engineering impasses of early speech recognition, where deterministic grammars proved fundamentally inadequate to capture the pervasive ambiguity and continuous variability of natural human expression. Early statistical approaches addressed this challenge through Markovian assumptions and discrete frequency estimation over local $n$-gram tuples, establishing the foundational paradigms of maximum likelihood training, likelihood evaluation, and autoregressive generation.

This chapter develops the theoretical, statistical, and algorithmic principles of language modeling. We begin by examining the historical imperatives that motivated probabilistic language modeling and formalize the joint distribution over word sequences via the probability chain rule. We then formulate the Markov assumption and $n$-gram models, trace parameter estimation via Maximum Likelihood Estimation on a concrete corpus, and systematically analyze the dual inference modes: sequence likelihood evaluation and autoregressive sequence generation. Finally, we dissect the fundamental limitations of statistical counting models—specifically combinatorial data sparsity, the zero-frequency collapse, and orthogonal symbolic representations—examine smoothing and discounting remedies, derive information-theoretic evaluation metrics including cross-entropy and perplexity, and establish the structural necessity of transitioning to continuous distributed word embeddings.

## 3.1 Introduction to language models and the modeling task

Understanding modern foundation architectures requires first examining how natural language was formalized as a statistical object. The formulation of language modeling emerged not from abstract linguistic taxonomy, but from concrete engineering challenges in telecommunications and signal processing that required machines to quantify the plausibility of textual hypotheses.

### 3.1.1 Historical motivation: the urge to model natural language

The mathematical formalization of language modeling originated in Claude Shannon's landmark 1948 treatise, *A Mathematical Theory of Communication* [27]. Shannon sought to quantify the fundamental limits of data compression and transmission across noisy telecommunication channels. Rather than viewing text as static philosophical prose, Shannon conceptualized natural language as a stochastic process generating discrete sequences of symbols chosen from a finite alphabet according to stationary statistical probabilities. By measuring the entropy of printed English through letter-level and word-level approximation experiments, Shannon demonstrated that natural text possesses substantial statistical redundancy, implying that preceding context exerts powerful probabilistic constraints on upcoming symbols.

In the decades that followed, the imperative to build automated language models transitioned from theoretical information theory to critical real-world signal processing domains:

1. **Acoustic speech recognition (ASR)**: In automated speech transcription, an input acoustic waveform must be mapped to a written sequence of words. However, human phonetics exhibits pervasive phonetic ambiguity: homophones and continuous connected speech yield acoustic signals that correspond equally well to multiple disparate word sequences. For example, the acoustic sequence corresponding to the phonetic sequence `/rek-uhg-nyze-beech/` can be transcribed as either *"recognize speech"* or *"wreck a nice beach"*. An acoustic sensor analyzing frequency spectra alone cannot reliably resolve this ambiguity. To achieve human-level accuracy, an ASR pipeline must factorize the problem into two complementary components:

$$W^* = \arg\max_W P(W \mid A) = \arg\max_W \frac{P(A \mid W)P(W)}{P(A)} = \arg\max_W P(A \mid W)P(W), \tag{3.1}$$

   where $P(A \mid W)$ denotes the acoustic likelihood model (how likely the sound $A$ is given words $W$), and $P(W)$ represents the language model prior. Because the phrase *"recognize speech"* occurs frequently in technical discourse while *"wreck a nice beach"* is exceptionally rare, the prior enforces $P(\text{"recognize speech"}) \gg P(\text{"wreck a nice beach"})$, allowing the system to effortlessly resolve phonetic noise.

2. **Machine translation (MT) and optical character recognition (OCR)**: In machine translation, transferring syntactic structures between disparate languages rarely proceeds via word-for-word substitution. An automated translator generating candidate lexical mappings requires a statistical prior over the target language to select phrase orderings that sound natural and grammatically fluent. Similarly, in optical character recognition, noisy visual scans that render ambiguous characters (such as distinguishing `'l'`, `'1'`, and `'I'`) are disambiguated by evaluating which lexical candidate forms a coherent word sequence within the sentence.

These engineering requirements triggered a philosophical pivot in computational linguistics. Early artificial intelligence research, guided by Chomskyan generative grammars [28], attempted to model natural language through deterministic, handcrafted formal rule sets that partitioned sentences strictly into grammatically valid or invalid categories. However, human language constantly breaks rigid grammatical rules through idioms, colloquialisms, poetry, and colloquial

speech while remaining completely intelligible. Statistical language modeling replaced this binary grammaticality dichotomy with a continuous, probabilistic spectrum of plausibility.

### 3.1.2 The language model task: formal probability distribution

Formally, let $\mathcal{V}$ denote a finite vocabulary comprising all discrete token primitives recognized by the system, with cardinality $|\mathcal{V}|$. An arbitrary natural language utterance $W$ is defined as an ordered tuple of $T$ tokens:

$$W = (w_1, w_2, \ldots, w_T), \quad w_t \in \mathcal{V}, \quad T \in \mathbb{N}^+. \tag{3.2}$$

The set of all possible finite sequences formed from vocabulary $\mathcal{V}$ is given by the Kleene closure $\mathcal{V}^* = \bigcup_{k=1}^{\infty} \mathcal{V}^k$.

A *Language Model* (LM) is a mathematical function that assigns a joint probability $P(W)$ to every possible sequence $W \in \mathcal{V}^*$, satisfying the standard Kolmogorov probability axioms:

$$P(W) \geq 0 \quad \forall W \in \mathcal{V}^*, \quad \text{and} \quad \sum_{W \in \mathcal{V}^*} P(W) = 1. \tag{3.3}$$

The probability distribution $P(W)$ can be conceptualized as defining a continuous density landscape across the discrete space of possible word combinations. In this space:

- **High-probability regions**: Encompass plausible, fluent, and syntactically well-formed sentences that frequently occur in human communication:

$$P(\texttt{"This is a reasonable sentence"}) \approx 10^{-1}. \tag{3.4}$$

- **Low-probability regions**: Encompass syntactically well-formed but semantically anomalous or bizarre sequences that violate common semantic expectations:

$$P(\texttt{"This is a purple sentence"}) \approx 10^{-3}. \tag{3.5}$$

- **Near-zero probability regions**: Encompass ungrammatical, disjoint, or corrupted strings of symbols that violate the structural syntax of the language:

$$P(\texttt{"This are a reasonable sentence"}) \approx 10^{-5}. \tag{3.6}$$

Because the combinatorial space of arbitrary symbol permutations is astronomically large, the overwhelming majority of sequences in $\mathcal{V}^*$ are ungrammatical noise. Consequently, natural human language occupies an extremely narrow, low-dimensional probability manifold within $\mathcal{V}^*$. The fundamental objective of language modeling is to accurately estimate this non-uniform probability distribution from finite empirical observations.

### 3.1.3 Factoring joint probabilities via the chain rule

Directly estimating the joint probability $P(w_1, w_2, \ldots, w_T)$ over an entire sentence simultaneously is computationally intractable because the number of possible complete sentences grows exponentially with sequence length $T$. To render this estimation feasible, we apply the general *chain rule of probability* (or Kolmogorov multiplication law), which factors any joint distribution into a product of conditional distributions:

$$\begin{aligned} P(W) &= P(w_1, w_2, \ldots, w_T) \\ &= P(w_1)\, P(w_2 \mid w_1)\, P(w_3 \mid w_1, w_2) \cdots P(w_T \mid w_1, \ldots, w_{T-1}) \\ &= \prod_{t=1}^{T} P(w_t \mid w_1, \ldots, w_{t-1}) = \prod_{t=1}^{T} P(w_t \mid w_{<t}), \end{aligned} \tag{3.7}$$

where $w_{<t} = (w_1, \ldots, w_{t-1})$ denotes the sequence prefix comprising all tokens preceding step $t$, with the base case defined as $P(w_1 \mid w_{<1}) = P(w_1)$.

Equation (3.7) provides a profound conceptual and operational shift. It demonstrates that the problem of assigning a joint probability to an entire sentence is mathematically identical to the sequential task of predicting the conditional probability distribution of the single next word $w_t$ given the observed historical context $w_{<t}$.

This formulation directly aligns language modeling with the classical *Cloze* (fill-in-the-blank) evaluation methodology in cognitive linguistics:

$$\text{Prompt 1:} \quad \texttt{"This is a reasonable [ ]"} \implies P\left(w_t = \texttt{"sentence"} \mid w_{<t}\right), \tag{3.8}$$

$$\text{Prompt 2:} \quad \texttt{"The black cat sat on the [ ]"} \implies P\left(w_t = \texttt{"mat"} \mid w_{<t}\right). \tag{3.9}$$

By framing language modeling as sequential next-token prediction, we can construct models that process text incrementally from left to right, evaluating sentence plausibility and generating synthetic continuations through successive conditional probability evaluations.

## 3.2 The Markov assumption, $n$-gram models, and operational inference

While Equation (3.7) is mathematically exact, it cannot be implemented directly in practice using raw frequency tables. As sequence length $t$ increases, the conditioning prefix $w_{<t}$ grows linearly, meaning that the historical context becomes unique. To construct practical statistical models, we must introduce simplifying independence assumptions that truncate the conditioning horizon.

### 3.2.1 The Markov assumption and combinatorial reduction

Consider estimating the conditional probability $P(w_t \mid w_1, \ldots, w_{t-1})$ using empirical counting over a text corpus. For a vocabulary of size $|\mathcal{V}| = 50{,}000$, a prefix of length $t - 1 = 10$ admits $|\mathcal{V}|^{10} \approx 9.76 \times 10^{46}$ distinct possible histories. Because no realistic training corpus contains more than $10^{10}$ to $10^{12}$ total words, the probability of encountering any specific 10-word prefix multiple times approaches zero, rendering empirical frequency ratios completely undefined.

To overcome this combinatorial barrier, statistical language modeling applies the *Markov assumption* [29]. Formally, an $(n-1)$-th order Markov chain posits that the conditional probability of the current state depends exclusively on the immediately preceding $n-1$ states, remaining conditionally independent of all earlier history:

$$P\left(w_t \mid w_1, w_2, \ldots, w_{t-1}\right) \approx P\left(w_t \mid w_{t-n+1}, \ldots, w_{t-1}\right). \tag{3.10}$$

Under the $(n-1)$-th order Markov assumption, an $n$-gram language model approximates the true joint probability of a sequence as:

$$P\left(w_1, \ldots, w_T\right) \approx \prod_{t=1}^{T} P\left(w_t \mid w_{t-n+1}, \ldots, w_{t-1}\right). \tag{3.11}$$

The choice of parameter $n$ defines the context horizon of the statistical model:

1. **Unigram model ($n = 1$)**: Assumes complete zeroth-order independence. The probability of each word is evaluated in total isolation from its context:

$$P\left(w_1, \ldots, w_T\right) \approx \prod_{t=1}^{T} P\left(w_t\right). \tag{3.12}$$

   A unigram model ignores all syntactic structure and word order, treating sentences as an unordered "bag of words".

2. **Bigram model ($n = 2$)**: Imposes a first-order Markov assumption. The probability of the next word depends strictly and exclusively on the single immediately preceding word:

$$P\left(w_1, \ldots, w_T\right) \approx P\left(w_1\right) \prod_{t=2}^{T} P\left(w_t \mid w_{t-1}\right). \tag{3.13}$$

3. **Trigram model ($n = 3$)**: Imposes a second-order Markov assumption, conditioning each word on the preceding two-word prefix:

$$P\left(w_1, \ldots, w_T\right) \approx P\left(w_1\right) P\left(w_2 \mid w_1\right) \prod_{t=3}^{T} P\left(w_t \mid w_{t-2}, w_{t-1}\right). \tag{3.14}$$

### 3.2.2 The worked transition matrix and parameter estimation

By truncating the conditioning horizon to a small constant $n$, the parameters of an $n$-gram model can be estimated directly from a reference corpus using *Maximum Likelihood Estimation* (MLE). Under MLE, the parameter values are chosen to maximize the likelihood of the observed training text, which equates to computing empirical frequency ratios.

For a bigram model ($n = 2$), the conditional probability of token $w_t$ given predecessor $w_{t-1}$ is evaluated by counting the joint occurrences of the bigram $(w_{t-1}, w_t)$ and dividing by the unigram frequency of the prefix $w_{t-1}$:

$$P_{\text{MLE}}(w_t \mid w_{t-1}) = \frac{C(w_{t-1}, w_t)}{\sum_{w \in \mathcal{V}} C(w_{t-1}, w)} = \frac{C(w_{t-1}, w_t)}{C(w_{t-1})}, \tag{3.15}$$

where $C(\cdot)$ denotes the count of exact occurrences in the training corpus.

*Worked pedagogical example.* To ground this formulation in concrete numbers, we analyze the exact toy corpus evaluated during course lectures. Consider a tiny reference corpus $C$ consisting of two sentences:

$$\text{Sentence 1:} \quad \texttt{"The cat chased the mouse happily."} \tag{3.16}$$

$$\text{Sentence 2:} \quad \texttt{"The mouse ate the cheese."} \tag{3.17}$$

Extracting all unique lexical tokens yields a vocabulary of cardinality $|\mathcal{V}| = 7$:

$$\mathcal{V} = \{\texttt{ate}, \texttt{cat}, \texttt{chased}, \texttt{cheese}, \texttt{happily}, \texttt{mouse}, \texttt{the}\}. \tag{3.18}$$

We construct the empirical transition matrix by scanning all adjacent bigram pairs across the corpus. Scanning Sentence 1 yields the bigram sequence:

$$(\texttt{the}, \texttt{cat}),\ (\texttt{cat}, \texttt{chased}),\ (\texttt{chased}, \texttt{the}),\ (\texttt{the}, \texttt{mouse}),\ (\texttt{mouse}, \texttt{happily}). \tag{3.19}$$

Scanning Sentence 2 yields:

$$(\texttt{the}, \texttt{mouse}),\ (\texttt{mouse}, \texttt{ate}),\ (\texttt{ate}, \texttt{the}),\ (\texttt{the}, \texttt{cheese}). \tag{3.20}$$

Summing these occurrences yields the absolute frequency matrix $C(w_{t-1}, w_t)$. Normalizing each row by the unigram count of the conditioning prefix ensures that $\sum_{w_t \in \mathcal{V}} P(w_t \mid w_{t-1}) = 1.0$ for all non-terminal rows, resulting in the complete bigram transition matrix presented in Table 3.1.

| $w_{t-1} \setminus w_t$ | **ate** | **cat** | **chased** | **cheese** | **happily** | **mouse** | **the** |
|---|---|---|---|---|---|---|---|
| **ate** | 0 | 0 | 0 | 0 | 0 | 0 | 1.00 (1/1) |
| **cat** | 0 | 0 | 1.00 (1/1) | 0 | 0 | 0 | 0 |
| **chased** | 0 | 0 | 0 | 0 | 0 | 0 | 1.00 (1/1) |
| **cheese** | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| **happily** | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| **mouse** | 0.50 (1/2) | 0 | 0 | 0 | 0.50 (1/2) | 0 | 0 |
| **the** | 0 | 0.25 (1/4) | 0 | 0.25 (1/4) | 0 | 0.50 (2/4) | 0 |

Table 3.1: Empirical Maximum Likelihood Estimation (MLE) bigram transition probability matrix computed from the lecture reference corpus. Each entry $(i, j)$ represents the conditional probability $P(w_j \mid w_i) = C(w_i, w_j)/C(w_i)$. Rows for terminal words (`cheese`, `happily`) sum to zero in the absence of explicit sentence boundary delimiters.

Inspecting Table 3.1 illustrates the statistical profile learned by the model:

- Following the word `"the"`, the model distributes probability mass across three nouns: $P(\texttt{mouse} \mid \texttt{the}) = 0.50$, $P(\texttt{cat} \mid \texttt{the}) = 0.25$, and $P(\texttt{cheese} \mid \texttt{the}) = 0.25$.
- Following the word `"mouse"`, the model assigns equal probability to the verb `"ate"` and the adverb `"happily"`: $P(\texttt{ate} \mid \texttt{mouse}) = 0.50$, $P(\texttt{happily} \mid \texttt{mouse}) = 0.50$.
- Deterministic transitions emerge where a word was observed with only a single successor: $P(\texttt{chased} \mid \texttt{cat}) = 1.00$, $P(\texttt{the} \mid \texttt{chased}) = 1.00$, and $P(\texttt{the} \mid \texttt{ate}) = 1.00$.

Once the transition matrix is populated and normalized, it functions as a complete, trained statistical language model. We can deploy this model in two fundamental operational modes: sequence likelihood evaluation and autoregressive text generation.

### 3.2.3 Inference: evaluating sentence likelihood

The first core inference task consists in evaluating the likelihood of a candidate word sequence. Given an unobserved test sentence, the language model computes its joint probability by taking the product of the corresponding transition probabilities from Table 3.1.

*Worked numerical evaluation.* Suppose we deploy the model to evaluate the probability of a candidate sentence that did not appear in the training corpus:

$$W_{\text{test}} = \texttt{"The cat chased the cheese."} \tag{3.21}$$

Applying the bigram factorization from Equation (3.11):

$$P\left(W_{\text{test}}\right) = P\left(\texttt{the}\right) \cdot P\left(\texttt{cat} \mid \texttt{the}\right) \cdot P\left(\texttt{chased} \mid \texttt{cat}\right) \cdot P\left(\texttt{the} \mid \texttt{chased}\right) \cdot P\left(\texttt{cheese} \mid \texttt{the}\right). \tag{3.22}$$

Assuming the unigram prior for the sentence-initial token is $P(\texttt{the}) = 1.0$ (since all training sentences in corpus $C$ begin with `"The"`), we substitute the empirical conditional probabilities from Table 3.1:

$$\begin{aligned} P\left(W_{\text{test}}\right) &= 1.0 \times \frac{1}{4} \times 1.0 \times 1.0 \times \frac{1}{4} \\ &= 1.0 \times 0.25 \times 1.0 \times 1.0 \times 0.25 = \frac{1}{16} = 0.0625. \end{aligned} \tag{3.23}$$

The model assigns a non-zero probability ($P = 0.0625$) to *"The cat chased the cheese"*. Crucially, this demonstrates that statistical language models can assign valid probability mass to novel combinations of words that were never observed verbatim during training, provided that all constituent transitions exist in the learned transition graph.

*Numerical stability via log-likelihood.* In practical applications involving sentences of realistic length ($T \geq 20$), multiplying dozens of conditional probabilities bounded within $[0, 1]$ leads directly to numerical underflow:

$$P(W) = \prod_{t=1}^{T} P\left(w_t \mid w_{t-1}\right) < 10^{-308}, \tag{3.24}$$

exceeding the precision of standard 64-bit IEEE double-precision floating-point arithmetic. To guarantee numerical stability, all probability computations are evaluated in the logarithmic domain:

$$\log P(W) = \log P\left(w_1\right) + \sum_{t=2}^{T} \log P\left(w_t \mid w_{t-1}\right). \tag{3.25}$$

Operating in log-space transforms unstable products of small fractions into stable sums of negative real numbers, preserving full analytical fidelity across sequences of arbitrary length.

### 3.2.4 Inference: autoregressive text generation and decoding dynamics

The second operational mode of a language model is *autoregressive text generation*. In this regime, the model does not score an existing sentence; instead, it synthesizes new text token by token. At each step $t$, the model conditions on the antecedent token $w_{t-1}$, retrieves the conditional categorical distribution $P(\cdot \mid w_{t-1})$ from its transition matrix, samples a token $w_t$, appends $w_t$ to the generated sequence, and feeds $w_t$ back into the model to condition the subsequent step:

$$w_t \sim P\left(\cdot \mid w_{t-1}\right). \tag{3.26}$$

*Step-by-step lecture generation trace.* We trace the complete generative execution evaluated in lecture, initiated with the prompt token $w_1 =$ `"The"`:

1. **Step 1**: Prompt $w_1 =$ `"The"`. The model queries row `"the"` in Table 3.1, obtaining the categorical distribution:

$$P\left(\texttt{cat} \mid \texttt{the}\right) = 0.25, \quad P\left(\texttt{cheese} \mid \texttt{the}\right) = 0.25, \quad P\left(\texttt{mouse} \mid \texttt{the}\right) = 0.50. \tag{3.27}$$

   Sampling stochastically from this distribution selects the candidate $w_2 =$ `"mouse"`.

$$\text{Current sequence:} \quad \texttt{"The mouse"} \tag{3.28}$$

2. **Step 2**: Preceding token $w_2 =$ `"mouse"`. The model queries row `"mouse"`:

$$P\left(\texttt{ate} \mid \texttt{mouse}\right) = 0.50, \quad P\left(\texttt{happily} \mid \texttt{mouse}\right) = 0.50. \tag{3.29}$$

   Sampling from this distribution selects $w_3 =$ `"ate"`.

$$\text{Current sequence:} \quad \texttt{"The mouse ate"} \tag{3.30}$$

3. **Step 3**: Preceding token $w_3 =$ `"ate"`. The model queries row `"ate"`:

$$P\left(\texttt{the} \mid \texttt{ate}\right) = 1.00. \tag{3.31}$$

   The transition is fully deterministic, yielding $w_4 =$ `"the"`.

$$\text{Current sequence:} \quad \texttt{"The mouse ate the"} \tag{3.32}$$

4. **Step 4**: Preceding token $w_4 =$ `"the"`. The model again queries row `"the"`. Although the model previously generated `"mouse"` at step 2, the bigram Markov assumption possesses zero memory of steps 1 to 3. Sampling from the identical distribution selects $w_5 =$ `"mouse"`.

$$\text{Current sequence:} \quad \texttt{"The mouse ate the mouse"} \tag{3.33}$$

5. **Step 5**: Preceding token $w_5 =$ `"mouse"`. Consulting row `"mouse"`, the model samples $w_6 =$ `"happily"`.

$$\text{Current sequence:} \quad \texttt{"The mouse ate the mouse happily"} \tag{3.34}$$

6. **Step 6**: Preceding token $w_6 =$ `"happily"`. Row `"happily"` contains strictly zero counts across all columns ($\sum_w C(\texttt{happily}, w) = 0$). In the transition graph, `"happily"` represents an *absorbing terminal state*. The model has no valid successor tokens to sample, and generation halts.

The final generated utterance is:

$$\texttt{"The mouse ate the mouse happily."} \tag{3.35}$$

This sentence never existed in the training corpus; it was synthesized dynamically through iterative sampling along the stochastic transition graph.

*Decoding strategies and pathological dynamics.* The generation trace highlights two critical operational aspects of autoregressive generation:

1. **Absorbing terminal states and explicit boundary modeling**: In the toy example, generation halted because the terminal word `"happily"` had zero outgoing transitions. In rigorous natural language processing, stopping conditions are formalized by augmenting the vocabulary with dedicated boundary tokens: `<BOS>` (Beginning of Sentence) and `<EOS>` (End of Sentence). Training sentences are padded as `<BOS>` $w_1 \ \ldots \ w_T$ `<EOS>`. The model learns explicit transition probabilities $P(\texttt{<EOS>} \mid w_t)$, allowing the network to decide autonomously when an utterance is complete.

2. **Greedy decoding versus stochastic sampling and cyclic limit cycles**: Instead of sampling stochastically, an alternative decoding strategy is *greedy decoding*, where the model deterministically selects the single token with maximum conditional probability:

$$w_t^* = \arg\max_{w \in \mathcal{V}} P\left(w \mid w_{t-1}\right). \tag{3.36}$$

   However, when greedy decoding is applied to finite transition matrices, the model frequently falls into *cyclic limit cycles*. In Table 3.1, starting from `"the"`, the greedy choice is $w_2$ = `"mouse"` ($P = 0.50$). If ties are broken in favor of `"ate"`, the sequence proceeds `"the"` $\rightarrow$ `"mouse"` $\rightarrow$ `"ate"` $\rightarrow$ `"the"` $\rightarrow$ `"mouse"` $\rightarrow$ `"ate"` $\ldots$, trapping the model in an infinite periodic loop. This pathology persists in modern foundation models, necessitating advanced decoding heuristics such as temperature scaling, top-$k$ sampling, and nucleus (top-$p$) truncation.

## 3.3 Limitations of statistical language models and the path to neural representations

Despite their mathematical simplicity and clear probabilistic interpretations, statistical $n$-gram models suffer from severe theoretical, combinatorial, and representational pathologies that fundamentally limit their scalability.

### 3.3.1 Combinatorial data sparsity and the zero-frequency collapse

The most fatal pathology of $n$-gram models is the curse of combinatorial data sparsity. An $n$-gram model over a realistic vocabulary $\mathcal{V}$ requires maintaining a contingency table whose parameter count scales exponentially with the context window size:

$$N_{\text{parameters}} = |\mathcal{V}|^n. \tag{3.37}$$

For a standard vocabulary of size $|\mathcal{V}| = 50{,}000$:

- **Bigram ($n = 2$)**: Requires $|\mathcal{V}|^2 = 2.5 \times 10^9$ parameters (requiring $\approx 10$ GB of memory under 32-bit floats).
- **Trigram ($n = 3$)**: Requires $|\mathcal{V}|^3 = 1.25 \times 10^{14}$ parameters (requiring $\approx 500$ TB of storage).
- **5-gram ($n = 5$)**: Requires $|\mathcal{V}|^5 = 3.125 \times 10^{23}$ parameters, far exceeding the physical storage capacity of global computing infrastructure.

Because human language allows words to combine in virtually boundless configurations, no finite corpus can ever observe more than a microscopic fraction of all valid $n$-grams. In any empirical training set, the vast majority of legitimate word combinations have an observed count of zero:

$$C(w_{t-n+1}, \ldots, w_t) = 0. \tag{3.38}$$

Under Maximum Likelihood Estimation, if a specific bigram $(w_{t-1}, w_t)$ was never seen during training, its assigned probability is identically zero:

$$P_{\text{MLE}}(w_t \mid w_{t-1}) = \frac{0}{C(w_{t-1})} = 0. \tag{3.39}$$

This leads directly to the *zero-frequency collapse*: when evaluating a candidate test sentence $W = (w_1, \ldots, w_T)$, if even a single constituent bigram is unobserved in the training corpus, the entire multiplicative chain collapses to zero:

$$P(W) = \prod_{t=1}^{T} P(w_t \mid w_{t-1}) = 0. \tag{3.40}$$

In log-space, $\log P(W) = \log 0 = -\infty$. Consequently, a model that encounters a single unobserved word combination declares the entire sequence impossible, causing evaluation metrics such as cross-entropy and perplexity to explode to infinity.

### 3.3.2 Smoothing and discounting techniques

To prevent probability collapse, statistical NLP developed *smoothing* (or discounting) algorithms. Smoothing techniques shave off a small fraction of probability mass from frequently observed events and reallocate it to unobserved events, ensuring that $P(w_t \mid w_{t-1}) > 0$ for all possible token combinations.

### Laplace and additive smoothing

The simplest smoothing formulation is *Laplace (Add-1) smoothing*, which artificially increments the count of every possible $n$-gram by 1:

$$P_{\text{Laplace}}(w_t \mid w_{t-1}) = \frac{C(w_{t-1}, w_t) + 1}{\sum_{w \in \mathcal{V}} (C(w_{t-1}, w) + 1)} = \frac{C(w_{t-1}, w_t) + 1}{C(w_{t-1}) + |\mathcal{V}|}. \tag{3.41}$$

A generalized variant, *additive ($\alpha$) smoothing* (or Lidstone smoothing), adds a fractional pseudo-count $0 < \alpha < 1$:

$$P_{\alpha}(w_t \mid w_{t-1}) = \frac{C(w_{t-1}, w_t) + \alpha}{C(w_{t-1}) + \alpha|\mathcal{V}|}. \tag{3.42}$$

*Analytical critique of Laplace smoothing.* While Laplace smoothing guarantees that no probability evaluates to zero, it distorts probability distributions disastrously when vocabulary size $|\mathcal{V}|$ is large. Suppose $|\mathcal{V}| = 50{,}000$ and an observed word appears with unigram count $C(w_{t-1}) = 100$. The denominator in Equation (3.41) increases from 100 to $100+50{,}000 = 50{,}100$. Consequently, an observed bigram that appeared 5 times has its estimated probability reduced from $5/100 = 0.05$ to $6/50{,}100 \approx 0.00012$, attenuating genuine empirical evidence by over 99%. Laplace smoothing steals far too much probability mass from observed events and scatters it indiscriminately across billions of unobserved combinations.

### Good-Turing discounting

To establish a more principled redistribution of probability mass, the *Good-Turing estimator* [30] utilizes the frequency of other frequencies. Let $N_r$ denote the number of distinct $n$-grams that were observed exactly $r$ times in the training corpus. Good-Turing re-estimates the effective count $r^*$ for an $n$-gram observed $r$ times as:

$$r^* = (r+1)\frac{N_{r+1}}{N_r}. \tag{3.43}$$

The total probability mass reserved for all unseen events ($r = 0$) is governed by the proportion of singleton $n$-grams (events seen exactly once, $N_1$):

$$P_{\text{GT}}(\text{unseen}) = \frac{N_1}{N}, \tag{3.44}$$

where $N = \sum_{r=1}^{\infty} rN_r$ is the total token count of the corpus. Good-Turing grounds probability estimation in empirical discovery rates: the likelihood of encountering an unobserved token is proportional to the number of tokens observed only once.

### Backoff and linear interpolation

When higher-order $n$-gram counts are scarce, models can borrow statistical strength from lower-order distributions. Two primary strategies exist:

1. **Jelinek-Mercer linear interpolation**: Computes a convex linear combination of trigram, bigram, and unigram probabilities:

$$P_{\text{interp}}(w_t \mid w_{t-2}, w_{t-1}) = \lambda_3 P_{\text{MLE}}(w_t \mid w_{t-2}, w_{t-1}) + \lambda_2 P_{\text{MLE}}(w_t \mid w_{t-1}) + \lambda_1 P_{\text{MLE}}(w_t), \tag{3.45}$$

   where interpolation coefficients satisfy $\sum_{i=1}^{3} \lambda_i = 1$ and $\lambda_i > 0$.

2. **Katz backoff**: Prioritizes the highest-order model whenever empirical counts exceed zero, backing off to discounted lower-order estimates only when the higher-order count is zero [31]:

$$P_{\text{Katz}}\left(w_t \mid w_{t-n+1}^{t-1}\right) = \begin{cases} P^*\left(w_t \mid w_{t-n+1}^{t-1}\right) & \text{if } C\left(w_{t-n+1}^{t}\right) > 0, \\ \alpha\left(w_{t-n+1}^{t-1}\right) P_{\text{Katz}}\left(w_t \mid w_{t-n+2}^{t-1}\right) & \text{if } C\left(w_{t-n+1}^{t}\right) = 0, \end{cases} \tag{3.46}$$

where $P^*$ denotes discounted probabilities and $\alpha(\cdot)$ is a normalization factor that ensures probabilities sum to 1.

#### Kneser-Ney smoothing and continuation probabilities

The gold standard statistical smoothing algorithm is *Modified Kneser-Ney smoothing* [32]. Kneser-Ney recognizes a critical flaw in standard backoff: lower-order unigram frequency $P(w)$ does not reflect the likelihood of a word completing an unfamiliar context.

Consider the proper noun `"Francisco"`. In a general news corpus, `"Francisco"` has a relatively high unigram count $C$(`"Francisco"`) because it occurs frequently in the bigram `"San Francisco"`. Under standard Jelinek-Mercer interpolation, if the model evaluates an unfamiliar prefix such as *"I want to eat a [ ]"*, the high unigram count of `"Francisco"` would cause the model to assign it substantial probability. However, in human language, `"Francisco"` almost never appears unless preceded by `"San"`.

Kneser-Ney resolves this by replacing raw unigram frequency with *continuation probability*, $P_{\text{cont}}(w)$, which measures how versatile a word is across diverse antecedents:

$$P_{\text{cont}}(w) = \frac{|\{w' : C(w', w) > 0\}|}{\sum_{w'' \in \mathcal{V}} |\{w' : C(w', w'') > 0\}|}. \tag{3.47}$$

Because `"Francisco"` appears almost exclusively after the single predecessor `"San"`, its continuation count is 1, resulting in a tiny continuation probability $P_{\text{cont}}$(`"Francisco"`). Conversely, words like `"apple"` or `"book"` follow hundreds of distinct words, receiving high continuation probabilities and correctly dominating lower-order backoff distributions.

### 3.3.3 Intrinsic evaluation metrics: cross-entropy and perplexity

To benchmark and compare different language models quantitatively without executing costly downstream applications, natural language processing relies on intrinsic, information-theoretic metrics: *Cross-Entropy* and *Perplexity* [27, 29].

#### Information-theoretic foundations: entropy and KL divergence

Let $\mathcal{X}$ denote a discrete sample space (the vocabulary $\mathcal{V}$), and let $P(x)$ represent the true, underlying, but inaccessible probability distribution of human language.

**Definition 3.1:** *Shannon entropy*

The *Shannon entropy* $H(P)$ measures the fundamental, irreducible uncertainty or average information content inherent in distribution $P$ [27]:

$$H(P) = -\sum_{x \in \mathcal{X}} P(x) \log_2 P(x) = \mathbb{E}_{x \sim P}\left[\log_2 \frac{1}{P(x)}\right]. \tag{3.48}$$

By Shannon's source coding theorem, $H(P)$ represents the absolute theoretical lower bound on the expected number of bits required to encode a symbol emitted by $P$ without

loss.

When constructing a language model, we approximate the true distribution $P$ with an estimated parameterized distribution $Q_\theta$. To quantify the statistical discrepancy between the true distribution $P$ and our approximating model $Q_\theta$, we employ the *Kullback-Leibler (KL) divergence*.

**Definition 3.2:** *Kullback-Leibler (KL) divergence*

The *KL divergence* (or relative entropy) from model distribution $Q$ to target distribution $P$ is:

$$D_{\mathrm{KL}}(P \parallel Q) = \sum_{x \in \mathcal{X}} P(x) \log_2 \left( \frac{P(x)}{Q(x)} \right) = \mathbb{E}_{x \sim P} \left[ \log_2 \frac{P(x)}{Q(x)} \right]. \tag{3.49}$$

**Theorem 3.1:** *Gibbs' inequality*

For any two discrete probability distributions $P$ and $Q$, the KL divergence is non-negative:

$$D_{\mathrm{KL}}(P \parallel Q) \geq 0, \tag{3.50}$$

with equality $D_{\mathrm{KL}}(P \parallel Q) = 0$ if and only if $P(x) = Q(x)$ for all $x \in \mathcal{X}$.

*Proof.* Expressing the divergence via the natural logarithm and utilizing the standard inequality $\ln(u) \leq u - 1$ for all $u > 0$ (with equality if and only if $u = 1$):

$$\begin{aligned} -D_{\mathrm{KL}}(P \parallel Q) &= \sum_{x \in \mathcal{X}} P(x) \log_2 \left( \frac{Q(x)}{P(x)} \right) = \frac{1}{\ln 2} \sum_{x \in \mathcal{X}} P(x) \ln \left( \frac{Q(x)}{P(x)} \right) \\ &\leq \frac{1}{\ln 2} \sum_{x \in \mathcal{X}} P(x) \left( \frac{Q(x)}{P(x)} - 1 \right) \\ &= \frac{1}{\ln 2} \left( \sum_{x \in \mathcal{X}} Q(x) - \sum_{x \in \mathcal{X}} P(x) \right) = \frac{1}{\ln 2} (1 - 1) = 0. \end{aligned} \tag{3.51}$$

Multiplying both sides by $-1$ reverses the inequality, yielding $D_{\mathrm{KL}}(P \parallel Q) \geq 0$.

**Cross-entropy decomposition and empirical sequence loss**

Expanding the logarithm in the definition of KL divergence demonstrates the fundamental relationship connecting Entropy, KL Divergence, and Cross-Entropy:

$$\begin{aligned} D_{\mathrm{KL}}(P \parallel Q) &= \sum_{x \in \mathcal{X}} P(x) \log_2 P(x) - \sum_{x \in \mathcal{X}} P(x) \log_2 Q(x) \\ &= -H(P) + \left( - \sum_{x \in \mathcal{X}} P(x) \log_2 Q(x) \right). \end{aligned} \tag{3.52}$$

Rearranging terms defines the *Cross-Entropy* $H(P, Q)$:

$$H(P, Q) \triangleq - \sum_{x \in \mathcal{X}} P(x) \log_2 Q(x) = H(P) + D_{\mathrm{KL}}(P \parallel Q). \tag{3.53}$$

Equation (3.53) provides the theoretical justification for cross-entropy minimization during training. Because the true language distribution $P$ is determined by human linguistic behavior, its entropy $H(P)$ is an invariant physical constant. When minimizing cross-entropy with respect to model parameters $\theta$:

$$\arg\min_{\theta} H(P, Q_\theta) = \arg\min_{\theta} (H(P) + D_{\mathrm{KL}}(P \parallel Q_\theta)) = \arg\min_{\theta} D_{\mathrm{KL}}(P \parallel Q_\theta). \tag{3.54}$$

Minimizing cross-entropy is mathematically identical to driving the KL divergence between model $Q_\theta$ and the true language distribution $P$ to zero.

Because we do not have closed-form access to the true distribution $P(x)$, the expected value is estimated empirically by sampling a long test sequence $W = (w_1, w_2, \ldots, w_T) \sim P$:

$$\mathcal{H}(W) = -\frac{1}{T} \log_2 Q(w_1, w_2, \ldots, w_T) = -\frac{1}{T} \sum_{t=1}^{T} \log_2 Q(w_t \mid w_{<t}). \tag{3.55}$$

Empirical cross-entropy measures the average number of bits required to compress each token of test text using model $Q$.

### Perplexity as an intrinsic evaluation metric

While cross-entropy evaluates performance on a logarithmic scale, researchers and practitioners report *Perplexity* (PPL), defined as the exponentiated cross-entropy loss:

$$\mathrm{PPL}(W) = 2^{\mathcal{H}(W)} = 2^{-\frac{1}{T}\sum_{t=1}^{T} \log_2 Q(w_t|w_{<t})} \tag{3.56}$$

$$= \left( \prod_{t=1}^{T} \frac{1}{Q(w_t \mid w_{<t})} \right)^{\frac{1}{T}}. \tag{3.57}$$

Perplexity represents the reciprocal geometric mean of the token probabilities assigned by the model across the test corpus.

**Theorem 3.2:** *Perplexity of a uniform $K$-ary distribution*

Let a language model $Q$ predict next tokens such that at each timestep $t$, the probability mass is distributed uniformly across exactly $K$ equally likely candidate tokens ($Q(w_t \mid w_{<t}) = \frac{1}{K}$ for each candidate). Then the perplexity on any generated sequence $W$ of length $T$ is identically $K$:

$$\mathrm{PPL}(W) = K. \tag{3.58}$$

*Proof.* Substituting the uniform prediction probability $Q(w_t \mid w_{<t}) = \frac{1}{K}$ into the empirical cross-entropy formula yields:

$$\mathcal{H}(W) = -\frac{1}{T} \sum_{t=1}^{T} \log_2 \left( \frac{1}{K} \right) = -\frac{1}{T} \sum_{t=1}^{T} (-\log_2 K)$$

$$= \frac{1}{T} (T \log_2 K) = \log_2 K. \tag{3.59}$$

Exponentiating the cross-entropy:

$$\mathrm{PPL}(W) = 2^{\mathcal{H}(W)} = 2^{\log_2 K} = K. \tag{3.60}$$

Theorem 3.2 establishes the physical interpretation of perplexity: it quantifies the *effective branching factor* of a language model. When a model evaluates a text corpus and achieves PPL = 42, this indicates that, on average, the model's predictive uncertainty at each token is mathematically equivalent to choosing uniformly at random from a shortlist of 42 equally plausible words. A uniform random guesser across vocabulary $\mathcal{V}$ achieves PPL = $|\mathcal{V}|$ (for example, PPL = 50,000), while an optimal oracle that predicts every token with probability 1.0 achieves the theoretical lower bound PPL = 1.0.

**Logarithmic base conversions: bits versus nats**

A practical mathematical distinction arises when connecting information-theoretic formulations (which employ base-2 logarithms) with deep learning frameworks such as PyTorch (which compute natural base-$e$ cross-entropy loss):

| Property | Information Theory (Bits) | Deep Learning / PyTorch (Nats) |
|---|---|---|
| Loss Formula | $\mathcal{H}_2 = -\frac{1}{T}\sum \log_2 Q(w_t \mid w_{<t})$ | $\mathcal{L}_{\text{CE}} = -\frac{1}{T}\sum \ln Q(w_t \mid w_{<t})$ |
| Unit | Bits per token | Nats per token |
| Perplexity Formula | $\text{PPL} = 2^{\mathcal{H}_2}$ | $\text{PPL} = \exp(\mathcal{L}_{\text{CE}}) = e^{\mathcal{L}_{\text{CE}}}$ |
| Conversion Identity | $\mathcal{H}_2 = \frac{\mathcal{L}_{\text{CE}}}{\ln 2} \approx \frac{\mathcal{L}_{\text{CE}}}{0.693147}$, | $\mathcal{L}_{\text{CE}} = \mathcal{H}_2 \cdot \ln 2 \approx 0.693147 \cdot \mathcal{H}_2$ |

To observe the effect of this conversion, suppose a neural model trained on code completion reports an evaluation loss of $\mathcal{L}_{\text{CE}} = 2.7726$ nats. Computing base-$e$ perplexity directly yields:

$$\text{PPL} = e^{2.7726} = \exp(2.7726) \approx 16.00. \tag{3.61}$$

Converting the loss to information bits produces:

$$\mathcal{H}_2 = \frac{2.7726}{\ln 2} = \frac{2.7726}{0.693147} = 4.00 \text{ bits/token}, \tag{3.62}$$

which confirms the identical perplexity value: $\text{PPL} = 2^{\mathcal{H}_2} = 2^{4.00} = 16.00$. Inadvertently evaluating $2^{\mathcal{L}_{\text{CE}}} = 2^{2.7726} \approx 6.83$ would severely underestimate the true branching factor by more than 57%, highlighting the necessity of matching logarithmic bases.

### 3.3.4 The structural ceiling and the bridge to continuous word embeddings

While advanced smoothing techniques such as Modified Kneser-Ney eliminate zero-frequency collapses and reduce empirical perplexity, statistical $n$-gram models remain trapped behind two insurmountable theoretical barriers:

1. **Context horizon truncation**: The $(n-1)$-th order Markov assumption discards all context residing beyond $n-1$ tokens. Increasing $n \geq 5$ yields rapidly diminishing returns because higher-order frequency tables become almost entirely empty, forcing the model to constantly back off to lower orders. Consequently, statistical $n$-gram models cannot maintain long-range syntactic agreement (e.g. subject-verb agreement across intervening subordinate clauses) or preserve document-level topic coherence.

2. **Discrete orthogonal representations and zero parameter sharing**: In statistical counting tables, words are represented as atomic, discrete symbolic indices. In vector space, this corresponds to treating words as standard basis vectors (one-hot vectors) in $\mathbb{R}^{|\mathcal{V}|}$:

$$\mathbf{e}_i^\top \mathbf{e}_j = 0, \quad \|\mathbf{e}_i - \mathbf{e}_j\|_2 = \sqrt{2}, \quad \forall i \neq j. \tag{3.63}$$

   Every word is geometrically orthogonal and equidistant to every other word. The model possesses no intrinsic notion of semantic similarity. Observing the training phrase *"The*

*cat chased the mouse"* provides strictly zero statistical evidence to assist the model when evaluating the semantically identical sentence *"The feline pursued the rodent"*. Because each lexical entry occupies an isolated, independent row in the transition matrix, the model cannot share statistical parameters between synonyms.

Smoothing techniques merely patch zero-frequency counts; they do not solve the fundamental lack of continuous semantic representation. Overcoming this representational ceiling requires replacing discrete, tabular counting models with continuous, parameterized mathematical functions [33]. Words must be projected into continuous Euclidean vector spaces $\mathbb{R}^d$ where geometric distance reflects semantic similarity. This profound conceptual leap—transitioning from discrete orthogonal indices to distributed continuous vector representations—forms the subject of Chapter 4.

CHAPTER 4

# Distributed word representations and Word2Vec

The preceding chapters introduced discrete language models, differentiable neural networks, and recurrent state updates. These models still require a representation that maps discrete tokens to numerical inputs on which the network can compute and optimize.

Tokens are discrete symbols, whereas neural networks operate on continuous numerical inputs. Assigning each word a one-hot vector makes the representation sparse; distinct words are orthogonal, and their parameters do not share updates through the input representation. Dense embeddings address this mismatch by representing tokens as vectors whose geometry can be learned from context.

Distributional learning uses context to organize token vectors, following the hypothesis associated with Harris and Firth. The chapter derives the Continuous Bag-of-Words (CBOW) and Skip-Gram objectives, then considers Hierarchical Softmax and negative sampling (SGNS) as alternatives to full-vocabulary normalization. It examines how FastText character $n$-grams represent morphology and unseen words, how vector geometry supports semantic clusters and analogies, and how subword tokenization and positional encodings are used in current language models.

## 4.1 The representation problem: why continuous embeddings are mandatory

Natural language and software engineering source code are fundamentally composed of discrete, symbolic tokens. In written English or in source files, programs and sentences present themselves as strings of alphanumeric characters: words such as `"compiler"`, `"optimizer"`, `"gradient"`, and keywords such as `"class"`, `"def"`, `"return"`.

Conversely, modern artificial neural networks are continuous, differentiable mathematical operators. An $L$-layer deep neural network $\mathcal{F}(\mathbf{x};\Theta)$ computes outputs through compositions of parameterized linear projections and non-linear activation functions:

$$\mathcal{F}(\mathbf{x};\Theta) = \sigma_L\Big(\mathbf{W}_L\,\sigma_{L-1}\big(\cdots\sigma_1(\mathbf{W}_1\mathbf{x}+\mathbf{b}_1)\cdots\big)+\mathbf{b}_L\Big), \tag{4.1}$$

where $\mathbf{W}_l \in \mathbb{R}^{d_l\times d_{l-1}}$ and $\mathbf{b}_l \in \mathbb{R}^{d_l}$. Such architectures rely fundamentally on the continuous mathematics of vector spaces $\mathbb{R}^d$, multivariable calculus, and gradient-based backpropagation. The optimization algorithm minimizes an empirical risk loss $\mathcal{L}(\Theta)$ by iteratively updating weight tensors along the negative gradient direction:

$$\Theta \leftarrow \Theta - \eta\,\nabla_\Theta\mathcal{L}(\Theta). \tag{4.2}$$

This mathematical reality creates an immediate, fundamental impasse: **discrete text strings cannot be differentiated**. The partial derivative of a scalar loss with respect to a raw string literal, $\frac{\partial\mathcal{L}}{\partial\texttt{"cat"}}$, is mathematically undefined. Discrete symbols possess no notion of infinitesimal perturbation, tangent spaces, or linear vector combinations. One cannot add `"cat"` to `"dog"`, nor compute an inner product between `"while"` and `"for"`.

To bridge this gap between discrete symbolic communication and continuous gradient-based computation, we must establish a formal mapping from the discrete vocabulary into a continuous vector space.

**Definition 4.1:** *Continuous word embedding*

Let $\mathcal{V} = \{w_1, w_2, \ldots, w_V\}$ denote a finite vocabulary of $V = |\mathcal{V}|$ distinct discrete symbolic tokens. A **continuous word embedding** is an injective mapping:

$$\phi : \mathcal{V} \longrightarrow \mathbb{R}^d, \quad \text{with } d \ll V, \tag{4.3}$$

that maps each discrete symbol $w_i \in \mathcal{V}$ to a low-dimensional, dense continuous vector $\mathbf{v}_i = \phi(w_i) \in \mathbb{R}^d$. The target vector space $(\mathbb{R}^d, \|\cdot\|, \langle\cdot,\cdot\rangle)$ is a continuous metric space equipped with an inner product, where geometric distance $D(\mathbf{v}_i, \mathbf{v}_j)$ and angular orientation $\cos(\mathbf{v}_i, \mathbf{v}_j)$ preserve semantic and syntactic relationships.

In typical natural language and software engineering applications, vocabulary sizes range from $V \approx 30{,}000$ to upwards of $V = 1{,}000{,}000$ unique tokens, while the latent embedding dimension is chosen to be compact: $d \in [100, 1024]$ (e.g., $d = 300$ in classical Word2Vec and FastText; $d = 768$ in standard Transformer baselines). By mapping discrete words into $\mathbb{R}^d$, language tokens become continuous variables that participate directly in matrix algebra, linear transformations, and gradient backpropagation.

To understand why dense continuous representations are indispensable, we must first examine the standard categorical baseline: local one-hot encoding. While intuitive and trivial to construct, one-hot vectors suffer from severe geometric and computational pathologies that fundamentally incapacitate neural learning.

## 4.2 The baseline approach: local one-hot encodings and why they fail

The most straightforward, historical baseline for converting discrete categorical symbols into numeric vectors is **one-hot encoding**. Under this local representational scheme, every word $w_i \in \mathcal{V}$ is mapped to a standard basis coordinate vector $\mathbf{e}_i$ in a $V$-dimensional Euclidean space $\mathbb{R}^V$:

$$\mathbf{e}_i = [\underbrace{0, \dots, 0}_{i-1 \text{ zeros}}, 1, \underbrace{0, \dots, 0}_{V-i \text{ zeros}}]^\top \in \{0,1\}^V, \quad \text{where } (\mathbf{e}_i)_k = \delta_{ik} = \begin{cases} 1 & \text{if } k = i, \\ 0 & \text{if } k \neq i. \end{cases} \tag{4.4}$$

For example, consider a canonical toy vocabulary of $V = 5$ tokens:

$$\mathcal{V} = \{\texttt{"dog"}, \texttt{"cat"}, \texttt{"fish"}, \texttt{"pen"}, \texttt{"pencil"}\}. \tag{4.5}$$

Under one-hot encoding, each word is assigned an isolated Cartesian coordinate axis:

$$\mathbf{e}_{\text{dog}} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{e}_{\text{cat}} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{e}_{\text{fish}} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{e}_{\text{pen}} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}, \quad \mathbf{e}_{\text{pencil}} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}. \tag{4.6}$$

### 4.2.1 Algebraic equivalence: one-hot matrix multiplication as embedding lookup

In the foundational guide *Build a Large Language Model from Scratch* [34], it is proven that feeding a one-hot vector into a dense linear layer is algebraically identical to retrieving a specific row or column from a weight lookup table.

**Theorem 4.1:** *Embedding lookup equivalence*

Let $\mathcal{V}$ be a vocabulary of size $V$, and let token $w_i \in \mathcal{V}$ be encoded as the standard basis vector $\mathbf{e}_i \in \{0,1\}^V$.

**Column-vector orientation (standard basis representation)**

Let $\mathbf{W}_{\text{in}} \in \mathbb{R}^{d \times V}$ be the embedding matrix whose $k$-th column corresponds to the $d$-dimensional embedding vector $\mathbf{w}_k = (\mathbf{W}_{\text{in}})_{:,k} \in \mathbb{R}^d$:

$$\mathbf{W}_{\text{in}} = \begin{bmatrix} | & | & & | \\ \mathbf{w}_1 & \mathbf{w}_2 & \cdots & \mathbf{w}_V \\ | & | & & | \end{bmatrix}. \tag{4.7}$$

Multiplying $\mathbf{W}_{\text{in}}$ by the one-hot column vector $\mathbf{e}_i$ extracts the $i$-th column of $\mathbf{W}_{\text{in}}$:

$$\mathbf{h} = \mathbf{W}_{\text{in}}\mathbf{e}_i = \sum_{k=1}^{V} (\mathbf{e}_i)_k (\mathbf{W}_{\text{in}})_{:,k} = 1 \cdot (\mathbf{W}_{\text{in}})_{:,i} + \sum_{k \neq i} 0 \cdot (\mathbf{W}_{\text{in}})_{:,k} = \mathbf{w}_i \in \mathbb{R}^d. \tag{4.8}$$

**Row-vector orientation (PyTorch convention)**

Let $\mathbf{W} \in \mathbb{R}^{V \times d}$ be an embedding table whose $k$-th row contains the embedding vector $\mathbf{w}_k^\top = \mathbf{W}_{k,:} \in \mathbb{R}^{1 \times d}$. Multiplying the one-hot row vector $\mathbf{e}_i^\top$ by $\mathbf{W}$ extracts the $i$-th row of $\mathbf{W}$:

$$\mathbf{e}_i^\top \mathbf{W} = \sum_{k=1}^{V} (\mathbf{e}_i)_k \mathbf{W}_{k,:} = 1 \cdot \mathbf{W}_{i,:} + \sum_{k \neq i} 0 \cdot \mathbf{W}_{k,:} = \mathbf{W}_{i,:} = \mathbf{w}_i^\top \in \mathbb{R}^{1 \times d}. \tag{4.9}$$

Hence, matrix multiplication of an embedding weight matrix by a one-hot indicator vector is identical to an indexed memory table lookup:

$$\mathbf{W}_{\text{in}}\mathbf{e}_i \equiv \texttt{lookup}(\mathbf{W}_{\text{in}}, i). \tag{4.10}$$

### 4.2.2 Systems-level justification: why frameworks use dedicated embedding lookups

Although dense matrix multiplication $\mathbf{e}_i^\top \mathbf{W}$ and table indexing $\texttt{lookup}(\mathbf{W}, i)$ are algebraically equivalent, no modern deep learning framework (PyTorch, TensorFlow, JAX) ever implements word embeddings as a dense matrix multiplication. Deep learning frameworks provide specialized lookup modules, such as `torch.nn.Embedding(num_embeddings=V, embedding_dim=d)`, for three decisive systems engineering reasons:

1. **Elimination of $O(V \cdot d)$ Wasted FLOPs**: Evaluating the matrix multiplication $\mathbf{e}_i^\top \mathbf{W}$ requires executing $V \cdot d$ floating-point multiplications and $(V-1) \cdot d$ additions. In a standard language model with vocabulary $V = 50{,}257$ (GPT-2 tokenizer) and hidden dimension $d = 768$, computing the input projection for a single token via dense matrix multiplication requires:
$$50{,}257 \times 768 \approx 3.86 \times 10^7 \text{ arithmetic operations per token.} \tag{4.11}$$
Of these 38.6 million operations, exactly 99.998% evaluate $0 \times W_{k,j} = 0$. By contrast, `torch.nn.Embedding` accepts integer token IDs $i \in \{0, \ldots, V-1\}$ and performs direct $O(1)$ pointer arithmetic into physical GPU/CPU memory:
$$\text{address}(\mathbf{w}_i) = \text{base_pointer} + i \times d \times \texttt{sizeof}(\text{float32}), \tag{4.12}$$
retrieving the $d$-dimensional embedding slice in $O(1)$ time with exactly zero arithmetic FLOPs.

2. **Massive VRAM Savings (25,000× Memory Reduction)**: Consider a standard training batch with batch size $B = 64$ and sequence context length $T = 1{,}024$. If input sequences were materialized as dense one-hot tensors, the input tensor $\mathbf{X}_{\text{onehot}}$ would have shape $(B, T, V) = (64, 1{,}024, 50{,}257)$. Storing this tensor in single-precision floating point (`float32`, 4 bytes per element) would require:
$$\text{VRAM}_{\text{one-hot}} = 64 \times 1{,}024 \times 50{,}257 \times 4 \text{ bytes} \approx 13{,}175{,}095{,}296 \text{ bytes} \approx 13.18\,\text{GB of VRAM,} \tag{4.13}$$
simply to hold the input batch before computing a single activation. Conversely, storing sequences as integer token IDs produces an `int64` tensor of shape $(B, T) = (64, 1{,}024)$, requiring:
$$\text{VRAM}_{\text{indices}} = 64 \times 1{,}024 \times 8 \text{ bytes} = 524{,}288 \text{ bytes} \approx 524\,\text{KB of VRAM.} \tag{4.14}$$
This constitutes a memory bandwidth reduction factor of $\frac{13.18\,\text{GB}}{524\,\text{KB}} \approx \mathbf{25{,}128}\times$, entirely preventing Out-Of-Memory (OOM) GPU crashes.

3. **Sparse Gradient Backpropagation**: Under dense matrix multiplication $\mathbf{y} = \mathbf{e}_i^\top \mathbf{W}$, the backpropagation adjoint with respect to the weight matrix is the outer product:
$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \mathbf{e}_i \left( \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \right) \in \mathbb{R}^{V \times d}. \tag{4.15}$$
This gradient matrix is massive ($V \times d$) and strictly sparse: only row $i$ contains non-zero values, while the remaining $V-1$ rows are populated entirely by zeros. Synchronizing such dense gradient tensors across multi-GPU distributed clusters saturates PCIe and NVLink bandwidth. `torch.nn.Embedding` executes sparse indexed gradient accumulation (`index_add_`), routing the upstream vector $\frac{\partial \mathcal{L}}{\partial \mathbf{y}} \in \mathbb{R}^d$ directly into the $i$-th row of $\mathbf{W}$.

### 4.2.3 The 5 structural failure modes of one-hot representations

The lookup-table implementation makes input projection computationally efficient, but one-hot vectors do not encode semantic relationships between words. They have five limitations:

#### Extreme sparsity and curse of dimensionality

As vocabulary size scales to web scale ($V \geq 50{,}000$ to $10^6$), one-hot vectors become very large sparse vectors where 99.998% of entries are zero. The dimensionality of the vector space equals the number of distinct words. Under this curse of dimensionality, the volume of the enclosing space grows exponentially, leaving data points isolated at the extreme vertices of independent axes.

#### Metric collapse and semantic orthogonality

Because one-hot vectors form the standard Cartesian basis of $\mathbb{R}^V$, every pair of distinct words $w_i \neq w_j$ is mutually perpendicular and equidistant:

$$\cos(\mathbf{e}_i, \mathbf{e}_j) = \frac{\mathbf{e}_i^\top \mathbf{e}_j}{\|\mathbf{e}_i\|_2 \|\mathbf{e}_j\|_2} = \frac{0}{1 \cdot 1} = 0, \quad \forall i \neq j, \tag{4.16}$$

$$d_E(\mathbf{e}_i, \mathbf{e}_j) = \|\mathbf{e}_i - \mathbf{e}_j\|_2 = \sqrt{(1)^2 + (-1)^2 + \sum_{k \neq i,j} 0^2} = \sqrt{2} \approx 1.414, \quad \forall i \neq j. \tag{4.17}$$

Under one-hot encoding, every word is orthogonal and equidistant to every other word:

$$d_E(\texttt{"pen"}, \texttt{"pencil"}) = d_E(\texttt{"pen"}, \texttt{"crocodile"}) = \sqrt{2}. \tag{4.18}$$

The underlying metric space collapses into a degenerate topology where open balls $B_r(\mathbf{e}_i) = \{\mathbf{x} \in \mathcal{V} : d_E(\mathbf{e}_i, \mathbf{x}) < r\}$ contain only $\{\mathbf{e}_i\}$ for $r \leq \sqrt{2}$, and immediately engulf the entire universe of vocabulary tokens for $r > \sqrt{2}$. There is zero geometric capacity to reflect semantic proximity, synonymy, or grammatical category.

#### Zero gradient parameter sharing during backpropagation

Because one-hot vectors activate strictly disjoint coordinates, training a neural network on an occurrence of token $w_i$ produces gradients exclusively for the parameters associated with index $i$:

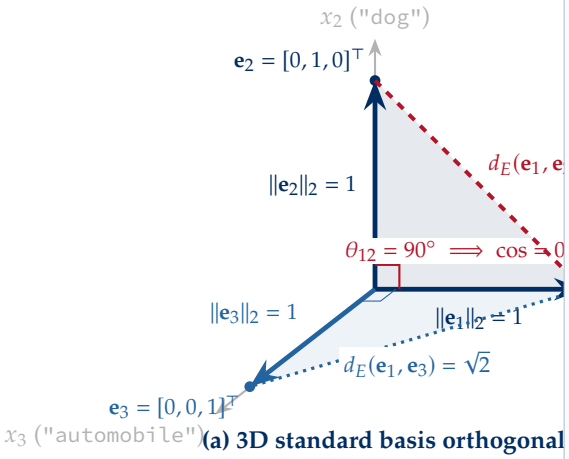
$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{k,:}} = \mathbf{0}, \quad \forall k \neq i. \tag{4.19}$$

If a model updates its weights upon observing the sentence *"The feline climbed the tree"*, the parameters associated with `"feline"` receive gradient updates, but the parameters for `"cat"`, `"kitten"`, or `"leopard"` remain completely unchanged. The model cannot transfer statistical strength across semantically synonymous or related words; it must encounter every word hundreds of times independently in every grammatical context.

#### Simplex confinement and absence of latent manifolds

One-hot vectors are confined strictly to the $V$ extreme vertices of the standard $(V-1)$-dimensional probability simplex:

$$\Delta^{V-1} = \left\{ \mathbf{x} \in \mathbb{R}^V : \sum_{i=1}^{V} x_i = 1, \; x_i \geq 0 \right\}. \tag{4.20}$$




**(a) 3D standard basis orthogonal**

**Pairwise metric equality matrices:**

• **Cosine similarity matrix** (Gramian $\mathbf{E}^\top\mathbf{E} = \mathbf{I}_V$):

$$\mathbf{S}_{\cos} = \begin{pmatrix} & \text{cat} & \text{dog} & \text{car} \\ \text{cat} & 1 & \mathbf{0} & \mathbf{0} \\ \text{dog} & \mathbf{0} & 1 & \mathbf{0} \\ \text{car} & \mathbf{0} & \mathbf{0} & 1 \end{pmatrix}$$

• **Euclidean distance matrix** ($\mathbf{D}_{ij} = \sqrt{2}(1 - \delta_{ij})$):

$$\mathbf{D}_{L_2} = \begin{pmatrix} & \text{cat} & \text{dog} & \text{car} \\ \text{cat} & 0 & \mathbf{\sqrt{2}} & \mathbf{\sqrt{2}} \\ \text{dog} & \mathbf{\sqrt{2}} & 0 & \mathbf{\sqrt{2}} \\ \text{car} & \mathbf{\sqrt{2}} & \mathbf{\sqrt{2}} & 0 \end{pmatrix}$$

**The geometric pathology:**
dist("cat", "dog") = dist("cat", "car") = $\sqrt{2}$
cos("cat", "dog") = cos("cat", "car") = 0

**(b) Semantic metric collapse**

**All tokens are equidistant and mutually perpendicular, completely destroying semantic metric topology.**

Figure 4.1: Visual proof of pairwise orthogonality and equidistance in one-hot vector representations. (a) In $V$-dimensional Euclidean space, each word is an orthogonal standard basis vector of unit norm ($\|\mathbf{e}_i\|_2 = 1$). By the Pythagorean theorem, every distinct pair forms a right isosceles triangle with origin $\mathbf{0}$, yielding an identical Euclidean distance of $\sqrt{1^2 + 1^2} = \sqrt{2}$ and an inner product of 0 ($\theta = 90°$). (b) The resulting Gramian matrix is the identity $\mathbf{I}_V$, treating related concepts like `"cat"` and `"dog"` with the exact same geometric distance as totally unrelated concepts like `"cat"` and `"automobile"`.

The interior and edges of the simplex correspond to fractional probabilistic mixtures, but distinct words themselves inhabit isolated, non-overlapping boundary corners. Words cannot reside on smooth, lower-dimensional semantic submanifolds, nor can they support linear translation arithmetic ($\mathbf{w}_{\text{king}} - \mathbf{w}_{\text{man}} + \mathbf{w}_{\text{woman}} \approx \mathbf{w}_{\text{queen}}$).

**Software engineering identifier polymorphism**

In software engineering source code, developers routinely express identical semantic intents through differing lexical notations, identifier casings, and API conventions:

- **Casing variants**: `getUserById` vs. `get_user_by_id` vs. `GetUserById`.
- **API synonyms**: `push()` vs. `append()` vs. `add()` vs. `enqueue()`.
- **Syntactic equivalents**: Type references such as `ArrayList` and `LinkedList` vs. unrelated punctuation symbols like `";"`.

Under one-hot encoding, each casing variation and API synonym is assigned a totally independent, mutually orthogonal basis vector. To a one-hot model:

$$\cos(\mathbf{e}_{\texttt{get_user_by_id}}, \mathbf{e}_{\texttt{getUserById}}) = 0, \quad d_E(\mathbf{e}_{\texttt{ArrayList}}, \mathbf{e}_{\texttt{LinkedList}}) = d_E(\mathbf{e}_{\texttt{ArrayList}}, \mathbf{e}_{\texttt{";"}}) = \sqrt{2}. \tag{4.21}$$

This lexical fragmentation completely prevents code-intelligence models from transferring refactoring patterns, bug-fix templates, or type semantics across codebases.

Because local one-hot encodings confine words to isolated, orthogonal coordinate axes where semantic similarity is geometrically undefinable, we require a fundamentally different representational paradigm. This paradigm shift rests on distributing linguistic meaning across continuous latent dimensions—guided by the foundational Distributional Hypothesis.

## 4.3 Distributed representations and the distributional hypothesis

To overcome the orthogonality and isolation of local representations, modern deep learning adopts **distributed representations**, a concept formalized in connectionist neural networks in 1986 [7].

**Definition 4.2:** *Local vs. distributed representations*

- **Local Representation**: Each concept or symbol is represented by a single dedicated computational unit (e.g., one coordinate axis in $\mathbb{R}^V$). If that unit is inactive, the concept is entirely absent. There is zero overlap between distinct symbols.
- **Distributed Representation**: Each concept is represented by a continuous pattern of activation across all $d$ dimensions of a shared vector space $\mathbb{R}^d$ ($d \ll V$). Conversely, each individual dimension participates in representing many distinct concepts.

In a distributed representation, meaning is not stored in an isolated slot; rather, it emerges from the continuous coordinates along latent dimensions. For example, latent dimensions might implicitly reflect attributes such as animacy, size, sentiment, syntactic part of speech, or software typing constructs.

### 4.3.1 The distributional hypothesis: context as meaning

How can an optimization algorithm discover meaningful continuous coordinates $\mathbf{v}_w \in \mathbb{R}^d$ for every word purely from raw text, without human supervision? The answer lies in the **Distributional Hypothesis**, formulated in structural linguistics in 1954 [35] and popularized in 1957 [36]:

> *"You shall know a word by the company it keeps."*
> — Distributional Hypothesis [36]

The distributional hypothesis posits that words that appear in similar linguistic contexts tend to share similar semantic and pragmatic meanings. Consider two sentences:

Sentence 1: "I used a **pencil** to write the essay."
Sentence 2: "You used my **pen** to write a letter."

Even if a machine has never encountered a dictionary definition of `"pencil"` or `"pen"`, it observes that both tokens co-occur with identical context words: `"used"`, `"write"`, `"essay"`, `"letter"`. Conversely, an unrelated word such as `"crocodile"` or `"refrigerator"` almost never occurs in the context of `"used a [...] to write"`.

Therefore, statistical co-occurrence within local contextual windows serves as an empirical, computable proxy for semantic similarity. If a learning algorithm is tasked with predicting surrounding context words from a target word (or vice versa), the learned vector coordinates must position words with similar co-occurrence profiles close together in vector space.

While the Distributional Hypothesis asserts that context defines meaning, it does not specify an optimization algorithm. To translate linguistic co-occurrence into optimized vector coordinates, we must frame a self-supervised prediction task and establish an end-to-end differentiable training loop.

## 4.4 The embedding training loop and semantic geometry evolution

To convert the distributional hypothesis into concrete numerical representations, we formulate a **self-supervised prediction cloze task** from unannotated text corpora.

### 4.4.1 The 7-step self-supervised optimization cycle

Given a raw, unannotated corpus of $T$ running words $C = (w_1, w_2, \ldots, w_T)$, we shift a sliding context window of half-width $C$ across the text. At each step $t$, the model solves a fill-in-the-blank prediction task: estimating the probability distribution over the vocabulary for a missing token given its surrounding context:

$$\mathbb{P}(w_t = w \mid w_{t-C}, \ldots, w_{t-1}, w_{t+1}, \ldots, w_{t+C}). \tag{4.22}$$

This self-supervised prediction task is executed through an iterative **7-step optimization cycle**:

1. **Initialize Parameter Matrices**: Allocate two weight matrices with small random values: the input embedding matrix $\mathbf{W}_{\text{in}} \in \mathbb{R}^{d \times V}$ and the output embedding matrix $\mathbf{W}_{\text{out}} \in \mathbb{R}^{d \times V}$.

2. **Context Lookup and Aggregation**: For the active context window, retrieve the input embedding vectors $\mathbf{v}_c = (\mathbf{W}_{\text{in}})_{:,c}$ and aggregate them into a single hidden context vector $\mathbf{h} \in \mathbb{R}^d$ (e.g., via vector summation or averaging).

3. **Output Candidate Projection**: Compute the dot product between the hidden context state $\mathbf{h}$ and every candidate output word vector $\mathbf{u}_w = (\mathbf{W}_{\text{out}})_{:,w}$ in the vocabulary:

$$z_w = \mathbf{u}_w^\top \mathbf{h}, \quad \text{yielding logit vector } \mathbf{z} = \mathbf{W}_{\text{out}}^\top \mathbf{h} \in \mathbb{R}^V. \tag{4.23}$$

4. **Probability Normalization**: Pass the similarity logits through the softmax activation function to produce a valid probability distribution $\hat{\mathbf{y}} \in \Delta^{V-1}$:

$$\hat{y}_w = \mathbb{P}(w \mid \text{context}) = \frac{\exp(z_w)}{\sum_{k=1}^{V} \exp(z_k)} = \frac{\exp(\mathbf{u}_w^\top \mathbf{h})}{\sum_{k=1}^{V} \exp(\mathbf{u}_k^\top \mathbf{h})}. \tag{4.24}$$

5. **Loss Evaluation**: Evaluate the prediction error against the true ground-truth target token $w^* = w_t$ using Categorical Cross-Entropy:

$$\mathcal{L}_{\text{CE}} = -\log \hat{y}_{w^*} = -\log \left( \frac{\exp(\mathbf{u}_{w^*}^\top \mathbf{h})}{\sum_{k=1}^{V} \exp(\mathbf{u}_k^\top \mathbf{h})} \right). \tag{4.25}$$

6. **Backpropagation Error Adjoints**: Apply reverse-mode automatic differentiation to compute error adjoints. The gradient with respect to logit vector $\mathbf{z}$ is the prediction residual $\boldsymbol{\delta} = \hat{\mathbf{y}} - \mathbf{y} \in \mathbb{R}^V$, where $\mathbf{y} = \mathbf{e}_{w^*}$. Propagating this residual backward yields parameter gradients:

$$\nabla_{\mathbf{W}_{\text{out}}} \mathcal{L} = \mathbf{h}\, \boldsymbol{\delta}^\top \in \mathbb{R}^{d \times V}, \quad \nabla_{\mathbf{h}} \mathcal{L} = \mathbf{W}_{\text{out}} \boldsymbol{\delta} \in \mathbb{R}^d, \quad \nabla_{\mathbf{W}_{\text{in}}} \mathcal{L} = (\nabla_{\mathbf{h}} \mathcal{L})\, \mathbf{e}_{\text{context}}^\top \in \mathbb{R}^{d \times V}. \tag{4.26}$$

7. **Stochastic Gradient Descent Parameter Update**: Adjust the coordinates in both embedding matrices by stepping along the negative gradient direction:

$$\mathbf{W}_{\text{out}} \leftarrow \mathbf{W}_{\text{out}} - \eta \nabla_{\mathbf{W}_{\text{out}}} \mathcal{L}, \qquad \mathbf{W}_{\text{in}} \leftarrow \mathbf{W}_{\text{in}} - \eta \nabla_{\mathbf{W}_{\text{in}}} \mathcal{L}. \tag{4.27}$$

Figure 4.2 illustrates this complete learning mechanism, juxtaposing the forward/backward computational dataflow in Panel (a) with the resulting geometric evolution of the embedding space across training epochs in Panel (b).

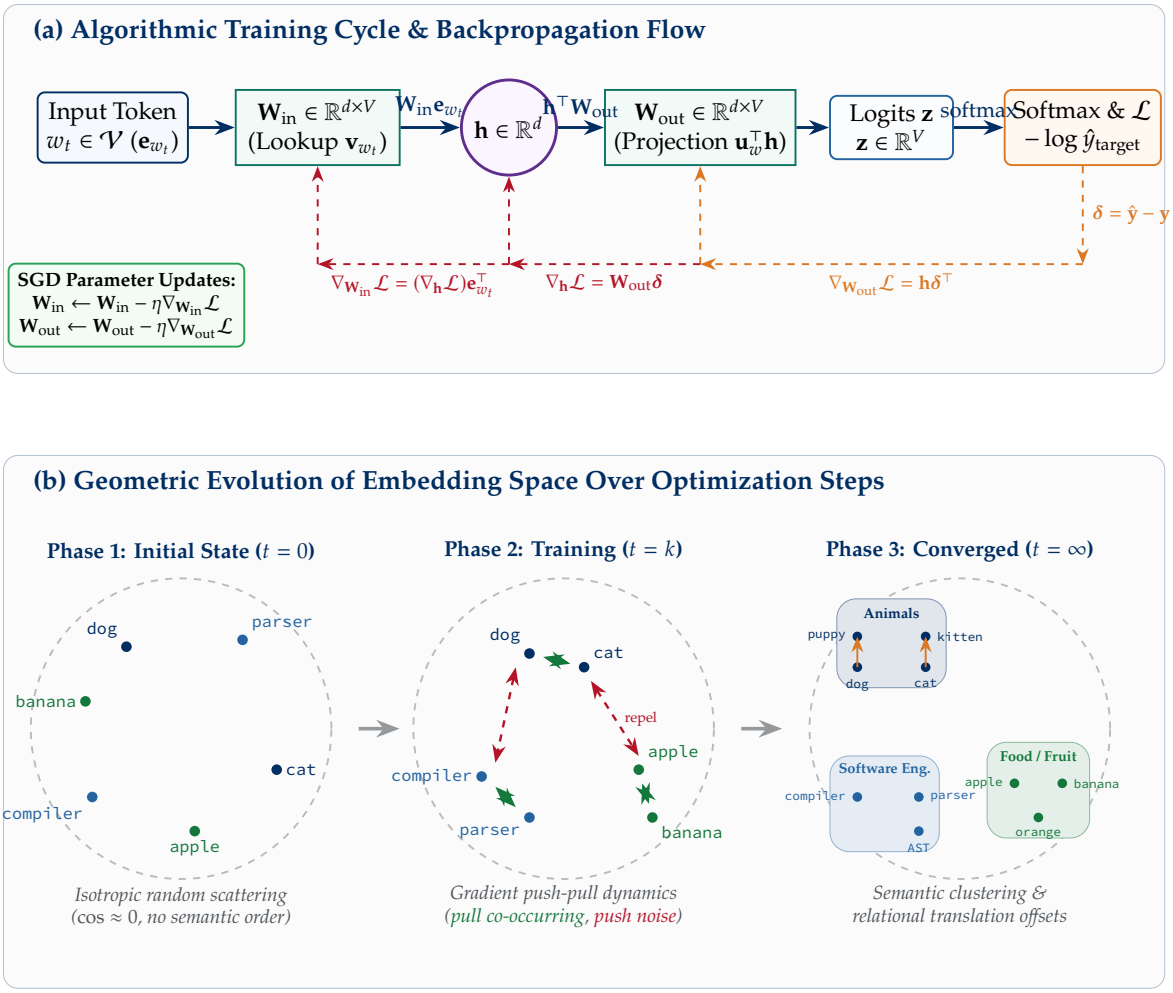



Figure 4.2: The embedding training loop and semantic space evolution. (a) Computational dataflow of the forward prediction pass and backward adjoint parameter updates. (b) Three-phase geometric evolution of embedding space from isotropic random scattering to clustered semantic topology with parallel relational offsets.

### 4.4.2 Geometric phase transition of the embedding space

As shown in Panel (b) of Figure 4.2, the continuous embedding space undergoes a profound topological self-organization over the course of training:

- **Phase 1: Initial State ($t = 0$).** Vectors are initialized using small Gaussian noise, $\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I}_d)$ with $\sigma \approx 0.01$ or $\sigma \approx 1/\sqrt{d}$. In high dimensions ($d \geq 100$), randomly sampled vectors are almost exactly orthogonal ($\cos \approx 0$). Tokens from disparate semantic domains—`"dog"`, `"compiler"`, `"apple"`—are scattered isotropically across the unit sphere with zero structural order.
- **Phase 2: Training Dynamics ($t = k$).** Whenever the model processes an observed sentence (e.g., *"The compiler parsed the syntax tree"*), backpropagation produces an attractive gradient force pulling the vectors of co-occurring words closer together. Concurrently, negative gradients or normalizer updates exert repulsive forces, pushing non-co-occurring words apart.
- **Phase 3: Converged Semantic Topology ($t = \infty$).** Upon convergence across millions of sentences, words coalesce into dense semantic neighborhoods (e.g., domestic animals, compiler construction terms, fruits). Remarkably, linear vector differences between related concepts become parallel across distinct pairs, giving rise to relational vector arithmetic (e.g., $\mathbf{v}_{\text{dog}} - \mathbf{v}_{\text{puppy}} \approx \mathbf{v}_{\text{cat}} - \mathbf{v}_{\text{kitten}}$).

### 4.4.3 Initialization dynamics and the hockey-stick loss curve

In educational analyses of neural language models and lookup tables [37], it is demonstrated that the scale of initial weights in $\mathbf{W}_{\text{in}}$ and $\mathbf{W}_{\text{out}}$ dictates optimization stability. At initialization, before the network has learned any linguistic patterns, the model should assign an approximately uniform probability to every word in the vocabulary:

$$\hat{y}_w \approx \frac{1}{V}, \quad \forall w \in \mathcal{V}. \tag{4.28}$$

Consequently, the theoretical expected initial cross-entropy loss is strictly:

$$\mathcal{L}_{\text{init}} = -\log\left(\frac{1}{V}\right) = \log V. \tag{4.29}$$

For a vocabulary of $V = 50{,}000$, the initial loss must be $\mathcal{L}_{\text{init}} = \ln(50{,}000) \approx 10.82$. If $\mathbf{W}_{\text{in}}$ or $\mathbf{W}_{\text{out}}$ are initialized with excessively large variance, logits $z_w = \mathbf{u}_w^\top \mathbf{h}$ take extreme values; the softmax distribution becomes peaky and confident on arbitrary words, causing the initial loss to spike drastically ($\mathcal{L} \gg 20$). The optimizer must then spend thousands of iterations flattening these unearned confidences—producing a pathological "hockey-stick" loss curve. Proper variance scaling ($\mathcal{N}(0, 0.01^2)$ or $\mathcal{N}(0, 1/d)$) ensures stable, well-conditioned gradient flow from step zero.

With the general self-supervised training cycle established, we must concretize the network architecture. The first foundational architecture is the Continuous Bag-of-Words (CBOW) model, which aggregates surrounding context words to predict an omitted target center word.

## 4.5 Word2Vec continuous bag-of-words (CBOW)

In 2013, **Word2Vec** was introduced by Mikolov et al. [38, 39] as a suite of log-linear models designed to scale distributed representation learning to web-scale corpora containing billions of tokens.

The first of these architectures is the **Continuous Bag-of-Words (CBOW)** model. In CBOW, the objective is to predict the omitted center target token $w_t$ given the symmetric surrounding context window:

$$C_t = \{w_{t-C}, \dots, w_{t-1}, w_{t+1}, \dots, w_{t+C}\} \setminus \{w_t\}. \tag{4.30}$$

### 4.5.1 Matrix formulation and tensor plumbing

The network consists of two weight matrices:

- $\mathbf{W}_{\text{in}} \in \mathbb{R}^{d \times V}$: The input embedding matrix, where each column $(\mathbf{W}_{\text{in}})_{:,k} = \mathbf{v}_k \in \mathbb{R}^d$ represents the vector of token $k$ when acting as context.
- $\mathbf{W}_{\text{out}} \in \mathbb{R}^{d \times V}$: The output embedding matrix, where each column $(\mathbf{W}_{\text{out}})_{:,k} = \mathbf{u}_k \in \mathbb{R}^d$ represents the vector of token $k$ when acting as the target.

Let $\mathbf{e} \in \{0,1\}^V$ denote the multi-hot context vector indicating the presence of words in $C_t$:

$$e_k = \sum_{j \in \{-C,\dots,C\} \setminus \{0\}} \mathbf{1}_{\{w_{t+j} = w_k\}}. \tag{4.31}$$

The hidden state $\mathbf{h} \in \mathbb{R}^d$ is obtained by projecting the context vector through $\mathbf{W}_{\text{in}}$ and averaging over the $2C$ context positions:

$$\mathbf{h} = \frac{1}{2C} \mathbf{W}_{\text{in}} \mathbf{e} = \frac{1}{2C} \sum_{\substack{-C \le j \le C \\ j \neq 0}} \mathbf{v}_{w_{t+j}} \in \mathbb{R}^d. \tag{4.32}$$

Next, the model scores candidate target words by projecting $\mathbf{h}$ against the columns of $\mathbf{W}_{\text{out}}$:

$$\hat{\mathbf{p}} = \mathbf{h}^\top \mathbf{W}_{\text{out}} \in \mathbb{R}^{1 \times V}, \quad \text{where } \hat{p}_k = \mathbf{u}_k^\top \mathbf{h}. \tag{4.33}$$

As emphasized in geometric visual analyses [40], the inner product $\mathbf{u}_k^\top \mathbf{h} = \|\mathbf{u}_k\|_2 \|\mathbf{h}\|_2 \cos\theta$ acts as a directional alignment sensor: it evaluates how closely the target word's representation points in the same direction as the accumulated context vector.

The raw similarity logits $\hat{\mathbf{p}}$ are normalized via the softmax function to form the conditional probability distribution:

$$\mathbb{P}(w_t = w \mid C_t) = \frac{\exp(\mathbf{u}_w^\top \mathbf{h})}{\sum_{k=1}^{V} \exp(\mathbf{u}_k^\top \mathbf{h})}. \tag{4.34}$$

The model is trained by minimizing the negative log-likelihood of the ground-truth target word $w^* = w_t$:

$$\mathcal{L}_{\text{CBOW}} = -\log \mathbb{P}(w_t = w^* \mid C_t) = -\mathbf{u}_{w^*}^\top \mathbf{h} + \log \sum_{k=1}^{V} \exp(\mathbf{u}_k^\top \mathbf{h}). \tag{4.35}$$

Figure 4.3 illustrates this computational flow.

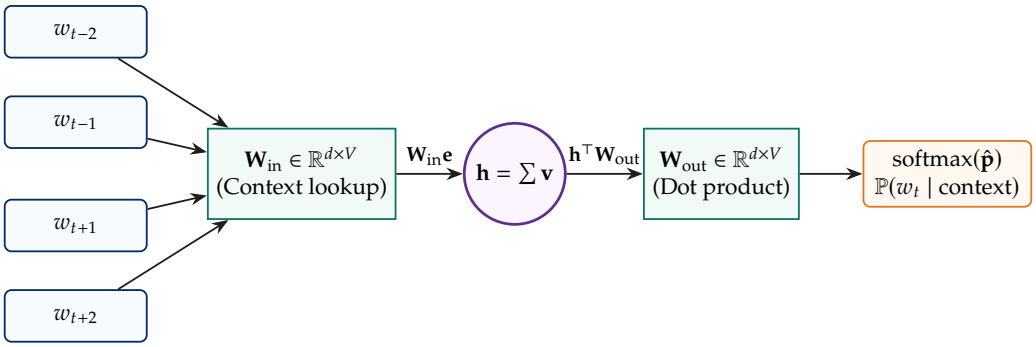



Figure 4.3: Continuous Bag-of-Words (CBOW) architecture. Context words $w_{t-c}, \ldots, w_{t+c}$ are looked up in $\mathbf{W}_{\text{in}}$ and summed to form context state $\mathbf{h} \in \mathbb{R}^d$. Inner products with output matrix $\mathbf{W}_{\text{out}}$ yield similarity logits normalized via softmax to predict the center target token $w_t$.

### 4.5.2 Comparison with earlier neural language models: the log-linear breakthrough

A neural language model learning distributed embeddings had been introduced a decade prior in 2003 [33]. However, that architecture had two major differences:

1. **Strict Causal Prediction**: In [33], language was modeled autoregressively, predicting $w_t$ strictly from past tokens $w_{t-n+1}, \ldots, w_{t-1}$. In Word2Vec, the window was opened symmetrically to both left and right context tokens $(w_{t-C}, \ldots, w_{t+C})$, doubling the statistical signal.

2. **Elimination of the Non-Linear Hidden Layer**: In [33], concatenated context embeddings were passed through a non-linear dense hidden layer $\tanh(\mathbf{W}_h \mathbf{x} + \mathbf{b})$. The matrix multiplication with $\mathbf{W}_h$ dominated computation time ($O(C \cdot d \cdot d_h)$). In Word2Vec, the non-linear activation was stripped away entirely, using a direct linear projection (vector averaging). This made Word2Vec a log-linear model, reducing compute costs by orders of magnitude and enabling training on billions of words in a few hours.

### 4.5.3 Strengths and representational bottlenecks of CBOW

**Strengths of CBOW**

- **Rapid Training Convergence**: Context averaging smooths the gradient signal, allowing CBOW to converge in fewer epochs over large corpora.

- **Robust Representations for Frequent Words**: For high-frequency tokens with abundant contextual observations, the smoothed averaging stabilizes coordinate estimates against noisy outliers.

**Representational bottlenecks of CBOW**

- **Loss of Sequential Word Order (Syntactic Blindness)**: Vector summation in Equation (4.32) is strictly commutative: $\mathbf{v}_a + \mathbf{v}_b = \mathbf{v}_b + \mathbf{v}_a$. The model is incapable of distinguishing *"the dog bit the man"* from *"the man bit the dog"*.

- **Rare Word Dilution**: Because context words are averaged into a single vector $\mathbf{h}$, if a rare technical token appears alongside multiple high-frequency stopwords (e.g., *"the **polymorphism** in the"*), the rare token's embedding is diluted by the dominant stopwords: $\frac{1}{2C}\mathbf{v}_{\text{rare}} \approx \mathbf{0}$. The gradient update to the rare word is severely attenuated.

Although CBOW trains rapidly by averaging context representations, this very averaging creates a representational bottleneck: it washes out rare vocabulary tokens and discards syntactic word order. To learn high-fidelity representations for infrequent words, inverting the predictive formulation yields the Continuous Skip-Gram architecture.

## 4.6 Word2Vec continuous skip-gram and the softmax bottleneck

To preserve the distinct gradient signal of rare words, the **Continuous Skip-Gram** architecture was introduced in [38]. Instead of aggregating context words to predict a single center token, Skip-Gram **inverts the predictive direction**: given a single center target word $w_t$, the model independently predicts each surrounding context word within window radius $C$.

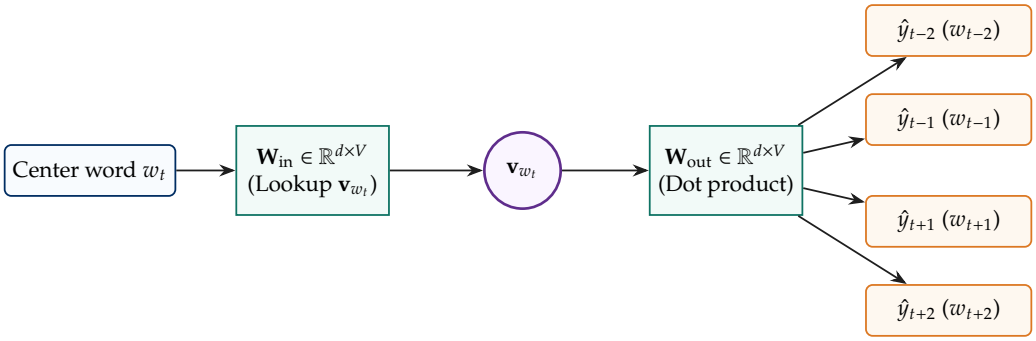



Figure 4.4: Continuous Skip-Gram architecture. A single center target word $w_t$ is projected via $\mathbf{W}_{\text{in}}$ into vector $\mathbf{v}_{w_t} \in \mathbb{R}^d$, which predicts the probability distribution of context words across window radius $c$ through output matrix $\mathbf{W}_{\text{out}}$.

**Definition 4.3:** *Skip-gram objective function*

Given a training sequence of $T$ words $w_1, w_2, \ldots, w_T$ and a context window of half-width $C$, the Skip-gram model maximizes the average log-likelihood:

$$\mathcal{L}_{\text{SG}} = \frac{1}{T} \sum_{t=1}^{T} \sum_{\substack{-C \le j \le C \\ j \neq 0}} \log \mathbb{P}(w_{t+j} \mid w_t). \tag{4.36}$$

Under standard multinomial softmax, the conditional probability of context word $w_O$ given center word $w_I$ is:

$$\mathbb{P}(w_O \mid w_I) = \frac{\exp\left(\mathbf{u}_{w_O}^\top \mathbf{v}_{w_I}\right)}{\sum_{w=1}^{V} \exp\left(\mathbf{u}_w^\top \mathbf{v}_{w_I}\right)}, \tag{4.37}$$

where $\mathbf{v}_w \in \mathbb{R}^d$ denotes the column of $\mathbf{W}_{\text{in}}$ (input embedding) and $\mathbf{u}_w \in \mathbb{R}^d$ denotes the column of $\mathbf{W}_{\text{out}}$ (output context embedding).

### 4.6.1 Why Skip-gram captures rare words

In Skip-Gram, context words are never averaged together. Instead, every context-target co-occurrence $(w_I, w_O)$ generates an independent, dedicated pairwise training instance. When an infrequent, domain-specific term (such as `"refactoring"` or `"polymorphism"`) appears in text, its vector $\mathbf{v}_{w_I}$ directly predicts each context word:

$$\nabla_{\mathbf{v}_{w_I}} \mathcal{L} = \sum_{w_O \in C} \left( \mathbf{u}_{w_O} - \sum_{k=1}^{V} \mathbb{P}(w_k \mid w_I) \mathbf{u}_k \right). \tag{4.38}$$

The gradient is never diluted by surrounding words. Consequently, Skip-Gram produces significantly higher-quality embeddings for rare words, specialized technical jargon, and

nuanced semantic distinctions than CBOW.

### 4.6.2 The shared computational wall: the full softmax bottleneck

Despite its empirical superiority, the standard formulation of Skip-Gram crashes into an prohibitive computational wall: **the full Softmax partition function bottleneck**.

Examining the denominator of Equation (4.37):

$$Z(w_I) = \sum_{w=1}^{V} \exp\left(\mathbf{u}_w^\top \mathbf{v}_{w_I}\right). \tag{4.39}$$

Computing this normalizing denominator requires evaluating an inner product $\mathbf{u}_w^\top \mathbf{v}_{w_I}$ and an exponential $\exp(\cdot)$ for **every single token $w$ in the entire vocabulary $\mathcal{V}$**.

- **Computational Complexity per Token**: Computing $Z(w_I)$ takes $O(V \cdot d)$ floating-point operations.
- **Total Corpus Cost**: For a window of radius $C = 5$, predicting $2C = 10$ context words per token over a corpus of $T = 10^9$ words with vocabulary $V = 10^5$ and dimension $d = 300$ requires:

$$10^9 \text{ tokens} \times 10 \text{ predictions} \times 10^5 \text{ words} \times 300 \text{ dims} = 3 \times 10^{17} \text{ FLOPs}. \tag{4.40}$$

- **Gradient Synchronization Bottleneck**: In backpropagation, the derivative $\frac{\partial \mathcal{L}}{\partial \mathbf{u}_w}$ touches every column $w \in \{1, \ldots, V\}$ of $\mathbf{W}_{\text{out}}$ on every single training step, forcing full-matrix memory writes.

This $O(V \cdot d)$ complexity renders standard multinomial softmax completely intractable for web-scale corpora.

While Continuous Skip-Gram excels at capturing nuanced semantic relationships and rare tokens, its standard formulation crashes into a severe computational barrier: normalizing over the full vocabulary requires an $O(V \cdot d)$ denominator calculation for every single token prediction. To scale to realistic vocabularies, we must circumvent this partition function bottleneck.

## 4.7 Scaling softmax: hierarchical softmax and negative sampling

To bypass the $O(V \cdot d)$ partition function barrier, two distinct scaling methods were formulated in [39]: **Hierarchical Softmax** (a structured tree-based approximation) and **Skip-Gram with Negative Sampling (SGNS)** (a sample-based binary discrimination formulation).

### 4.7.1 Hierarchical softmax ($O(\log_2 V)$)

First developed in 2005 [41] and adapted in [39], **Hierarchical Softmax** decomposes the flat multiclass probability distribution into a hierarchical sequence of binary routing decisions over a binary tree.

- **Binary Tree Topology**: The $V$ vocabulary words form the leaves of a binary tree (constructed as a binary Huffman tree where frequent words have shorter code lengths).
- **Path Specification**: For any word $w \in \mathcal{V}$, let $L(w)$ denote the depth of the leaf from the root. There exists a unique path of nodes from the root node $n(w, 1) =$ root to the leaf node $n(w, L(w)) = w$.
- **Internal Node Parameters**: Each internal node $j \in \{1, \ldots, L(w)-1\}$ is assigned a continuous parameter vector $\mathbf{v}'_{n(w,j)} \in \mathbb{R}^d$.
- **Binary Branching Probability**: At each internal node, the probability of branching to the left child is parameterized by the logistic sigmoid function:

$$\mathbb{P}(\text{branch left} \mid \mathbf{v}_{w_I}) = \sigma\left(\mathbf{v}'^{\top}_{n(w,j)} \mathbf{v}_{w_I}\right) = \frac{1}{1 + \exp\left(-\mathbf{v}'^{\top}_{n(w,j)} \mathbf{v}_{w_I}\right)}. \tag{4.41}$$

  The probability of branching to the right child is:

$$\mathbb{P}(\text{branch right} \mid \mathbf{v}_{w_I}) = 1 - \sigma\left(\mathbf{v}'^{\top}_{n(w,j)} \mathbf{v}_{w_I}\right) = \sigma\left(-\mathbf{v}'^{\top}_{n(w,j)} \mathbf{v}_{w_I}\right). \tag{4.42}$$

- **Path Probability Product**: The conditional probability of reaching leaf word $w$ is the product of binary decision probabilities along its unique path:

$$\mathbb{P}(w \mid w_I) = \prod_{j=1}^{L(w)-1} \sigma\left(\text{sign}(w, j) \cdot \mathbf{v}'^{\top}_{n(w,j)} \mathbf{v}_{w_I}\right), \tag{4.43}$$

  where $\text{sign}(w, j) = +1$ if the path branches left at node $j$, and $-1$ if it branches right.

Because probabilities sum to 1 at each binary branching step ($\sigma(z) + \sigma(-z) = 1$), telescoping products guarantee that the sum of probabilities over all leaves equals 1 identically:

$$\sum_{w \in \mathcal{V}} \mathbb{P}(w \mid w_I) \equiv 1. \tag{4.44}$$

No global partition function $Z(w_I)$ ever needs to be computed.

Evaluating $\mathbb{P}(w \mid w_I)$ and its gradients requires traversing only the $L(w) - 1$ nodes along the path. In a balanced or Huffman binary tree, the average path length is $O(\log_2 V)$. For a vocabulary of $V = 100{,}000$:

$$\log_2(100{,}000) \approx 16.6 \implies \frac{100{,}000}{17} \approx \mathbf{5{,}880}\times \text{ acceleration}. \tag{4.45}$$

### 4.7.2 Skip-gram negative sampling (SGNS)

While Hierarchical Softmax scales to $O(\log_2 V)$, an even more efficient and empirically superior alternative was formulated in [39]: **Negative Sampling (SGNS)**, derived as a simplified approximation to Noise Contrastive Estimation (NCE).

Instead of treating context prediction as a massive multinomial classification task over $V$ mutually exclusive categories, SGNS reformulates the problem as $1 + K$ **independent binary logistic classifications**:

1. **Positive Sample ($y = 1$):** The true center-context pair $(w_c, w_o)$ observed in the text corpus. The model maximizes the probability that this pair is a true co-occurrence.
2. **Negative Samples ($y = 0$):** $K$ noise words $\{w_{n_1}, w_{n_2}, \ldots, w_{n_K}\}$ drawn at random from a noise distribution $P_n(w)$. The model maximizes the probability that these randomly sampled pairs are synthetic noise.

**Definition 4.4:** *Skip-gram negative sampling objective*

For an observed center-context token pair $(w_c, w_o)$ and $K$ negative samples $\{w_{n_1}, \ldots, w_{n_K}\} \sim P_n(w)$, the SGNS loss is:

$$\mathcal{L}_{\text{SGNS}} = -\log \sigma\left(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c}\right) - \sum_{k=1}^{K} \log \sigma\left(-\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c}\right), \tag{4.46}$$

where $\sigma(z) = \frac{1}{1+e^{-z}}$ denotes the standard logistic sigmoid function.

### 4.7.3 Step-by-step display math analytical gradient derivations

To understand how SGNS shapes the geometry of the embedding space, we derive the exact analytical gradients of $\mathcal{L}_{\text{SGNS}}$ with respect to all parameter vectors.

Recall the fundamental identities for the logistic sigmoid derivative:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)), \tag{4.47}$$

$$\frac{d}{dz} \log \sigma(z) = \frac{\sigma'(z)}{\sigma(z)} = 1 - \sigma(z), \tag{4.48}$$

$$\frac{d}{dz} \log \sigma(-z) = \frac{-\sigma'(-z)}{\sigma(-z)} = \frac{-\sigma(-z)(1 - \sigma(-z))}{\sigma(-z)} = -(1 - \sigma(-z)) = -\sigma(z). \tag{4.49}$$

**Gradient with respect to positive target vector**

The positive target vector $\mathbf{u}_{w_o}$ appears exclusively in the first term of Equation (4.46):

$$\begin{aligned} \frac{\partial \mathcal{L}_{\text{SGNS}}}{\partial \mathbf{u}_{w_o}} &= -\frac{\partial}{\partial \mathbf{u}_{w_o}} \log \sigma(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c}) \\ &= -\left[1 - \sigma(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c})\right] \frac{\partial(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c})}{\partial \mathbf{u}_{w_o}} \\ &= \left[\sigma(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c}) - 1\right] \mathbf{v}_{w_c}. \end{aligned} \tag{4.50}$$

Defining the positive prediction error as $e_{\text{pos}} = \sigma(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c}) - 1 = \hat{y}_{\text{pos}} - 1$, this simplifies to:

$$\frac{\partial \mathcal{L}_{\text{SGNS}}}{\partial \mathbf{u}_{w_o}} = (\hat{y}_{\text{pos}} - 1)\mathbf{v}_{w_c}. \tag{4.51}$$

**Gradient with respect to negative noise vector**

Each negative sample vector $\mathbf{u}_{w_{n_k}}$ appears in exactly one term of the summation in Equation (4.46):

$$\begin{aligned}\frac{\partial \mathcal{L}_{\text{SGNS}}}{\partial \mathbf{u}_{w_{n_k}}} &= -\frac{\partial}{\partial \mathbf{u}_{w_{n_k}}} \log \sigma(-\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c}) \\ &= -\left[-\sigma(\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c})\right] \frac{\partial(\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c})}{\partial \mathbf{u}_{w_{n_k}}} \\ &= \sigma(\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c})\, \mathbf{v}_{w_c}. \end{aligned} \tag{4.52}$$

Defining the negative prediction error as $e_{\text{neg},k} = \sigma(\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c}) - 0 = \hat{y}_{\text{neg},k}$, this simplifies to:

$$\frac{\partial \mathcal{L}_{\text{SGNS}}}{\partial \mathbf{u}_{w_{n_k}}} = \hat{y}_{\text{neg},k}\, \mathbf{v}_{w_c}. \tag{4.53}$$

**Gradient with respect to center word vector**

The center word vector $\mathbf{v}_{w_c}$ appears in both the positive term and all $K$ negative terms. Applying the chain rule to the complete loss:

$$\begin{aligned}\frac{\partial \mathcal{L}_{\text{SGNS}}}{\partial \mathbf{v}_{w_c}} &= -\frac{\partial}{\partial \mathbf{v}_{w_c}} \log \sigma(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c}) - \sum_{k=1}^{K} \frac{\partial}{\partial \mathbf{v}_{w_c}} \log \sigma(-\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c}) \\ &= \underbrace{\left[\sigma(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c}) - 1\right] \mathbf{u}_{w_o}}_{\text{Attractive Component (Pull Context)}} + \underbrace{\sum_{k=1}^{K} \sigma(\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c})\, \mathbf{u}_{w_{n_k}}}_{\text{Repulsive Component (Push Noise)}}. \end{aligned} \tag{4.54}$$

### 4.7.4 Physical vector mechanics interpretation

Under gradient descent, parameter updates subtract the gradient: $\mathbf{v}_{w_c} \leftarrow \mathbf{v}_{w_c} - \eta \nabla_{\mathbf{v}_{w_c}} \mathcal{L}$:

$$\mathbf{v}_{w_c} \leftarrow \mathbf{v}_{w_c} + \eta \left(1 - \hat{y}_{\text{pos}}\right) \mathbf{u}_{w_o} - \eta \sum_{k=1}^{K} \hat{y}_{\text{neg},k}\, \mathbf{u}_{w_{n_k}}. \tag{4.55}$$

This update formula admits a clean, intuitive **physical vector mechanics** interpretation:

- **The Attractive Pull Force**: Since $\hat{y}_{\text{pos}} = \sigma(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c}) \in (0, 1)$, the coefficient $(1 - \hat{y}_{\text{pos}})$ is strictly positive. The update adds a vector component in the direction $+\mathbf{u}_{w_o}$, **pulling the center word vector toward the true context vector**. If the model is already confident ($\hat{y}_{\text{pos}} \approx 1$), the force vanishes; if the model is surprised ($\hat{y}_{\text{pos}} \approx 0$), the force reaches maximum strength $+1 \cdot \mathbf{u}_{w_o}$.
- **The Repulsive Push Force**: For every negative sample $k$, $\hat{y}_{\text{neg},k} = \sigma(\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c}) \in (0, 1)$ is strictly positive. The update subtracts a vector component along each noise vector $-\mathbf{u}_{w_{n_k}}$, **pushing the center word vector away from noise words**. If the noise word is far away ($\hat{y}_{\text{neg},k} \approx 0$), no force is exerted; if the noise word is erroneously close ($\hat{y}_{\text{neg},k} \approx 1$), a strong repulsive kick $-\mathbf{u}_{w_{n_k}}$ repels the points.

Crucially, the computational complexity of evaluating Equation (4.55) is $O(K \cdot d)$, which is **entirely independent of vocabulary size** $V$. In practice, choosing $K \in [5, 20]$ for small corpora and $K \in [2, 5]$ for web-scale corpora provides sufficient negative contrast, reducing training time from days to hours.

### 4.7.5 Theoretical justification of the $3/4$ unigram noise exponent

How should negative samples be drawn? In [39], the noise distribution $P_n(w)$ is defined as a smoothed unigram distribution raised to the power of $\alpha = 3/4 = 0.75$:

$$P_n(w) = \frac{f(w)^{3/4}}{\sum_{w' \in \mathcal{V}} f(w')^{3/4}}, \tag{4.56}$$

where $f(w)$ denotes the raw unigram occurrence frequency of token $w$ in the corpus.

**Theoretical justification of the 3/4 unigram noise exponent**

1. **Failure of Raw Frequency ($\alpha = 1.0$)**: Natural language word frequencies follow Zipf's law ($f(r) \propto 1/r$). A tiny fraction of stopwords (`"the"`, `"of"`, `"and"`, `"to"`) account for over 50% of all token occurrences in a typical corpus. If $\alpha = 1.0$, more than half of all negative samples drawn are uninformative stopwords. The model wastes its entire discriminative budget separating words from `"the"`, while rare content words (`"polymorphism"`, `"compiler"`) have near-zero probability of ever being selected as negative samples, receiving virtually zero negative gradient updates.

2. **Failure of Uniform Sampling ($\alpha = 0.0$)**: Under a uniform distribution $P_n(w) = 1/V$, every vocabulary word has equal probability. Because the vast majority of words in any natural vocabulary reside in the long tail of rare words, sampling uniformly selects almost exclusively rare words as negatives. The network rarely practices distinguishing moderately frequent content words from one another.

3. **The Sub-linear Compromise ($\alpha = 0.75$)**: The exponent 3/4 acts as a concave, sub-linear dampener. It compresses the gap between ultra-frequent stopwords and rare content words.

Consider a frequent stopword $w_{\text{freq}}$ with corpus frequency $f(w_{\text{freq}}) = 0.01$ (e.g., $10^6$ occurrences in $10^8$ words) and a rare technical term $w_{\text{rare}}$ with corpus frequency $f(w_{\text{rare}}) = 10^{-6}$ (e.g., 100 occurrences in $10^8$ words):

- Under raw unigram frequency ($\alpha = 1.0$), the sampling ratio is:

$$\frac{f(w_{\text{freq}})}{f(w_{\text{rare}})} = \frac{10^{-2}}{10^{-6}} = 10{,}000. \tag{4.57}$$

  The stopword is sampled 10,000 times more frequently than the rare word.

- Under the smoothed distribution ($\alpha = 0.75$), the ratio becomes:

$$\frac{f(w_{\text{freq}})^{0.75}}{f(w_{\text{rare}})^{0.75}} = \left(\frac{10^{-2}}{10^{-6}}\right)^{0.75} = (10{,}000)^{0.75} = (10^4)^{3/4} = 10^3 = 1{,}000. \tag{4.58}$$

By applying the 3/4 power, the relative probability of sampling the rare word is amplified by a factor of:

$$\frac{10{,}000}{1{,}000} = \mathbf{10}\times. \tag{4.59}$$

This 10× boost provides rare words with sufficient negative gradient pushes to carve out accurate decision boundaries, while still reflecting broad corpus statistics.

### 4.7.6 Global count matrix factorization: GloVe

Around the same time, **GloVe (Global Vectors for Word Representation)** was introduced in [42]. Whereas Word2Vec is a local window-based predictive model that processes streaming word pairs, GloVe demonstrated that distributed embeddings could be trained by directly factorizing a **global co-occurrence matrix** $\mathbf{X} \in \mathbb{R}^{V \times V}$, where $X_{ij}$ counts the number of times word $j$ appears in the context window of word $i$.

GloVe's core insight is that the semantic distinction between two words $i$ and $j$ relative to a probe word $k$ is encoded not by raw probabilities $P_{ik} = X_{ik}/X_i$, but by the **ratio of co-occurrence probabilities**:

$$\frac{P_{ik}}{P_{jk}} = \frac{P(k \mid i)}{P(k \mid j)}. \tag{4.60}$$

Table 4.1 reproduces the seminal demonstration from [42] comparing $i$ = `"ice"` and $j$ = `"steam"` across probe words $k$.

Table 4.1: Co-occurrence probability ratios illustrating semantic discrimination in GloVe [42].

| **Probability & Ratio** | $k$ = `"solid"` | $k$ = `"gas"` | $k$ = `"water"` | $k$ = `"fashion"` |
|---|---|---|---|---|
| $P(k \mid$ `"ice"`$)$ | $1.9 \times 10^{-4}$ | $6.6 \times 10^{-5}$ | $3.0 \times 10^{-3}$ | $1.7 \times 10^{-5}$ |
| $P(k \mid$ `"steam"`$)$ | $2.2 \times 10^{-5}$ | $7.8 \times 10^{-4}$ | $2.2 \times 10^{-3}$ | $1.8 \times 10^{-5}$ |
| $P(k \mid$ `"ice"`$)/P(k \mid$ `"steam"`$)$ | **8.9** | **0.085** | 1.36 | 0.96 |

When $k$ = `"solid"` is strongly related to `"ice"` but not `"steam"`, the ratio is large ($8.9 \gg 1$). When $k$ = `"gas"` is related to `"steam"` but not `"ice"`, the ratio is tiny ($0.085 \ll 1$). When $k$ = `"water"` is equally related to both (or $k$ = `"fashion"` is related to neither), the ratio hovers near 1.0.

GloVe enforces this relationship by fitting a log-bilinear model where dot products approximate log co-occurrence counts, minimized via weighted least squares:

$$J = \sum_{i=1}^{V} \sum_{j=1}^{V} f(X_{ij}) \left( \mathbf{w}_i^\top \tilde{\mathbf{w}}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2, \tag{4.61}$$

where $f(X_{ij}) = \min(1, (X_{ij}/x_{\max})^\alpha)$ is a sub-linear weighting function preventing frequent stopwords from dominating the loss. Both Word2Vec and GloVe construct semantically structured vector spaces, with SGNS operating on streaming text and GloVe operating on aggregated count statistics.

Skip-Gram Negative Sampling and GloVe resolve the computational bottleneck, making large-scale distributed representation learning feasible. However, Word2Vec operates under an implicit architectural assumption: that the word is the minimal, indivisible atomic unit of language. In real-world deployment, this assumption creates severe structural vulnerabilities.

## 4.8 Limitations of Word2Vec and the out-of-vocabulary (OOV) barrier

Despite their revolutionary impact, classical Word2Vec and GloVe architectures suffer from fundamental linguistic and software engineering limitations.

### 4.8.1 The atomic word token assumption and out-of-vocabulary (OOV) failure

Word2Vec models assume a closed, fixed vocabulary $\mathcal{V} = \{w_1, \ldots, w_V\}$ established during training. Each word is treated as an indivisible, atomic entity: its character composition is completely ignored. This creates the **Out-of-Vocabulary (OOV) catastrophe**:

- **Unseen Words at Test Time**: In open-domain text and evolving codebases, users constantly produce novel tokens: technical neologisms, composite identifiers (e.g., `AsyncHttpClientFactory`), misspelled words (e.g., `"recusive"`), and inflected stems.
- **Mapping to `<UNK>`**: When a trained Word2Vec model encounters an OOV word at inference, it has no entry in $\mathbf{W}_{\text{in}}$ or $\mathbf{W}_{\text{out}}$. Deep learning pipelines are forced to map all OOV tokens to a generic unknown token placeholder:

$$w_{\text{oov}} \longmapsto \texttt{<UNK>} \implies \mathbf{v}_{w_{\text{oov}}} = \mathbf{v}_{\texttt{<UNK>}}. \tag{4.62}$$

  This discards all semantic identity. To Word2Vec, a typo like `"algorithmm"` is as completely devoid of meaning as an alien string.

### 4.8.2 Morphological blindness

Word2Vec is entirely blind to internal word morphology. In natural languages with rich inflectional systems (e.g., Italian, German, Turkish) as well as programming languages, words share sub-morphemic stems, prefixes, and suffixes:

Natural language: `"develop"`, `"developer"`, `"developing"`, `"undeveloped"`,
Software identifiers: `"parseFile"`, `"parseTokens"`, `"parseTree"`.

Under Word2Vec, each of these variants is assigned a totally disjoint, independent column in $\mathbf{W}_{\text{in}}$. The model possesses zero architectural mechanism to share parameters across words containing the common root `"develop"` or `"parse"`. If `"undeveloped"` occurs only twice in the corpus, its vector remains poorly trained, unable to inherit semantic properties from the frequent token `"develop"`.

### 4.8.3 Polysemy collapse: static context invariance

Word2Vec assigns exactly **one static vector $\mathbf{v}_w \in \mathbb{R}^d$ to each word string** $w$, regardless of the surrounding runtime context in which it appears. This results in **polysemy collapse**.

**Definition 4.5:** *Polysemy collapse*

Let word $w$ possess $M$ distinct semantic senses $\mathcal{S}_1, \ldots, \mathcal{S}_M$ with respective corpus mixture proportions $\pi_1, \ldots, \pi_M$ ($\sum \pi_m = 1$). Because Word2Vec minimizes prediction error across all occurrences using a single vector $\mathbf{v}_w$, the optimal vector converges to the convex combination of distinct sense centroids:

$$\mathbf{v}_w^* \approx \sum_{m=1}^{M} \pi_m \mathbf{v}_{\mathcal{S}_m}. \tag{4.63}$$

For example, consider the polysemous word `"bat"`:

Sense 1 (Zoological): "A **bat** is a nocturnal flying mammal."
Sense 2 (Sports Equipment): "The baseball player swung the wooden **bat**."

In Word2Vec, $\mathbf{v}_{\text{bat}}$ is pulled toward biological animals on some iterations, and toward sports equipment on others. The resulting point converges to an intermediate centroid in $\mathbb{R}^d$ that represents neither animal nor sports gear accurately, degrading cosine similarity to both senses.

In software engineering, polysemy is ubiquitous:

- `"Python"`: A biological serpent vs. a high-level dynamic programming language.
- `"Spring"`: A season of the year vs. a popular Java enterprise application framework.
- `"rust"`: Iron oxide corrosion vs. a memory-safe systems programming language.

A static Word2Vec model maps `"Rust"` to a compromise vector that conflates oxidation chemistry with memory safety guarantees.

While contextual ambiguity requires sequence models that dynamically modulate representations based on full sentence context, the Out-of-Vocabulary barrier and morphological blindness can be solved at the token representation level. By abandoning the word as an atomic unit and descending into subword character $n$-grams, FastText establishes a morphologically aware embedding paradigm.

## 4.9 FastText: subword character $n$-gram embeddings

To conquer the OOV barrier and enable morphological parameter sharing, **FastText** was introduced by Bojanowski et al. [43].

### 4.9.1 Character $n$-gram decomposition and boundary markers

Rather than treating a word as an atomic token, FastText represents each word as a **bag of character $n$-grams**.

1. **Boundary Enclosure**: To distinguish prefixes and suffixes from character sequences occurring in the interior of a word, each word $w$ is bracketed by special boundary symbols `"<"` and `">"`:
$$\tilde{w} = \texttt{"<"} \circ w \circ \texttt{">"}. \tag{4.64}$$
This ensures that the prefix sequence `"<re"` (as in `"<refactor>"`) is mathematically distinct from the internal substring `"re"` (as in `"<where>"`).

2. **Subword Extraction**: All character $n$-grams of length $n \in [n_{\min}, n_{\max}]$ (typically $n \in [3, 6]$) are extracted, along with the special sequence containing the entire word itself $\tilde{w}$.

**Subword decomposition worked example**

Consider the word `"where"` decomposed into character trigrams ($n = 3$):

$$\texttt{"<where>"} \implies \mathcal{G}_{\texttt{where}} = \Big\{\texttt{"<wh"}, \texttt{"whe"}, \texttt{"her"}, \texttt{"ere"}, \texttt{"re>"}, \texttt{"<where>"}\Big\}. \tag{4.65}$$

### 4.9.2 Vector sum composition

Each subword $n$-gram $g$ is assigned a latent continuous embedding vector $\mathbf{z}_g \in \mathbb{R}^d$. The final representation of word $w$ is defined as the **sum of its constituent subword vectors**:

$$\mathbf{v}_w = \sum_{g \in \mathcal{G}_w} \mathbf{z}_g \in \mathbb{R}^d. \tag{4.66}$$

Under the Skip-Gram Negative Sampling objective, the similarity score between center word $w$ and context word $c$ becomes:

$$s(w, c) = \mathbf{v}_w^\top \mathbf{u}_c = \sum_{g \in \mathcal{G}_w} \mathbf{z}_g^\top \mathbf{u}_c. \tag{4.67}$$

During backpropagation, the prediction error with respect to $\mathbf{v}_w$ distributes gradients to every constituent subword vector $\mathbf{z}_g$.

### 4.9.3 Resolving OOV tokens dynamically and morphological sharing in code

This simple compositional formulation yields two decisive capabilities:

1. **Zero-Shot Synthesis of Out-of-Vocabulary Tokens**: When an unseen word $w_{\text{oov}}$ appears at test time (e.g., a compound identifier like `"refactorable"` or a code typo like `"recusive"`), even if the full token was never seen during training, its constituent character $n$-grams (`"<re"`, `"ref"`, `"fact"`, `"able>"`) were updated across other vocabulary words. FastText

synthesizes an accurate embedding vector on the fly by summing the vectors of its known subwords:

$$\mathbf{v}_{w_{\text{oov}}} = \sum_{g \in \mathcal{G}_{w_{\text{oov}}}} \mathbf{z}_g. \tag{4.68}$$

The OOV barrier is completely eliminated.

2. **Morphological Parameter Sharing**: In natural language, prefixes like `"un-"`, suffixes like `"-tion"`, and inflections like `"-ing"` accumulate shared vector representations across hundreds of distinct words. In software engineering, identifier substrings automatically share semantics:

$$\texttt{"parseXmlDocument"} \quad \text{and} \quad \texttt{"parseJsonDocument"} \tag{4.69}$$

share the character subwords for `"parse"` and `"Document"`. The model immediately recognizes that both identifiers describe parsing operations on structured documents.

**Memory bounding via the Fowler-Noll-Vo hash trick**

Because the universe of all character $n$-grams ($n \in [3, 6]$) across large vocabularies exceeds tens of millions of unique strings, allocating an unbounded matrix would cause memory blowout. FastText bounds the embedding table to $K = 2 \times 10^6$ rows using the Fowler-Noll-Vo (FNV-1a) hashing function:

$$\text{index}(g) = \text{FNV-1a}(g) \pmod{K}. \tag{4.70}$$

Colliding $n$-grams share bucket rows; due to the high dimensionality ($d = 300$) and independent sum aggregation, hash collisions act as benign regularizing noise that does not disrupt semantic retrieval.

### 4.9.4 Connection to modern subword tokenizers: BPE and WordPiece

As highlighted in pedagogical breakdowns of language models [44], the subword philosophy pioneered by FastText directly inspired the tokenization mechanisms powering modern Large Language Models:

- **Byte Pair Encoding (BPE)** [45, 46]: Iteratively merges the most frequent pairs of adjacent characters or bytes to form a deterministic subword vocabulary (used in GPT-2, GPT-4, and LLaMA).
- **SentencePiece** [47]: Treats input text as a raw byte stream without whitespace pre-segmentation, training unigram language model segmentations.

FastText operates continuous character $n$-gram summations at the embedding layer, whereas BPE operates discrete token mergers during preprocessing. Both architectures fundamentally exist to dismantle the brittle, isolated word-level assumption and establish robust subword parameter sharing.

By decomposing words into character $n$-grams, FastText conquers the Out-of-Vocabulary barrier and enables morphological parameter sharing. With our embedding models fully trained, we can now step back and examine the remarkable mathematical geometry that spontaneously emerges within these dense vector spaces.

## 4.10 Emergent vector space geometry: clustering, metrics and relational analogies

When Word2Vec and FastText models are trained over large text corpora, the resulting continuous vector spaces exhibit profound emergent mathematical properties: high-dimensional semantic clustering, metric equivalence, and linear relational translations.

### 4.10.1 Semantic clustering and dimensionality reduction

In high-dimensional embedding space ($d = 300$), words with similar functional roles organize into distinct, tightly bound clusters.

Projecting 300-dimensional FastText vectors into two dimensions via Principal Component Analysis (PCA) reveals distinct semantic clusters:

- **Household Items**: `"table"`, `"chair"`, `"desk"`, `"sofa"`.
- **Mammals**: `"dog"`, `"cat"`, `"tiger"`, `"elephant"`.
- **Birds**: `"eagle"`, `"sparrow"`, `"parrot"`, `"pigeon"`.

The leading principal components preserve categorical variance: domestic entities, animal taxonomy, and biological sub-branches form linearly separable clusters in the compressed 2D projection as well as in the original 300-dimensional manifold.

### 4.10.2 Linear relational translations (vector analogy arithmetic)

The most celebrated emergent property of continuous word embeddings is that semantic and syntactic relationships manifest as **linear vector translation offsets**:

$$\mathbf{v}_{w_2} - \mathbf{v}_{w_1} \approx \vec{d}_{\text{relation}}. \tag{4.71}$$

#### Country-to-capital linear offsets

For instance, subtracting the vector for `"Berlin"` from `"Germany"` produces a directional displacement vector $\vec{d}_{\text{capital_of}}$:

$$\mathbf{v}_{\text{Germany}} - \mathbf{v}_{\text{Berlin}} \approx \vec{d}_{\text{capital_of}}. \tag{4.72}$$

Applying this learned offset vector to another country predicts its capital city via simple vector addition:

$$\mathbf{v}_{\text{Spain}} + (\mathbf{v}_{\text{Germany}} - \mathbf{v}_{\text{Berlin}}) \approx \mathbf{v}_{\text{Madrid}}. \tag{4.73}$$

#### The gender-royalty parallelogram

The canonical demonstration established in [39] is the gender-royalty parallelogram:

$$\mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}} + \mathbf{v}_{\text{woman}} \approx \mathbf{v}_{\text{queen}}. \tag{4.74}$$

Figure 4.5 illustrates this parallelogram in two dimensions.

#### Software engineering relational analogies

In code representation learning (e.g., CodeBERT [48], OpenAI Codex embeddings [49]), relational vector arithmetic reflects runtime architectures and language toolchains:

$$\mathbf{v}_{\text{Java}} - \mathbf{v}_{\text{JVM}} + \mathbf{v}_{\text{Python}} \approx \mathbf{v}_{\text{CPython}}, \tag{4.75}$$

$$\mathbf{v}_{\text{C++}} - \mathbf{v}_{\text{g++}} + \mathbf{v}_{\text{Rust}} \approx \mathbf{v}_{\text{rustc}}. \tag{4.76}$$

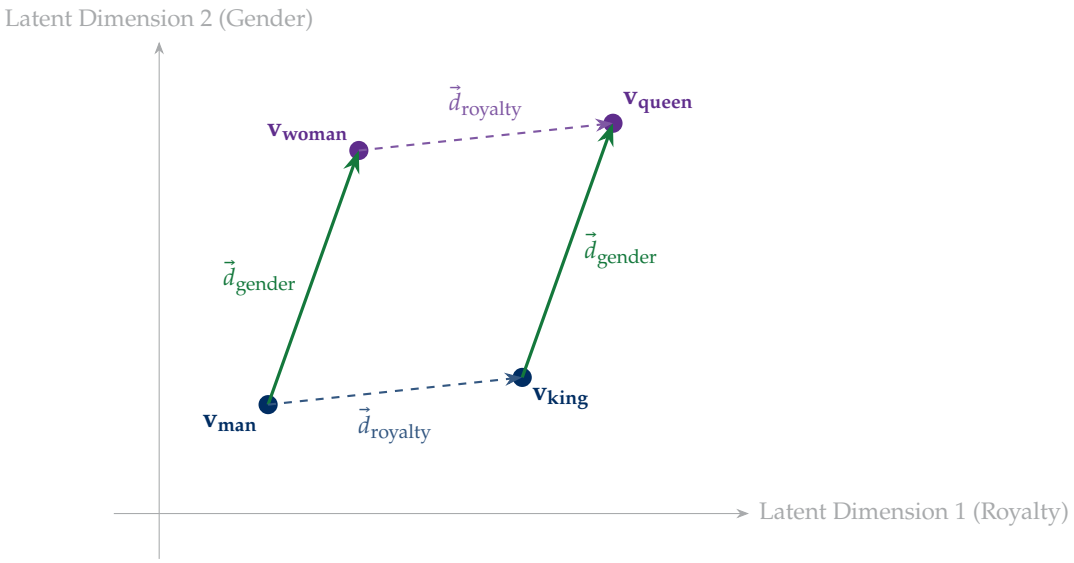



Figure 4.5: Geometric parallelogram in continuous embedding space demonstrating relational vector translations: $\mathbf{v}_{\text{King}} - \mathbf{v}_{\text{Man}} \approx \mathbf{v}_{\text{Queen}} - \mathbf{v}_{\text{Woman}}$. The directional displacement $\vec{d}_{\text{gender}}$ isolates the feminine gender transformation, while $\vec{d}_{\text{royalty}}$ isolates the monarchical status offset.

### 4.10.3 Metric geometry of embedding spaces: cosine vs. Euclidean distance

When querying nearest neighbors or evaluating semantic similarity between two continuous vectors $\mathbf{u}, \mathbf{v} \in \mathbb{R}^d$, three metrics are commonly considered:

1. **Euclidean ($L_2$) Distance**:

$$D_{L_2}(\mathbf{u}, \mathbf{v}) = \|\mathbf{u} - \mathbf{v}\|_2 = \sqrt{\sum_{i=1}^{d} (u_i - v_i)^2}. \tag{4.77}$$

2. **Dot Product (Inner Product)**:

$$\langle \mathbf{u}, \mathbf{v} \rangle = \mathbf{u}^\top \mathbf{v} = \sum_{i=1}^{d} u_i v_i = \|\mathbf{u}\|_2 \|\mathbf{v}\|_2 \cos\theta. \tag{4.78}$$

3. **Cosine Distance**:

$$D_{\cos}(\mathbf{u}, \mathbf{v}) = 1 - \cos(\mathbf{u}, \mathbf{v}) = 1 - \frac{\mathbf{u}^\top \mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2} \in [0, 2]. \tag{4.79}$$

**Theorem 4.2:** *Monotonic equivalence under $L_2$ normalization*

Let $\mathbf{u}, \mathbf{v} \in \mathbb{R}^d$ be normalized to unit Euclidean length ($\|\mathbf{u}\|_2 = \|\mathbf{v}\|_2 = 1$). Then squared Euclidean distance is strictly proportional to Cosine distance:

$$\|\mathbf{u} - \mathbf{v}\|_2^2 = \|\mathbf{u}\|_2^2 + \|\mathbf{v}\|_2^2 - 2\mathbf{u}^\top \mathbf{v} = 1 + 1 - 2\cos(\mathbf{u}, \mathbf{v}) = 2(1 - \cos(\mathbf{u}, \mathbf{v})) = 2D_{\cos}(\mathbf{u}, \mathbf{v}). \tag{4.80}$$

Consequently, nearest neighbor rankings under Euclidean distance, Cosine distance, and Dot Product are mathematically identical on the unit hypersphere $\mathbb{S}^{d-1}$.

**Why cosine similarity dominates metric evaluations**

As demonstrated in educational analyses [44], when word embeddings are trained without explicit length regularization, the Euclidean norm $\|\mathbf{v}_w\|_2$ strongly correlates with unigram token frequency. Highly frequent words undergo millions of gradient pushes, systematically expanding their vector magnitude, whereas rare terms remain close to the origin. Cosine distance normalizes out vector magnitude, projecting all embeddings onto $\mathbb{S}^{d-1}$. It evaluates **pure angular orientation** (conceptual topic) rather than **vector length** (frequency artifact).

### 4.10.4 Dimensionality trade-offs and the Johnson-Lindenstrauss lemma

Why do word embeddings operate effectively in dimensions $d \in [100, 300]$, and modern LLMs in $d \in [768, 4096]$?

- **Under-parameterization ($d < 50$)**: Severe geometric crowding occurs. The space lacks enough orthogonal degrees of freedom to separate independent semantic concepts, forcing unrelated topics into spurious proximity.
- **Over-parameterization ($d \gg 10{,}000$)**: The space suffers from the curse of dimensionality. The volume of the unit ball concentrates entirely in a thin outer shell, distance distributions become uniform, and memory bandwidth demands explode.

The mathematical guarantee underwriting this stability is the **Johnson-Lindenstrauss Lemma**:

**Theorem 4.3:** *Johnson-Lindenstrauss Lemma*

Given $0 < \epsilon < 1$, any set of $N$ points in arbitrary Euclidean space can be embedded into a Euclidean space of dimension:

$$d = O\left(\frac{\log N}{\epsilon^2}\right), \tag{4.81}$$

such that all pairwise Euclidean distances are preserved within a distortion factor of $1 \pm \epsilon$:

$$(1-\epsilon)\|\mathbf{u}-\mathbf{v}\|_2^2 \leq \|f(\mathbf{u}) - f(\mathbf{v})\|_2^2 \leq (1+\epsilon)\|\mathbf{u}-\mathbf{v}\|_2^2. \tag{4.82}$$

For a vocabulary of $N = 100{,}000$ tokens and distortion tolerance $\epsilon = 0.2$, $\log(10^5)/0.04 \approx 287$, precisely justifying why $d = 300$ serves as the golden standard for dense static word vectors.

## 4.11 Modern subword tokenization and input pipelines for foundation models

In Sections 4.1 through 4.10, we formulated the mathematical foundations of classical distributed word representations (Word2Vec, GloVe, FastText). While continuous embeddings solved the differentiability barrier and geometric similarity, classical architectures operated under a fragmented, static paradigm: input text was naively pre-split on whitespace into word-level units, vocabulary sizes ballooned into hundreds of thousands of entries, rare or misspelled tokens collapsed into the uninformative `<UNK>` placeholder, and input sequences were treated as unordered bags or static lookup tables.

Modern Large Language Models (LLMs) such as GPT-2, GPT-3, GPT-4, LLaMA, and Claude discard this isolated design in favor of an integrated, end-to-end **data preprocessing, tokenization, and input embedding pipeline**. As formalized in *Build a Large Language Model from Scratch* [34], preparing raw text for foundation model pretraining requires an indivisible four-stage architectural pipeline:

1. **Subword Tokenization**: Segmenting raw text into atomic subword units using Byte Pair Encoding (BPE), resolving punctuation and whitespace losslessly.
2. **Token ID Mapping**: Converting discrete subwords into numerical indices via an invertible vocabulary lookup table, enriched with special operational context markers.
3. **Continuous Embedding Projection**: Retrieving dense vector representations for each token index from a learned embedding table $\mathbf{W}_{\text{tok}} \in \mathbb{R}^{V \times d}$.
4. **Composite Positional Synthesis**: Injecting sequential order into permutation-invariant neural layers by additively combining continuous token embeddings with positional embeddings: $\mathbf{X}_{\text{in}} = \mathbf{E}_{\text{tok}} + \mathbf{E}_{\text{pos}}$.

Figure 4.6 illustrates this end-to-end data pipeline from raw textual strings to the final continuous tensor ingested by Transformer architectures.

### 4.11.1 The tokenization spectrum: character, word, and subword trade-offs

Converting raw natural language or software code into discrete units reveals a fundamental trade-off across three representational granularities: character-level, word-level, and subword tokenization.

#### Limitations of whitespace-based splitting

A naive baseline approach to text tokenization splits raw strings on whitespace characters (‘\s’). However, whitespace splitting fails substantially on punctuation:

$$\texttt{"Hello, world. This is a test."} \overset{\text{whitespace split}}{\Longrightarrow} \texttt{['Hello,', 'world.', 'This', 'is', 'a', 'test.']}$$

The comma is fused to `"Hello"`, and periods are fused to `"world"` and `"test"`. Consequently, the model assigns distinct vocabulary indices to `"world"`, `"world."`, `"world,"`, and `"world?"`. This fragments training counts, bloats the vocabulary with redundant punctuation variants, and destroys syntactic generalization.

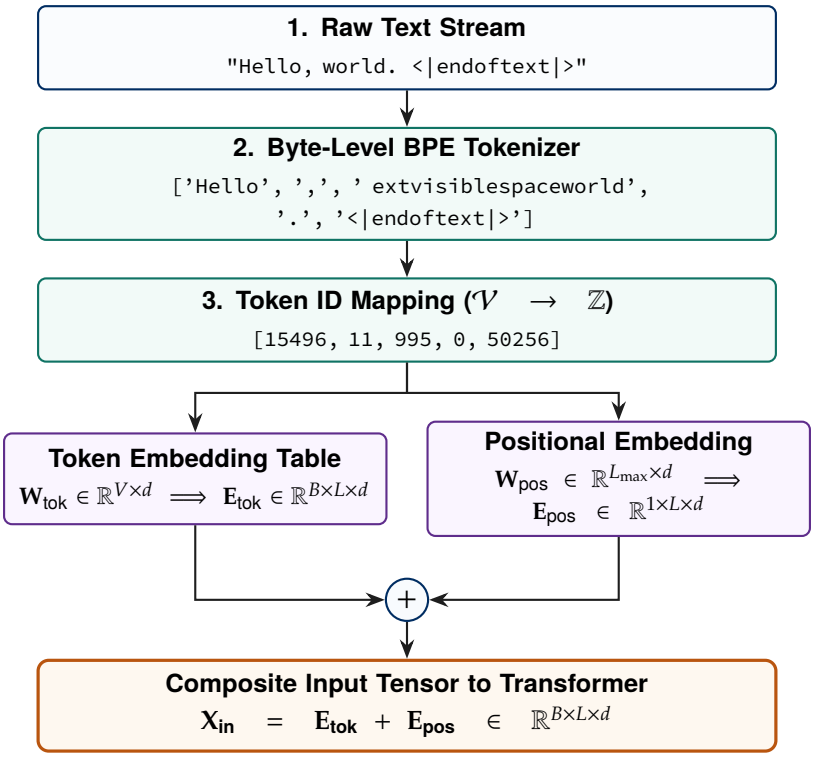



Figure 4.6: The modern foundation model tokenization and input preparation pipeline. Raw text is segmented into byte-level BPE subwords, mapped to unique integer token IDs, and converted into continuous latent vectors where token embeddings and positional embeddings combine additively.

Table 4.2: Comprehensive taxonomy and trade-off analysis of tokenization paradigms in natural language processing and modern foundation models.

| Paradigm | Vocab ($V$) | Length ($L$) | OOV | Linguistic & Systems Limitations | Representative Models |
|---|---|---|---|---|---|
| **Character-Level** | 100 – 256 | Massive (4 – 6×) | Zero | High sequence length explodes computational complexity $O(L^2)$ in self-attention; lacks word-level semantic priors. | Char-RNN, Canine, ByT5 |
| **Word-Level** | 50k – 1M+ | Minimal (1×) | Severe | Out-of-vocabulary words collapse to `<UNK>`; morphologically blind; massive embedding matrix $\mathbf{W} \in \mathbb{R}^{V \times d}$. | Word2Vec, GloVe, N-gram LMs |
| **Subword-Level** | 32k – 100k | Moderate (1.2 – 1.4×) | Zero | Requires statistical offline pre-tokenization merge training; optimal Pareto frontier of length and vocabulary. | BPE (GPT-2/3/4, LLaMA), WordPiece (BERT) |

**Regex-based tokenization with punctuation isolation**

To isolate punctuation while preserving them as distinct semantic tokens, a regular expression splitting rule can be formulated [34] that captures punctuation symbols, em-dashes, and whitespaces:

```python
import re

class SimpleTokenizerV1:
    def __init__(self, vocab):
        self.str_to_int = vocab
        self.int_to_str = {i: s for s, i in vocab.items()}

    def encode(self, text):
        # Split on whitespace, punctuation marks, and special dash patterns
        tokens = re.split(r'([,.:;?_."()\']|--|\s)', text)
        tokens = [t.strip() for t in tokens if t.strip()]
        token_ids = [self.str_to_int[t] for t in tokens]
        return token_ids

    def decode(self, token_ids):
        text = " ".join([self.int_to_str[i] for i in token_ids])
        # Clean up spaces before punctuation marks
        text = re.sub(r'\s+([,.?.])', r'\1', text)
        return text

# Building the vocabulary from unique preprocessed tokens
text = "Hello, world. Is this-- a test?"
raw_tokens = re.split(r'([,.:;?_."()\']|--|\s)', text)
tokens = [t.strip() for t in raw_tokens if t.strip()]
vocab = {token: idx for idx, token in enumerate(sorted(set(tokens)))}
tokenizer = SimpleTokenizerV1(vocab)
encoded = tokenizer.encode("Hello, world.")
print("Encoded IDs:", encoded)
print("Decoded text:", tokenizer.decode(encoded))
```

Listing 4.1: Basic regular expression text tokenization with punctuation isolation and vocabulary construction [34].

### 4.11.2 Special context tokens: document boundaries, padding, and masked batching

In practical machine learning and software engineering workflows, a vocabulary cannot consist solely of natural language words. The model requires **special context tokens** to encode structural and operational metadata:

1. **Unknown Word Token** (<|unk|>): In fixed-vocabulary systems, words unseen during training cannot be mapped to an integer index. Such tokens are mapped to <|unk|>. While this prevents code execution errors, it irreversibly destroys semantic information: a technical identifier or typo is flattened into a generic void.

2. **Document Boundary Delimiter** (<|endoftext|>): In large-scale autoregressive pretraining, billions of tokens spanning Wikipedia articles, books, and GitHub repositories are concatenated into continuous sequential streams. Without an explicit separator, the causal attention mechanism would assume that the first sentence of an article is the causal continuation of an unrelated previous document. The token <|endoftext|> signals that all preceding context is decoupled from subsequent tokens, allowing clean document demarcation.

3. **Padding Token** (<|pad|>): Mini-batch processing on GPU hardware requires rectangular tensors $\mathbf{X} \in \mathbb{Z}^{B \times L}$. When batching sequences of varying lengths ($L_1 \neq L_2$), shorter sequences are padded with <|pad|> tokens up to maximum length $L$. Crucially, the model's self-attention mechanism is configured with an **attention mask**:

$$M_{ij} = \begin{cases} 0 & \text{if token } j \text{ is a genuine text token,} \\ -\infty & \text{if token } j \text{ is a <|pad|> token.} \end{cases} \tag{4.83}$$

Adding $M_{ij}$ to attention logits before Softmax drives pad attention weights to $\frac{e^{-\infty}}{\sum e^z} = 0$, completely preventing padded positions from corrupting token representations.

### 4.11.3 Byte Pair Encoding (BPE): algorithmic mechanics and merge trees

To conquer the Out-of-Vocabulary barrier without blowing up sequence length, modern LLMs deploy **Byte Pair Encoding (BPE)**. Originally invented for data compression in [50], BPE was adapted to natural language processing in [45].

BPE constructs a subword vocabulary through an iterative bottom-up statistical merge procedure partitioned into distinct training and inference phases:

1. **Base Alphabet Initialization**: Initialize base vocabulary $\mathcal{V}_0$ with all individual characters occurring across the training corpus $C$, augmenting each word with an explicit end-of-word boundary marker (e.g., '</w>'):

$$\mathcal{V}_0 = \{c \mid c \in w,\ w \in C\} \cup \{\texttt{</w>}\}. \tag{4.84}$$

Every word in $C$ is initially split into a sequence of individual character symbols.

2. **Adjacent Pair Frequency Tally**: At iteration $k$, construct the co-occurrence frequency histogram over all adjacent symbol pairs $(p_1, p_2)$ across all tokenized words in $C$:

$$(p_1^*, p_2^*) = \arg\max_{(p_1,p_2)} \sum_{w \in C} \text{count}(w) \cdot \mathbb{I}\{(p_1, p_2) \subseteq w\}. \tag{4.85}$$

3. **Greedy Merge and Priority Rule Logging**: Merge the globally most frequent pair $(p_1^*, p_2^*)$ into an atomic new subword unit $p_{\text{new}} = p_1^* \circ p_2^*$. Crucially, record this merge operation in an ordered priority dictionary $\mathcal{M}$:

$$\mathcal{M}[k] = ((p_1^*, p_2^*) \mapsto p_{\text{new}}). \tag{4.86}$$

4. **In-Place Corpus Contraction**: Substitute every contiguous occurrence of $(p_1^*, p_2^*)$ across corpus $C$ with the merged symbol $p_{\text{new}}$, and expand the vocabulary:

$$\mathcal{V}_k = \mathcal{V}_{k-1} \cup \{p_{\text{new}}\}. \tag{4.87}$$

5. **Termination and Inference Application**: Repeat Steps 2–4 until reaching the target vocabulary budget $|\mathcal{V}_k| = V_{\text{target}}$. At inference time, any arbitrary input word is first split into its constituent base characters, and the learned merge rules in $\mathcal{M}$ are applied sequentially according to their training rank.

**BPE training merge mechanics and vocabulary growth**

To illustrate how BPE resolves out-of-vocabulary tokens without explosion, consider a small transparent corpus consisting of four words with their occurrence frequencies:

$$C = \{\texttt{"hug"} : 10,\ \texttt{"pug"} : 5,\ \texttt{"pun"} : 12,\ \texttt{"hugs"} : 5\}. \tag{4.88}$$

The total corpus count is $10 + 5 + 12 + 5 = 32$ word instances. The initial base vocabulary consists of 7 symbols: $\mathcal{V}_0 = \{\texttt{'h'}, \texttt{'u'}, \texttt{'g'}, \texttt{'p'}, \texttt{'n'}, \texttt{'s'}, \texttt{'</w>'}\}$. The segmented corpus begins as:

```
h u g </w> :  10    p u g </w> :  5    p u n </w> :  12    h u g s </w> :  5
```

- **Iteration 1**: We evaluate the frequency of all adjacent bigrams across the corpus:
  - ('u','g') : 10(hug) + 5(pug) + 5(hugs) = 20.
  - ('p','u') : 5(pug) + 12(pun) = 17.
  - ('h','u') : 10(hug) + 5(hugs) = 15.
  - ('g','</w>') : 10(hug) + 5(pug) = 15.
  - ('u','n') : 12(pun), ('n','</w>') : 12(pun).
  - ('g','s') : 5(hugs), ('s','</w>') : 5(hugs).

  The most frequent pair is ('u','g') with frequency 20. We log Merge Rule 1: ('u','g') → 'ug' and update $\mathcal{V}_1 = \mathcal{V}_0 \cup \{\texttt{'ug'}\}$. The corpus contracts to:

```
h ug </w> :  10    p ug </w> :  5    p u n </w> :  12    h ug s </w> :  5
```

- **Iteration 2**: We recompute adjacent pair frequencies with the merged symbol 'ug':
  - ('h','ug') : 10 + 5 = 15.
  - ('ug','</w>') : 10 + 5 = 15.
  - ('p','u') : 12(pun), ('u','n') : 12(pun), ('n','</w>') : 12(pun).
  - ('p','ug') : 5(pug), ('ug','s') : 5(hugs), ('s','</w>') : 5(hugs).

  The highest frequency is 15. Selecting ('h','ug'), we log Merge Rule 2: ('h','ug') → 'hug' and update $\mathcal{V}_2 = \mathcal{V}_1 \cup \{\texttt{'hug'}\}$. The corpus becomes:

```
hug </w> :  10    p ug </w> :  5    p u n </w> :  12    hug s </w> :  5
```

**Inference segmentation walkthrough on unseen tokens**

Now consider what happens when a model deployed in production encounters the word "pugs" (the plural of pug). Notice that "pugs" **never appeared in the training corpus as a complete word** (the training data only contained "pug" and "hugs").

1. Under a traditional word-level tokenizer, "pugs" is completely absent from vocabulary $\mathcal{V}$ and collapses into the information-destroying token <UNK>.
2. Under Byte Pair Encoding:
   - The word is initialized into base characters: ['p','u','g','s'].
   - The tokenizer queries the learned merge rules in priority order. Merge Rule 1 ('u','g') → 'ug' matches immediately. The sequence is rewritten as:

$$[\texttt{'p'}, \texttt{'u'}, \texttt{'g'}, \texttt{'s'}] \xrightarrow{\text{Merge 1}} [\texttt{'p'}, \texttt{'ug'}, \texttt{'s'}].$$

- Merge Rule 2 (`'h','ug'`) → `'hug'` is queried next. It does not match because the preceding symbol is `'p'`, not `'h'`.
- No further merge rules match adjacent symbols.

3. The final emitted token sequence is:

$$\texttt{encode("pugs")} = \texttt{['p', 'ug', 's']}. \tag{4.89}$$

The Out-of-Vocabulary barrier is overcome: the model represents the novel word via known morphological primitives—the initial consonant `'p'`, the shared stem `'ug'`, and the plural suffix `'s'`.

Figure 4.7 visualizes both the bottom-up hierarchical merge tree constructed during training and the top-down decomposition of unseen words at inference time.

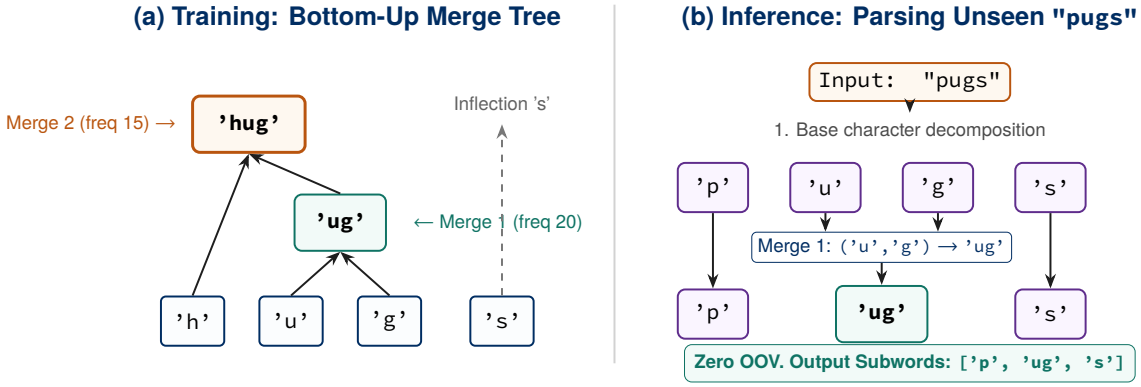



Figure 4.7: Byte Pair Encoding (BPE) mechanics. (a) During training, adjacent high-frequency symbol pairs are greedily merged into subwords. (b) At inference time, unseen words (e.g., `"pugs"`) are parsed by applying learned merge rules in priority order, resolving morphology without out-of-vocabulary collapse.

### 4.11.4 Byte-level BPE: the 256 UTF-8 byte foundation and elimination of unknown tokens

Despite the success of character-level BPE, applying it directly to Unicode strings faces a severe scaling barrier: the Unicode standard contains over 150,000 unique characters spanning hundreds of languages and symbol sets. Initializing $\mathcal{V}_0$ with every conceivable character creates an unmanageably large base alphabet, yet still fails when a user inputs rare historical glyphs or newly released emojis.

#### The 256 UTF-8 base byte formulation

In GPT-2, this dilemma was solved in [46] by introducing **Byte-Level Byte Pair Encoding (BBPE)**. Instead of characters, the base alphabet $\mathcal{V}_0$ is initialized strictly with the **256 distinct byte values** ($0x00$ through $0xFF$) of the UTF-8 encoding standard:

$$\mathcal{V}_0 = \{0x00, 0x01, \ldots, 0xFF\}, \quad |\mathcal{V}_0| = 256. \tag{4.90}$$

**Theorem 4.4:** *Total elimination of unknown tokens in byte-level BPE*

Under Byte-Level BPE with base alphabet $\mathcal{V}_0 = \{0x00, \ldots, 0xFF\}$, the probability of an Out-of-Vocabulary (OOV) token is strictly zero:

$$\mathbb{P}(w_{\text{oov}}) = 0 \quad \forall S \in \Sigma^*. \tag{4.91}$$

By definition of the Unicode transformation format (UTF-8), any arbitrary character or sequence of symbols in the Unicode code space ($U+0000$ to $U+10FFFF$) encodes deterministically into a sequence of between 1 and 4 raw bytes in $\{0x00, \ldots, 0xFF\}$. Because the base alphabet $\mathcal{V}_0$ contains all 256 possible 8-bit byte values, every byte sequence can be parsed. If an input string contains an unseen compound word, a programming syntax construct, a severe typo, or an unknown emoji, the tokenizer attempts to apply merged subword rules; if no higher-level merge rule applies, the tokenizer simply falls back to emitting individual byte tokens from $\mathcal{V}_0$. Because no input byte can ever fall outside $\{0x00, \ldots, 0xFF\}$, no token can ever be unmapped. The token `<|unk|>` is completely eliminated. ■

**Lossless whitespace preservation**

Standard natural language processing pipelines strip whitespace or split on spaces, making exact detokenization ambiguous. In Byte-Level BPE, whitespaces are treated as regular characters/bytes. In GPT-2 and Hugging Face tokenizers, a leading whitespace is mapped to a special character (e.g., ' `extvisiblespace`' or byte $0x20$), binding the space directly to the succeeding word:

$$\texttt{"Hello world"} \overset{\text{BBPE}}{\Longrightarrow} \texttt{['Hello', ' extvisiblespaceworld']}$$

This guarantees that tokenization is **strictly lossless and fully invertible**:

$$\texttt{decode}(\texttt{encode}(S)) \equiv S \quad \forall S \in \Sigma^*. \tag{4.92}$$

**Production implementations: OpenAI's tiktoken**

In modern production LLM engineering, tokenization must process terabytes of training data without creating CPU bottlenecks. OpenAI's open-source library `tiktoken` implements byte-level BPE in pure Rust, executing tokenization orders of magnitude faster than Python regular expressions:

- **GPT-2 Tokenizer** (`gpt2`): Vocabulary size $V = 50{,}257$ (composed of 256 base byte tokens +50,000 learned BPE merges +1 special `<|endoftext|>` token).
- **GPT-4 Tokenizer** (`cl100k_base`): Vocabulary size $V = 100{,}277$. Expanding vocabulary from 50k to 100k reduces the total token count of natural language and code by $15 - 20\%$, directly reducing sequence length $L$, decreasing GPU memory footprint, and slashing API inference costs.

```python
import tiktoken

# Instantiate the GPT-2 tokenizer (vocab size 50,257)
tokenizer = tiktoken.get_encoding("gpt2")

text = "Hello, do you like tea? <|endoftext|> In the sunlit terraces of
    someunknownPlace."
# Disallow special tokens check or explicitly allow specific ones
integers = tokenizer.encode(text, allowed_special={"<|endoftext|>"})
```

```
print("Token IDs:", integers)

# Zero-shot handling of unknown words:
# 'someunknownPlace' is decomposed into subwords: ['some', 'unknown', 'Place']
for token_id in integers:
    print(f"{token_id:>5} -> {tokenizer.decode([token_id]).r}")

# Lossless roundtrip reconstruction
decoded_text = tokenizer.decode(integers)
assert decoded_text == text, "Decoded text must match original string identically."
print("Lossless decoding confirmed.")
```

Listing 4.2: Byte-level BPE tokenization using OpenAI's `tiktoken` library [34].

Having established how Byte-Level BPE eliminates the out-of-vocabulary barrier and deterministically encodes raw text streams into discrete integer token IDs $\mathbf{T} = [t_0, t_1, \ldots, t_{L-1}] \in \mathbb{Z}^L$, we now confront the final representational bridge: how do discrete numerical indices enter the continuous, differentiable computational graph of modern neural networks? In the following subsections, we first establish the high-level architecture of the language model input pipeline, and then embark on an exhaustive investigation into positional representations.

### 4.11.5 From token IDs to composite inputs: high-level architecture of the foundation model input pipeline

Having transformed raw text into sequences of discrete token IDs $\mathbf{T} \in \mathbb{Z}^L$ (batched into mini-batch integer tensors $\mathbf{X} \in \mathbb{Z}^{B \times L}$), modern foundation models execute a unified input preparation pipeline that maps discrete token indices into continuous, differentiable vector representations equipped with sequential order prior to Transformer ingestion.

#### Delimiter and architectural control tokens

Before projecting indices into continuous embedding space, language models inject dedicated **special control tokens** into the sequence to regulate generation boundaries and downstream task execution:

- **Sequence demarcation tokens**:
  - ⟨BOS⟩ (*Beginning of Sequence*): Prepended to every sequence to prime autoregressive decoders, signaling the onset of generation.
  - ⟨EOS⟩ (*End of Sequence*): Appended to target text; during inference, emitting this token instructs the generation loop to terminate.
- **Architectural and task-specific delimiter tokens**:
  - [CLS] (*Classification token*): Introduced in bidirectional encoder architectures such as BERT. Prepend-ed at sequence index 0, its final contextual representation aggregates whole-sequence semantics for sentence classification or regression tasks.
  - [SEP] (*Separator token*): Delimits distinct segments within a single combined input sequence (for example, demarcating the boundary between a Question and an Answer, or between a code docstring and its method body), enabling the model to learn segment-specific interaction patterns.

### The token embedding lookup layer

In PyTorch, continuous token representations are instantiated via an embedding matrix:

```
token_embedding_layer = torch.nn.Embedding(num_embeddings=vocab_size, embedding_dim=d)
```

Passing batch tensor $\mathbf{X} \in \mathbb{Z}^{B \times L}$ retrieves the corresponding continuous embedding vectors:

$$\mathbf{E}_{\text{tok}} = \texttt{Embedding}(\mathbf{X}) \in \mathbb{R}^{B \times L \times d}, \tag{4.93}$$

where slice $(\mathbf{E}_{\text{tok}})_{b,t,:} = \mathbf{w}_{X_{b,t}} \in \mathbb{R}^d$ is the $d$-dimensional embedding vector for the token at position $t$ in batch item $b$.

### Composite Transformer input synthesis

To endow the model with sequential awareness, the input pipeline combines the lexical token representation $\mathbf{E}_{\text{tok}}$ and a dedicated positional tensor $\mathbf{E}_{\text{pos}} \in \mathbb{R}^{1 \times L \times d}$ via **elementwise addition** rather than concatenation:

$$\mathbf{X}_{\text{in}} = \mathbf{E}_{\text{tok}} + \mathbf{E}_{\text{pos}} \in \mathbb{R}^{B \times L \times d}, \tag{4.94}$$

where for each batch sample $b \in \{0, \ldots, B-1\}$ and temporal coordinate $pos \in \{0, \ldots, L-1\}$:

$$(\mathbf{X}_{\text{in}})_{b,pos,:} = \mathbf{w}_{X_{b,pos}} + \mathbf{p}_{pos} \in \mathbb{R}^d. \tag{4.95}$$

Listing 4.3 demonstrates how this composite synthesis is executed concretely in PyTorch using tensor broadcasting.

```
import torch

vocab_size = 50257     # GPT-2 BPE vocabulary size
output_dim = 256       # Latent embedding dimension d
context_length = 4     # Sequence length L

# 1. Allocate trainable token embedding layer
token_embedding_layer = torch.nn.Embedding(vocab_size, output_dim)

# 2. Allocate positional embedding layer (e.g., learned or precomputed sinusoidal)
pos_embedding_layer = torch.nn.Embedding(context_length, output_dim)

# Example batch of token IDs: [batch_size=2, context_length=4]
inputs = torch.tensor([[40, 367, 2885, 1464],
                       [1807, 3619, 402, 271]])

# Extract token embeddings: [2, 4, 256]
token_embeddings = token_embedding_layer(inputs)

# Generate position indices: tensor([0, 1, 2, 3]) -> [4, 256]
pos_indices = torch.arange(context_length)
pos_embeddings = pos_embedding_layer(pos_indices)

# 3. Additive composite synthesis with PyTorch tensor broadcasting
# token_embeddings: [2, 4, 256] + pos_embeddings: [4, 256] -> [2, 4, 256]
input_embeddings = token_embeddings + pos_embeddings

print("Token Embeddings Shape:", token_embeddings.shape)  # torch.Size([2, 4, 256])
print("Pos Embeddings Shape:  ", pos_embeddings.shape)    # torch.Size([4, 256])
print("Final Composite Shape: ", input_embeddings.shape)  # torch.Size([2, 4, 256])
```

Listing 4.3: PyTorch synthesis of token embeddings and positional embeddings [34].

While Equation (4.94) and Listing 4.3 summarize the high-level computational contract of the LM input pipeline, they raise foundational questions: *Why is the positional tensor* $\mathbf{E}_{pos}$ *strictly mandatory? Why do naive indexing schemes fail substantially? And what mathematical principles govern its construction?* We now dedicate the next subsection to an exhaustive investigation into positional representations.

### 4.11.6 Positional representations: intuition, mathematical formulation, and implementations

**Permutation invariance dilemma in self-attention**

In classical Recurrent Neural Networks (RNNs), sequential order is implicitly baked into the architecture: the hidden state transitions recurrently via $\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t)$, guaranteeing that token $\mathbf{x}_t$ is processed strictly after token $\mathbf{x}_{t-1}$. In Convolutional Neural Networks (CNNs), sequential order is preserved through localized spatial receptive fields.

In contrast, the cornerstone mathematical operator of the Transformer—**Self-Attention**—is strictly **permutation-equivariant**:

**Theorem 4.5:** *Permutation equivariance of self-attention*

Let $\mathbf{X} \in \mathbb{R}^{L\times d}$ be an input sequence of $L$ vectors, and let $\mathbf{P} \in \{0,1\}^{L\times L}$ be an arbitrary permutation matrix (a square matrix having exactly one entry 1 per row and column, satisfying $\mathbf{P}^\top = \mathbf{P}^{-1}$). Under standard dot-product self-attention:

$$\texttt{Attention}(\mathbf{X}) = \text{softmax}\left(\frac{\mathbf{X}\mathbf{W}_Q(\mathbf{X}\mathbf{W}_K)^\top}{\sqrt{d_k}}\right)\mathbf{X}\mathbf{W}_V, \tag{4.96}$$

the operator is equivariant to input permutation:

$$\texttt{Attention}(\mathbf{P}\mathbf{X}) = \mathbf{P}\,\texttt{Attention}(\mathbf{X}). \tag{4.97}$$

*Proof.* Let $\mathbf{X}' = \mathbf{P}\mathbf{X}$. The query and key projections evaluate to $\mathbf{Q}' = \mathbf{P}\mathbf{X}\mathbf{W}_Q$ and $\mathbf{K}' = \mathbf{P}\mathbf{X}\mathbf{W}_K$. The unscaled attention score matrix becomes:

$$\mathbf{Q}'(\mathbf{K}')^\top = (\mathbf{P}\mathbf{X}\mathbf{W}_Q)(\mathbf{P}\mathbf{X}\mathbf{W}_K)^\top = \mathbf{P}(\mathbf{X}\mathbf{W}_Q\mathbf{W}_K^\top\mathbf{X}^\top)\mathbf{P}^\top. \tag{4.98}$$

Because the Softmax function is evaluated independently along each row, applying a permutation matrix $\mathbf{P}$ on the left permutes the rows, while $\mathbf{P}^\top$ on the right permutes the columns:

$$\text{softmax}\left(\frac{\mathbf{P}(\mathbf{X}\mathbf{W}_Q\mathbf{W}_K^\top\mathbf{X}^\top)\mathbf{P}^\top}{\sqrt{d_k}}\right) = \mathbf{P}\,\text{softmax}\left(\frac{\mathbf{X}\mathbf{W}_Q\mathbf{W}_K^\top\mathbf{X}^\top}{\sqrt{d_k}}\right)\mathbf{P}^\top. \tag{4.99}$$

Finally, multiplying by the value projection $\mathbf{V}' = \mathbf{P}\mathbf{X}\mathbf{W}_V$:

$$\begin{aligned}\texttt{Attention}(\mathbf{P}\mathbf{X}) &= \left[\mathbf{P}\,\text{softmax}\left(\frac{\mathbf{X}\mathbf{W}_Q\mathbf{W}_K^\top\mathbf{X}^\top}{\sqrt{d_k}}\right)\mathbf{P}^\top\right](\mathbf{P}\mathbf{X}\mathbf{W}_V)\\ &= \mathbf{P}\,\text{softmax}\left(\frac{\mathbf{X}\mathbf{W}_Q\mathbf{W}_K^\top\mathbf{X}^\top}{\sqrt{d_k}}\right)(\mathbf{P}^\top\mathbf{P})\mathbf{X}\mathbf{W}_V\\ &= \mathbf{P}\,\texttt{Attention}(\mathbf{X}),\end{aligned} \tag{4.100}$$

where the identity $\mathbf{P}^\top\mathbf{P} = \mathbf{I}$ collapses the internal permutation. ■ □

Theorem 4.5 establishes that if we permute the input tokens, the output vectors undergo the exact same permutation with *identical values*. The attention mechanism treats the input sequence as an **unordered multiset** (a "bag of vectors") with zero intrinsic knowledge of temporal order.

To understand the substantial consequence of this property, consider two contrasting sentence pairs:

$$\begin{aligned} \textbf{Sentence 1}\text{: } \textit{"I ate an apple"} &\implies \{\mathbf{e}_{\text{I}}, \mathbf{e}_{\text{ate}}, \mathbf{e}_{\text{an}}, \mathbf{e}_{\text{apple}}\} \\ \textbf{Sentence 2}\text{: } \textit{"An apple, I ate"} &\implies \{\mathbf{e}_{\text{an}}, \mathbf{e}_{\text{apple}}, \mathbf{e}_{\text{I}}, \mathbf{e}_{\text{ate}}\} \end{aligned}$$

Or similarly:

$$\begin{aligned} \textbf{Sentence A}\text{: } \textit{"The dog bit the man"} &\implies \{\mathbf{e}_{\text{The}}, \mathbf{e}_{\text{dog}}, \mathbf{e}_{\text{bit}}, \mathbf{e}_{\text{the}}, \mathbf{e}_{\text{man}}\} \\ \textbf{Sentence B}\text{: } \textit{"The man bit the dog"} &\implies \{\mathbf{e}_{\text{The}}, \mathbf{e}_{\text{man}}, \mathbf{e}_{\text{bit}}, \mathbf{e}_{\text{the}}, \mathbf{e}_{\text{dog}}\} \end{aligned}$$

In both pairs, the sentences express fundamentally different or opposite semantic realities. Yet, because each pair comprises the exact same multiset of token embeddings, an unaugmented Transformer evaluates identical attention weights. To the model, subject, verb, and object become entirely conflated. Sequential order must therefore be injected into the representations explicitly.

### Why naive positional indexing fails

Before arriving at the modern formulation, it is vital to understand why naive indexing schemes break down:

1. **Naive Integer Addition ($pos \in \{0, 1, 2, \ldots, L-1\}$)**: If we add scalar indices $pos$ to token embeddings, early tokens receive tiny perturbations (+0, +1), while token 2048 receives +2048. Because embedding vectors are initialized with mean 0 and unit variance, large coordinates overwhelm the semantic representations completely, and models fail to generalize to sequence lengths beyond training.

2. **Normalized Coordinates ($pos/L \in [0, 1]$)**: Normalizing positions by sequence length bounds values within $[0, 1]$. However, the physical meaning of a step size becomes variable: in a 10-token sentence, an offset of $\Delta = 0.1$ corresponds to an adjacent word, whereas in a 1000-token document, $\Delta = 0.1$ spans 100 words. The model cannot learn consistent relative distances.

3. **Coordinate Concatenation ($[\mathbf{w}_t; \mathbf{p}_t]$)**: Concatenating positional coordinates expands embedding dimensionality to $d + d_{\text{pos}}$, increasing parameter counts across all subsequent projection matrices ($\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V, \mathbf{W}_O$) and preventing seamless parameter sharing.

### The geometric clock intuition for positional encoding

How can we construct an encoding that remains strictly bounded within $[-1, 1]$, assigns a unique signature to every position, and preserves consistent geometric notions of distance?

*From binary counting to continuous multi-scale clocks.*

To build geometric intuition, consider how a digital computer represents numbers using bounded bits $\{0, 1\}$. In a 3-bit binary counter:

- The lowest bit (Bit 0) alternates every single step: $0, 1, 0, 1, 0, 1 \ldots$ (high-frequency wave).
- The middle bit (Bit 1) alternates every two steps: $0, 0, 1, 1, 0, 0 \ldots$ (medium-frequency wave).
- The highest bit (Bit 2) alternates every four steps: $0, 0, 0, 0, 1, 1 \ldots$ (low-frequency wave).

Every integer receives an unambiguous signature, even though individual bits are bounded in $\{0, 1\}$. However, discrete step functions are non-differentiable. Sinusoidal positional encoding operates as the smooth, continuous, differentiable generalization of binary counting.

Rather than binary square waves, we assign continuous trigonometric waves oscillating at geometrically decreasing frequencies across embedding channels:

- **Fast Clocks (Seconds Hand)**: Oscillate rapidly across low dimension indices, capturing fine-grained local syntax and adjacent word order.
- **Medium Clocks (Minutes Hand)**: Oscillate at intermediate frequencies, capturing phrase and clause boundaries.
- **Slow Clocks (Hours Hand)**: Oscillate extremely slowly across high dimension indices, capturing document-level flow and paragraphs.

Why can we not simply use sine waves with varying frequencies? Consider an analog clock where you can only observe the vertical height of the hand ($Y = \sin\theta$). Because $\sin(\theta) = \sin(\pi - \theta)$, the hand is at the exact same vertical height at 2:00 and 10:00. Furthermore, $\sin(0) = \sin(\pi) = 0$. A single 1D wave suffers from severe directional ambiguity (aliasing).

To establish an unambiguous position, we need both coordinates of the clock face:

$$(X, Y) = (\cos\theta, \sin\theta). \tag{4.101}$$

Together, $(\sin\theta, \cos\theta)$ traces a point along the unit circle $S^1$. On a 2D circle, every angular orientation $\theta \in [0, 2\pi)$ has an unambiguous coordinate. This is why Transformer positional encodings alternate sin and cos across dimension pairs.

Figure 4.8 illustrates this multi-frequency clock mechanism and demonstrates how pairing sine and cosine eliminates 1D wave aliasing.

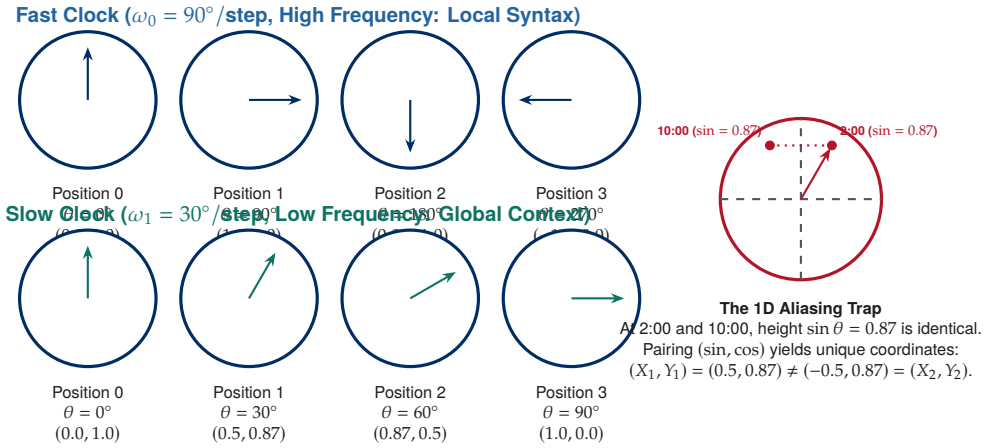



Figure 4.8: The geometric clock intuition behind positional encodings. Each dimension pair acts as an analog clock hand rotating at an angular frequency $\omega_i$. Fast clocks disambiguate immediate neighbors (local syntax), while slow clocks preserve long-range sequence context. Pairing sin and cos prevents 1D wave aliasing by establishing a unique 2D coordinate on the unit circle $S^1$.

### Taxonomy of positional representations

Modern architectures implement positional awareness through three primary paradigms:

1. **Fixed Sinusoidal Positional Encodings [1]**: Fully deterministic, parameter-free functions computed from geometric progressions of trigonometric frequencies.

2. **Learned Absolute Positional Embeddings [46, 51, 52]**: Trainable parameter weights stored in an embedding matrix $\mathbf{W}_{\text{pos}} \in \mathbb{R}^{L_{\max} \times d}$ optimized via backpropagation (used in GPT-2, GPT-3, and BERT).
3. **Relative and Rotary Positional Embeddings (RoPE / ALiBi) [53–55]**: Encodings that inject positional information directly into attention scores as a function of relative distance $|i - j|$. Notably, Rotary Position Embedding (RoPE) [54] directly generalizes the 2D rotation matrix property of sinusoidal encodings into query and key representations.

### Sinusoidal positional encodings mathematical derivation

In the seminal architecture introduced in [1], positional coordinates are generated deterministically:

**Definition 4.6:** *Sinusoidal positional encoding*

For a token at temporal sequence coordinate $pos \in \{0, 1, \ldots, L-1\}$ and channel index $i \in \{0, 1, \ldots, \frac{d}{2} - 1\}$, the components of the positional vector $\mathbf{p}_{pos} = \mathsf{PE}_{(pos,:)} \in \mathbb{R}^d$ are defined by:

$$\mathsf{PE}_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right), \tag{4.102}$$

$$\mathsf{PE}_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right). \tag{4.103}$$

Let us examine every term in this formulation:

- $pos$: The integer position of the token within the sequence $(0, 1, 2, \ldots, L-1)$.
- $i$: The frequency channel index $(0 \leq i < d/2)$.
- $2i$ and $2i + 1$: Even indices receive sine values, odd indices receive cosine values.
- Angular frequency $\omega_i$: Defined as $\omega_i = \frac{1}{10000^{2i/d}} = 10000^{-2i/d}$.
- Wavelength $\lambda_i$: The wavelength $\lambda_i = \frac{2\pi}{\omega_i} = 2\pi \cdot 10000^{2i/d}$ forms a geometric progression:

$$\text{At } i = 0: \quad \lambda_0 = 2\pi \cdot 10000^0 = 2\pi \approx 6.28 \text{ tokens}, \tag{4.104}$$

$$\text{At } i = \frac{d}{2} - 1: \quad \lambda_{\max} \approx 2\pi \cdot 10000^1 \approx 62{,}832 \text{ tokens}. \tag{4.105}$$

- **Why the base 10000?** As highlighted by the professor, 10,000 guarantees that within typical context lengths (up to $\approx 60{,}000$ tokens), the slowest frequency does not complete a single cycle. This guarantees that every position vector in the operational horizon is globally unique.

Sinusoidal positional encodings satisfy three pivotal theoretical properties:

1. **Positional uniqueness**: Because each channel $i$ oscillates at a distinct frequency $\omega_i$, no two positional vectors are identical. The vectors remain distinct for at least $2\pi \cdot 10000 \approx 62{,}832$ positions before periodic recurrence can occur.
2. **The linear relative shift property**: For any fixed spatial offset $k \in \mathbb{Z}$, the positional vector at position $pos + k$ can be computed as a direct linear transformation of the vector at position $pos$:

**Theorem 4.6:** *Linear relative transformation property*

For any frequency $\omega_i = 10000^{-2i/d}$ and spatial offset $k \in \mathbb{Z}$, there exists an orthogonal rotation matrix $\mathbf{M}_{i,k} \in \mathbb{R}^{2\times 2}$ independent of $pos$ such that:

$$\begin{bmatrix} \mathsf{PE}_{(pos+k,2i)} \\ \mathsf{PE}_{(pos+k,2i+1)} \end{bmatrix} = \begin{bmatrix} \cos(\omega_i k) & \sin(\omega_i k) \\ -\sin(\omega_i k) & \cos(\omega_i k) \end{bmatrix} \begin{bmatrix} \mathsf{PE}_{(pos,2i)} \\ \mathsf{PE}_{(pos,2i+1)} \end{bmatrix}. \tag{4.106}$$

*Proof.* Applying the standard trigonometric angle addition identities:

$$\sin(\omega_i(pos+k)) = \sin(\omega_i pos)\cos(\omega_i k) + \cos(\omega_i pos)\sin(\omega_i k), \tag{4.107}$$
$$\cos(\omega_i(pos+k)) = \cos(\omega_i pos)\cos(\omega_i k) - \sin(\omega_i pos)\sin(\omega_i k). \tag{4.108}$$

Expressing this system in matrix notation yields Equation (4.106). Because the rotation matrix $\mathbf{M}_{i,k}$ depends strictly on offset $k$ and is independent of absolute position $pos$, linear attention projection layers ($\mathbf{W}_Q, \mathbf{W}_K$) can easily learn to attend to relative positions. ■ □

3. **Metric similarity and distance decay**: In self-attention, query and key vectors interact via inner products. The dot product between the 2D positional components of two tokens at positions $p_1$ and $p_2$ evaluates to:

$$\mathbf{p}_{p_1}^{(i)} \cdot \mathbf{p}_{p_2}^{(i)} = \sin(\omega_i p_1)\sin(\omega_i p_2) + \cos(\omega_i p_1)\cos(\omega_i p_2) = \cos(\omega_i(p_1 - p_2)). \tag{4.109}$$

The dot product depends **strictly on the relative distance** $\Delta p = p_1 - p_2$:

- When $p_1 = p_2$, $\cos(0) = 1$ (maximum correlation along the diagonal).
- As $|p_1 - p_2|$ increases, the similarity smoothly decays.

Computing the complete pairwise cosine similarity matrix across 500 positions demonstrates this property empirically:

- The 2D Gram matrix exhibits a bright, luminous main diagonal (cos = 1).
- A horizontal slice along row $pos = 100$ reveals a sharp symmetric peak at index 100, followed by a smooth, monotonic decay as distance $|pos - 100|$ expands.

**Numerical calculation of sinusoidal coordinates**

To illustrate this formulation concretely, consider a canonical numerical configuration: sequence position $pos = 1$ (the second token, under 0-indexed coordinates) within a standard latent embedding space of dimension $d = 768$:

1. **Lowest channel pair ($i = 0$, high-frequency regime)**:

$$2i = 0 \implies \mathsf{PE}_{(1,0)} = \sin\left(\frac{1}{10000^{0/768}}\right) = \sin(1) \approx 0.8415, \tag{4.110}$$
$$2i + 1 = 1 \implies \mathsf{PE}_{(1,1)} = \cos\left(\frac{1}{10000^{0/768}}\right) = \cos(1) \approx 0.5403. \tag{4.111}$$

2. **Subsequent even channel ($i = 2$, channel index $2i = 4$)**:

$$2i = 4 \implies \mathsf{PE}_{(1,4)} = \sin\left(\frac{1}{10000^{4/768}}\right) = \sin\left(\frac{1}{10000^{0.005208}}\right)$$
$$\approx \sin\left(\frac{1}{1.0490}\right) = \sin(0.9533) \approx 0.8153. \tag{4.112}$$

These exact values (0.8415, 0.5403, and 0.8153) illustrate how trigonometric arguments shift smoothly across embedding indices, providing each positional coordinate with an unambiguous, deterministic signature.

### Learned positional embeddings (GPT convention)

An alternative approach, adopted by architectures such as GPT-2 and GPT-3 [2, 46], treats positional encodings as trainable parameter weights rather than fixed sinusoidal functions:

$$\mathbf{W}_{\text{pos}} \in \mathbb{R}^{L_{\max} \times d}. \tag{4.113}$$

Each integer coordinate $p \in \{0, 1, \ldots, L_{\max} - 1\}$ indexes a row vector $\mathbf{p}_p = (\mathbf{W}_{\text{pos}})_{p,:} \in \mathbb{R}^d$ optimized via gradient descent alongside token embeddings.

Table 4.3: Architectural comparison: sinusoidal positional encodings vs. learned positional embeddings.

| Property | Sinusoidal encodings [1] | Learned embeddings [46] |
|---|---|---|
| **Parameters** | 0 trainable weights (fully deterministic) | $L_{\max} \times d$ trainable parameters |
| **Extrapolation** | Can theoretically extrapolate to sequence lengths $> L_{\max}$ | Strictly confined to context length $L \leq L_{\max}$ |
| **Relative shift** | Exact orthogonal rotation $\mathbf{M}_{i,k}$ provable analytically | Must be approximated empirically from data |
| **Empirical similarity** | Pure smooth monotonic distance decay around diagonal | Smooth decay near diagonal with data-driven periodic peaks |
| **Empirical performance** | Comparable translation BLEU in [1] | Standard in modern autoregressive decoders |

Computing the cosine similarity matrix of learned GPT-2 embeddings reveals high values along the main diagonal, accompanied by periodic secondary peaks at larger distances. These secondary peaks reflect statistical regularities in the training corpus (such as paragraph transitions, section breaks, or dialogue conventions).

### Walkthrough: disambiguating opposite sequential orders

To crystallize all of these principles into an intuitive interpretation, let us trace how a mini 2-clock system disambiguates our contrasting sentences:

**Sentence 1**: *"The dog bit the man."* vs. **Sentence 2**: *"The man bit the dog."*

Let $d = 4$ (consisting of 2 clocks):

- **Clock 1 (Fast Clock, $\omega_0 = 90°$/step)**: Rotates 90° clockwise per step.
- **Clock 2 (Slow Clock, $\omega_1 = 30°$/step)**: Rotates 30° clockwise per step.

Evaluating the positional vectors $\mathbf{PE}(pos) = [\sin(\omega_0 pos), \cos(\omega_0 pos), \sin(\omega_1 pos), \cos(\omega_1 pos)]$ yields:

Now consider the additive inputs:

- In **Sentence 1**: $\mathbf{x}_{\text{dog}} = \mathbf{E}_{\text{dog}} + \mathbf{p}_1$, $\mathbf{x}_{\text{bit}} = \mathbf{E}_{\text{bit}} + \mathbf{p}_2$, $\mathbf{x}_{\text{man}} = \mathbf{E}_{\text{man}} + \mathbf{p}_4$.
- In **Sentence 2**: $\mathbf{x}_{\text{man}} = \mathbf{E}_{\text{man}} + \mathbf{p}_1$, $\mathbf{x}_{\text{bit}} = \mathbf{E}_{\text{bit}} + \mathbf{p}_2$, $\mathbf{x}_{\text{dog}} = \mathbf{E}_{\text{dog}} + \mathbf{p}_4$.

Table 4.4: Exact numerical positional vectors for positions 0 through 4 under the 2-clock system.

| **Position ($pos$)** | **Clock 1 ($\omega_0 = 90° \cdot pos$)** | **Clock 2 ($\omega_1 = 30° \cdot pos$)** | **Positional Vector $\mathbf{p}_{pos}$** |
|---|---|---|---|
| 0 | $\theta = 0° \implies [0.0, 1.0]$ | $\theta = 0° \implies [0.0, 1.0]$ | $[0.0, 1.0, 0.0, 1.0]$ |
| 1 | $\theta = 90° \implies [1.0, 0.0]$ | $\theta = 30° \implies [0.5, 0.87]$ | $[1.0, 0.0, 0.5, 0.87]$ |
| 2 | $\theta = 180° \implies [0.0, -1.0]$ | $\theta = 60° \implies [0.87, 0.5]$ | $[0.0, -1.0, 0.87, 0.5]$ |
| 3 | $\theta = 270° \implies [-1.0, 0.0]$ | $\theta = 90° \implies [1.0, 0.0]$ | $[-1.0, 0.0, 1.0, 0.0]$ |
| 4 | $\theta = 360° \implies [0.0, 1.0]$ | $\theta = 120° \implies [0.87, -0.5]$ | $[0.0, 1.0, 0.87, -0.5]$ |

In English syntax, the grammatical subject of the active verb "bit" resides 1 step prior (relative offset $k = -1$). At position $pos = 2$, Clock 1 points South (180°, vector $[0.0, -1.0]$). To identify the doer of the action, an attention head querying offset $k = -1$ rotates 90° counterclockwise:

$$180° - 90° = 90° \implies [1.0, 0.0]. \tag{4.114}$$

The attention query searches for the token whose Clock 1 points East ($[1.0, 0.0]$):

- In **Sentence 1**, the token at position 1 with East-pointing clock is **"dog"**. The attention mechanism routes information from "dog" to "bit", resolving the dog as the subject.
- In **Sentence 2**, the token at position 1 with East-pointing clock is **"man"**. The attention mechanism routes information from "man" to "bit", resolving the man as the subject.

The ambiguity is completely resolved: through sinusoidal clock rotations, the Transformer deciphers who bit whom.

CHAPTER 5

# The Transformer architecture and attention mechanisms

Chapter 4 described how tokenization and embedding layers map discrete input sequences to vectors with positional information. Chapter 2 showed how recurrent networks process these vectors by updating a hidden state over time.

Recurrence imposes a sequential dependency: computing $\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t)$ requires the state from the preceding position, which limits parallel computation across a sequence. In encoder-decoder networks, compressing the input into a fixed-size state can also restrict the information available to the decoder. Attention addresses the latter limitation by allowing representations at one position to use information from other positions; the Transformer applies attention without a recurrent state update.

Vaswani et al. introduced the Transformer in 2017, using attention in place of recurrence [1]. This chapter derives scaled dot-product attention and examines bidirectional encoder self-attention, causal decoder self-attention, and cross-attention. It then develops multi-head projections, residual connections, Pre-LN and Post-LN normalization, and relative positional representations. The discussion also compares encoder-decoder models such as T5, encoder-only models such as BERT, and decoder-only autoregressive models, before treating sampling methods for sequence generation.

## 5.1 The Self-Attention Primitive

For nearly a decade, sequence modeling was dominated by Recurrent Neural Networks (RNNs) and their gated variants, Long Short-Term Memory (LSTM) networks and Gated Recurrent Units (GRUs). As established in Chapter 2, recurrent models represent a sequence of tokens by recursively updating an internal hidden state vector:

$$\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t), \quad t \in \{1, 2, \dots, L\}, \tag{5.1}$$

where $\mathbf{h}_t \in \mathbb{R}^{d_h}$ encapsulates the historical context observed up to time step $t$, and $\mathbf{x}_t \in \mathbb{R}^d$ is the input vector representation at time $t$.

While gating mechanisms and Constant Error Carousels mitigated the vanishing gradient problem over moderate temporal spans, recurrence inherently imposes two severe, prohibitive bottlenecks that halted further scaling in language modeling:

#### The sequential execution wall

The transition equation $\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t)$ enforces an unbroken, strictly sequential causal dependency chain: computation of $\mathbf{h}_t$ cannot commence until the physical memory write of $\mathbf{h}_{t-1}$ is completed. Consequently, an input sequence of $L$ tokens mandates $O(L)$ sequential computational operations. Modern deep learning accelerators (GPUs and TPUs) derive their performance from massive data parallelism, executing thousands of arithmetic matrix operations concurrently across high-bandwidth tensor cores. Under recurrent processing, hardware parallelism across the temporal dimension is impossible: the accelerator is forced to idle between consecutive time steps, creating a computational wall that restricts training across massive datasets.

#### The information compression bottleneck of Seq2Seq encoders

In classical sequence-to-sequence architectures, an encoder compresses an input sequence of arbitrary length $L$ into a single, fixed-size vector $\mathbf{h}_L \in \mathbb{R}^{d_h}$ (or a tuple $(\mathbf{h}_L, \mathbf{c}_L)$ in LSTMs). This single vector is subsequently handed over to the decoder to initiate target sequence generation. As the input length $L$ expands—such as in multi-turn dialogues, technical documentation, or software engineering source code files spanning thousands of tokens—compressing all semantic, syntactic, and structural relationships into a finite-dimensional vector becomes physically impossible. Information from early time steps undergoes continuous dissipation, inducing severe contextual amnesia.

These limitations raised a fundamental question: *Can we design a neural architecture that completely eliminates recurrence, operates across all tokens in parallel with $O(1)$ sequential operations, and allows every token to directly communicate with every other token in the sequence without a fixed-size information bottleneck?* In 2017, this question was answered by the introduction of the **Transformer** [1], an architecture built entirely upon multi-head attention mechanisms.

## 5.2 Transformer Architecture Overview

The foundational Transformer proposed in [1] adopts an **Encoder-Decoder** paradigm, structured to eliminate the recurrent compression bottleneck while enabling end-to-end parallel training. Figure 5.1 reproduces the canonical architectural diagram from the landmark paper [1].

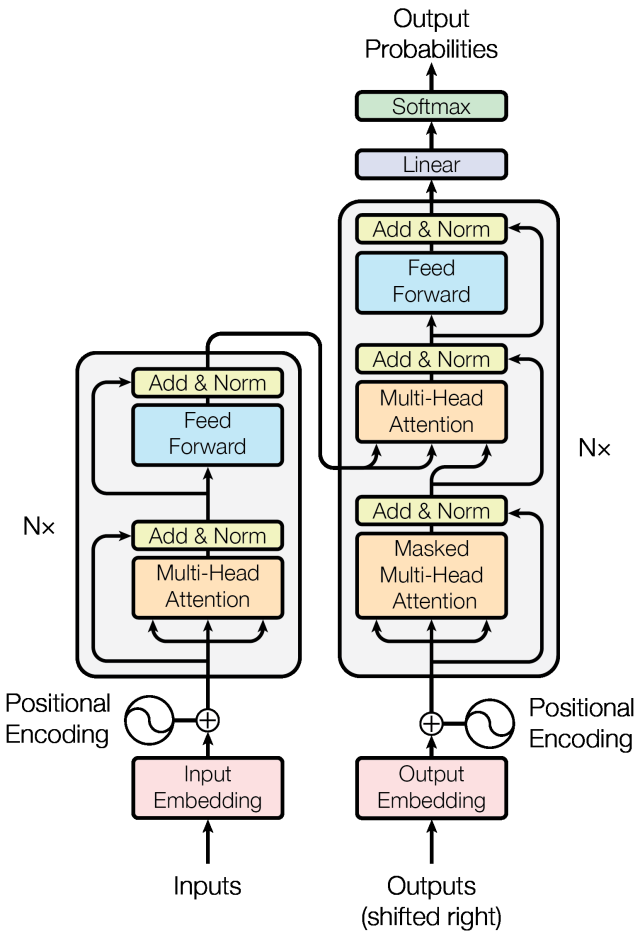



Figure 5.1: The original Transformer model architecture as published in [1]. The left stack corresponds to the Encoder ($N = 6$ identical layers), and the right stack corresponds to the Decoder ($N = 6$ identical layers), equipped with causal masked self-attention and cross-attention over the encoder representations.

### The Encoder stack: uncompressed contextual representations

The Encoder consists of a stack of $N$ identical layers (typically $N = 6$ in the base model). Crucially, the Encoder does *not* compress the input sequence into a single summary vector. Instead, it takes the input sequence of position-augmented embeddings $\mathbf{X} \in \mathbb{R}^{L_{\text{enc}} \times d_{\text{model}}}$ and transforms it into an equally sized sequence of contextual representations:

$$\mathbf{H}_{\text{enc}} = [\mathbf{h}_1, \mathbf{h}_2, \ldots, \mathbf{h}_{L_{\text{enc}}}] \in \mathbb{R}^{L_{\text{enc}} \times d_{\text{model}}}. \tag{5.2}$$

Every output vector $\mathbf{h}_i$ possesses a global receptive field that incorporates information from every other token in the sequence through bidirectional self-attention, preserving fine-grained token-level semantics without information loss.

### The Decoder stack: autoregressive generation and cross-attention

The Decoder also consists of a stack of $N$ identical layers, operating autoregressively at inference time:

1. **Autoregressive sequence initialization**: Generation is primed by feeding the special beginning-of-sequence delimiter ⟨BOS⟩ as the initial token.
2. **Causal step-by-step prediction**: The Decoder processes the tokens generated so far, attends to the encoder representations $\mathbf{H}_{\text{enc}}$, and emits logits over vocabulary $\mathcal{V}$. The highest-scoring token (or sampled token) $y_1$ is generated.
3. **Prefix feedback loop**: The newly emitted token is appended to the decoder input, forming the context $[\langle\text{BOS}\rangle, y_1]$ to predict $y_2$. This loop repeats until the model outputs the end-of-sequence delimiter ⟨EOS⟩, terminating generation.

### The training phase: Teacher Forcing and sequence parallelization

At inference, the decoder must generate tokens sequentially. However, during training, the entire target sequence $[y_0, y_1, \ldots, y_{L_{\text{dec}}-1}]$ is already known. By employing **Teacher Forcing** combined with causal attention masking (which prevents tokens from attending to subsequent target tokens), all $L_{\text{dec}}$ generation steps are computed simultaneously in a single forward pass. The decoder does not unroll an iterative loop during training: the loss across all output positions is evaluated concurrently on the GPU.

## 5.3 Attention Principles

### 5.3.1 Why we need the attention mechanism: from RNN bottlenecks to dynamic interaction

To understand the conceptual necessity of the attention mechanism, we must examine the architectural bottleneck that afflicted sequential models prior to the Transformer, as formalized by Sebastian Raschka [34].

When translating natural language text (e.g., from German or Italian to English) or performing code generation from formal specifications, literal word-by-word substitution is fundamentally impossible. Grammatical structures, morphological agreements, and clause ordering diverge substantially between linguistic systems. Consequently, sequence transduction requires a deep model consisting of two interacting submodules: an **Encoder** that ingests and understands the source sequence, and a **Decoder** that synthesizes the target sequence.

Prior to Transformers, the dominant paradigm for sequence-to-sequence modeling was the recurrent neural network (RNN, Chapter 2). In an encoder-decoder RNN:

1. The encoder ingests source tokens $\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_L$ sequentially, updating its recurrent hidden state at each time step via $\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t)$.

2. At the conclusion of the source sequence, the encoder attempts to compress the entire semantic and syntactic content of the input into its final hidden state vector $\mathbf{h}_L \in \mathbb{R}^{d_h}$.

3. This single vector $\mathbf{h}_L$ is handed over to the decoder as its initial context vector $\mathbf{c} = \mathbf{h}_L$, from which the decoder must generate the translated output token by token.

This architecture suffers from a substantial mathematical limitation: the **information compression bottleneck** [34]. Forcing an arbitrary-length sentence—spanning tens, hundreds, or thousands of tokens—into a single fixed-dimensional embedding vector creates an information-theoretic bottleneck. As sequence length expands:

- The decoder possesses *zero direct access* to intermediate representations $\mathbf{h}_1, \mathbf{h}_2, \ldots, \mathbf{h}_{L-1}$ from earlier encoder steps.
- Gradients backpropagating through time undergo exponential decay or explosion across recurrent chain products $\prod_{k=t+1}^{L} \frac{\partial \mathbf{h}_k}{\partial \mathbf{h}_{k-1}}$.
- Information from early tokens undergoes continuous dissipative dilution, causing severe context amnesia on long sequences.

To solve this dilemma, Bahdanau et al. [26] introduced the attention mechanism for RNNs, providing the decoder with dynamic, direct connections to *all* intermediate encoder states $\mathbf{h}_1, \ldots, \mathbf{h}_L$ at every generation step. Three years later, Vaswani et al. [1] realized that recurrence itself was entirely superfluous. By introducing **Self-Attention**, the model allows every position within a sequence to directly attend to and dynamically interact with every other position in the sequence across an $O(1)$ sequential path length, computing rich contextual embeddings where each word's representation is dynamically updated based on the relevant context of the entire sentence [34].

### 5.3.2 The associative memory analogy: queries, keys, and values

To understand how attention operates mathematically from first principles, it is instructive to draw an analogy to associative retrieval structures in computer science [29, 56].

**Classical dictionary lookup (discrete hash table)**

In computer systems, a dictionary or associative array stores information as a collection of $m$ discrete key-value pairs:

$$\mathcal{D} = \{(\kappa_1, v_1), (\kappa_2, v_2), \ldots, (\kappa_m, v_m)\}. \tag{5.3}$$

When a discrete search query $q$ is issued, the dictionary performs an exact binary match against the keys:

$$\texttt{Lookup}(q, \mathcal{D}) = \begin{cases} v_k & \text{if } \exists\, k \text{ such that } q = \kappa_k, \\ \texttt{None} & \text{otherwise.} \end{cases} \tag{5.4}$$

In this classical paradigm, retrieval is binary (0 or 1) and non-differentiable: if query $q$ is slightly perturbed, the lookup either matches completely or fails substantially.

**Soft, differentiable key-value retrieval**

Attention generalizes dictionary lookup to continuous metric vector spaces $\mathbb{R}^d$:

- A **Query** vector $\mathbf{q} \in \mathbb{R}^{d_k}$ represents the information need: *"What linguistic, syntactic, or semantic attributes am I searching for?"*
- A set of $m$ **Key** vectors $\mathbf{k}_j \in \mathbb{R}^{d_k}$ represent content identifiers or addresses: *"What attributes do I offer?"*
- A set of $m$ **Value** vectors $\mathbf{v}_j \in \mathbb{R}^{d_v}$ represent the informative content or payload: *"What information do I communicate if selected?"*

Instead of checking exact equality $q = \kappa_k$, we evaluate a continuous compatibility score $s(\mathbf{q}, \mathbf{k}_j) \in \mathbb{R}$ using vector inner products:

$$s_j = \mathbf{q}^\top \mathbf{k}_j. \tag{5.5}$$

The raw alignment scores $[s_1, s_2, \ldots, s_m]$ are mapped onto the probability simplex via the softmax operator, yielding non-negative attention weights that sum to unity:

$$\alpha_j = \frac{\exp(s_j)}{\sum_{l=1}^{m} \exp(s_l)}, \quad \sum_{j=1}^{m} \alpha_j = 1, \quad \alpha_j \geq 0. \tag{5.6}$$

Finally, the retrieved representation is computed as the **convex combination** (weighted sum) of all value vectors:

$$\mathbf{o} = \sum_{j=1}^{m} \alpha_j \mathbf{v}_j \in \mathbb{R}^{d_v}. \tag{5.7}$$

Because inner products, exponentials, and summations are smooth operations, the entire lookup pipeline is fully differentiable with well-defined gradients $\frac{\partial \mathbf{o}}{\partial \mathbf{q}}$, $\frac{\partial \mathbf{o}}{\partial \mathbf{k}_j}$, and $\frac{\partial \mathbf{o}}{\partial \mathbf{v}_j}$.

**Illustrative example: contextual displacement of a fluffy blue creature**

To understand how soft key-value retrieval transforms static representations into rich contextual embeddings in high-dimensional vector spaces, consider the visual intuition formalized by Grant Sanderson [56] and illustrated in Figure 5.2:

1. **The static embedding**: Initially, each word in the phrase *"A fluffy blue creature roamed the verdant forest"* begins as an isolated unigram embedding vector $\mathbf{e} \in \mathbb{R}^{d_{\text{model}}}$. The static vector for *"creature"* merely occupies a generic semantic coordinate indicating an animal or organism, completely devoid of context regarding its visual appearance.

2. **Queries as informational requests**: When passed into an attention block, linear projection $\mathbf{W}_Q$ maps the vector for *"creature"* into a Query vector $\mathbf{q}_{\text{creature}}$. Geometrically, this query acts as a directional probe asking: *"What descriptive adjectives precede me?"*

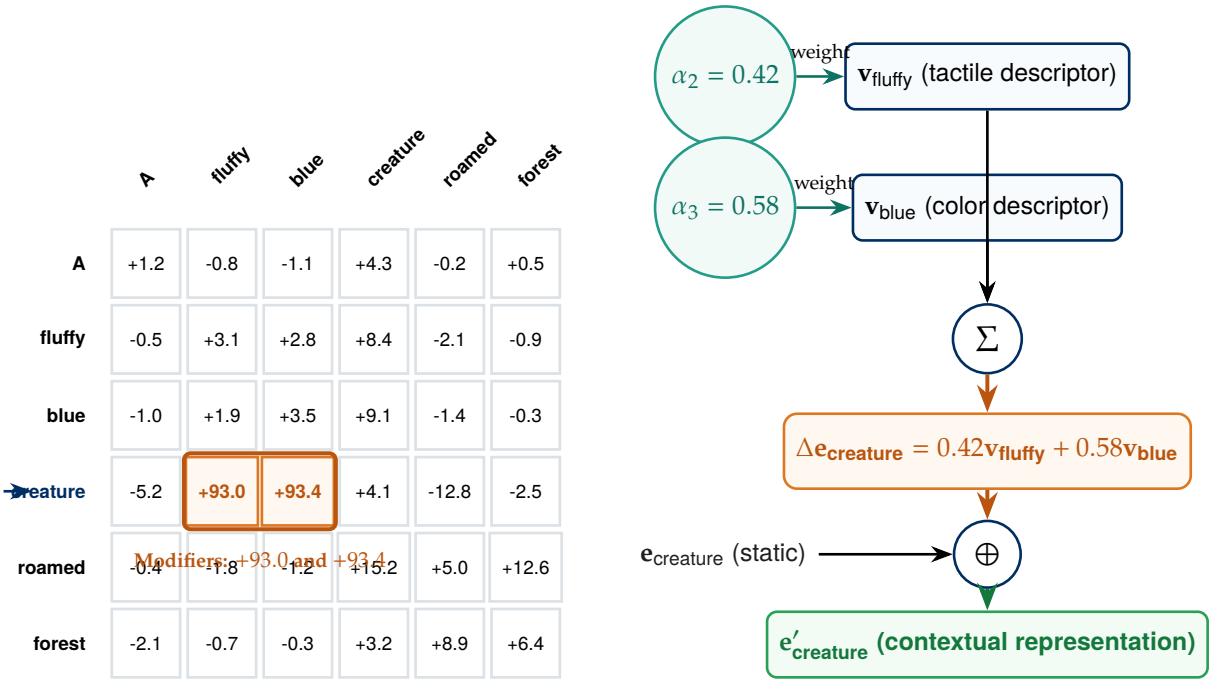
3. **Keys as feature advertisements**: Concurrently, projection matrix $\mathbf{W}_K$ maps all tokens into Key vectors. The preceding adjectives *"fluffy"* and *"blue"* produce keys $\mathbf{k}_{\text{fluffy}}$ and $\mathbf{k}_{\text{blue}}$ advertising: *"I am a tactile descriptor"* and *"I am a visual color descriptor"*.

4. **Inner-product compatibility**: Computing inner products $\mathbf{q}_{\text{creature}}^{\top}\mathbf{k}_j$ evaluates geometric alignment (Figure 5.2a). Because the query explicitly seeks adjective descriptors, it achieves high positive inner products with $\mathbf{k}_{\text{fluffy}}$ (+93.0) and $\mathbf{k}_{\text{blue}}$ (+93.4), but low or negative alignment with irrelevant tokens like *"roamed"* or *"forest"*.

5. **Value routing and displacement**: Linear projection $\mathbf{W}_V$ extracts the informative payload: Value vectors $\mathbf{v}_{\text{fluffy}}$ and $\mathbf{v}_{\text{blue}}$ encode specific sensory features. Taking their weighted sum produces a contextual displacement vector (Figure 5.2b):

$$\Delta\mathbf{e}_{\text{creature}} = \alpha_{\text{fluffy}}\mathbf{v}_{\text{fluffy}} + \alpha_{\text{blue}}\mathbf{v}_{\text{blue}}. \tag{5.8}$$

When added to the initial representation via residual connection, $\mathbf{e}_{\text{creature}} + \Delta\mathbf{e}_{\text{creature}}$ physically moves the point in embedding space into the specialized manifold region corresponding to a *fluffy, blue creature*.




(a) Query-Key compatibility matrix ($\mathbf{QK}^{\top}$) (b) Value matrix weighting and vector update ($\Delta\mathbf{e}$)

Figure 5.2: Geometric attention dynamics and matrix transformations in 3Blue1Brown's pedagogical breakdown [56]. (a) Pairwise Query-Key dot products evaluate compatibility across all token pairs in *"A fluffy blue creature roamed the verdant forest"*; the Query for *"creature"* strongly matches the Keys for descriptive modifiers *"fluffy"* (+93.0) and *"blue"* (+93.4). (b) The attention table routes the corresponding Value vectors: after softmax normalization (0.42 and 0.58), the weighted sum produces a displacement vector $\Delta\mathbf{E}_4 = 0.42\mathbf{V}_2 + 0.58\mathbf{V}_3$. Adding $\Delta\mathbf{E}_4$ to the generic creature vector $\mathbf{E}_4$ displaces the representation into the specialized manifold coordinates of a fluffy blue creature.

## 5.4 Scaled Dot-Product Attention

### 5.4.1 Definition of the scaled attention mechanism

When computing alignment scores between queries and keys, standard dot product $s = \mathbf{q}^\top \mathbf{k}$ suffers from a critical numerical failure mode as vector dimensionality increases. To resolve this, the landmark formulation in [1] introduced **Scaled Dot-Product Attention**.

**Definition 5.1:** *Scaled dot-product attention*

Let $\mathbf{Q} \in \mathbb{R}^{L_Q \times d_k}$, $\mathbf{K} \in \mathbb{R}^{L_K \times d_k}$, and $\mathbf{V} \in \mathbb{R}^{L_K \times d_v}$ denote Query, Key, and Value matrices. The Scaled Dot-Product Attention operator evaluates:

$$\texttt{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} + \mathbf{M}\right)\mathbf{V}, \tag{5.9}$$

where $\mathbf{M} \in \{0, -\infty\}^{L_Q \times L_K}$ is an optional masking matrix, and softmax is applied row-wise.

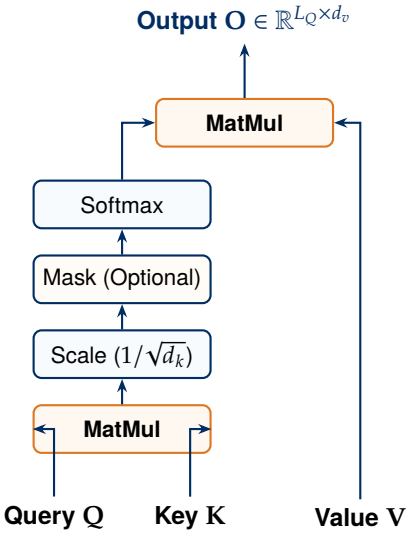



Figure 5.3: Computational dataflow of Scaled Dot-Product Attention [1]. The dot-product matrix is scaled by $1/\sqrt{d_k}$ to prevent gradient saturation before row-wise softmax normalization and value multiplication.

### 5.4.2 Why linear projections are mandatory

In self-attention, input tokens start as embeddings $\mathbf{x}_i \in \mathbb{R}^{d_{\text{model}}}$. If we computed dot products directly between raw token embeddings $\mathbf{x}_i^\top \mathbf{x}_j$, two critical limitations would cripple model expressivity:

1. **Symmetry trap**: Because the dot product is commutative ($\mathbf{x}_i^\top \mathbf{x}_j = \mathbf{x}_j^\top \mathbf{x}_i$), the unprojected affinity between token $i$ and token $j$ would be strictly symmetric. A token could not express an asymmetric relationship, such as a verb actively seeking its direct object without the direct object reciprocating with identical strength.

2. **Role collapse**: A token vector would be forced to play three fundamentally distinct roles

simultaneously: formulating a search request (Query), advertising its semantic contents (Key), and providing the contextual payload (Value).

To break this symmetry and enable functional specialization, the model projects each input vector into three distinct latent subspaces using learned weight matrices:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}_V, \tag{5.10}$$

where $\mathbf{W}_Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $\mathbf{W}_K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, and $\mathbf{W}_V \in \mathbb{R}^{d_{\text{model}} \times d_v}$.

### 5.4.3 Complete numerical walkthrough: reproducing the lecture calculations

To illustrate these matrix projections and attention operations concretely, we walk through step-by-step the complete numerical worked example, tracking intermediate values down to four decimal places.

Consider an input sequence of $L = 3$ tokens in a 2-dimensional latent embedding space ($d_{\text{model}} = 2$), with projection dimensions $d_k = d_v = 2$:

$$\mathbf{X} = \begin{bmatrix} 0.1 & 0.5 \\ 0.1 & 0.2 \\ 0.9 & 0.9 \end{bmatrix} \in \mathbb{R}^{3\times 2}, \quad \mathbf{W}_Q = \begin{bmatrix} 0.9 & 0.9 \\ 0.5 & 0.3 \end{bmatrix}, \quad \mathbf{W}_K = \begin{bmatrix} 1.0 & 0.9 \\ 1.0 & 0.6 \end{bmatrix}, \quad \mathbf{W}_V = \begin{bmatrix} 0.8 & 0.9 \\ 0.9 & 1.0 \end{bmatrix}. \tag{5.11}$$

**Linear projection into Queries, Keys, and Values**

Multiplying $\mathbf{X}$ by each projection matrix yields:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q = \begin{bmatrix} 0.1(0.9) + 0.5(0.5) & 0.1(0.9) + 0.5(0.3) \\ 0.1(0.9) + 0.2(0.5) & 0.1(0.9) + 0.2(0.3) \\ 0.9(0.9) + 0.9(0.5) & 0.9(0.9) + 0.9(0.3) \end{bmatrix} = \begin{bmatrix} 0.34 & 0.24 \\ 0.19 & 0.15 \\ 1.26 & 1.08 \end{bmatrix}, \tag{5.12}$$

$$\mathbf{K} = \mathbf{X}\mathbf{W}_K = \begin{bmatrix} 0.1(1.0) + 0.5(1.0) & 0.1(0.9) + 0.5(0.6) \\ 0.1(1.0) + 0.2(1.0) & 0.1(0.9) + 0.2(0.6) \\ 0.9(1.0) + 0.9(1.0) & 0.9(0.9) + 0.9(0.6) \end{bmatrix} = \begin{bmatrix} 0.60 & 0.39 \\ 0.30 & 0.21 \\ 1.80 & 1.35 \end{bmatrix}, \tag{5.13}$$

$$\mathbf{V} = \mathbf{X}\mathbf{W}_V = \begin{bmatrix} 0.1(0.8) + 0.5(0.9) & 0.1(0.9) + 0.5(1.0) \\ 0.1(0.8) + 0.2(0.9) & 0.1(0.9) + 0.2(1.0) \\ 0.9(0.8) + 0.9(0.9) & 0.9(0.9) + 0.9(1.0) \end{bmatrix} = \begin{bmatrix} 0.53 & 0.59 \\ 0.26 & 0.29 \\ 1.53 & 1.71 \end{bmatrix}. \tag{5.14}$$

**Scaled attention logits computation**

Evaluating the full matrix multiplication $\mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{3\times 3}$ directly:

$$\mathbf{Q}\mathbf{K}^\top = \begin{bmatrix} 0.34 & 0.24 \\ 0.19 & 0.15 \\ 1.26 & 1.08 \end{bmatrix} \begin{bmatrix} 0.60 & 0.30 & 1.80 \\ 0.39 & 0.21 & 1.35 \end{bmatrix} = \begin{bmatrix} 0.2976 & 0.1524 & 0.9360 \\ 0.1725 & 0.0885 & 0.5445 \\ 1.1772 & 0.6048 & 3.7260 \end{bmatrix}. \tag{5.15}$$

Alongside this matrix computation, examining row 1 reveals the individual scalar dot-product compatibility scores for the first token ($\mathbf{q}_1 = [0.34, 0.24]$):

$$s_{1,1} = \mathbf{q}_1^\top \mathbf{k}_1 = 0.34(0.60) + 0.24(0.39) = 0.2040 + 0.0936 = 0.2976, \tag{5.16}$$

$$s_{1,2} = \mathbf{q}_1^\top \mathbf{k}_2 = 0.34(0.30) + 0.24(0.21) = 0.1020 + 0.0504 = 0.1524, \tag{5.17}$$

$$s_{1,3} = \mathbf{q}_1^\top \mathbf{k}_3 = 0.34(1.80) + 0.24(1.35) = 0.6120 + 0.3240 = 0.9360. \tag{5.18}$$

Next, we scale the complete matrix by $\frac{1}{\sqrt{d_k}} = \frac{1}{\sqrt{2}} \approx 0.7071$:

$$\mathbf{S} = \frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{2}} = \begin{bmatrix} \frac{0.2976}{\sqrt{2}} & \frac{0.1524}{\sqrt{2}} & \frac{0.9360}{\sqrt{2}} \\ \frac{0.1725}{\sqrt{2}} & \frac{0.0885}{\sqrt{2}} & \frac{0.5445}{\sqrt{2}} \\ \frac{1.1772}{\sqrt{2}} & \frac{0.6048}{\sqrt{2}} & \frac{3.7260}{\sqrt{2}} \end{bmatrix} \approx \begin{bmatrix} 0.2104 & 0.1078 & 0.6619 \\ 0.1220 & 0.0626 & 0.3850 \\ 0.8324 & 0.4277 & 2.6347 \end{bmatrix}. \tag{5.19}$$

**Matrix softmax normalization onto the probability simplex**

Applying the row-wise softmax operator $\mathbf{A} = \text{softmax}(\mathbf{S})$ exponentiates each row and normalizes by its partition sum. For row 1:

$$\exp(0.2104) \approx 1.2342, \quad \exp(0.1078) \approx 1.1138, \quad \exp(0.6619) \approx 1.9385, \tag{5.20}$$

$$\sum_{j=1}^{3} \exp(\tilde{s}_{1,j}) = 1.2342 + 1.1138 + 1.9385 = 4.2865, \tag{5.21}$$

$$\alpha_{1,1} = \frac{1.2342}{4.2865} \approx 0.2879, \quad \alpha_{1,2} = \frac{1.1138}{4.2865} \approx 0.2598, \quad \alpha_{1,3} = \frac{1.9385}{4.2865} \approx 0.4522. \tag{5.22}$$

Evaluating rows 2 and 3 identically yields the full row-stochastic attention weight matrix:

$$\mathbf{A} = \text{softmax}\left(\frac{\mathbf{QK}^\top}{\sqrt{d_k}}\right) = \begin{bmatrix} 0.2879 & 0.2598 & 0.4522 \\ 0.3083 & 0.2906 & 0.4011 \\ 0.1294 & 0.0863 & 0.7843 \end{bmatrix}, \tag{5.23}$$

where each row sums precisely to 1.0000.

**Matrix value aggregation**

The full sequence output matrix is obtained by matrix multiplication $\mathbf{O} = \mathbf{AV} \in \mathbb{R}^{3\times 2}$:

$$\mathbf{O} = \mathbf{AV} = \begin{bmatrix} 0.2879 & 0.2598 & 0.4522 \\ 0.3083 & 0.2906 & 0.4011 \\ 0.1294 & 0.0863 & 0.7843 \end{bmatrix} \begin{bmatrix} 0.53 & 0.59 \\ 0.26 & 0.29 \\ 1.53 & 1.71 \end{bmatrix} \approx \begin{bmatrix} 0.91 & 1.02 \\ 0.85 & 0.95 \\ 1.29 & 1.44 \end{bmatrix}. \tag{5.24}$$

For token 1, this matrix product directly computes the linear combination of value rows:

$$\mathbf{o}_1 = 0.2879 \begin{bmatrix} 0.53 \\ 0.59 \end{bmatrix} + 0.2598 \begin{bmatrix} 0.26 \\ 0.29 \end{bmatrix} + 0.4522 \begin{bmatrix} 1.53 \\ 1.71 \end{bmatrix} \approx \begin{bmatrix} 0.91 \\ 1.02 \end{bmatrix}. \tag{5.25}$$

This reproduces identically the exact tensor results displayed in the calculations.

### 5.4.4 Dimensional constraints across projection matrices

The dimensional compatibility of the projection matrices is governed by three fundamental algebraic constraints:

- **Queries and Keys must share identical dimensionality ($d_k$)**: The dot product $\mathbf{q}^\top\mathbf{k}$ is an inner product between two vectors. An inner product is mathematically undefined unless both vectors live in the identical Euclidean vector space $\mathbb{R}^{d_k}$. Therefore, $\mathbf{W}_Q$ and $\mathbf{W}_K$ must always share identical column dimension $d_k$.
- **Values can possess independent dimensionality ($d_v$)**: Value vectors are never multiplied with queries or keys. They are only scaled by scalar attention weights $\alpha_j \in [0, 1]$ and summed together. Consequently, $d_v$ could mathematically differ from $d_k$. In standard architectures (including [1]), practitioners choose $d_k = d_v = d_{\text{model}}/h$ purely for architectural symmetry and implementation simplicity.
- **Sequence lengths in self-attention vs. cross-attention**: In self-attention, $L_Q = L_K = L_V = L$. In cross-attention, queries stem from the decoder ($L_Q = L_{\text{dec}}$), while keys and values stem from the encoder ($L_K = L_V = L_{\text{enc}}$). The dot-product matrix $\mathbf{QK}^\top$ has shape $L_{\text{dec}} \times L_{\text{enc}}$, and multiplying by $\mathbf{V} \in \mathbb{R}^{L_{\text{enc}}\times d_v}$ yields the required decoder output shape $L_{\text{dec}} \times d_v$.

### 5.4.5 Mathematical derivation: theoretical necessity of the $\frac{1}{\sqrt{d_k}}$ scaling factor

Why must the dot product $\mathbf{q}^\top \mathbf{k}$ be scaled specifically by $\frac{1}{\sqrt{d_k}}$? To understand the mathematical necessity of this normalization, consider the variance behavior of unscaled dot products as dimensionality expands.

Assume that the components of query vector $\mathbf{q} \in \mathbb{R}^{d_k}$ and key vector $\mathbf{k} \in \mathbb{R}^{d_k}$ are independent, identically distributed (i.i.d.) random variables with zero mean and unit variance:

$$\mathbb{E}[q_i] = 0, \quad \mathbb{V}\mathrm{ar}(q_i) = 1, \quad \mathbb{E}[k_i] = 0, \quad \mathbb{V}\mathrm{ar}(k_i) = 1, \quad \forall i \in \{1, 2, \ldots, d_k\}. \tag{5.26}$$

The unscaled dot product is the random variable $z = \mathbf{q}^\top \mathbf{k} = \sum_{i=1}^{d_k} q_i k_i$. By linearity of expectation and independence:

$$\mathbb{E}[z] = \mathbb{E}\left[\sum_{i=1}^{d_k} q_i k_i\right] = \sum_{i=1}^{d_k} \mathbb{E}[q_i]\,\mathbb{E}[k_i] = \sum_{i=1}^{d_k} (0 \cdot 0) = 0. \tag{5.27}$$

To compute the variance of $z$, we expand the definition $\mathbb{V}\mathrm{ar}(z) = \sum_{i=1}^{d_k} \mathbb{V}\mathrm{ar}(q_i k_i)$. Using identity $\mathbb{V}\mathrm{ar}(XY) = \mathbb{E}[X^2]\,\mathbb{E}[Y^2] - (\mathbb{E}[X]\,\mathbb{E}[Y])^2$:

$$\begin{aligned} \mathbb{V}\mathrm{ar}(q_i k_i) &= \mathbb{E}[q_i^2]\,\mathbb{E}[k_i^2] - (\mathbb{E}[q_i]\,\mathbb{E}[k_i])^2 \\ &= \left(\mathbb{V}\mathrm{ar}(q_i) + (\mathbb{E}[q_i])^2\right)\left(\mathbb{V}\mathrm{ar}(k_i) + (\mathbb{E}[k_i])^2\right) - 0 \\ &= (1+0)(1+0) = 1. \end{aligned} \tag{5.28}$$

Summing over all $d_k$ independent dimensions:

$$\mathbb{V}\mathrm{ar}(z) = \sum_{i=1}^{d_k} \mathbb{V}\mathrm{ar}(q_i k_i) = \sum_{i=1}^{d_k} 1 = d_k. \tag{5.29}$$

Thus, the standard deviation of the dot product is $\sigma_z = \sqrt{d_k}$.

For large dimensions (e.g., $d_k = 64$ or 128), the variance $d_k$ is substantial, causing dot products to take values with magnitude $|z| \gg 1$. Recall that the derivative of the softmax function with respect to logit $z_i$ is given by:

$$\frac{\partial\,\mathrm{softmax}(z)_i}{\partial z_j} = \mathrm{softmax}(z)_i \left(\delta_{ij} - \mathrm{softmax}(z)_j\right). \tag{5.30}$$

When logits $z$ have large absolute variance, the exponential $\exp(z_m)$ of the maximal logit completely dominates the sum. The softmax output degenerates into a nearly one-hot indicator distribution ($\alpha_m \approx 1$, and $\alpha_{j \neq m} \approx 0$). In this regime:

$$\frac{\partial\,\mathrm{softmax}(z)_i}{\partial z_j} \approx 0, \quad \forall i, j. \tag{5.31}$$

The gradients vanish, arresting backpropagation through the attention module. By scaling $z$ by scalar factor $\frac{1}{\sqrt{d_k}}$, we renormalize the variance back to unity:

$$\mathbb{V}\mathrm{ar}\left(\frac{z}{\sqrt{d_k}}\right) = \frac{1}{(\sqrt{d_k})^2}\,\mathbb{V}\mathrm{ar}(z) = \frac{1}{d_k} \cdot d_k = 1. \tag{5.32}$$

Renormalizing the variance preserves healthy gradient dynamics across arbitrarily deep networks.

## 5.5 Attention Mechanism Types

Within the complete Transformer architecture, attention is deployed across three distinct operational settings [1], summarized in Figure 5.4.

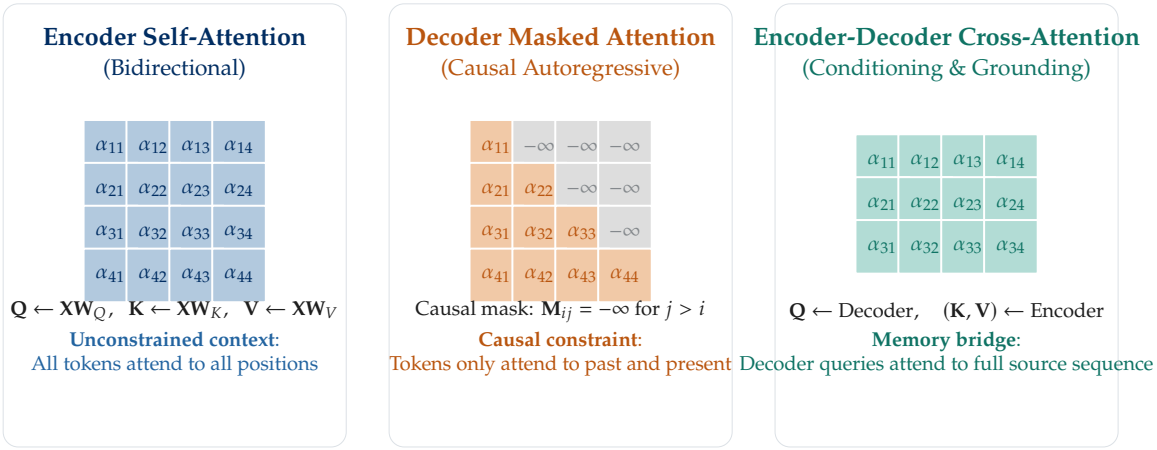



Figure 5.4: The triad of attention mechanisms across the Transformer architecture. (a) Encoder self-attention processes inputs bidirectionally with an unconstrained attention matrix. (b) Decoder masked self-attention applies a lower-triangular causal mask ($\mathbf{M}_{ij} = -\infty$ for $j > i$) to enforce autoregressive generation without future information leakage. (c) Encoder-decoder cross-attention allows decoder queries to attend across all encoder key-value pairs, bridging representations across different sequence lengths.

### 5.5.1 Encoder self-attention: unconstrained bidirectional contextualization

In natural language understanding, the meaning of a word is profoundly shaped by tokens that appear both *before* and *after* it. For example, consider syntactic ambiguity, pronoun reference, or polysemy: in the sentence *"The bank of the river overflowed"*, the polysemous noun *"bank"* cannot be resolved into a geological landform rather than a financial institution without inspecting the downstream word *"river"*. Because the Encoder's sole objective is representation learning rather than autoregressive word generation, there is no risk of label leakage. Therefore, the Encoder must have access to the entire sentence simultaneously: the attention matrix $\mathbf{A} \in \mathbb{R}^{L_{\text{enc}} \times L_{\text{enc}}}$ is completely unconstrained ($\mathbf{M} = \mathbf{0}$), granting every token an instantaneous $O(1)$ global receptive field across all $L_{\text{enc}}$ positions.

In the Encoder, queries, keys, and values are all linearly projected from the preceding encoder layer's representations:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}_V, \tag{5.33}$$

where $\mathbf{X} \in \mathbb{R}^{L_{\text{enc}} \times d_{\text{model}}}$, $\mathbf{W}_Q, \mathbf{W}_K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, and $\mathbf{W}_V \in \mathbb{R}^{d_{\text{model}} \times d_v}$. No masking matrix is applied ($\mathbf{M} = \mathbf{0}$). Every token $i$ can attend freely to all other tokens $j \in \{1, 2, \ldots, L_{\text{enc}}\}$, producing unconstrained bidirectional representations.

**Worked example: bidirectional polysemy disambiguation**

Consider an English input sentence of $L_{\text{enc}} = 3$ tokens:

$$X = [\text{"bank"}_1, \text{"of"}_2, \text{"river"}_3]. \tag{5.34}$$

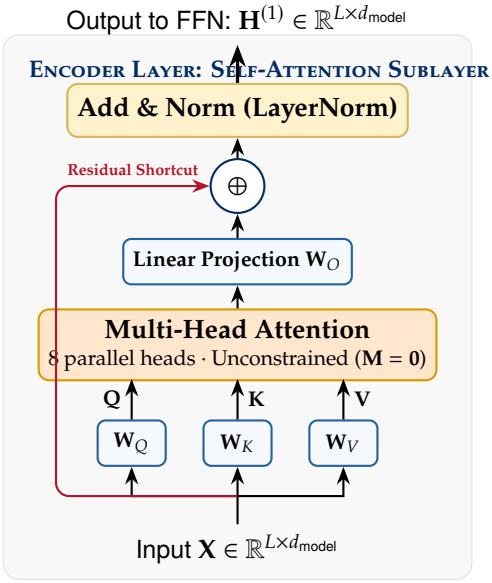



Figure 5.5: Architectural schematic of the Encoder Self-Attention sublayer [1]. Input tokens $\mathbf{X}$ are projected into queries $\mathbf{Q}$, keys $\mathbf{K}$, and values $\mathbf{V}$. Unconstrained multi-head attention evaluates all-to-all token visibility without masking ($\mathbf{M} = \mathbf{0}$), followed by linear output projection $\mathbf{W}_O$, identity residual addition, and Layer Normalization.

The token *"bank"* is polysemous. Without inspecting the future token *"river"*, disambiguation is impossible. Let $d_k = d_v = 2$. Suppose the linear projections $\mathbf{X}\mathbf{W}_Q$, $\mathbf{X}\mathbf{W}_K$, and $\mathbf{X}\mathbf{W}_V$ yield:

$$\mathbf{Q} = \begin{bmatrix} 1.0 & 0.5 \\ 0.2 & 0.8 \\ 0.9 & 1.2 \end{bmatrix}, \quad \mathbf{K} = \begin{bmatrix} 0.8 & 0.4 \\ 0.1 & 0.9 \\ 1.0 & 1.1 \end{bmatrix}, \quad \mathbf{V} = \begin{bmatrix} 1.5 & 0.2 \\ 0.1 & 0.8 \\ 0.4 & 2.0 \end{bmatrix}. \tag{5.35}$$

We execute the attention mechanism step by step:

1. **Raw dot-product computation**: Evaluating unscaled inner products $\mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{3\times 3}$:

$$\mathbf{Q}\mathbf{K}^\top = \begin{bmatrix} (1.0)(0.8) + (0.5)(0.4) & (1.0)(0.1) + (0.5)(0.9) & (1.0)(1.0) + (0.5)(1.1) \\ (0.2)(0.8) + (0.8)(0.4) & (0.2)(0.1) + (0.8)(0.9) & (0.2)(1.0) + (0.8)(1.1) \\ (0.9)(0.8) + (1.2)(0.4) & (0.9)(0.1) + (1.2)(0.9) & (0.9)(1.0) + (1.2)(1.1) \end{bmatrix}. \tag{5.36}$$

   Evaluating each scalar entry yields:

$$\mathbf{Q}\mathbf{K}^\top = \begin{bmatrix} 1.00 & 0.55 & 1.55 \\ 0.48 & 0.74 & 1.08 \\ 1.20 & 1.17 & 2.22 \end{bmatrix}. \tag{5.37}$$

2. **Scaling by $\sqrt{d_k} = \sqrt{2} \approx 1.414$**:

$$\mathbf{S} = \frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{2}} \approx \begin{bmatrix} 0.707 & 0.389 & 1.096 \\ 0.339 & 0.523 & 0.764 \\ 0.849 & 0.827 & 1.570 \end{bmatrix}. \tag{5.38}$$

3. **Bidirectional softmax normalization**: Since no mask is present ($\mathbf{M} = \mathbf{0}$), we compute row-wise softmax directly across all columns: For row 1 (corresponding to *"bank"*):

$$\exp(0.707) \approx 2.028, \quad \exp(0.389) \approx 1.475, \quad \exp(1.096) \approx 2.992, \tag{5.39}$$

$$Z_1 = 2.028 + 1.475 + 2.992 = 6.495, \tag{5.40}$$

$$\alpha_{11} = \frac{2.028}{6.495} \approx 0.312, \quad \alpha_{12} = \frac{1.475}{6.495} \approx 0.227, \quad \alpha_{13} = \frac{2.992}{6.495} \approx 0.461. \tag{5.41}$$

Applying this across all rows yields the dense attention matrix:

$$\mathbf{A}_{\text{enc}} = \begin{bmatrix} 0.312 & 0.227 & 0.461 \\ 0.268 & 0.322 & 0.410 \\ 0.248 & 0.243 & 0.510 \end{bmatrix}. \tag{5.42}$$

4. **Contextual value aggregation**: Multiplying $\mathbf{A}_{\text{enc}}$ by $\mathbf{V} \in \mathbb{R}^{3\times 2}$ yields the updated contextual representations:

$$\mathbf{O}_{\text{enc}} = \mathbf{A}_{\text{enc}}\mathbf{V} = \begin{bmatrix} 0.312\mathbf{v}_1 + 0.227\mathbf{v}_2 + 0.461\mathbf{v}_3 \\ 0.268\mathbf{v}_1 + 0.322\mathbf{v}_2 + 0.410\mathbf{v}_3 \\ 0.248\mathbf{v}_1 + 0.243\mathbf{v}_2 + 0.510\mathbf{v}_3 \end{bmatrix} \approx \begin{bmatrix} 0.675 & 1.165 \\ 0.598 & 1.131 \\ 0.600 & 1.263 \end{bmatrix}. \tag{5.43}$$

Notice that token 1 (*"bank"*) allocates its largest attention weight ($\alpha_{13} = 0.461$) to token 3 (*"river"*), looking *ahead* into future context. This unconstrained bidirectional information exchange allows *"bank"* to instantly absorb the geological semantics of *"river"* into its contextual output $\mathbf{o}_1$.

### 5.5.2 Decoder masked self-attention: causal autoregressive constraint

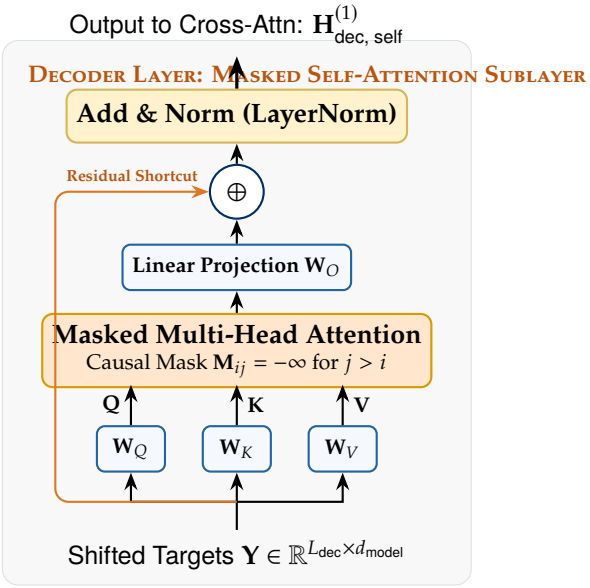



Figure 5.6: Architectural schematic of the Decoder Masked Self-Attention sublayer [1]. Causal masking enforces autoregressive conditioning by blinding all future positions with $-\infty$, ensuring query $i$ attends exclusively to preceding tokens $j \leq i$.

Unlike the Encoder, the Decoder is tasked with sequential language generation: producing the next token $y_t$ conditioned strictly on the prefix of tokens emitted so far $[y_1, y_2, \ldots, y_{t-1}]$. At test time, future tokens $y_{t+1}, y_{t+2}, \ldots$ do not exist. During training, however, the entire target sequence is supplied to the model simultaneously to enable parallel execution via Teacher Forcing. If unconstrained self-attention were permitted in the decoder, a token at position $t$ could directly attend to position $t+1$ and cheat by copying the ground-truth answer. The model would suffer from substantial information leakage, failing to learn any predictive distribution over future tokens. To enforce the arrow of time while preserving parallel tensor execution on hardware, a **causal mask M** is added to the attention logits prior to softmax, strictly setting all future attention weights to zero.

To preserve causality while retaining parallel training under Teacher Forcing, a **causal mask** $\mathbf{M} \in \mathbb{R}^{L_{\text{dec}} \times L_{\text{dec}}}$ is added to the scaled dot products prior to the softmax operator:

$$M_{i,j} = \begin{cases} 0 & \text{if } j \leq i, \\ -\infty & \text{if } j > i. \end{cases} \tag{5.44}$$

In matrix form for a sequence of length 3:

$$\mathbf{M} = \begin{bmatrix} 0 & -\infty & -\infty \\ 0 & 0 & -\infty \\ 0 & 0 & 0 \end{bmatrix}. \tag{5.45}$$

Because $\exp(-\infty) = 0$, evaluating the row-wise softmax guarantees:

$$\alpha_{i,j} = 0 \quad \forall j > i. \tag{5.46}$$

From a hardware efficiency standpoint, computing the full dense GEMM operation $\mathbf{QK}^\top$ followed by a tensor addition with $\mathbf{M}$ fully saturates GPU tensor cores and memory buses, outperforming jagged triangular loops.

**Worked example: autoregressive generation under causal masking**

Now consider the Decoder generating the Italian translation prefix of length $L_{\text{dec}} = 3$:

$$Y = [y_1 = \langle \text{BOS} \rangle,\ y_2 = \text{“riva”},\ y_3 = \text{“del”}]. \tag{5.47}$$

All queries, keys, and values are projected from the decoder's target token embeddings ($d_k = d_v = 2$):

$$\mathbf{Q} = \begin{bmatrix} 0.5 & 0.5 \\ 1.2 & 0.3 \\ 0.8 & 1.0 \end{bmatrix}, \quad \mathbf{K} = \begin{bmatrix} 0.6 & 0.4 \\ 1.0 & 0.2 \\ 0.7 & 0.9 \end{bmatrix}, \quad \mathbf{V} = \begin{bmatrix} 0.2 & 0.5 \\ 1.8 & 0.4 \\ 0.6 & 1.4 \end{bmatrix}. \tag{5.48}$$

Let us trace the computation step by step:

1. **Raw scaled dot products (unmasked)**: Computing $\mathbf{S} = \frac{\mathbf{QK}^\top}{\sqrt{2}}$:

$$\mathbf{S} = \frac{1}{\sqrt{2}} \begin{bmatrix} 0.50 & 0.60 & 0.80 \\ 0.84 & 1.26 & 1.11 \\ 0.88 & 1.00 & 1.46 \end{bmatrix} \approx \begin{bmatrix} 0.354 & 0.424 & 0.566 \\ 0.594 & 0.891 & 0.785 \\ 0.622 & 0.707 & 1.032 \end{bmatrix}. \tag{5.49}$$

   Notice that unmasked score $S_{13} = 0.566$ is the highest in the first row: without masking, $\langle \text{BOS} \rangle$ would strongly attend to future token *“del”*.

2. **Applying the causal mask $\mathbf{M}$**: We add $\mathbf{M}$ to $\mathbf{S}$:

$$\mathbf{S}_{\text{masked}} = \mathbf{S} + \mathbf{M} = \begin{bmatrix} 0.354 & -\infty & -\infty \\ 0.594 & 0.891 & -\infty \\ 0.622 & 0.707 & 1.032 \end{bmatrix}. \tag{5.50}$$

3. **Row-wise softmax under masking**:

   - **Row 1 ($y_1 = \langle \text{BOS} \rangle$)**: $\exp(0.354) \approx 1.425$, while $\exp(-\infty) = 0$:

$$\alpha_{11} = \frac{1.425}{1.425 + 0 + 0} = 1.000, \quad \alpha_{12} = 0.000, \quad \alpha_{13} = 0.000. \tag{5.51}$$

     Position 1 can *only* attend to itself.

- **Row 2 ($y_2$ = "riva")**: $\exp(0.594) \approx 1.811$, $\exp(0.891) \approx 2.438$, $\exp(-\infty) = 0$:

$$Z_2 = 1.811 + 2.438 = 4.249, \tag{5.52}$$

$$\alpha_{21} = \frac{1.811}{4.249} \approx 0.426, \quad \alpha_{22} = \frac{2.438}{4.249} \approx 0.574, \quad \alpha_{23} = 0.000. \tag{5.53}$$

  Position 2 attends strictly to the prefix $\{y_1, y_2\}$; future token $y_3$ is zeroed out.
- **Row 3 ($y_3$ = "del")**: $\exp(0.622) \approx 1.863$, $\exp(0.707) \approx 2.028$, $\exp(1.032) \approx 2.807$:

$$Z_3 = 1.863 + 2.028 + 2.807 = 6.698 \implies \alpha_{31} \approx 0.278, \quad \alpha_{32} \approx 0.303, \quad \alpha_{33} \approx 0.419. \tag{5.54}$$

This produces the strictly lower-triangular attention matrix:

$$\mathbf{A}_{\text{dec}} = \begin{bmatrix} 1.000 & 0.000 & 0.000 \\ 0.426 & 0.574 & 0.000 \\ 0.278 & 0.303 & 0.419 \end{bmatrix}. \tag{5.55}$$

4. **Causal value aggregation**: Multiplying by $\mathbf{V} \in \mathbb{R}^{3\times 2}$:

$$\mathbf{O}_{\text{dec}} = \mathbf{A}_{\text{dec}}\mathbf{V} = \begin{bmatrix} 1.000\mathbf{v}_1 \\ 0.426\mathbf{v}_1 + 0.574\mathbf{v}_2 \\ 0.278\mathbf{v}_1 + 0.303\mathbf{v}_2 + 0.419\mathbf{v}_3 \end{bmatrix} \approx \begin{bmatrix} 0.200 & 0.500 \\ 1.118 & 0.443 \\ 0.852 & 0.847 \end{bmatrix}. \tag{5.56}$$

Contrast this directly with Encoder self-attention: despite $S_{13} = 0.566$ having high compatibility, the causal mask completely extinguishes its influence ($\alpha_{13} = 0$). Output $\mathbf{o}_1$ is computed strictly from $\mathbf{v}_1$, preserving the causality required for autoregressive sequence modeling.

### 5.5.3 Encoder-decoder cross-attention: dynamic source grounding

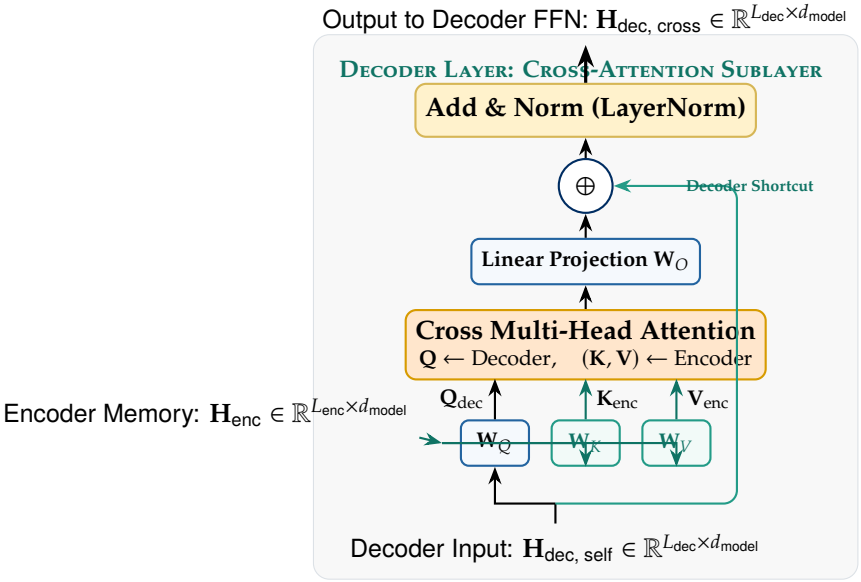



Figure 5.7: Architectural schematic of the Encoder-Decoder Cross-Attention sublayer [1]. The decoder generates Query vectors $\mathbf{Q}$ from its self-attended states, while Key $\mathbf{K}$ and Value $\mathbf{V}$ vectors are projected from the final encoder representations, allowing the decoder to dynamically attend across the entire source sequence.

In sequence-to-sequence translation, the decoder is generating tokens in the target language, while the encoder holds the semantic context of the source sentence. Recall the fundamental roles of the attention triad:

- A **Query** asks: *"What information am I currently searching for to generate my next word?"* Because the entity generating the next word is the decoder, the queries $\mathbf{Q}$ must be projected from the decoder's current representation.
- A **Key** advertises: *"What features do I describe in the source sentence?"*
- A **Value** provides: *"What source meaning should be retrieved?"*

Because the source content is held exclusively in the encoder, the keys $\mathbf{K}$ and values $\mathbf{V}$ must originate from the encoder's output representations. If queries were taken from the encoder instead, the source sentence would be searching the target words, which completely contradicts the directional objective of translating source meaning into target text. By projecting queries from the decoder and keys/values from the encoder, cross-attention acts as a dynamic search engine allowing each generated target token to query the entire uncompressed source sentence.

In cross-attention, the model bridges the target sequence with the source sequence. The queries originate from the decoder's intermediate representations $\mathbf{H}_{\text{dec}} \in \mathbb{R}^{L_{\text{dec}} \times d_{\text{model}}}$, while keys and values originate from the uncompressed output of the encoder stack $\mathbf{H}_{\text{enc}} \in \mathbb{R}^{L_{\text{enc}} \times d_{\text{model}}}$:

$$\mathbf{Q} = \mathbf{H}_{\text{dec}}\mathbf{W}_Q^{\text{cross}} \in \mathbb{R}^{L_{\text{dec}} \times d_k}, \quad \mathbf{K} = \mathbf{H}_{\text{enc}}\mathbf{W}_K^{\text{cross}} \in \mathbb{R}^{L_{\text{enc}} \times d_k}, \quad \mathbf{V} = \mathbf{H}_{\text{enc}}\mathbf{W}_V^{\text{cross}} \in \mathbb{R}^{L_{\text{enc}} \times d_v}. \tag{5.57}$$

Crucially:

- **Asymmetric sequence lengths**: The number of queries $L_{\text{dec}}$ does not need to equal the number of keys/values $L_{\text{enc}}$. In translation, sentences in different languages rarely have identical token counts.
- **No causal mask on the source**: The source sentence was fully processed by the encoder before decoding began. Thus, every decoder query position can inspect *all* $L_{\text{enc}}$ source tokens ($\mathbf{M} = \mathbf{0}$).

**Worked example: cross-lingual alignment with asymmetric lengths**

Suppose our decoder has generated a target prefix of length $L_{\text{dec}} = 2$ ([⟨BOS⟩, "riva"]), and cross-attends to our earlier English source sentence of length $L_{\text{enc}} = 3$ (["bank", "of", "river"]). The decoder provides $L_{\text{dec}} = 2$ queries $\mathbf{Q} \in \mathbb{R}^{2\times 2}$ (taken from the first two rows of (5.48)), while the encoder provides $L_{\text{enc}} = 3$ keys and values $\mathbf{K}, \mathbf{V} \in \mathbb{R}^{3\times 2}$ (from (5.35)):

$$\mathbf{Q} = \begin{bmatrix} 0.5 & 0.5 \\ 1.2 & 0.3 \end{bmatrix}, \quad \mathbf{K} = \begin{bmatrix} 0.8 & 0.4 \\ 0.1 & 0.9 \\ 1.0 & 1.1 \end{bmatrix}, \quad \mathbf{V} = \begin{bmatrix} 1.5 & 0.2 \\ 0.1 & 0.8 \\ 0.4 & 2.0 \end{bmatrix}. \tag{5.58}$$

Let us trace the rectangular cross-attention computation step by step:

1. **Rectangular dot-product matrix ($L_{\text{dec}} \times L_{\text{enc}} = 2 \times 3$):**

$$\mathbf{QK}^\top = \begin{bmatrix} (0.5)(0.8) + (0.5)(0.4) & (0.5)(0.1) + (0.5)(0.9) & (0.5)(1.0) + (0.5)(1.1) \\ (1.2)(0.8) + (0.3)(0.4) & (1.2)(0.1) + (0.3)(0.9) & (1.2)(1.0) + (0.3)(1.1) \end{bmatrix}. \tag{5.59}$$

   Evaluating the pairwise inner products yields:

$$\mathbf{QK}^\top = \begin{bmatrix} 0.60 & 0.50 & 1.05 \\ 1.08 & 0.39 & 1.53 \end{bmatrix}. \tag{5.60}$$

2. **Scaling by $\sqrt{d_k} = \sqrt{2}$:**

$$\mathbf{S}_{\text{cross}} = \frac{\mathbf{QK}^\top}{\sqrt{2}} \approx \begin{bmatrix} 0.424 & 0.354 & 0.742 \\ 0.764 & 0.276 & 1.082 \end{bmatrix} \in \mathbb{R}^{2\times 3}. \tag{5.61}$$

3. **Row-wise softmax normalization (over $L_{\mathbf{enc}} = 3$ source keys)**:

   - **Row 1 ($\mathbf{q}_{\langle \mathbf{BOS} \rangle}$)**: $\exp(0.424) \approx 1.528$, $\exp(0.354) \approx 1.425$, $\exp(0.742) \approx 2.100$:

   $$Z_1 = 1.528 + 1.425 + 2.100 = 5.053 \implies \mathbf{A}_{1,:} \approx [0.302,\ 0.282,\ 0.416]. \tag{5.62}$$

   - **Row 2 ($\mathbf{q}_{\text{“riva”}}$)**: $\exp(0.764) \approx 2.147$, $\exp(0.276) \approx 1.318$, $\exp(1.082) \approx 2.951$:

   $$Z_2 = 2.147 + 1.318 + 2.951 = 6.416 \implies \mathbf{A}_{2,:} \approx [0.335,\ 0.205,\ 0.460]. \tag{5.63}$$

   This yields the rectangular cross-attention matrix:

   $$\mathbf{A}_{\text{cross}} = \begin{bmatrix} 0.302 & 0.282 & 0.416 \\ 0.335 & 0.205 & 0.460 \end{bmatrix} \in \mathbb{R}^{2\times 3}. \tag{5.64}$$

4. **Cross-attention context retrieval**: Multiplying $\mathbf{A}_{\text{cross}} \in \mathbb{R}^{2\times 3}$ by encoder values $\mathbf{V} \in \mathbb{R}^{3\times 2}$ yields the grounded representations $\mathbf{O}_{\text{cross}} \in \mathbb{R}^{2\times 2}$:

   $$\mathbf{O}_{\text{cross}} = \mathbf{A}_{\text{cross}}\mathbf{V} = \begin{bmatrix} 0.302\mathbf{v}_1 + 0.282\mathbf{v}_2 + 0.416\mathbf{v}_3 \\ 0.335\mathbf{v}_1 + 0.205\mathbf{v}_2 + 0.460\mathbf{v}_3 \end{bmatrix} \approx \begin{bmatrix} 0.648 & 1.117 \\ 0.706 & 1.151 \end{bmatrix}. \tag{5.65}$$

Query $\mathbf{q}_{\text{“riva”}}$ strongly aligns with source key $\mathbf{k}_{\text{“river”}}$ ($\alpha_{23} = 0.460$) and source key $\mathbf{k}_{\text{“bank”}}$ ($\alpha_{21} = 0.335$). The decoder directly pulls the exact semantic coordinates of the geological shore from the source representation without compressing the source into a single bottleneck vector.

## 5.6 Multi-Head Attention

In single-head attention, the model computes a single set of attention weights $\alpha_{i,j}$, averaging all values into a single weighted sum. However, a single token often participates in multiple simultaneous linguistic relationships:

- A verb must track both its grammatical subject (often located earlier in the sentence) and its direct object (often located downstream).
- In programming languages, an identifier must link to its type declaration, its scope boundary, and its mutating assignments.

A single attention head cannot focus simultaneously on disparate positions with different semantic intents: the softmax distribution is forced to average across them, diluting specific relational signals.

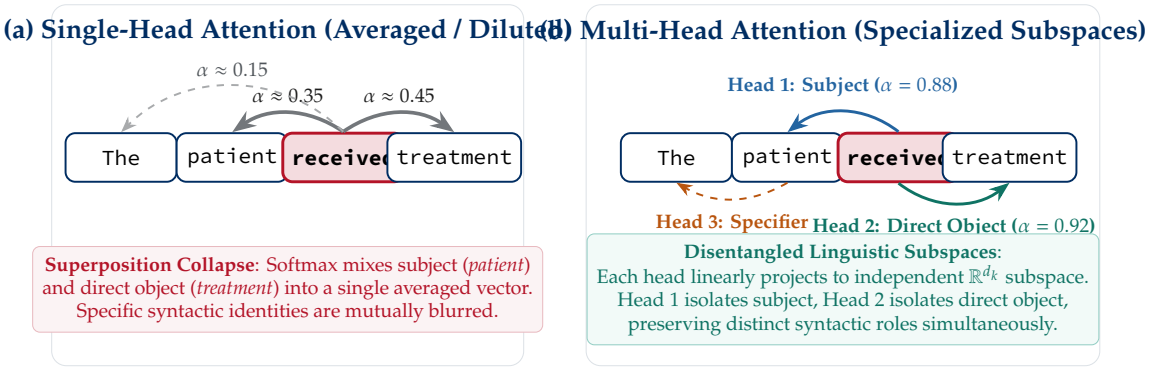



Figure 5.8: Linguistic relationship modeling: Single-Head vs. Multi-Head Attention. In Single-Head Attention (a), a token involved in multiple syntactic dependencies must compromise its single softmax distribution, averaging disparate features and diluting specific relational roles. In Multi-Head Attention (b), projection into $h$ distinct representation subspaces enables separate heads to concurrently isolate orthogonal linguistic relationships (such as grammatical subject tracking and direct object binding) without mutual interference.

Multi-Head Attention (MHA) resolves this by linearly projecting queries, keys, and values $h$ times with different, learned parameter matrices:

**Definition 5.2:** *Multi-head attention*

Let $h$ denote the number of attention heads. Multi-Head Attention projects $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ into $h$ distinct representation subspaces of dimension $d_k = d_{\text{model}}/h$ and $d_v = d_{\text{model}}/h$:

$$\texttt{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = [\text{head}_1 \parallel \text{head}_2 \parallel \cdots \parallel \text{head}_h]\,\mathbf{W}^O, \tag{5.66}$$

where each individual head is evaluated via:

$$\text{head}_i = \texttt{Attention}(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \mathbf{V}\mathbf{W}_i^V), \quad i \in \{1, 2, \ldots, h\}, \tag{5.67}$$

with parameter projection matrices $\mathbf{W}_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $\mathbf{W}_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $\mathbf{W}_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$, and output projection matrix $\mathbf{W}^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$.

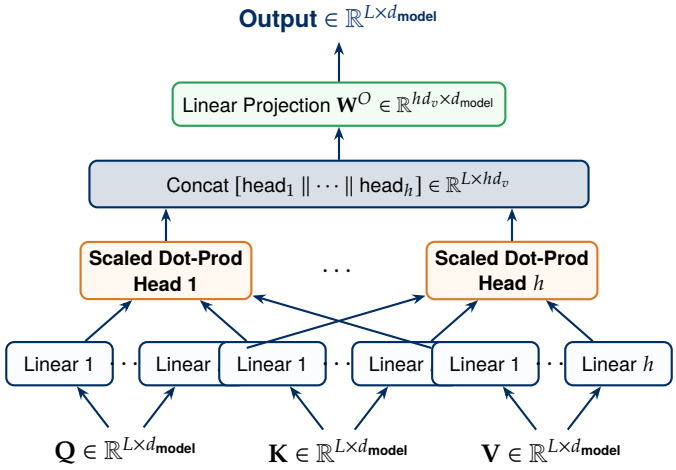



Figure 5.9: Architecture of Multi-Head Attention (MHA) [1]. Projections map inputs into $h$ parallel representation subspaces of reduced dimension $d_k = d_v = d_{\text{model}}/h$, whose outputs are concatenated and linearly projected back to $d_{\text{model}}$.

### Parameter equivalence and computational efficiency

Due to the reduced dimension of each head ($d_k = d_v = d_{\text{model}}/h$), the total computational cost of multi-head attention is identical to that of single-head attention with full dimensionality $d_{\text{model}}$:

$$h \times \left( d_{\text{model}} \times \frac{d_{\text{model}}}{h} \right) = d_{\text{model}}^2 \text{ operations for projections.} \tag{5.68}$$

In standard configurations (e.g., $d_{\text{model}} = 512, h = 8 \implies d_k = d_v = 64$, or $d_{\text{model}} = 768, h = 12 \implies d_k = d_v = 64$), parallel heads learn specialized sub-behaviors: some heads attend to local syntactic structure, others to long-range dependencies, and others to delimiter tokens.

## 5.7 Residual Connections and Normalization

In deep Transformer architectures comprising dozens of layers, stacking attention blocks and non-linear feed-forward networks directly would lead to optimization failure due to exploding or vanishing gradients. To ensure stable gradient propagation, each Transformer sublayer is wrapped in a residual connection followed by Layer Normalization:

$$\mathbf{y} = \texttt{LayerNorm}(\mathbf{x} + \texttt{Sublayer}(\mathbf{x})). \tag{5.69}$$

### Residual skip connections

First introduced in [57] for deep residual networks (ResNets), identity shortcut connections pass the unmodified input $\mathbf{x}$ around the sublayer (Figure 5.10):

$$\mathbf{y} = \mathbf{x} + \mathcal{F}(\mathbf{x}), \tag{5.70}$$

where $\mathcal{F}(\mathbf{x})$ represents the multi-head attention or feed-forward transformation.

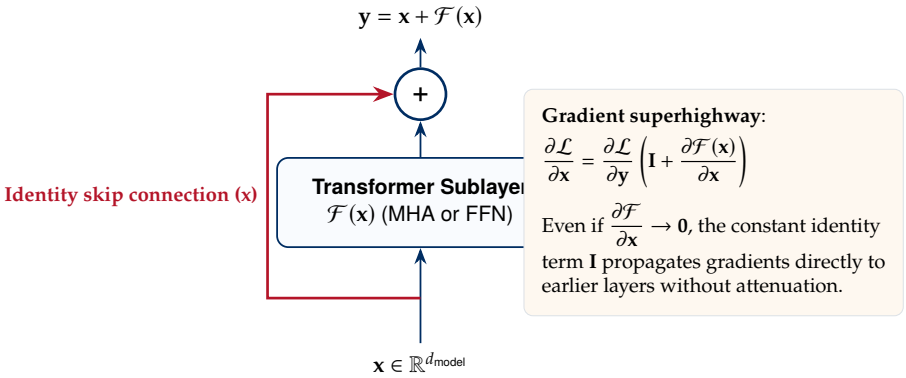



Figure 5.10: Residual skip connection and gradient propagation dynamics. The input $\mathbf{x}$ bypasses the non-linear sublayer $\mathcal{F}(\mathbf{x})$ via an identity shortcut. During backpropagation, the derivative $\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \mathbf{I} + \frac{\partial \mathcal{F}}{\partial \mathbf{x}}$ ensures that the gradient $\frac{\partial \mathcal{L}}{\partial \mathbf{y}}$ flows unimpeded across arbitrary layer depths via the identity term $\mathbf{I}$.

During backpropagation, the gradient of the loss $\mathcal{L}$ with respect to input $\mathbf{x}$ expands via the chain rule to:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \left( \mathbf{I} + \frac{\partial \mathcal{F}(\mathbf{x})}{\partial \mathbf{x}} \right) = \frac{\partial \mathcal{L}}{\partial \mathbf{y}} + \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \frac{\partial \mathcal{F}(\mathbf{x})}{\partial \mathbf{x}}. \tag{5.71}$$

The presence of the identity matrix $\mathbf{I}$ guarantees an unobstructed **gradient superhighway**: even if the Jacobian $\frac{\partial \mathcal{F}}{\partial \mathbf{x}}$ vanishes (for instance, when weights are initialized small or activations saturate), term $\frac{\partial \mathcal{L}}{\partial \mathbf{y}}$ flows directly back to earlier layers in a single step, completely preventing gradient extinction.

### Layer Normalization

Unlike Batch Normalization (which computes statistics across the mini-batch dimension, making it sensitive to batch size and variable sequence lengths), **Layer Normalization** [58] computes normalization statistics across the feature dimensions for each individual token independently (Figure 5.11):

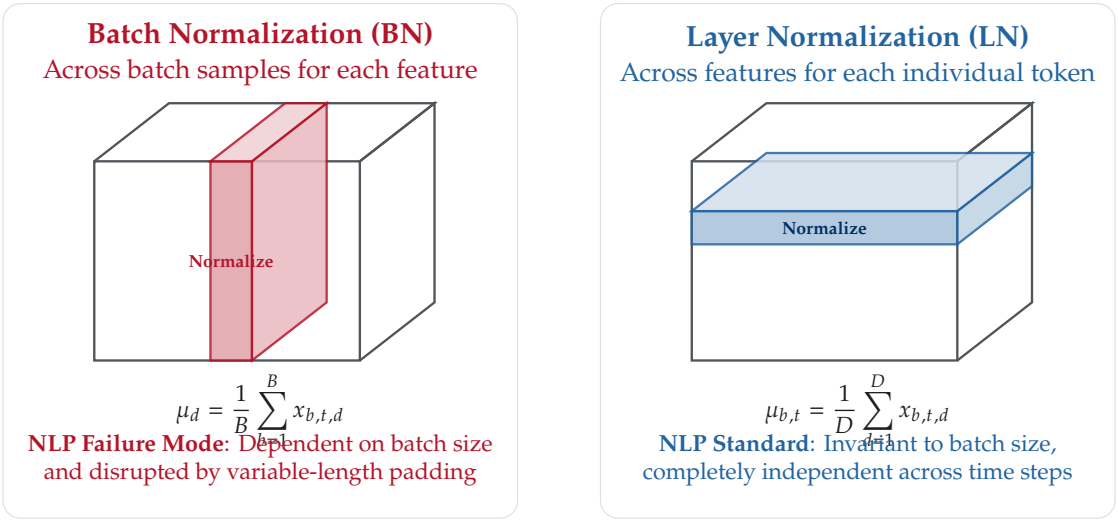



Figure 5.11: Comparison of normalization geometries in 3D activation tensor space $\mathcal{X} \in \mathbb{R}^{B\times T\times D}$. (Left) Batch Normalization computes statistics across all batch instances $B$ for each feature channel $d$, creating inter-sample dependencies and breaking down under variable sequence lengths and sequence padding. (Right) Layer Normalization computes statistics across all feature channels $D$ for each token vector $\mathbf{x}_{b,t}$ independently, ensuring strict batch-size invariance and identical execution during training and autoregressive inference.

**Definition 5.3:** *Layer Normalization*

For a token representation $\mathbf{x} = [x_1, x_2, \ldots, x_d]^\top \in \mathbb{R}^d$, Layer Normalization evaluates:

$$\mu = \frac{1}{d}\sum_{j=1}^{d} x_j, \quad \sigma^2 = \frac{1}{d}\sum_{j=1}^{d}(x_j - \mu)^2, \tag{5.72}$$

$$\texttt{LayerNorm}(\mathbf{x}) = \frac{\mathbf{x} - \mu}{\sqrt{\sigma^2 + \epsilon}} \odot \boldsymbol{\gamma} + \boldsymbol{\beta}, \tag{5.73}$$

where $\epsilon > 0$ is a small numerical stabilization constant, and $\boldsymbol{\gamma}, \boldsymbol{\beta} \in \mathbb{R}^d$ are learnable affine gain and bias parameters initialized to $\boldsymbol{\gamma} = \mathbf{1}$ and $\boldsymbol{\beta} = \mathbf{0}$.

Because Layer Normalization operates on a per-token basis without batch-level pooling, its behavior is identical during training and evaluation, providing exceptional stability across variable-length sequences.

### Position-wise Feed-Forward Networks (FFN)

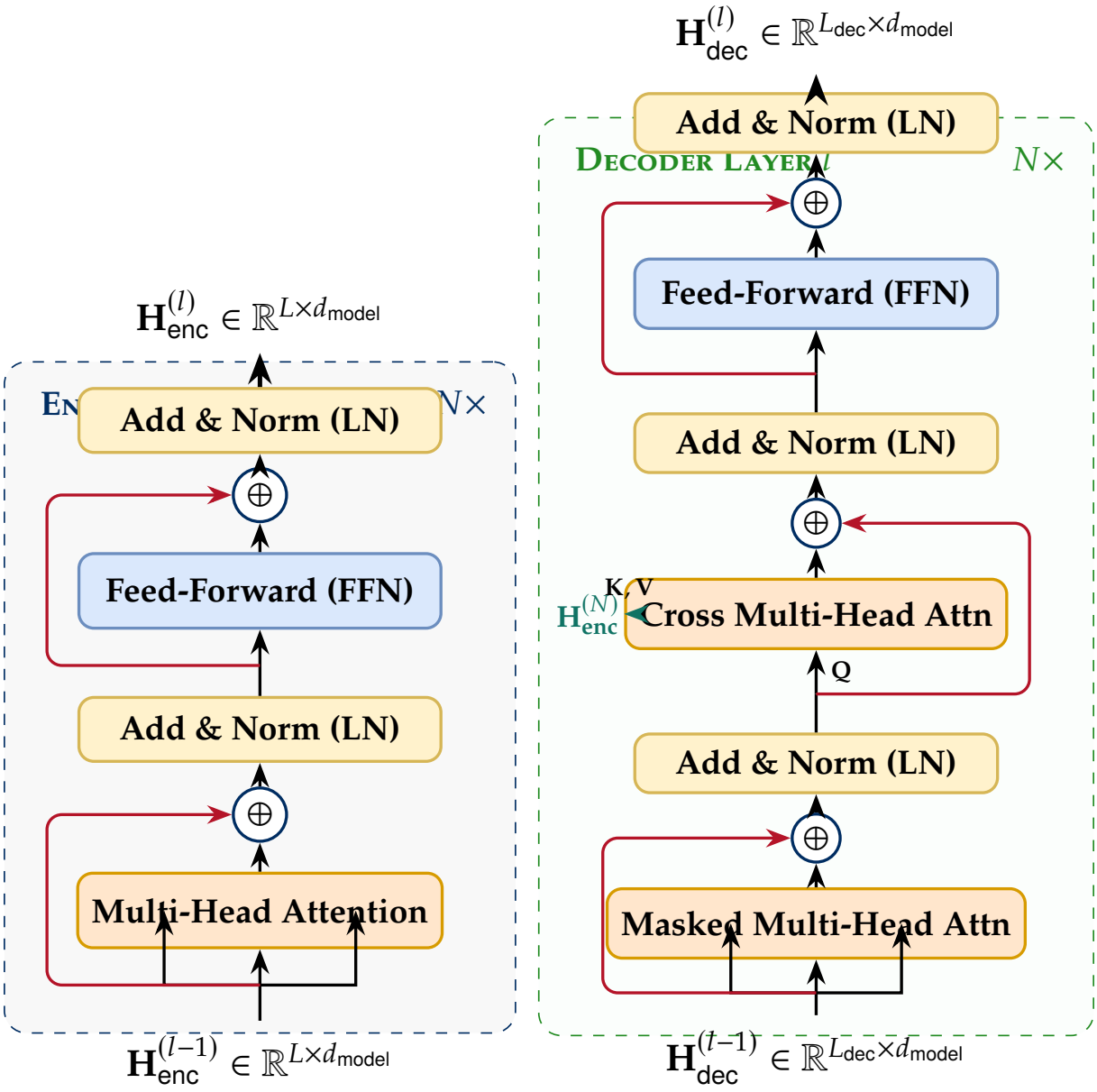
In addition to attention sublayers, each layer in the encoder and decoder contains a fully connected **Feed-Forward Network** applied to each position separately and identically:

$$\texttt{FFN}(\mathbf{x}) = \max(0, \mathbf{x}\mathbf{W}_1 + \mathbf{b}_1)\,\mathbf{W}_2 + \mathbf{b}_2, \tag{5.74}$$

where $\mathbf{W}_1 \in \mathbb{R}^{d_{\text{model}}\times d_{\text{ff}}}$, $\mathbf{b}_1 \in \mathbb{R}^{d_{\text{ff}}}$, $\mathbf{W}_2 \in \mathbb{R}^{d_{\text{ff}}\times d_{\text{model}}}$, and $\mathbf{b}_2 \in \mathbb{R}^{d_{\text{model}}}$. Typically, the inner hidden dimension is expanded by a factor of 4 ($d_{\text{ff}} = 4d_{\text{model}}$, e.g., $d_{\text{ff}} = 2048$ for $d_{\text{model}} = 512$). While attention routes information across different temporal positions, the FFN provides non-linear feature transformation within each individual position.

### 5.7.1 End-to-end sublayer integration: the complete mathematical pipeline

Having analyzed Multi-Head Attention, residual skip connections, Layer Normalization, and Feed-Forward Networks individually, we now formalize how these mathematical operators chain together within complete Transformer layers, tracing their exact locations in the canonical architecture (Figure 5.12).




(a) Encoder layer stack ($N\times$)

(b) Decoder layer stack ($N\times$)

Figure 5.12: End-to-end sublayer connectivity within the canonical Transformer architecture [1]. Every attention and feed-forward block is coupled with an identity residual shortcut and an "Add & Norm" operator.

#### Complete Encoder layer pipeline

An encoder layer $l \in \{1, \ldots, N\}$ receives tensor $\mathbf{H}_{\text{enc}}^{(l-1)} \in \mathbb{R}^{L_{\text{enc}} \times d_{\text{model}}}$ (with $\mathbf{H}_{\text{enc}}^{(0)} = \mathbf{X}_{\text{enc}}$) and applies two composite sublayers:

1. **Sublayer 1 (Bidirectional Multi-Head Self-Attention)**:

$$\mathbf{Z}^{(l)} = \texttt{MultiHead}\left(\mathbf{H}_{\text{enc}}^{(l-1)}, \mathbf{H}_{\text{enc}}^{(l-1)}, \mathbf{H}_{\text{enc}}^{(l-1)}\right), \tag{5.75}$$

$$\tilde{\mathbf{H}}_{\text{enc}}^{(l)} = \texttt{LayerNorm}\left(\mathbf{H}_{\text{enc}}^{(l-1)} + \mathbf{Z}^{(l)}\right). \tag{5.76}$$

2. **Sublayer 2 (Position-wise Feed-Forward Network)**:

$$\mathbf{F}^{(l)} = \texttt{FFN}\left(\tilde{\mathbf{H}}_{\text{enc}}^{(l)}\right), \tag{5.77}$$

$$\mathbf{H}_{\text{enc}}^{(l)} = \texttt{LayerNorm}\left(\tilde{\mathbf{H}}_{\text{enc}}^{(l)} + \mathbf{F}^{(l)}\right). \tag{5.78}$$

**Complete Decoder layer pipeline**

A decoder layer $l \in \{1, \ldots, N\}$ receives target representation $\mathbf{H}_{\text{dec}}^{(l-1)} \in \mathbb{R}^{L_{\text{dec}} \times d_{\text{model}}}$ along with the terminal encoder output $\mathbf{H}_{\text{enc}}^{(N)} \in \mathbb{R}^{L_{\text{enc}} \times d_{\text{model}}}$, executing three consecutive sublayers:

1. **Sublayer 1 (Causal Masked Multi-Head Self-Attention)**:

$$\mathbf{Z}_{\text{self}}^{(l)} = \texttt{MaskedMultiHead}\left(\mathbf{H}_{\text{dec}}^{(l-1)}, \mathbf{H}_{\text{dec}}^{(l-1)}, \mathbf{H}_{\text{dec}}^{(l-1)}\right), \tag{5.79}$$

$$\mathbf{H}_{\text{dec},1}^{(l)} = \texttt{LayerNorm}\left(\mathbf{H}_{\text{dec}}^{(l-1)} + \mathbf{Z}_{\text{self}}^{(l)}\right). \tag{5.80}$$

2. **Sublayer 2 (Encoder-Decoder Multi-Head Cross-Attention)**:

$$\mathbf{Z}_{\text{cross}}^{(l)} = \texttt{MultiHead}\left(\mathbf{H}_{\text{dec},1}^{(l)}, \mathbf{H}_{\text{enc}}^{(N)}, \mathbf{H}_{\text{enc}}^{(N)}\right), \tag{5.81}$$

$$\mathbf{H}_{\text{dec},2}^{(l)} = \texttt{LayerNorm}\left(\mathbf{H}_{\text{dec},1}^{(l)} + \mathbf{Z}_{\text{cross}}^{(l)}\right). \tag{5.82}$$

   Here, queries are projected from $\mathbf{H}_{\text{dec},1}^{(l)}$, while keys and values are projected from $\mathbf{H}_{\text{enc}}^{(N)}$.

3. **Sublayer 3 (Position-wise Feed-Forward Network)**:

$$\mathbf{F}_{\text{dec}}^{(l)} = \texttt{FFN}\left(\mathbf{H}_{\text{dec},2}^{(l)}\right), \tag{5.83}$$

$$\mathbf{H}_{\text{dec}}^{(l)} = \texttt{LayerNorm}\left(\mathbf{H}_{\text{dec},2}^{(l)} + \mathbf{F}_{\text{dec}}^{(l)}\right). \tag{5.84}$$

**Post-LN vs. Pre-LN parameter stabilization**

The formulation above represents the original **Post-LN** architecture of [1], where normalization is placed on the residual sum: $\mathbf{y} = \texttt{LN}(\mathbf{x} + \mathcal{F}(\mathbf{x}))$. While mathematically elegant, deep Post-LN models require delicate learning rate warm-up schedules because gradients passing through the unnormalized residual additions can destabilize early training. Modern implementations (such as GPT-2, LLaMA, and Mistral) predominantly adopt the **Pre-LN** formulation:

$$\mathbf{y} = \mathbf{x} + \mathcal{F}\left(\texttt{LayerNorm}(\mathbf{x})\right), \tag{5.85}$$

which preserves an unbroken, completely unnormalized identity shortcut throughout the entire depth of the network, enabling stable optimization without learning rate warm-up.

## 5.8 Positional Representations

In Chapter 2, we introduced absolute positional encodings—both fixed sinusoidal functions [1] and learned absolute embedding vectors. Under absolute encoding, each index in the sequence ($pos = 0, 1, 2, \dots$) is assigned a dedicated vector added directly to the token embedding before entering the Transformer.

In natural language and software engineering source code, the precise absolute coordinate of a token is frequently less important than its **relative displacement** with respect to other tokens. For example, whether a verb appears at index 12 or index 84, its grammatical subject is typically found 2 or 3 tokens earlier. To incorporate this inductive bias directly into the attention mechanism, Shaw et al. [53] formulated **Relative Positional Representations**.

Instead of adding absolute vectors to the input, relative positional embeddings inject pairwise distance information directly into the scaled dot-product attention logits :

$$\texttt{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top + \mathbf{S}_{\text{rel}}}{\sqrt{d_k}}\right)\mathbf{V}, \tag{5.86}$$

where $\mathbf{S}_{\text{rel}} \in \mathbb{R}^{L\times L}$ is a relative position bias matrix. For query position $i$ and key position $j$, the entry is defined by:

$$S_{\text{rel}}[i, j] = \mathbf{q}_i^\top \mathbf{a}^K_{\text{clip}(i-j,k)}, \tag{5.87}$$

where $\mathbf{a}^K$ is a learned embedding representing the relative offset $i - j$, and $\text{clip}(x, k) = \max(-k, \min(k, x))$ truncates relative distances beyond a maximum clipping horizon $k$.

### Asymmetry of relative positional embeddings

Relative positional embeddings make sense exclusively for self-attention, and cannot be naturally defined for cross-attention:

- **Self-attention**: Both queries and keys originate from the same sequence. The scalar distance $i - j$ represents a physical, well-defined offset within that sequence (e.g., token $j$ occurs 2 steps before token $i$).
- **Cross-attention**: Queries stem from the target sequence (e.g., generated English words in machine translation), while keys stem from the source sequence (e.g., input Italian words). Because source and target sentences possess different lengths, word orders, and grammatical alignments, there is no shared coordinate frame: asking whether an English target word is "three positions ahead" of an Italian source word is mathematically meaningless. Consequently, cross-attention cannot leverage relative positional biases.

## 5.9 Computational Complexity

To appreciate why the Transformer superseded Recurrent Neural Networks, we analyze three foundational computational metrics: computational complexity per layer, sequential operations, and maximum signal path length [1], summarized in Table 5.1.

Table 5.1: Computational complexity and operational parallelism across sequence modeling layers [1].

| Layer type | Complexity per layer | Sequential operations | Maximum path length |
|---|---|---|---|
| **Self-Attention** | $O(L^2 \cdot d)$ | $O(1)$ | $O(1)$ |
| **Recurrent (RNN/LSTM)** | $O(L \cdot d^2)$ | $O(L)$ | $O(L)$ |
| **Convolutional** | $O(k \cdot L \cdot d^2)$ | $O(1)$ | $O(\log_k(L))$ |

#### Sequential operations: shattering the recurrence barrier

In an RNN, computing the representation at step $L$ requires traversing an unbroken chain of $L$ sequential state updates ($O(L)$ sequential operations). In Self-Attention, computing all pairwise dot products $\mathbf{QK}^\top$ and value weighted sums involves dense matrix multiplications that execute in a single parallel step ($O(1)$ sequential operations) across GPU tensor cores.

#### Maximum path length: constant-time gradient propagation

In recurrent networks, information and backpropagation gradients traveling between token 1 and token $L$ must traverse $L$ intermediate state transitions. As proven by the Jacobian product $\prod_{k=t+1}^{T} \frac{\partial \mathbf{h}_k}{\partial \mathbf{h}_{k-1}}$ in Chapter 2, long path lengths cause exponential gradient decay. In Self-Attention, the distance between any arbitrary pair of tokens $(i, j)$ is strictly $O(1)$: the attention weight $\alpha_{i,j}$ directly links token $i$ to token $j$ in a single operation. This constant path length completely eliminates topological distance as an obstacle to learning long-range dependencies.

#### Asymptotic trade-off: sequence length vs. latent dimension

Self-attention incurs a quadratic time and memory complexity $O(L^2 \cdot d)$ with respect to sequence length $L$. When sequence length $L$ is smaller than embedding dimension $d$ (as was typical in early machine translation benchmarks where $L \approx 50$ and $d = 512$), $O(L^2 \cdot d)$ is strictly faster than the recurrent cost $O(L \cdot d^2)$. While long context windows ($L > 8192$) eventually encounter the $O(L^2)$ memory wall, the massive parallel speedup during training outweighs this cost for standard operational contexts.

CHAPTER 6

# Transformer Architectures

Chapter 5 developed the core mathematical primitives of the Transformer, including scaled dot-product attention, multi-head projections, residual streams, and normalization layers. The unified formulation of self-attention permits multiple structural arrangements depending on how attention masks are configured and how sublayers are connected.

Following the initial encoder-decoder Transformer, natural language processing bifurcated into three distinct architectural paradigms. Full encoder-decoder models maintain separate bidirectional encoding and autoregressive decoding stacks for sequence-to-sequence transduction. Pure encoder-only models utilize unmasked bidirectional self-attention to generate contextual representations for classification and extraction. Pure decoder-only models apply causal lower-triangular masks to perform autoregressive next-token prediction.

This chapter examines the three architectural archetypes. We analyze full encoder-decoder networks through the T5 framework, examine encoder-only models through BERT and Masked Language Modeling, and trace the emergence of decoder-only models through the GPT architecture family. Finally, we formalize autoregressive decoding strategies, contrasting greedy search, beam search, temperature scaling, and nucleus sampling.

## 6.1 Encoder-Decoder Models

Following the introduction of the unified Encoder-Decoder Transformer in [1], the machine learning community recognized that many tasks do not require both components simultaneously. Maintaining both stacks incurs unnecessary parameter bloat and memory overhead when the target problem involves either pure sequence transduction with distinct input and output vocabularies, or pure feature representation and bidirectional sequence understanding. Consequently, the architecture bifurcated into specialized paradigms: full Encoder-Decoder architectures and pure Encoder-Only architectures. (We will address pure autoregressive Decoder-Only architectures in the subsequent chapter dedicated to large-scale autoregressive pre-training and scaling laws).

### 6.1.1 The unified text-to-text framework

The full Encoder-Decoder architecture was revitalized by Raffel et al. [59] through the **Text-to-Text Transfer Transformer (T5)** (Figure 6.1).

#### The unified text-to-text framing

Rather than designing separate network architectures for different NLP tasks (e.g., classification heads for sentiment analysis, sequence taggers for named entity recognition, and autoregressive decoders for summarization), T5 casts **every** problem into an identical format: text-to-text sequence generation. The input string encodes both the task directive and the input data via a prepended **task prefix**:

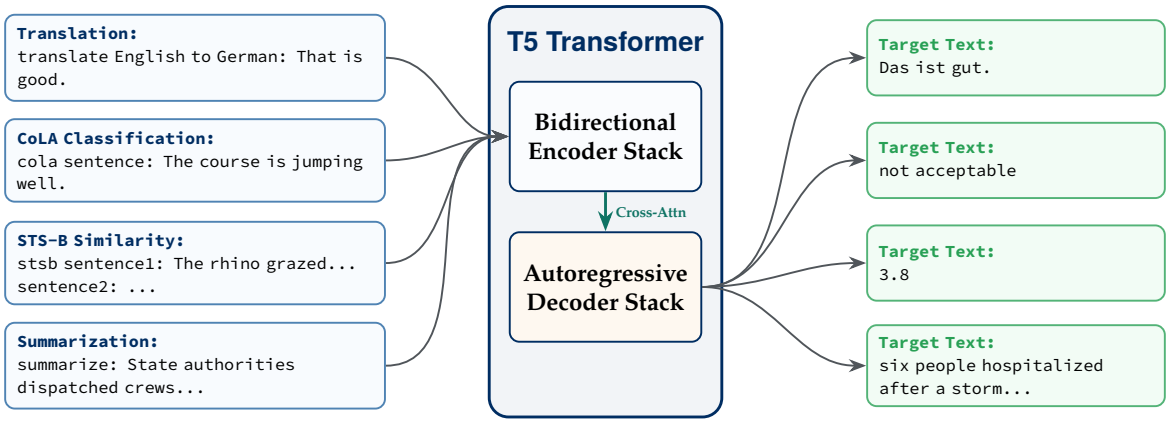



Figure 6.1: The T5 unified text-to-text framework [59]. Every natural language problem—including machine translation, linguistic acceptability (CoLA), sentence similarity scoring (STS-B), and document summarization—is framed as receiving a task-conditioned text prompt with a fixed prefix and autoregressively generating a target text string.

- **Machine translation**: `"translate English to German: That is good."` ⟶ Output: `"Das ist gut."`
- **Summarization**: `"summarize: State authorities dispatched emergency crews..."` ⟶ Output: `"Crews responded to flood alerts."`
- **Linguistic acceptability (CoLA)**: `"cola sentence: The course is great."` ⟶ Output: `"acceptable"`

#### Task conditioning vs. instruction tuning

While T5's prefix-based behavior superficially resembles modern conversational assistants, **T5 is not instruction-tuned**. T5 was trained strictly on fixed, rigid task prefixes. The model learns to recognize these specific keywords and route its internal representations accordingly. However, it exhibits no zero-shot generalization to unseen instructions or altered formulations:

- A slightly verbose variation, such as `"Can you translate English to German the following sentence: That is good"`, causes the model to break down and emit ungrammatical gibberish.
- Prompts requesting novel reasoning, such as `"Compute 2 + 2"`, fail completely because arithmetic was not included in its pre-training task templates.

### 6.1.2 Trade-offs of full encoder-decoder architectures

While the unified encoder-decoder architecture provides exceptional flexibility for arbitrary sequence transduction, maintaining both an encoder and an autoregressive decoder introduces significant computational and representational overhead when the downstream task does not involve sequence generation. In sequence classification, token tagging, or representation retrieval, running an autoregressive decoder step-by-step is completely wasteful. This motivates isolating the encoder stack into a standalone, deeply bidirectional architecture.

## 6.2 Encoder-Only Models

When the objective is sequence understanding, document classification, or semantic code search rather than autoregressive text generation, the decoder stack is completely superfluous. Devlin et al. [52] isolated the bidirectional encoder stack to create **BERT** (Figure 6.2).

To master BERT from first principles, it is essential to systematically decompose and analyze each of its core architectural and training components:

1. **Input representation**: Why we must analyze the tripartite embedding construction (Token, Segment, Position) and special tokens (`[CLS]`, `[SEP]`), which enables a single unified Transformer stack to ingest either isolated sentences or structured sentence pairs without architectural modifications.
2. **Masked Language Model (MLM)**: Why we must analyze bidirectional Cloze pre-training, resolving the fundamental paradox that standard autoregressive next-token prediction causes trivial label leakage in bidirectional self-attention networks.
3. **Next Sentence Prediction (NSP)**: Why intra-sentence bidirectional token conditioning alone is insufficient for discourse-level relational reasoning, requiring explicit cross-sentence relationship pre-training.
4. **Attention map anatomy and emergent linguistic patterns**: Why we must inspect the empirical behavior across the 144 attention heads, discovering how self-attention spontaneously organizes into syntactic dependency trackers, coreference resolvers, and delimiter registers without supervised parse trees.
5. **Downstream fine-tuning mechanisms**: Why we must formalize the adaptation mechanics (sequence-level pooling vs. token-level classification heads), demonstrating how pre-trained bidirectional representations transfer to diverse task topologies with minimal task-specific parameters.

### 6.2.1 Input representation and special tokens

A central challenge in designing a universal sequence understanding backbone is handling diverse downstream input topologies: single text sequences (e.g., sentiment analysis, spam detection), paired sequences with asymmetric roles (e.g., Question Answering premise-hypothesis pairs), and span-level entities. Rather than designing disparate input pipelines for each task type, BERT unifies all inputs into a single continuous stream. Analyzing this construction reveals how lexical identity, sentence membership, and ordinal sequence position are simultaneously communicated to permutation-invariant self-attention layers.

BERT supports pairs of sentences $(A, B)$ packed into a single sequence (Figure 6.3). Every input token representation is constructed by summing three distinct embedding vectors:

$$\mathbf{x}_i = \texttt{TokenEmbedding}(w_i) + \texttt{SegmentEmbedding}(s_i) + \texttt{PositionEmbedding}(i), \tag{6.1}$$

where:

- **Token embeddings**: Subword tokens derived from WordPiece tokenization (with vocabulary cardinality $|\mathcal{V}| = 30{,}522$).
- **Segment embeddings**: Distinguishes whether the token belongs to Sentence $A$ ($E_A$) or Sentence $B$ ($E_B$). This allows self-attention queries to identify cross-sentence relationships and apply sentence-specific relational transformations.

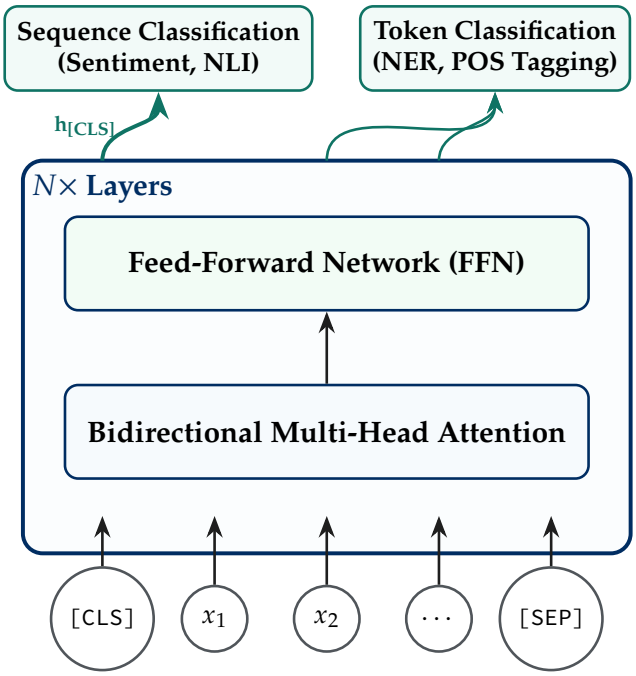



Figure 6.2: The Encoder-only architecture (BERT) [52]. The autoregressive decoder is discarded. Stacked bidirectional Transformer encoder layers process the full context simultaneously. Downstream tasks branch directly off the contextual representations: sequence-level tasks utilize the pooled $\mathbf{h}_{[\text{CLS}]}$ representation, while token-level tasks attach classification heads to individual token outputs $\mathbf{h}_i$.

- **Position embeddings**: Learned absolute positional vectors $(E_0, \ldots, E_{511})$ that inject spatial sequence ordering.

BERT introduces two canonical special tokens:

- `[CLS]` (**Classification**): Always prepended as the very first token ($i = 0$). Because self-attention is bidirectional, the final hidden state $\mathbf{h}_{[\text{CLS}]} \in \mathbb{R}^{d_{\text{model}}}$ aggregates context from the entire sequence and serves as the pooled representation for sentence-level classification.
- `[SEP]` (**Separator**): Appended to mark the boundary between Sentence $A$ and Sentence $B$, as well as the termination of the sequence.

### 6.2.2 Pre-training task 1: Masked Language Model (MLM)

Analyzing the MLM objective is crucial because bidirectional representations cannot be pre-trained using standard autoregressive next-token prediction. In a bidirectional self-attention model, token $i$ can attend to all tokens $j > i$. If tasked with predicting $x_{i+1}$, the network would simply route attention to the next position and trivial identity-copy the answer, collapsing gradient learning. The MLM task formulates a non-trivial, label-leakage-free self-supervised objective inspired by the Cloze procedure [52].

In each input sequence, a subset $\mathcal{M} \subset \{1, 2, \ldots, L\}$ representing approximately 15% of all input tokens is uniformly selected to provide pre-training supervision. However, how should these selected positions be presented to the network?

The naive Cloze procedure would simply replace every selected token in $\mathcal{M}$ with the artificial marker `[MASK]`. While conceptually straightforward, this naive strategy introduces a important limitation known as the **pre-training vs. fine-tuning discrepancy** (or domain shift):

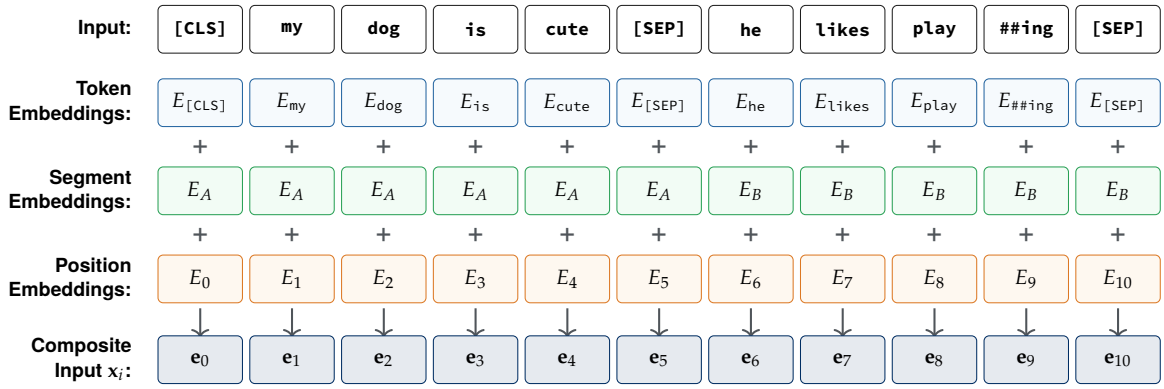



Figure 6.3: BERT input representation structure [52]. Each input vector $\mathbf{e}_i$ is constructed by the element-wise sum of its Token Embedding (WordPiece vocabulary), Segment Embedding ($E_A$ for Sentence $A$, $E_B$ for Sentence $B$), and Position Embedding ($E_0, \ldots, E_{10}$). Sequences begin with `[CLS]` and separate sentences with `[SEP]`. Distinct learned segment embeddings allow the self-attention mechanism to discriminate between paired sentences.

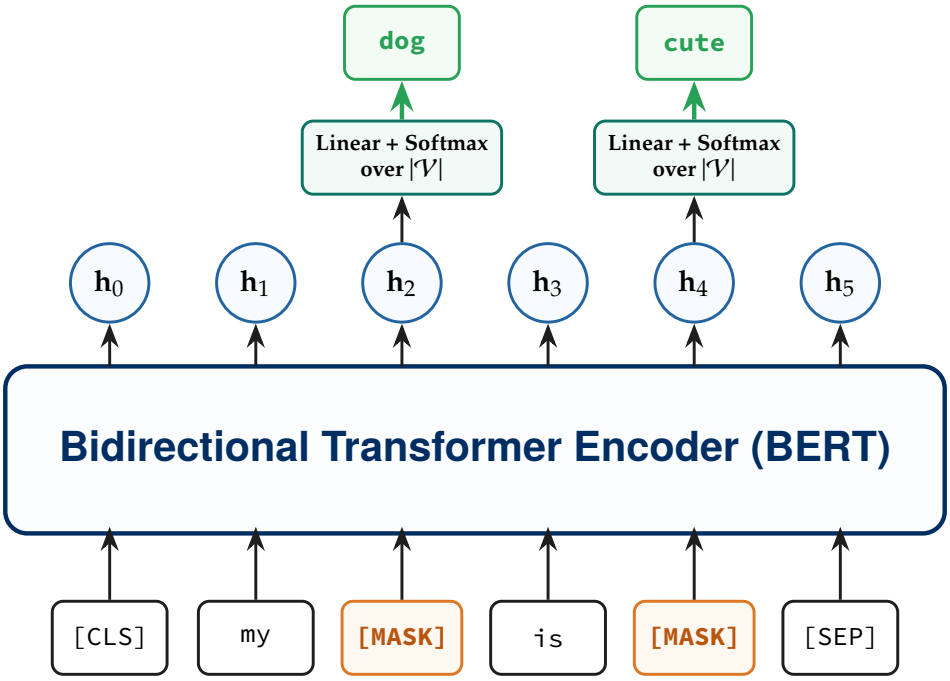



Figure 6.4: The Masked Language Model (MLM) pre-training task in BERT. 15% of input tokens are corrupted using the 80/10/10 masking rule. The model passes corrupted sequences through bidirectional self-attention and trains linear classification heads over the full vocabulary $\mathcal{V}$ specifically on hidden representations $\mathbf{h}_i$ at masked positions to reconstruct the original ground-truth tokens.

1. **Absence of `[MASK]` in downstream tasks**: When BERT is subsequently fine-tuned on real-world applications (e.g., sequence classification, sentiment analysis, named entity recognition, or question answering), the input text consists exclusively of genuine natural language tokens. The artificial delimiter `[MASK]` *never appears during inference*.

2. **Pathological representation specialization**: If the encoder were exposed to `[MASK]` whenever a prediction was required, its internal self-attention heads would learn a degenerative heuristic: treat all regular words as unquestioned, static context, and only activate deep predictive feature extraction when encountering the explicit `[MASK]` token. Consequently, during fine-tuning, the network would fail to generate optimal contextual representations for normal tokens.

3. **Neglect of the observed token**: If target positions are always overwritten with a mask symbol, the model never learns to evaluate or preserve the identity of the token physically occupying position $i$.

To force the model to build high-quality contextual representations for *all* tokens—while keeping the model uncertain about which positions are being graded—Devlin et al. [52] formulated the **80/10/10 corruption policy**. Once the 15% target positions $\mathcal{M}$ are chosen, they are perturbed according to three distinct operations:

1. **The 80% replacement with `[MASK]` (Cloze semantic reconstruction)**: In 80% of the selected positions (12% of total sequence tokens), the original token is replaced with `[MASK]`:

$$\texttt{"my dog is cute"} \longrightarrow \texttt{"my [MASK] is cute.}$$

   *Motivation*: This forms the core Cloze engine. Because the ground-truth token is physically removed from the input stream, the model cannot copy it. It is compelled to synthesize bidirectional semantic and syntactic clues from the entire left and right contexts (e.g., using `"my"` and `"is cute"` to infer a domestic pet like `"dog"`).

2. **The 10% replacement with a random token (Denoising and contextual validation)**: In 10% of the selected positions (1.5% of total sequence tokens), the target token is replaced with an arbitrary token drawn uniformly from the vocabulary $w_{\text{rand}} \sim \mathcal{V}$:

$$\texttt{"my dog is cute"} \longrightarrow \texttt{"my apple is cute.}$$

   *Motivation*: If the model only ever saw `[MASK]` or correct words, it would assume that every non-masked word in the sentence is guaranteed to be 100% truthful and contextually valid. By injecting random corrupted words, self-attention heads are forced to act as an active **anomaly detector** and **denoiser**. When the encoder encounters `"apple"` framed by `"my"` and `"is cute"`, it must detect that `"apple"` creates a semantic contradiction, discount the corrupted surface form, and reconstruct the true underlying concept (`"dog"`). This prevents the network from blindly trusting unmasked tokens.

3. **The 10% retention of the original token (Grounding and representation anchoring)**: In the remaining 10% of selected positions (1.5% of total sequence tokens), the token is kept strictly unchanged:

$$\texttt{"my dog is cute"} \longrightarrow \texttt{"my dog is cute.}$$

   *Motivation*: If target positions were *always* altered (either masked or replaced with random noise), the encoder would learn that the input symbol at target position $i$ is never the right answer, incentivizing it to completely discard the token's own embedding. By leaving the correct word intact 10% of the time, the model is taught that the observed word may well be genuine. This forces the final hidden representation $\mathbf{h}_i$ to remain faithfully anchored to the actual lexical token observed at that position, directly bridging the gap to downstream fine-tuning where all input tokens are real.

Because of this tri-fold distribution, when BERT computes self-attention across any token $x_i$, it operates under total uncertainty:

- The network does not know whether token $x_i$ is one of the supervised targets being evaluated by cross-entropy loss.
- If $x_i$ is a target, the network does not know whether it represents an uncorrupted word, an anomalous impostor, or an explicit mask.

This deliberate ambiguity eliminates any possibility of lazy shortcut heuristics, compelling the Transformer encoder to maintain an expressive, contextually verified representation $\mathbf{h}_i$ for **every single token** across the entire sequence.

For each selected position $i \in \mathcal{M}$, the topmost encoder output $\mathbf{h}_i \in \mathbb{R}^{d_{\text{model}}}$ is projected onto the full vocabulary $\mathcal{V}$ through a linear classification layer parameterized by $\mathbf{W}_{\text{MLM}} \in \mathbb{R}^{|\mathcal{V}| \times d_{\text{model}}}$ (whose weights are tied with the input token embedding matrix $\mathbf{E}_{\text{tok}}$) and bias $\mathbf{b}_{\text{MLM}} \in \mathbb{R}^{|\mathcal{V}|}$:

$$P(w \mid \tilde{\mathbf{X}}; \theta) = \frac{\exp(\mathbf{w}_w^\top \mathbf{h}_i + b_w)}{\sum_{v \in \mathcal{V}} \exp(\mathbf{w}_v^\top \mathbf{h}_i + b_v)}, \quad \forall w \in \mathcal{V}, \tag{6.2}$$

where $\tilde{\mathbf{X}}$ denotes the corrupted input sequence. The MLM pre-training objective minimizes the cross-entropy loss aggregated across all masked positions:

$$\mathcal{L}_{\text{MLM}}(\theta) = -\sum_{i \in \mathcal{M}} \log P(w_i^* \mid \tilde{\mathbf{X}}; \theta), \tag{6.3}$$

where $w_i^*$ is the original ground-truth token at position $i$.

### 6.2.3 Pre-training task 2: Next Sentence Prediction (NSP)

While the Masked Language Model objective forces the encoder to capture deep intra-sentence lexical and syntactic relationships, many critical downstream tasks—such as Question Answering (QA), Natural Language Inference (NLI), and code-comment retrieval—depend fundamentally on understanding logical coherence across *pairs* of sentences. Analyzing NSP explains how the network develops cross-sentence relational awareness, and motivates why subsequent architectures (e.g., RoBERTa) later audited whether NSP was truly necessary or suboptimal.

During corpus pre-training, input sequences are constructed by sampling pairs of sentences $(A, B)$ concatenated with segment markers (Figure 6.5):

- **50% of the time (Label $y = 1$, IsNext)**: Sentence $B$ is the actual contiguous sentence immediately succeeding Sentence $A$ in the source document.
- **50% of the time (Label $y = 0$, NotNext)**: Sentence $B$ is a random sentence sampled independently from an entirely unrelated document in the pre-training corpus.

To classify the sentence pair without introducing an ad-hoc pooling mechanism, BERT leverages the special leading token representation $\mathbf{h}_{[\text{CLS}]} \in \mathbb{R}^{d_{\text{model}}}$. Because self-attention is bidirectional, $\mathbf{h}_{[\text{CLS}]}$ computes attention over every token in both Sentence $A$ and Sentence $B$. This pooled vector is projected through a non-linear dense transformation:

$$\mathbf{h}_{\text{pool}} = \tanh(\mathbf{W}_{\text{pool}} \mathbf{h}_{[\text{CLS}]} + \mathbf{b}_{\text{pool}}), \quad \mathbf{W}_{\text{pool}} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}, \ \mathbf{b}_{\text{pool}} \in \mathbb{R}^{d_{\text{model}}}. \tag{6.4}$$

The binary prediction probability is emitted by a logistic regression head:

$$\hat{y}_{\text{NSP}} = \sigma(\mathbf{w}_{\text{NSP}}^\top \mathbf{h}_{\text{pool}} + b_{\text{NSP}}), \quad \mathbf{w}_{\text{NSP}} \in \mathbb{R}^{d_{\text{model}}}, \ b_{\text{NSP}} \in \mathbb{R}, \tag{6.5}$$

and evaluated under the binary cross-entropy loss:

$$\mathcal{L}_{\text{NSP}}(\theta) = -y \log \hat{y}_{\text{NSP}} - (1 - y) \log(1 - \hat{y}_{\text{NSP}}). \tag{6.6}$$

The unified pre-training objective jointly optimizes the composite loss across all sampled sequences:

$$\mathcal{L}_{\text{BERT}}(\theta) = \mathcal{L}_{\text{MLM}}(\theta) + \mathcal{L}_{\text{NSP}}(\theta). \tag{6.7}$$

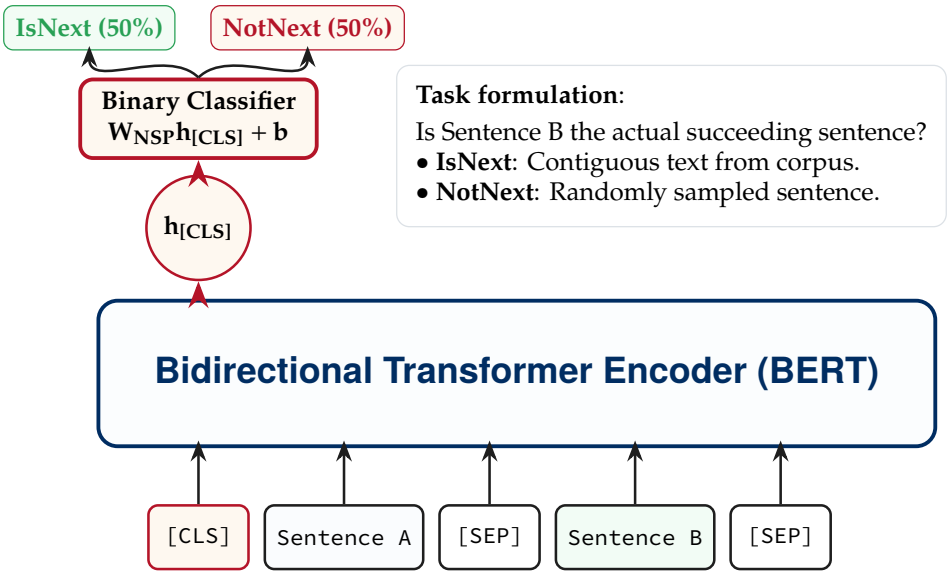



Figure 6.5: The Next Sentence Prediction (NSP) pre-training task in BERT. Two sentences are packed with delimiters `[CLS]` Sentence $A$ `[SEP]` Sentence $B$ `[SEP]`. The pooled final representation $\mathbf{h}_{[\text{CLS}]}$ is projected through a binary classification head $\mathbf{W}_{\text{NSP}}$ to predict whether Sentence $B$ logically follows Sentence $A$.

### 6.2.4 Attention map anatomy and emergent linguistic structures

Analyzing internal attention maps demystifies the "black box" of deep Transformer encoders. It allows researchers to verify whether the statistical optimizations of MLM and NSP genuinely induce human-like linguistic and syntactic competencies, or whether the model merely relies on shallow statistical surface correlations. By visualizing the attention matrices $\mathbf{A}^{(l,h)}$, we observe how individual attention heads autonomously specialize into interpretable linguistic operators.

A standard BERT-Base model comprises $L = 12$ stacked encoder layers with $h = 12$ attention heads per layer, operating across a hidden dimension of $d_{\text{model}} = 768$ ($d_k = d_{\text{model}}/h = 64$). Consequently, for any input sequence, the network computes:

$$12 \times 12 = 144 \text{ distinct attention heads (and corresponding attention maps).} \tag{6.8}$$

For an input sequence of length $N$, each attention head generates an $N \times N$ row-stochastic attention weight matrix $\mathbf{A}^{(l,h)} \in [0,1]^{N \times N}$ satisfying $\sum_{j=1}^{N} A_{ij}^{(l,h)} = 1$.

Consider the canonical linguistic probe evaluated in [60]:

`[CLS] The dog ate the food because it was hungry [SEP]` ($N = 11$ tokens).

Across the 144 attention maps, empirical probing reveals that self-attention layers autonomously partition into highly specialized linguistic registers without explicit syntactic supervision (Figure 6.6):

1. **Coreference and anaphora resolution (intermediate layers, e.g., Layer 5, Head 4)**: Specific attention heads in the middle layers specialize in resolving pronominal anaphora across clausal boundaries. In the running example, when computing attention from the pronoun query token `"it"`, the head routes over 82% of its total attention mass directly to the antecedent noun key `"dog"` ($\alpha = 0.82$). This allows the contextual representation of `"it"` to absorb the semantic properties of the entity it replaces.

2. **Syntactic dependency tracking (early-to-intermediate layers)**: Distinct heads capture grammatical relations: verb-object heads route strong attention from transitive verbs to

their direct objects (e.g., "ate" → "food" with $\alpha = 0.74$), determiner heads link articles to head nouns ("The" → "dog", "the" → "food"), and predicate heads connect adjectives to sentence subjects ("hungry" → "dog").

3. **Surface sequential tracking (early layers)**: In early layers, several heads attend predominantly to adjacent tokens ($i \to i-1$ or $i \to i+1$), preserving localized $n$-gram ordering within the permutation-invariant self-attention matrix.

4. **The delimiter and neutral register phenomenon (deep layers, e.g., Layer 10, Head 8)**: In deeper layers, Clark et al. [60] and Michel et al. [61] observed that multiple attention heads allocate between 50% and 90% of their total attention weight directly to the special delimiter token [SEP]. This behavior arises from a fundamental mathematical constraint: the softmax operator strictly enforces $\sum_{j=1}^{N} A_{ij} = 1$. When a specialized attention head (such as a coreference head) processes a token that does not participate in that specific linguistic relation (e.g., the conjunction "because" or punctuation tokens), the head must still distribute a probability mass of 1.0. Distributing this mass across arbitrary semantic words would inject irrelevant noise and distort their representations. Therefore, the network designates [SEP] as a **neutral default register / "no-op" target**: unassigned attention is dumped onto [SEP], leaving the contextual representation of the source token virtually unaltered ($\Delta \mathbf{h} \approx \mathbf{0}$).

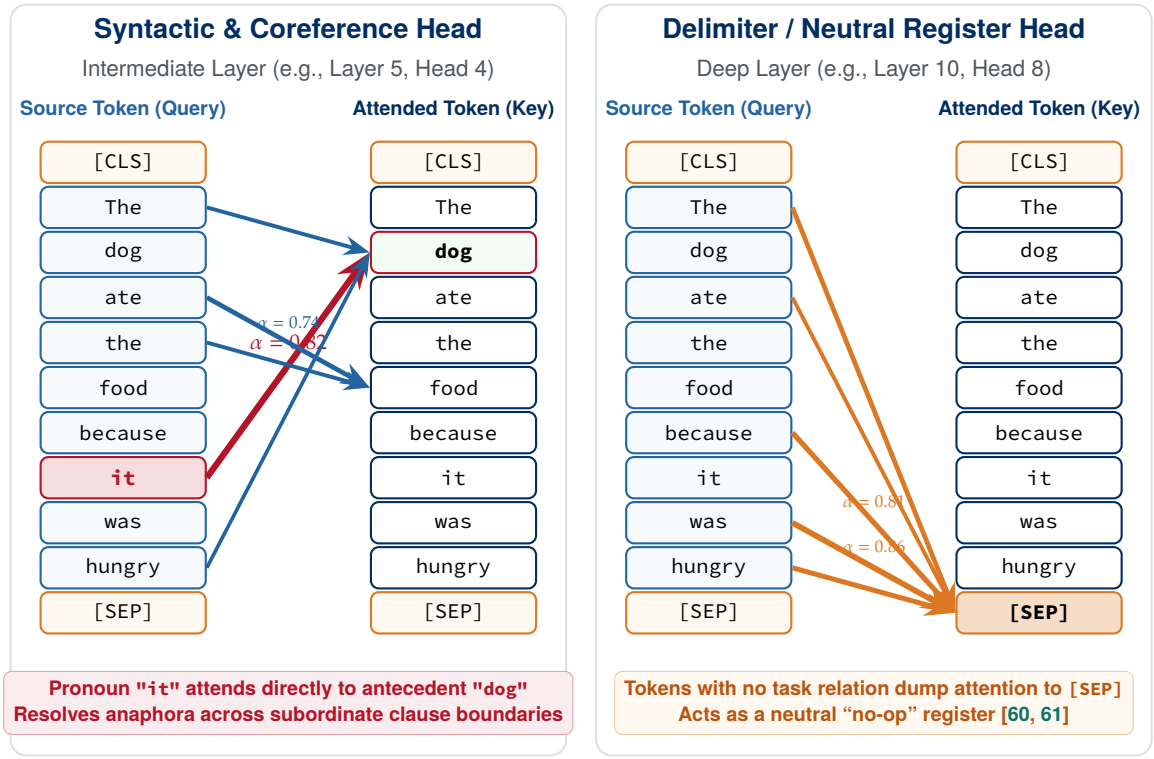
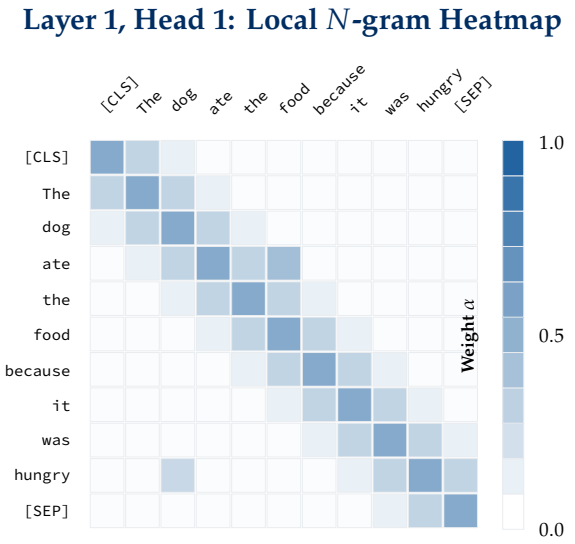


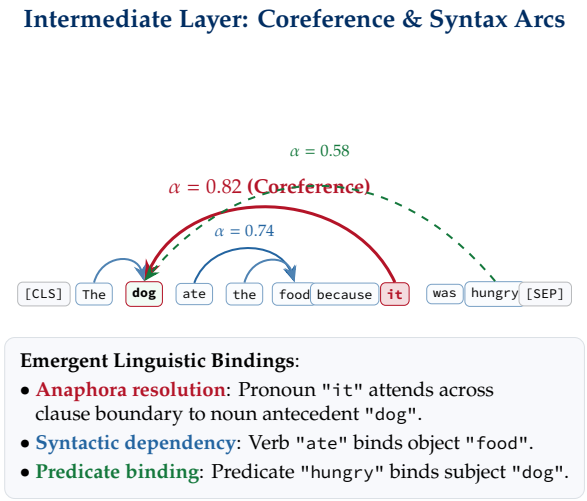

Figure 6.6: Emergent linguistic specialization and delimiter behavior across BERT attention heads for the input sequence "[CLS] The dog ate the food because it was hungry [SEP]" (11 tokens, evaluated across $12 \times 12 = 144$ attention maps). Left panel: An intermediate-layer head (Layer 5, Head 4) resolves anaphoric coreference by routing strong attention from pronoun "it" to antecedent "dog" ($\alpha = 0.82$), alongside verb-object dependencies ("ate" → "food"). Right panel: A deep-layer head (Layer 10, Head 8) exhibits delimiter pooling where unassigned tokens dump over 80% of their attention mass onto [SEP], functioning as a neutral no-op register that leaves vector representations undisturbed [60, 61].

### Empirical layer-wise emergence: syntax and delimiter registers

To deeply understand how representations transform across the encoder depth, we inspect the actual attention heatmaps across the entire network hierarchy (Figures 6.7a and 6.8). This reveals a pronounced, progressive stratification of linguistic functions across depth:




(a) Attention heatmap for Layer 1, Head 1. Attention mass concentrates tightly along the diagonal ($i \to i, i \pm 1$), functioning like local convolutional filters to preserve immediate sequential n-grams.




(b) Intermediate-layer attention head resolving ambiguous pronominal coreference. Query token "it" allocates massive attention mass ($\alpha = 0.82$) to antecedent "dog", allowing BERT to accurately bind pronominal entities across clausal boundaries.

Figure 6.7: Emergent internal representation geometry across BERT attention heads. Early layers form localized n-gram filters, while intermediate layers synthesize non-local syntactic and pronominal coreference dependencies without supervised parse trees.

The emergent patterns across the layers decompose into three canonical phases:

1. **Early layers (Layers 1–3: Surface and Local N-gram Geometry)**: As shown in Figure 6.7a, attention weights are concentrated heavily along the main diagonal ($i \to i$) and adjacent off-diagonals ($i \to i-1$ and $i \to i+1$). The model first reconstructs sequential positional adjacency, functioning much like convolutional filters with small receptive fields. Furthermore, basic lexical dependencies begin forming (e.g., "ate" linking with "food").

2. **Middle layers (Layers 4–8: Syntactic Trees and Anaphora Resolution)**: In the intermediate layers, attention patterns become markedly sparse and structured (Figure 6.7b). The heads no longer attend to adjacent words; instead, they bridge long-range syntactic gaps. Dedicated heads track verb-object dependencies, noun-modifier bindings, and pronoun coreferences across subordinate conjunctions ("it" $\to$ "dog"). This proves that the multi-head mechanism divides labor: different heads track different syntactic relations concurrently.

3. **Deep layers (Layers 9–12: High-Level Task Abstraction and Delimiter Pooling)**: In the final layers, attention distributions shift again. Many heads show broad, diffuse patterns, while a significant fraction dump 80%–90% of their attention mass onto the special token [SEP]. Because the final layers synthesize task-level features, tokens that have already aggregated their required syntactic context utilize [SEP] as an inert sink. This preserves their accumulated representation vectors without absorbing unwanted semantic drift in the topmost layers before classification or MLM loss computation.

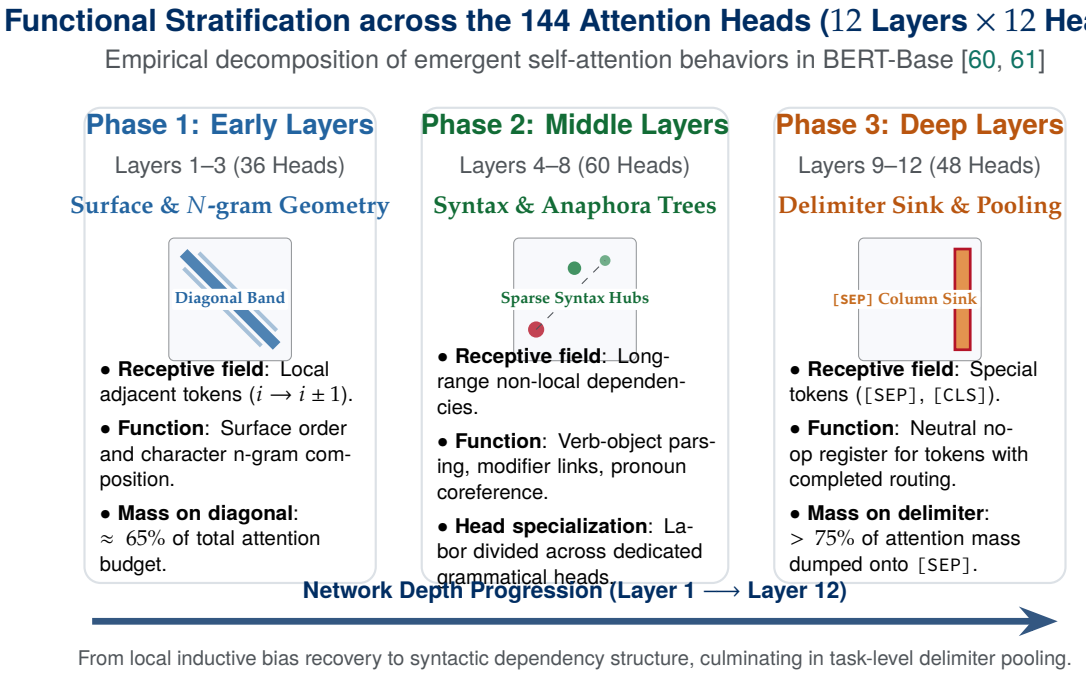



Figure 6.8: The emergent landscape of $12 \times 12 = 144$ attention maps across BERT-Base. Rather than an uninterpretable black box, self-attention exhibits clear layer-wise functional stratification: early layers act as local n-gram filters (diagonal band); middle layers construct sparse syntactic trees and resolve pronominal coreference; final layers pool inactive representations into the `[SEP]` delimiter register [60, 61].

### 6.2.5 Downstream fine-tuning mechanisms

A pre-trained foundation model is only as valuable as its adaptability to real-world tasks. Analyzing BERT's downstream adaptation demonstrates the paradigm shift from task-specific feature engineering to task-agnostic representation fine-tuning. By modifying only the topmost prediction layer while allowing gradients to backpropagate through all 110 million parameters, BERT achieves state-of-the-art performance across diverse natural language tasks with minimal training time.

Following pre-training on MLM and NSP, the generic pre-training classification layers are discarded. The pre-trained Transformer weights are retained, and minimal task-specific projection heads are appended to adapt the model to downstream NLP applications with end-to-end gradient updates:

1. **Sequence-level classification** (e.g., sentiment analysis, code defect prediction, natural language inference): The pooled sentence representation $\mathbf{h}_{\text{pool}} \in \mathbb{R}^{d_{\text{model}}}$ from Equation (6.4) is projected through a task-specific linear head:

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{W}_{\text{seq}}\mathbf{h}_{\text{pool}} + \mathbf{b}_{\text{seq}}), \quad \mathbf{W}_{\text{seq}} \in \mathbb{R}^{C \times d_{\text{model}}}, \ \mathbf{b}_{\text{seq}} \in \mathbb{R}^{C}, \tag{6.9}$$

   where $C$ denotes the number of target classes. All parameters (both $\mathbf{W}_{\text{seq}}$ and the encoder backbone) are fine-tuned via cross-entropy loss.

2. **Token-level sequence labeling** (e.g., Named Entity Recognition NER, Part-of-Speech tagging, code token vulnerability tagging): Instead of pooling, the final contextual representation $\mathbf{h}_i \in \mathbb{R}^{d_{\text{model}}}$ of each individual token $i \in \{1, 2, \ldots, L\}$ is mapped to a discrete label distribution:

$$\hat{\mathbf{y}}_i = \text{softmax}(\mathbf{W}_{\text{tok}}\mathbf{h}_i + \mathbf{b}_{\text{tok}}), \quad \mathbf{W}_{\text{tok}} \in \mathbb{R}^{K \times d_{\text{model}}}, \ \mathbf{b}_{\text{tok}} \in \mathbb{R}^{K}, \tag{6.10}$$

where $K$ is the number of entity or syntactic tags.

3. **Extractive Question Answering** (e.g., SQuAD): Conditioned on a passage and a question concatenated as Sentence $A$ and Sentence $B$, the model identifies the continuous answer span by learning a start vector $\mathbf{w}_{\text{start}} \in \mathbb{R}^{d_{\text{model}}}$ and an end vector $\mathbf{w}_{\text{end}} \in \mathbb{R}^{d_{\text{model}}}$. The probability that token $i$ is the start of the answer span is:

$$P_{\text{start}}(i) = \frac{\exp(\mathbf{w}_{\text{start}}^{\top} \mathbf{h}_i)}{\sum_{j=1}^{L} \exp(\mathbf{w}_{\text{start}}^{\top} \mathbf{h}_j)}, \tag{6.11}$$

with an identical formulation for $P_{\text{end}}(i)$.

While the Encoder-Decoder paradigm addresses sequence transduction and the Encoder-Only paradigm excels at bidirectional representation learning and sequence classification, pure autoregressive generative tasks (such as open-ended text synthesis and conversational reasoning) rely on Decoder-Only architectures. We defer the comprehensive architectural analysis of large-scale Decoder-Only models to Chapter 6, where scaling laws and autoregressive pre-training are developed in full detail.

However, regardless of whether generation occurs within the decoder stack of an Encoder-Decoder model or a dedicated autoregressive language decoder, the fundamental problem of selecting discrete tokens from continuous output distributions remains mathematically invariant. In the subsequent section, we formalize the probabilistic mechanics governing autoregressive decoding.

## 6.3 Decoding Strategies

At each step $t$ of autoregressive sequence generation, an autoregressive language decoder projects its topmost contextual hidden state $\mathbf{h}_t \in \mathbb{R}^{d_{\text{model}}}$ through an unembedding linear transformation $\mathbf{W}_U \in \mathbb{R}^{|\mathcal{V}| \times d_{\text{model}}}$ parameterized with vocabulary bias $\mathbf{b}_U \in \mathbb{R}^{|\mathcal{V}|}$. This maps the continuous latent manifold into raw unnormalized affinities over discrete token space:

$$\mathbf{z}_t = \mathbf{W}_U \mathbf{h}_t + \mathbf{b}_U \in \mathbb{R}^{|\mathcal{V}|}, \tag{6.12}$$

where $\mathbf{z}_t = [z_{t,1}, z_{t,2}, \ldots, z_{t,|\mathcal{V}|}]^\top \in \mathbb{R}^{|\mathcal{V}|}$ denotes the unnormalized log-affinity (logit) vector across the vocabulary $\mathcal{V} = \{v_1, v_2, \ldots, v_{|\mathcal{V}|}\}$. Passing $\mathbf{z}_t$ through the canonical softmax operator $\text{softmax} : \mathbb{R}^{|\mathcal{V}|} \to \Delta^{|\mathcal{V}|-1}$ maps the continuous unbounded logits onto the standard $(|\mathcal{V}| - 1)$-dimensional probability simplex $\Delta^{|\mathcal{V}|-1} = \{\mathbf{p} \in \mathbb{R}^{|\mathcal{V}|} : p_i \geq 0, \sum_{i=1}^{|\mathcal{V}|} p_i = 1\}$.

This defines the discrete categorical conditional probability mass function governing the next-token random variable $X_t \sim \text{Categorical}(\mathbf{p}_t)$:

$$\begin{aligned} P(X_t = v_i \mid w_{<t}) &= [\text{softmax}(\mathbf{z}_t)]_i \\ &= \frac{\exp(z_{t,i})}{\sum_{j=1}^{|\mathcal{V}|} \exp(z_{t,j})} \\ &= \frac{\exp\left(\mathbf{w}_{U,i}^\top \mathbf{h}_t + b_{U,i}\right)}{\sum_{j=1}^{|\mathcal{V}|} \exp\left(\mathbf{w}_{U,j}^\top \mathbf{h}_t + b_{U,j}\right)}, \quad \forall v_i \in \mathcal{V}, \end{aligned} \tag{6.13}$$

where $\mathbf{w}_{U,i}^\top \in \mathbb{R}^{1 \times d_{\text{model}}}$ denotes the $i$-th row vector of the unembedding matrix $\mathbf{W}_U$ (often tied with the input embedding matrix $\mathbf{E}_{\text{tok}}^\top$ in weight-tied models), and $b_{U,i} \in \mathbb{R}$ represents the corresponding vocabulary bias term. By construction, the probabilities satisfy the Kolmogorov axioms:

$$P(X_t = v_i \mid w_{<t}) \in (0, 1) \quad \text{and} \quad \sum_{i=1}^{|\mathcal{V}|} P(X_t = v_i \mid w_{<t}) = 1. \tag{6.14}$$

In modern language models, the vocabulary cardinality $|\mathcal{V}|$ spans tens of thousands of subword units (for example, $|\mathcal{V}| = 30{,}522$ in BERT's WordPiece, $|\mathcal{V}| = 50{,}257$ in GPT-2's Byte-Pair Encoding). Consequently, $\mathbf{p}_t$ represents a massive, highly non-uniform probability distribution over tens of thousands of discrete outcomes.

Once this dense probability distribution $P(w_i \mid w_{<t})$ is obtained, the fundamental challenge of autoregressive inference arises: **how do we select the discrete token $\hat{w}_t$ to append to the generated sequence?** The choice of decoding policy governs the trade-off between deterministic precision and stochastic diversity, directly deciding whether the model produces factual truth, creative narrative, generic clichés, or ungrammatical gibberish.

To establish an identical, controlled empirical testbed across all decoding policies studied in this section, we condition each strategy on the opening prompt drawn from Arthur Conan Doyle's *The Red-Headed League* (as examined in lecture):

> *"I had called upon my friend, Mr. Sherlock Holmes, one day in the autumn of last year and found him in deep conversation with a very stout, florid-faced, elderly gentleman with fiery red hair. With an apology for my intrusion, I was about to withdraw when Holmes pulled me abruptly into the room and closed the door behind me."*

In the following subsections, we analyze the six canonical decoding paradigms in strict linear progression: from deterministic greedy selection and beam optimization to stochastic sampling, entropy modulation, and dynamic truncation filters.

### 6.3.1 Greedy decoding (argmax) and the degeneration trap

The simplest and computationally most direct decoding policy is **Greedy Search**. At each temporal step $t$, the algorithm deterministically selects the single token assigned the highest instantaneous conditional probability:

$$\hat{w}_t = \text{argmax}_{w \in \mathcal{V}} P(w \mid w_{<t}). \tag{6.15}$$

Greedy decoding requires zero hyperparameters and incurs minimal computational overhead: it performs a single $O(|\mathcal{V}|)$ reduction across the logit vector at each time step.

Suppose after processing the prompt, the model emits the normalized probabilities across a representative candidate subset shown in Table 6.1.

Table 6.1: Worked probability distribution for next-token selection at step $t = 1$.

| **Token Candidate** $w$ | `"I` | `"He` | `"The` | `"Mr.` | `"What` | ... [Remaining 50,252 tokens] |
|---|---|---|---|---|---|---|
| **Logit** $z$ | 14.20 | 13.50 | 12.80 | 11.90 | 10.10 | $< 8.00$ |
| **Softmax** $P(w \mid w_{<t})$ | **0.521** | 0.260 | 0.130 | 0.053 | 0.009 | $\sum = 0.027$ |

Under greedy search, the algorithm evaluates $\text{argmax}_w P(w \mid w_{<t})$ and deterministically emits token $\hat{w}_t =$ `"I`. All remaining hypotheses are discarded permanently.

#### Failure mode: greedy myopia and cyclic repetition

Despite its superficial appeal, greedy decoding exhibits two substantial failure modes in open-ended language generation:

1. **Mathematical Myopia (Local Greediness ≠ Global Sequence Optimality)**: Sequence likelihood factorizes via the probabilistic chain rule as $P(w_{1:T}) = \prod_{t=1}^{T} P(w_t \mid w_{<t})$. A greedy local maximum at step $t$ frequently leads into a barren probability subspace at subsequent steps:

$$\text{argmax}_{w_1,\ldots,w_T} \prod_{t=1}^{T} P(w_t \mid w_{<t}) \neq \prod_{t=1}^{T} \text{argmax}_{w_t} P(w_t \mid w_{<t}). \tag{6.16}$$

   As illustrated in Figure 6.9, committing to token $A$ because $P(A) = 0.60 > P(B) = 0.40$ traps the generator if $A$ can only be followed by low-probability tokens ($P(C \mid A) = 0.10$, yielding path likelihood 0.06), whereas selecting the locally sub-optimal token $B$ uncovers a highly fluent sequence ($P(D \mid B) = 0.80$, yielding path likelihood 0.32).

2. **Neural Text Degeneration**: Holtzman et al. [62] demonstrated that greedy decoding on neural language models triggers self-reinforcing attractor loops. Common syntactic markers (e.g., pronouns, auxiliary verbs) dominate local argmax selections. Once a repetitive n-gram is appended to the context, self-attention attending back to those identical tokens artificially elevates their conditional probabilities even higher at subsequent steps, locking the model into an inescapable loop of cyclic degeneration:

   ```
   "I am sorry, Mr. Holmes," I said. "I am sorry, Mr. Holmes, for having
   been so rude to you. I am sorry, Mr. Holmes, for having been so rude
   to you. I am sorry, Mr. Holmes, for having been so rude to you..."
   ```

Because greedy search suffers from irrecoverable path pruning, we require an algorithm capable of looking ahead and exploring multiple alternative sequences concurrently. This directly motivates **Beam Search**.

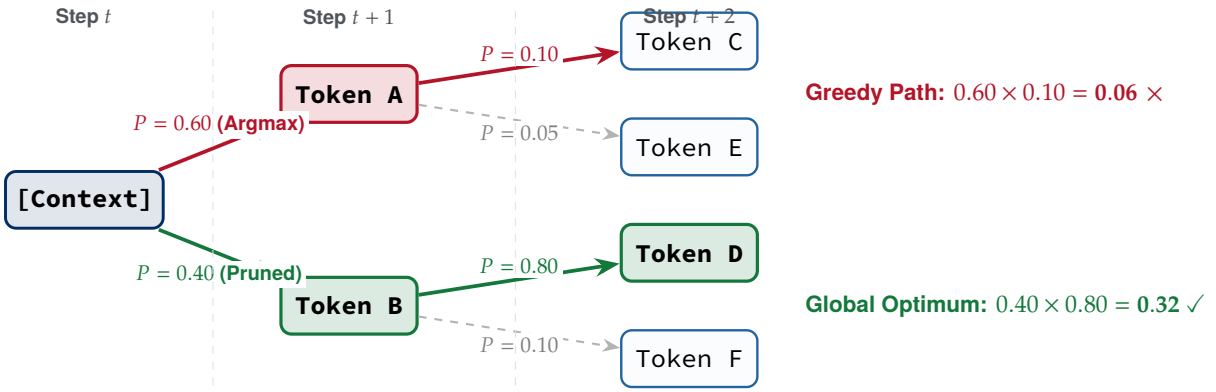



Figure 6.9: The failure mechanism of greedy decoding. By greedily selecting `Token A` ($P = 0.60$) over `Token B` ($P = 0.40$) at step $t+1$, the search space collapses into low-probability continuations ($P = 0.06$). The global optimum sequence through `Token B` ($P = 0.32$) is permanently pruned.

### 6.3.2 Beam search: exploring parallel hypotheses

**Beam Search** avoids the strict myopia of greedy decoding by maintaining a fixed set of $B$ top candidate sequences (termed the *beam width* $B$, or $k$ in literature) at each time step.

Let $\mathcal{Y}_t = \{\mathbf{y}_t^{(1)}, \mathbf{y}_t^{(2)}, \ldots, \mathbf{y}_t^{(B)}\}$ denote the set of active partial hypotheses of length $t$. At generation step $t+1$:

1. Every hypothesis $\mathbf{y}_t^{(b)} \in \mathcal{Y}_t$ is expanded across all $|\mathcal{V}|$ vocabulary candidates, forming $B \times |\mathcal{V}|$ child candidate sequences $\mathbf{y}_{t+1} = (\mathbf{y}_t^{(b)}, w)$.

2. The cumulative score of each sequence is evaluated in logarithmic space to prevent numerical underflow:
$$\text{Score}(\mathbf{y}_{1:t+1}) = \sum_{i=1}^{t+1} \log P(y_i \mid \mathbf{y}_{<i}). \tag{6.17}$$

3. The expanded set of $B \cdot |\mathcal{V}|$ candidates is sorted, and only the top-$B$ candidates with the highest cumulative log-probabilities are preserved:
$$\mathcal{Y}_{t+1} = \text{argtop-B}_{\mathbf{y} \in \mathcal{Y}_t \times \mathcal{V}} \text{Score}(\mathbf{y}). \tag{6.18}$$
All other $B \cdot (|\mathcal{V}| - 1)$ hypotheses are permanently pruned.

#### Length penalty normalization

Because $\log P(y_i \mid \mathbf{y}_{<i}) \leq 0$ for all valid probabilities, the objective in Equation (6.17) is strictly non-increasing with sequence length. Consequently, unconstrained beam search inherently penalizes longer generations, favoring unnaturally short, truncated outputs. To counteract this bias, standard implementations incorporate a length normalization heuristic:

$$\text{Score}_{\text{norm}}(Y) = \frac{\sum_{i=1}^{|Y|} \log P(y_i \mid \mathbf{y}_{<i})}{\text{LP}(Y)}, \quad \text{where} \quad \text{LP}(Y) = \frac{(5 + |Y|)^\alpha}{(5+1)^\alpha}, \tag{6.19}$$

with length penalty parameter $\alpha \in [0.6, 1.0]$. When $\alpha = 0$, no length normalization occurs; when $\alpha = 1.0$, the score is normalized by length.

Consider beam search with beam width $B = 2$ expanding from the root token `"The"`, exactly replicating the lecture derivation:

1. **Step** $t = 1$: From `"The"`, candidate probabilities are:

$$P(\texttt{"nice"} \mid \texttt{"The"}) = 0.50, \quad P(\texttt{"dog"} \mid \texttt{"The"}) = 0.40, \quad P(\texttt{"car"} \mid \texttt{"The"}) = 0.10. \quad (6.20)$$

The top-2 beams retained are $\mathcal{Y}_1 = \{(\texttt{"The"}, \texttt{"nice"}), (\texttt{"The"}, \texttt{"dog"})\}$. The branch `"car"` is pruned.

2. **Step** $t = 2$: Each of the 2 active beams expands its candidates:
   - From `"The dog"`: `"has"` ($P = 0.90 \implies$ joint $0.40 \times 0.90 = 0.36$), `"and"` ($P = 0.05 \implies$ joint 0.02), `"run"` ($P = 0.05 \implies$ joint 0.02).
   - From `"The nice"`: `"woman"` ($P = 0.40 \implies$ joint $0.50 \times 0.40 = 0.20$), `"house"` ($P = 0.30 \implies$ joint 0.15), `"guy"` ($P = 0.30 \implies$ joint 0.15).

   Across all 6 expanded paths, the top-2 global sequences are `"The dog has"` ($P = 0.36$) and `"The nice woman"` ($P = 0.20$). All other 4 paths are pruned.

Figure 6.10 illustrates this exact operational trellis.

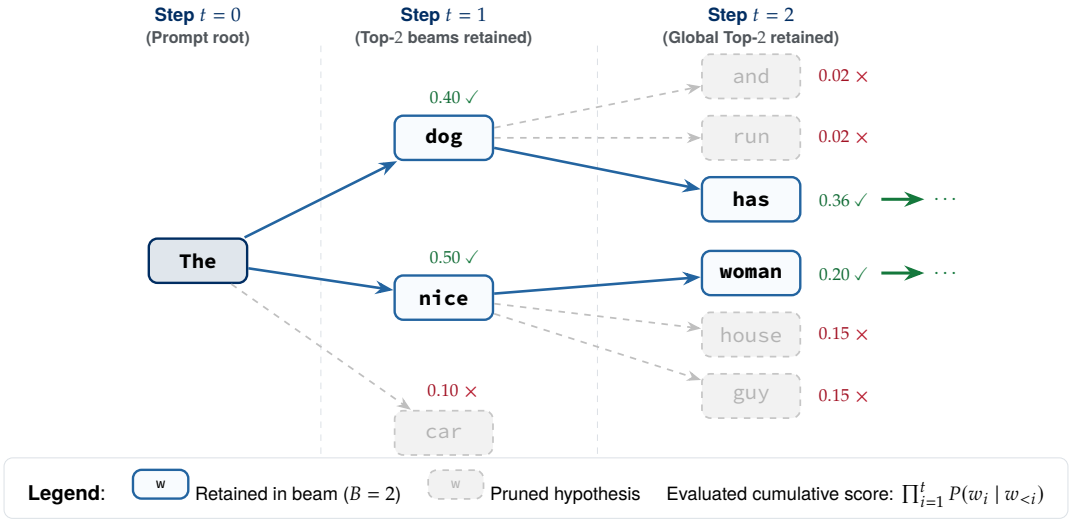



Figure 6.10: Beam search decoding with beam width $B = 2$. At each generation step, all $B$ candidate sequences are expanded across vocabulary candidates. The algorithm prunes lower-probability branches, maintaining only the top-$B$ cumulative probability hypotheses (here, `"The dog has"` with $P = 0.36$ and `"The nice woman"` with $P = 0.20$).

### Failure mode: beam search degradation in open-ended generation

While beam search is the gold standard for closed, goal-oriented tasks with narrow target spaces (such as neural machine translation and formal code generation), it fails strikingly in open-ended creative text generation:

- **Deterministic Blandness and Clichés**: Human communication does not optimize for maximum likelihood at every step. Natural human language contains an alternating rhythm of predictable syntactic connectors (low information entropy) and surprising, domain-specific words (high information entropy). Maximizing cumulative likelihood forces the model into high-probability clichés and generic sentences.
- **Repetitive Stagnation**: Holtzman et al. [62] demonstrated that increasing the beam width $B$ in open-ended language generation actually *degrades* perceived quality: as $B \to \infty$, the search uncovers pathological repeating loops that accumulate high probability mass over extended horizons.

To escape deterministic clichés and inject human-like entropy, we must transition from deterministic maximization to **stochastic generation**. This leads directly to **Pure Random Sampling**.

### 6.3.3 Pure random sampling and the catastrophic tail risk

Instead of deterministically tracking top-ranking candidates, **Pure Random Sampling** (Multinomial Sampling) draws the next token directly from the categorical probability distribution generated by the softmax unembedding:

$$w_t \sim \text{Categorical}\left(P(w_1 \mid w_{<t}), P(w_2 \mid w_{<t}), \ldots, P(w_{|\mathcal{V}|} \mid w_{<t})\right). \tag{6.21}$$

Sampling is implemented computationally via Inverse Transform Sampling: given a uniform pseudo-random draw $u \sim \mathcal{U}[0,1)$, select index $\hat{\imath}$ satisfying:

$$\sum_{j=1}^{\hat{\imath}-1} P(w_j \mid w_{<t}) \leq u < \sum_{j=1}^{\hat{\imath}} P(w_j \mid w_{<t}). \tag{6.22}$$

Returning to the distribution from Table 6.1:

- Under greedy search, `"I` is selected with 100% probability.
- Under random sampling:

$$P(w_t = \texttt{"I"}) = 52.1\%, \quad P(w_t = \texttt{"He"}) = 26.0\%, \quad P(w_t = \texttt{"The"}) = 13.0\%, \quad \ldots \tag{6.23}$$

  If the pseudo-random variable evaluates to $u = 0.650$, since $0.521 \leq 0.650 < (0.521 + 0.260 = 0.781)$, the generator selects token $\hat{w}_t =$ `"He"`. The generated continuation gains immediate narrative variety.

#### Failure mode: long tail instability and hallucination cascades

While random sampling completely eliminates repetitive loops, it introduces a severe vulnerability: **the substantial tail hazard**. In language models with $|\mathcal{V}| \approx 50{,}000$, although individual tokens in the tail have infinitesimal probabilities (e.g., $P(w) < 10^{-4}$), the *cumulative aggregate mass* of the thousands of tail tokens commonly sums to 15% to 30% of the entire probability simplex (Figure 6.11).

Consequently, when generating a multi-token sequence, the probability of sampling *at least one* erratic tail token over $N$ steps is $1 - (1 - P_{\text{tail}})^N \approx 1$. Once an aberrant token is sampled into the context, self-attention attends to it, irrevocably corrupting the topic representation and causing severe semantic collapse:

```
"...and my amazement owled wild-it seems an impossibility-and while it
was gone in an stupor and good hearing from me I pretended to know his
name, assuming some personal information which I would..."
```

Before resorting to hard truncation of the vocabulary, can we smoothly reshape the probability landscape itself, suppressing the tail while accentuating dominant modes? This leads to **Temperature Scaling**.

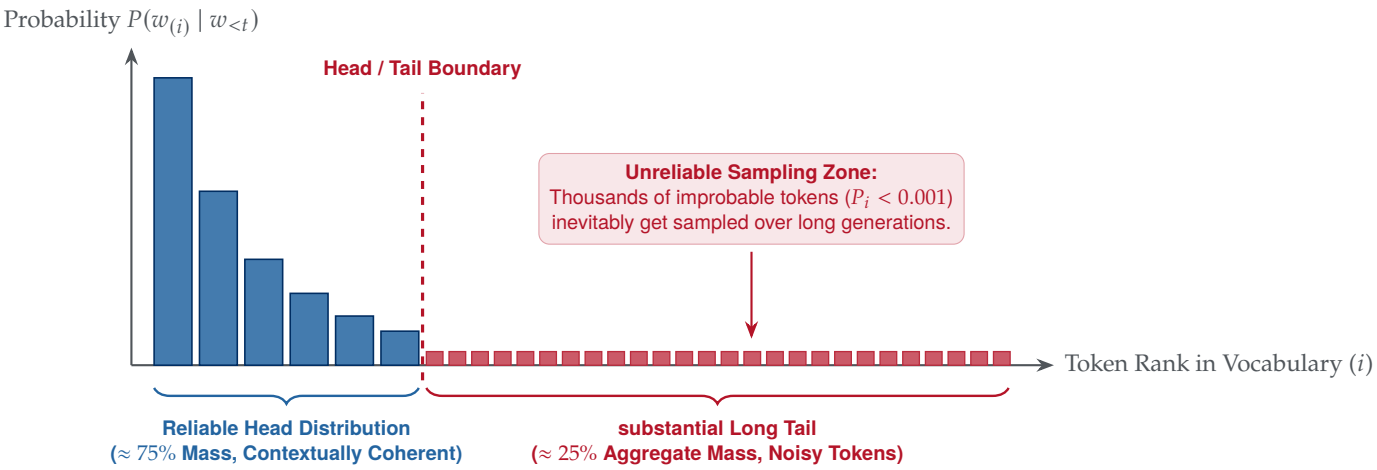



Figure 6.11: The long-tail vulnerability of pure multinomial random sampling. Although individual low-probability tokens have negligible probabilities, their cumulative probability mass (≈ 25%) guarantees that over a horizon of dozens of tokens, the generator will sample an erratic, nonsensical token from the tail, corrupting the subsequent context history.

### 6.3.4 Temperature scaling: thermodynamic entropy modulation

**Temperature Scaling** modifies the sharpness of the categorical probability distribution by introducing a continuous thermodynamic parameter $T > 0$ that scales raw logits prior to exponentiation:

$$P(w_i; T) = \frac{\exp(z_i/T)}{\sum_{j=1}^{|\mathcal{V}|} \exp(z_j/T)}, \quad \forall w_i \in \mathcal{V}. \tag{6.24}$$

The parameter $T$ acts as a continuous entropy dial:

- **Low Temperature ($T < 1.0$)**: Amplifies relative logit differences $z_i - z_j$, concentrating probability mass on the highest-scoring tokens (reducing Shannon entropy).
- **Standard Temperature ($T = 1.0$)**: Preserves the original, calibrated categorical distribution learned during maximum-likelihood pre-training.
- **High Temperature ($T > 1.0$)**: Dampens logit disparities, flattening the distribution toward maximum-entropy uniformity.

Consider a model evaluating three candidate tokens with raw logits $\mathbf{z} = [10.0, 8.0, 5.0]^\top$. Table 6.2 computes the resulting probabilities across four temperature regimes.

Table 6.2: Behavior of probabilities under temperature scaling for logits $\mathbf{z} = [10.0, 8.0, 5.0]^\top$.

| Regime | Scaled Logits $\mathbf{z}/T$ | $\exp(\mathbf{z}/T)$ | Probabilities $P(w;T)$ | Shannon Ent |
|---|---|---|---|---|
| $T = 0.5$ (Low / Sharp) | $[20.0, 16.0, 10.0]$ | $[4.85 \cdot 10^8, 8.89 \cdot 10^6, 2.20 \cdot 10^4]$ | $[\mathbf{0.9820}, 0.0180, 0.00004]$ | 0.092 na |
| $T = 1.0$ (Default) | $[10.0, 8.0, 5.0]$ | $[22026, 2981, 148]$ | $[\mathbf{0.8756}, 0.1185, 0.0059]$ | 0.388 na |
| $T = 2.0$ (High / Soft) | $[5.0, 4.0, 2.5]$ | $[148.4, 54.6, 12.2]$ | $[\mathbf{0.6896}, 0.2537, 0.0567]$ | 0.784 na |
| $T = 5.0$ (Extreme) | $[2.0, 1.6, 1.0]$ | $[7.39, 4.95, 2.72]$ | $[\mathbf{0.4907}, 0.3287, 0.1806]$ | 1.042 na |

As demonstrated, decreasing $T$ from 1.0 to 0.5 drives the top token's probability from 87.6% to 98.2%, virtually suppressing all tail tokens. Conversely, increasing $T$ to 5.0 flattens the distribution toward equal weighting ($1/3 \approx 0.333$).

Figure 6.12 visualizes this transformation across the vocabulary spectrum.

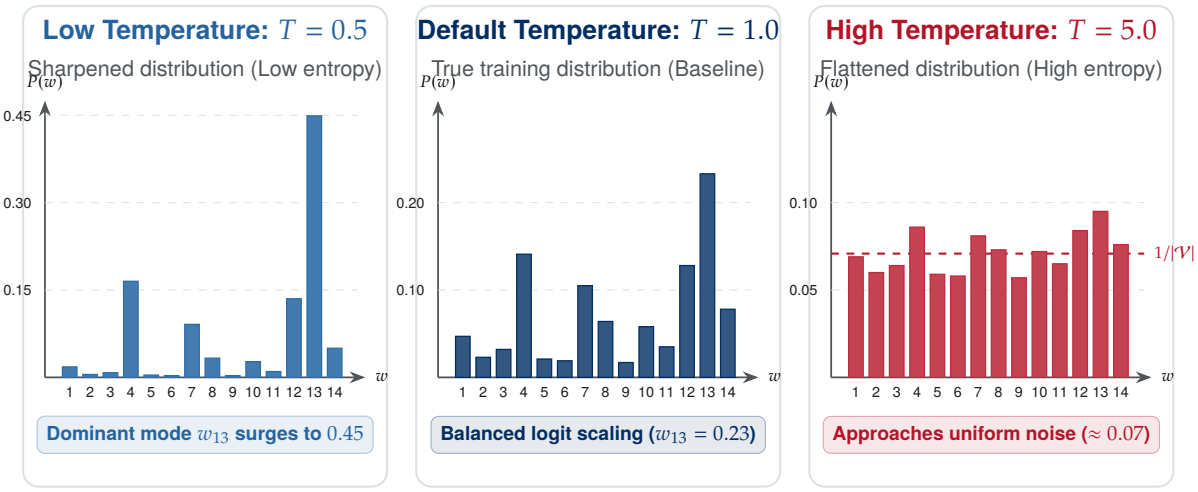



Figure 6.12: Temperature scaling of the softmax distribution. Scaling logits by temperature $T$ modulates distribution entropy: $T < 1.0$ (left) exponentially sharpens differences toward the dominant mode ($w_{13}$), approaching deterministic greedy decoding as $T \to 0$; $T = 1.0$ (center) preserves the model's true training logits; $T > 1.0$ (right) flattens logit disparities toward a uniform distribution, producing erratic and ungrammatical token sequences.

### Analytical limits of temperature scaling

The behavior of temperature scaling is bounded by two rigorous analytical limits:

1. **Low-Temperature Limit ($T \to 0^+$, Greedy Convergence)**: Let $z^* = \max_j z_j$ denote the unique maximal logit. Factoring out $\exp(z^*/T)$ from the numerator and denominator of Equation (6.24):

$$\lim_{T\to 0^+} P(w_i;T) = \lim_{T\to 0^+} \frac{\exp((z_i - z^*)/T)}{\sum_{j=1}^{|\mathcal{V}|} \exp((z_j - z^*)/T)}. \tag{6.25}$$

   For the maximal token where $z_i = z^*$, $(z_i - z^*)/T = 0$, so $\exp(0) = 1$. For any sub-optimal token $k$ where $z_k < z^*$, the numerator satisfies $z_k - z^* = -\delta < 0$ ($\delta > 0$). As $T \to 0^+$, $-\delta/T \to -\infty$, yielding $\exp(-\delta/T) \to 0$. Therefore:

$$\lim_{T\to 0^+} P(w_i;T) = \begin{cases} 1 & \text{if } z_i = \max_j z_j, \\ 0 & \text{if } z_i < \max_j z_j. \end{cases} \tag{6.26}$$

   Temperature sampling converges continuously to deterministic **Greedy decoding**.

2. **High-Temperature Limit ($T \to \infty$, Uniform Collapse)**: As $T \to \infty$, for every finite logit $z_i$, the argument $z_i/T \to 0$, implying $\exp(z_i/T) \to 1$. Consequently:

$$\lim_{T\to\infty} P(w_i;T) = \frac{1}{\sum_{j=1}^{|\mathcal{V}|} 1} = \frac{1}{|\mathcal{V}|}. \tag{6.27}$$

   The distribution collapses into a maximal-entropy uniform distribution across all $|\mathcal{V}|$ vocabulary tokens.

3. **Monotonicity of Shannon Entropy**: The Shannon entropy of the distribution is defined as $H(P;T) = -\sum_{i=1}^{|\mathcal{V}|} P_i(T) \log P_i(T)$. Differentiating with respect to $T$ yields:

$$\frac{\partial H(P;T)}{\partial T} = \frac{1}{T^3} \mathrm{Var}_P(z) = \frac{1}{T^3} \left( \sum_{i=1}^{|\mathcal{V}|} P_i(T) z_i^2 - \left( \sum_{i=1}^{|\mathcal{V}|} P_i(T) z_i \right)^2 \right) \geq 0. \tag{6.28}$$

Because variance is strictly non-negative ($\mathrm{Var}_P(z) \geq 0$), entropy increases strictly monotonically with $T$ whenever logits are non-identical.

Although temperature scaling compresses or flattens the distribution, **it never zeros out the probability of any token**: every token in the vocabulary retains $P(w_i; T) > 0$. In low-entropy contexts where only a single token is correct, even a lowered temperature leaves non-zero probability mass in the tail, still leaving room for improbable hallucinations. Conversely, lowering temperature globally to combat tail noise also suppresses legitimate synonyms in creative passages.

To guarantee that low-probability tokens cannot be sampled under any circumstances, we must apply explicit **truncation** to the support of the distribution. This motivates **Top-$k$ Sampling**.

### 6.3.5 Top-$k$ sampling: hard cardinality truncation

Introduced by Fan et al. [63], **Top-$k$ Sampling** bounds tail risk by strictly restricting the sampling candidate set to the $k$ most probable tokens at each generation step.

**Definition 6.1:** *Top-k sampling*

Let $\mathcal{V}^{(k)} \subset \mathcal{V}$ denote the subset of size $|\mathcal{V}^{(k)}| = k$ containing the tokens with the highest probabilities under $P(w \mid w_{<t})$:

$$\sum_{w \in \mathcal{V}^{(k)}} P(w \mid w_{<t}) \quad \text{is maximized subject to } |\mathcal{V}^{(k)}| = k. \tag{6.29}$$

All tokens outside $\mathcal{V}^{(k)}$ have their probabilities truncated to zero. The remaining $k$ probabilities are renormalized by dividing by their cumulative mass $S_k = \sum_{w_j \in \mathcal{V}^{(k)}} P(w_j)$:

$$P'(w_i) = \begin{cases} \frac{P(w_i)}{\sum_{w_j \in \mathcal{V}^{(k)}} P(w_j)} & \text{if } w_i \in \mathcal{V}^{(k)}, \\ 0 & \text{otherwise.} \end{cases} \tag{6.30}$$

The next token is sampled from this truncated categorical distribution: $w_t \sim \text{Categorical}(P')$.

Suppose a model evaluates the token distribution in Table 6.3 under a top-$k$ policy with $k = 3$.

Table 6.3: Worked example of Top-$k$ truncation ($k = 3$) and probability renormalization.

| **Token Candidate** $w$ | `"dog"` | `"cat"` | `"bird"` | `"fish"` | `"horse"` | `...` [Tail] |
|---|---|---|---|---|---|---|
| **Original** $P(w)$ | 0.40 | 0.30 | 0.15 | 0.08 | 0.04 | $\sum = 0.03$ |
| **In Top-3 Set** $\mathcal{V}^{(k)}$**?** | ✓ | ✓ | ✓ | × | × | × |
| **Truncated Mass** | 0.40 | 0.30 | 0.15 | 0.00 | 0.00 | 0.00 |
| **Renormalized** $P'(w)$ | $\frac{0.40}{0.85} = \mathbf{0.471}$ | $\frac{0.30}{0.85} = \mathbf{0.353}$ | $\frac{0.15}{0.85} = \mathbf{0.176}$ | 0.000 | 0.000 | 0.000 |

The cumulative mass of the top-3 candidates is $S_3 = 0.40 + 0.30 + 0.15 = 0.85$. Dividing each retained candidate by 0.85 rescales the probabilities so that $P'(\texttt{"dog"}) + P'(\texttt{"cat"}) + P'(\texttt{"bird"}) = 1.000$. All tail candidates (accounting for 15% of original probability mass) are strictly excluded.

Figure 6.13 illustrates this truncation and renormalization geometry.

Evaluating top-$k$ sampling with $k = 50$ on the Sherlock Holmes prompt eliminates bizarre tail tokens and produces coherent dialogue:

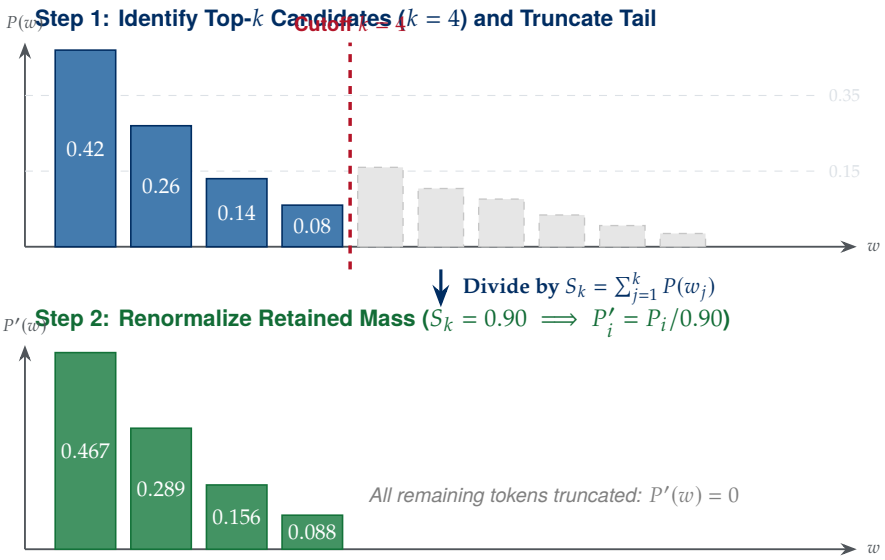



Figure 6.13: The mechanics of Top-$k$ sampling ($k = 4$). The algorithm isolates the top-$k$ tokens, strictly zeroing out the remaining probability mass. Renormalizing by the sum of retained mass rescales the top-$k$ probabilities so they sum to 1.0, shielding the generator from tail errors.

```
"'You will think, Doctor, that a man of my age would be so much of a menace
to me, if I were to refuse a pass for him.  I should rather get a few things
away from him than any which might injure his reputation...'"
```

**Failure mode: structural rigidity of static top-$k$ truncation**

Despite its widespread adoption, top-$k$ sampling suffers from an inherent architectural rigidity: **a static threshold $k$ cannot adapt to varying distribution entropy across different contexts**:

1. **Low-Entropy (Peaked / Factual Context)**: Suppose the prompt is `"The capital of France is"`. The model assigns $P(\texttt{"Paris"}) = 0.98$, while all other tokens are factual errors. If $k = 50$, top-$k$ sampling is forced to retain 49 improbable, factually erroneous tokens. Because their collective mass is renormalized, the chance of sampling an error is artificially elevated.

2. **High-Entropy (Flat / Creative Context)**: Suppose the prompt is `"She looked out into the vast horizon and saw"`. Hundreds of valid English nouns and verbs (e.g., `"mountains"`, `"ships"`, `"stars"`, `"glaciers"`) share roughly equal probabilities ($\sim 0.005$). If $k = 10$ or $k = 20$, top-$k$ prematurely truncates dozens of perfectly fluent, highly creative candidate tokens.

To overcome the inflexibility of a fixed cardinality $k$, the candidate pool size must become **dynamic**, expanding in high-entropy situations and contracting in low-entropy situations. This motivates **Top-$p$ (Nucleus) Sampling**.

### 6.3.6 Top-$p$ (nucleus) sampling: dynamic cumulative mass truncation

To eliminate the rigid cardinality constraint of top-$k$, Holtzman et al. [62] proposed **Top-$p$ (Nucleus) Sampling**. Instead of fixing the number of tokens $k$, nucleus sampling fixes the **cumulative probability mass threshold** $p \in (0, 1]$.

**Definition 6.2:** *Top-$p$ (Nucleus) sampling*

Let $p \in (0, 1]$ be a cumulative probability threshold. Let $\mathcal{V}^{(p)} \subset \mathcal{V}$ denote the smallest subset of vocabulary tokens whose cumulative probability mass exceeds threshold $p$:

$$\mathcal{V}^{(p)} = \text{argmin}_{V' \subseteq \mathcal{V}} |V'| \quad \text{subject to} \quad \sum_{w \in V'} P(w \mid w_{<t}) \geq p. \tag{6.31}$$

Equivalently, sorting tokens in descending order of probability $P(w_{(1)}) \geq P(w_{(2)}) \geq \cdots \geq P(w_{(|\mathcal{V}|)})$, the nucleus cutoff index $k^*(p)$ is:

$$k^*(p) = \min \left\{ k \in \{1, \ldots, |\mathcal{V}|\} \;\middle|\; \sum_{i=1}^{k} P(w_{(i)} \mid w_{<t}) \geq p \right\}. \tag{6.32}$$

All tokens with rank $i > k^*(p)$ are zeroed out, and the probabilities within $\mathcal{V}^{(p)}$ are renormalized:

$$P'(w_i) = \begin{cases} \frac{P(w_i)}{\sum_{w_j \in \mathcal{V}^{(p)}} P(w_j)} & \text{if } w_i \in \mathcal{V}^{(p)}, \\ 0 & \text{otherwise.} \end{cases} \tag{6.33}$$

The next token is sampled from $P'$.

**Worked example: adaptive candidate cardinality**

To demonstrate the dynamic adaptivity of nucleus sampling, consider setting threshold $p = 0.90$ across two different contexts:

- **Context A (Peaked / Low Entropy)**: Prompt: `"The capital of France is"`.

$$P(\texttt{"Paris"}) = 0.92, \quad P(\texttt{"the"}) = 0.03, \quad P(\texttt{"a"}) = 0.01, \quad \ldots$$
$$\text{Cumulative sum}: \quad P(\texttt{"Paris"}) = 0.92 \geq 0.90.$$

  The nucleus contains **only 1 token**: $\mathcal{V}^{(p)} = \{\texttt{"Paris"}\}$. The candidate pool automatically collapses to greedy behavior, preventing hallucinations.

- **Context B (Flat / High Entropy)**: Prompt: `"She looked through the telescope and saw"`. Suppose tokens $w_{(1)}$ through $w_{(15)}$ each carry $P \approx 0.06$. To accumulate $\sum P \geq 0.90$, the nucleus retains $k^*(0.90) = 15$ tokens. The candidate pool dynamically expands, allowing rich lexical variation.

Figure 6.14 demonstrates this dynamic expansion and contraction across high-entropy and low-entropy distributions.

Evaluating nucleus sampling with $p = 0.90$ on the Sherlock Holmes prompt produces fluent, highly natural prose that maintains the atmospheric literary tone of Arthur Conan Doyle without degenerate loops or tail gibberish:

```
"Mr.  Holmes had been talking for awhile and seemed very engrossed in the
conversation.  He wished to conduct his last spoke through the eyes of
another, and by that he meant to tell me the whole of the publick story
that had been passed from one of them into the other..."
```

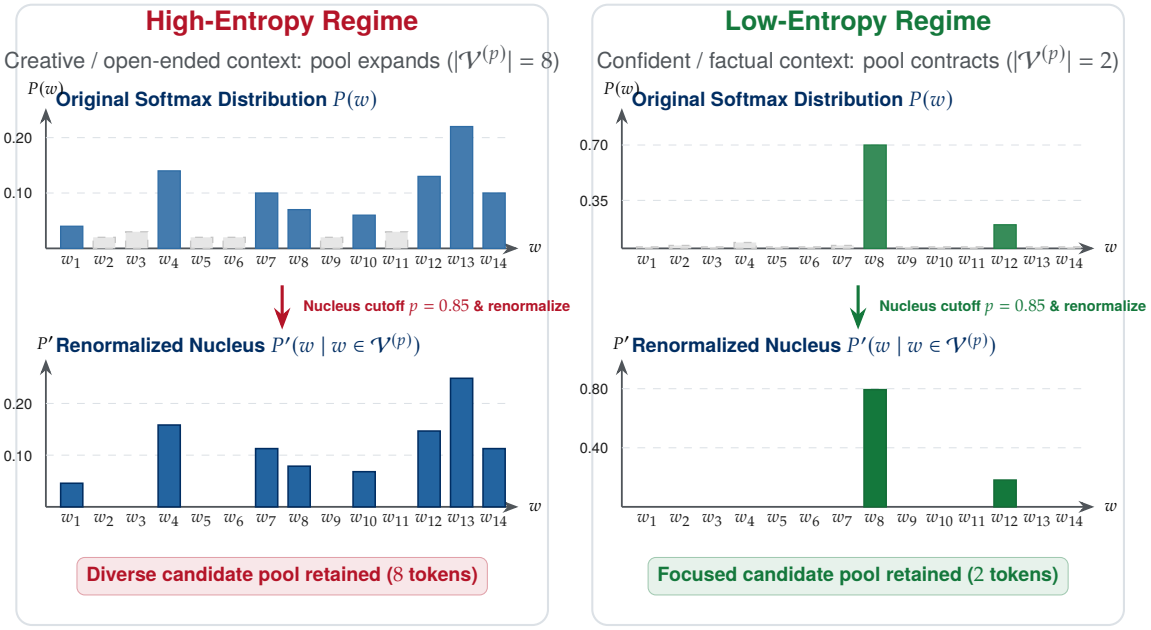



Figure 6.14: Top-$p$ (Nucleus) sampling mechanism. Instead of a fixed $k$, the candidate pool dynamically expands or contracts to cover cumulative probability mass $p$. In high-entropy regimes (left), the pool expands to preserve lexical diversity (8 tokens); in low-entropy regimes (right), the pool automatically contracts to dominant tokens (2 tokens), preventing tail hallucinations [62].

### 6.3.7 Empirical synthesis: the Sherlock Holmes spectrum and decoding selection

**Empirical decoding comparison: Sherlock Holmes generation spectrum**

To directly observe how temperature interacts with sampling mechanics, Table 6.4 summarizes the empirical outputs obtained across temperature and truncation regimes conditioned on the identical Sherlock Holmes prefix.

**Algorithmic selection guide for SE and NLP tasks**

In practical production deployments, no single decoding policy dominates all use cases. The optimal strategy depends strictly on whether the downstream task prioritizes **invariance to randomness** (factual precision, exact syntax) or **stochastic diversity** (creative exploration, synthetic data generation):

1. **Deterministic Tasks (Code Generation, Unit Test Synthesis, Mathematical Reasoning, Machine Translation)**: Require strict reproducibility and syntactic validity. Standard practice employs **Greedy Decoding** ($T = 0$) or **Beam Search** with small beam width ($B \in [2, 5]$) and length penalty $\alpha \in [0.6, 0.8]$.

2. **Open-Ended Creative Tasks (Conversational Agents, Story Generation, Synthetic Documentation)**: Require human-like cadence and vocabulary richness. Standard practice employs **Top-$p$ (Nucleus) Sampling** ($p \in [0.90, 0.95]$) coupled with moderate temperature ($T \in [0.6, 0.8]$).

3. **Hybrid Pipeline: Combining Temperature, Top-$k$, and Top-$p$**: Modern inference engines (e.g., vLLM, Hugging Face 'transformers') execute decoding as a multi-stage sequential filter pipeline:

$$\mathbf{z} \xrightarrow{\text{Temperature } T} \frac{\mathbf{z}}{T} \xrightarrow{\text{Top-}k \text{ Truncation}} \mathcal{V}^{(k)} \xrightarrow{\text{Top-}p \text{ Nucleus}} \mathcal{V}^{(p)} \xrightarrow{\text{Softmax \& Categorical Draw}} \hat{w}_t. \quad (6.34)$$

Table 6.4: Empirical generation spectrum across decoding policies on the Sherlock Holmes testbed.

| Decoding Policy | Configuration | Observed Output Behavior |
|---|---|---|
| **Greedy (Argmax)** | Deterministic ($T \to 0$) | Collapses into cyclic repetition: *"I am sorry, Mr. Holmes... I am sorry, Mr. Holmes..."* |
| **Beam Search** | Width $B = 4$, $\alpha = 0.8$ | Grammatically pristine but highly generic and formulaic; lacks human stylistic variance. |
| **Pure Random** | Multinomial ($T = 1.0$) | substantial tail contamination: emits ungrammatical fragments (*"owled wild–it seems..."*). |
| **Temperature** | $T = 0.1$ (Near Greedy) | Repetition loop persists identically to greedy decoding. |
| **Temperature** | $T = 0.5$ (Balanced Low) | Highly coherent, narrative continuity: *"It was a very sad moment, but a true one..."* |
| **Temperature** | $T = 1.1$ (High Entropy) | Stylistic drift; bizarre metaphorical imagery (*"grim gulags..."*). |
| **Temperature** | $T = 1.5$ (Extreme Noise) | Complete entropy collapse into gibberish subwords (*"IMLEG it off... doctorm Hiraghbene"*). |
| **Top-$k$** | $k = 50$, $T = 0.8$ | Fluent dialogue, eliminates bizarre tails, but risks hallucinating if context is strictly factual. |
| **Top-$p$ (Nucleus)** | $p = 0.90$, $T = 0.7$ | Optimal human-like balance; dynamic candidate pool preserves narrative voice while preventing collapse. |

Scaling logits by $T$ first modulates relative entropy; filtering by Top-$k$ removes extreme vocabulary outliers (e.g., $k = 50$); filtering by Top-$p$ dynamically adapts candidate size for the remaining distribution; and sampling from the renormalized subset provides controlled, diverse outputs.

## 6.4 Architectural Paradigms Synthesis

Table 6.5 synthesizes the architectural trade-offs across the three Transformer families studied in this chapter.

Table 6.5: Comprehensive comparison of Transformer architectural paradigms.

| Feature | Encoder-Decoder (e.g., T5) | Encoder-Only (e.g., BERT) | Decoder-Only (Preview: Chapter 6) |
|---|---|---|---|
| **Attention types** | Self-Attention, Masked Self-Attention, Cross-Attention | Bidirectional Self-Attention only | Causal Masked Self-Attention only |
| **Pre-training loss** | Sequence-to-Sequence span corruption | Masked LM (15%) + Next Sentence Prediction | Autoregressive Next-Token Prediction |
| **Receptive field** | Bidirectional (Enc) + Causal (Dec) | Fully Bidirectional across all tokens | Strictly Causal (preceding tokens only) |
| **Best suited for** | Translation, summarization, structured text-to-text | Sentence classification, NER, semantic code search | Open-ended text generation, dialogue, few-shot prompting |
| **Inference style** | Autoregressive decoding conditioned on encoder | Single forward pass (non-autoregressive) | Autoregressive generation loop with sampling |

While encoder-only models dominated semantic representation, syntax parsing, and discriminative classification, decoder-only models introduced a profound architectural unification: their pre-training objective (autoregressive next-token prediction) is identical to their inference task. By pairing decoder-only transformers with stochastic sampling strategies—such as nucleus sampling ($p \in [0.9, 0.95]$) and temperature scaling ($T \in [0.6, 0.8]$)—researchers uncovered an extraordinary emergent capability: as model parameter count, training token volume, and computational budget scale upward, language models exhibit few-shot and zero-shot reasoning without task-specific parameter fine-tuning. In the subsequent chapter, we formalize the empirical principles and mathematical boundaries governing this regime: **Scaling Laws and Large-Scale Pre-training**.

## 6.5 Architectural Paradigms

Prior to the advent of large-scale pre-training, deep learning for natural language processing was dominated by supervised, task-specific architectures. Practitioners collected human-annotated datasets for individual tasks—such as sentiment analysis (SST-2), natural language inference (SNLI, MNLI), named entity recognition (CoNLL-2003), or question answering (SQuAD)—and trained specialized neural networks from scratch or initialized word vectors using static lookup tables such as Word2Vec or GloVe.

This classical paradigm suffered from three substantial structural bottlenecks:

1. **The annotation bottleneck**: Supervised datasets require expensive, human-in-the-loop labeling. Real-world labeled corpora typically span only $10^4$ to $10^5$ samples. Such small training volumes severely constrain the parameter capacity of neural models, inducing severe overfitting when scaling beyond several million parameters.

2. **Static representation collapse**: Static embeddings map each word to a single spatial coordinate $\mathbf{w} \in \mathbb{R}^d$, completely ignoring polysemy and local syntactic context (e.g., the word "bank" in "river bank" versus "investment bank").

3. **Architectural siloing**: Every distinct NLP task required custom output layers, specialized pooling operations, or task-specific bi-directional recurrent architectures, preventing positive transfer across related domains.

The solution to these bottlenecks emerged with **self-supervised learning**, wherein the supervisory signal is derived autonomously from raw, unannotated text without human labels. By exploiting the inherent sequential structure of natural language, models can be pre-trained on billions or trillions of tokens, learning grammatical syntax, world knowledge, semantic coherence, and logical reasoning patterns.

### 6.5.1 The tripartite architectural fork

Following the introduction of the Transformer by Vaswani et al. [1], the NLP community underwent a major architectural fork based on how the self-attention mechanism is masked and connected:

- **Encoder-only architectures (e.g., BERT [52], RoBERTa)**: Encoder-only models utilize full bidirectional self-attention across the sequence: every token attends to all previous, current, and subsequent tokens simultaneously. The attention mask $\mathbf{M} \in \mathbb{R}^{T \times T}$ is all zeros (no masking). Pre-training relies on the **Masked Language Modeling (MLM)** objective: a random subset (typically 15%) of tokens is corrupted (replaced by a special $\langle \text{MASK} \rangle$ token, random words, or kept unchanged), and the model must reconstruct the original tokens:

$$\mathcal{L}_{\text{MLM}}(\theta) = -\sum_{i \in \mathcal{M}} \log \mathbb{P}_\theta(w_i \mid \mathbf{w}_{\setminus \mathcal{M}}), \tag{6.35}$$

  where $\mathcal{M}$ is the set of masked indices and $\mathbf{w}_{\setminus \mathcal{M}}$ denotes the unmasked context. Encoder-only models are suited to discriminative tasks such as sentence classification, extractive question answering, and token classification. They do not generate text autoregressively in a single forward pass: generating a sequence of length $L$ requires $L$ separate forward iterations, and predicting a masked token conditional on future masked tokens does not define an autoregressive factorization.

- **Encoder-decoder architectures (e.g., T5 [59], BART)**: Encoder-decoder models retain the original Vaswani et al. [1] duality. A bidirectional encoder processes the input

prefix $\mathbf{x} = [x_1, \ldots, x_{T_{\text{in}}}]$, and an autoregressive decoder generates the target sequence $\mathbf{y} = [y_1, \ldots, y_{T_{\text{out}}}]$ token by token using causal self-attention and cross-attention over the encoder's top hidden states $\mathbf{H}_{\text{enc}}$. Pre-training typically uses a span-corruption objective. In T5, for example, arbitrary text spans are replaced with sentinel tokens such as $\langle X \rangle$ and $\langle Y \rangle$, and the decoder is trained to emit the missing spans. This text-to-text framework supports sequence-transduction tasks such as machine translation and document summarization. For open-ended generation, however, the two-stack design increases parameter memory and requires cross-attention at each decoding step.

- **Decoder-only architectures (e.g., GPT [64], GPT-2 [46], GPT-3 [2])**: Decoder-only models eliminate the encoder entirely, consisting of a single, stacked Transformer decoder equipped with **causal (lower-triangular) self-attention masking**:

$$\mathbf{M}_{i,j} = \begin{cases} 0 & \text{if } j \le i \\ -\infty & \text{if } j > i. \end{cases} \tag{6.36}$$

The causal mask ensures that the representation of token $t$ depends strictly on tokens at positions $\le t$. Pre-training optimizes the **Causal Language Modeling (CLM)** objective, predicting the next token $w_t$ conditioned on historical prefix $w_{<t}$:

$$\mathcal{L}_{\text{CLM}}(\theta) = -\frac{1}{T} \sum_{t=1}^{T} \log \mathbb{P}_\theta(w_t \mid w_1, w_2, \ldots, w_{t-1}). \tag{6.37}$$

### 6.5.2 Convergence on the causal decoder-only paradigm

Encoder-only and encoder-decoder models initially achieved state-of-the-art results on NLU benchmarks such as GLUE and SuperGLUE. Subsequent generative systems increasingly adopted the **decoder-only** architecture because of its computational and algorithmic properties:

1. **Unified training and generative alignment**: Causal next-token prediction matches the physical inference process. Every training token serves as both input context and ground-truth target, maximizing data efficiency without requiring artificial mask tokens that induce train-test distribution mismatches.

2. **Simplified memory and KV caching**: In decoder-only models, key and value tensors for prior tokens are simply appended to a single, contiguous cache tensor ($\mathbf{K}, \mathbf{V} \in \mathbb{R}^{B \times H \times T \times d_k}$), avoiding the dual-cache synchronization required in encoder-decoder cross-attention.

3. **Emergence of in-context learning**: As model scale expands, the causal objective forces the model to encode implicit reasoning, syntactic parsing, and world knowledge into its left-to-right activation trajectory, enabling zero-shot and few-shot task execution without gradient updates.

Having established why the decoder-only causal framework is uniquely suited for general-purpose sequence modeling, Figure 6.15 charts the complete chronological timeline of foundation model milestones from 2017 to 2024 with monthly granularity, establishing the historical roadmap across pre-training foundations, in-context emergence, compute scaling laws, and modern open foundation models. We now trace each milestone in detail, beginning with OpenAI's Generative Pre-trained Transformer series.

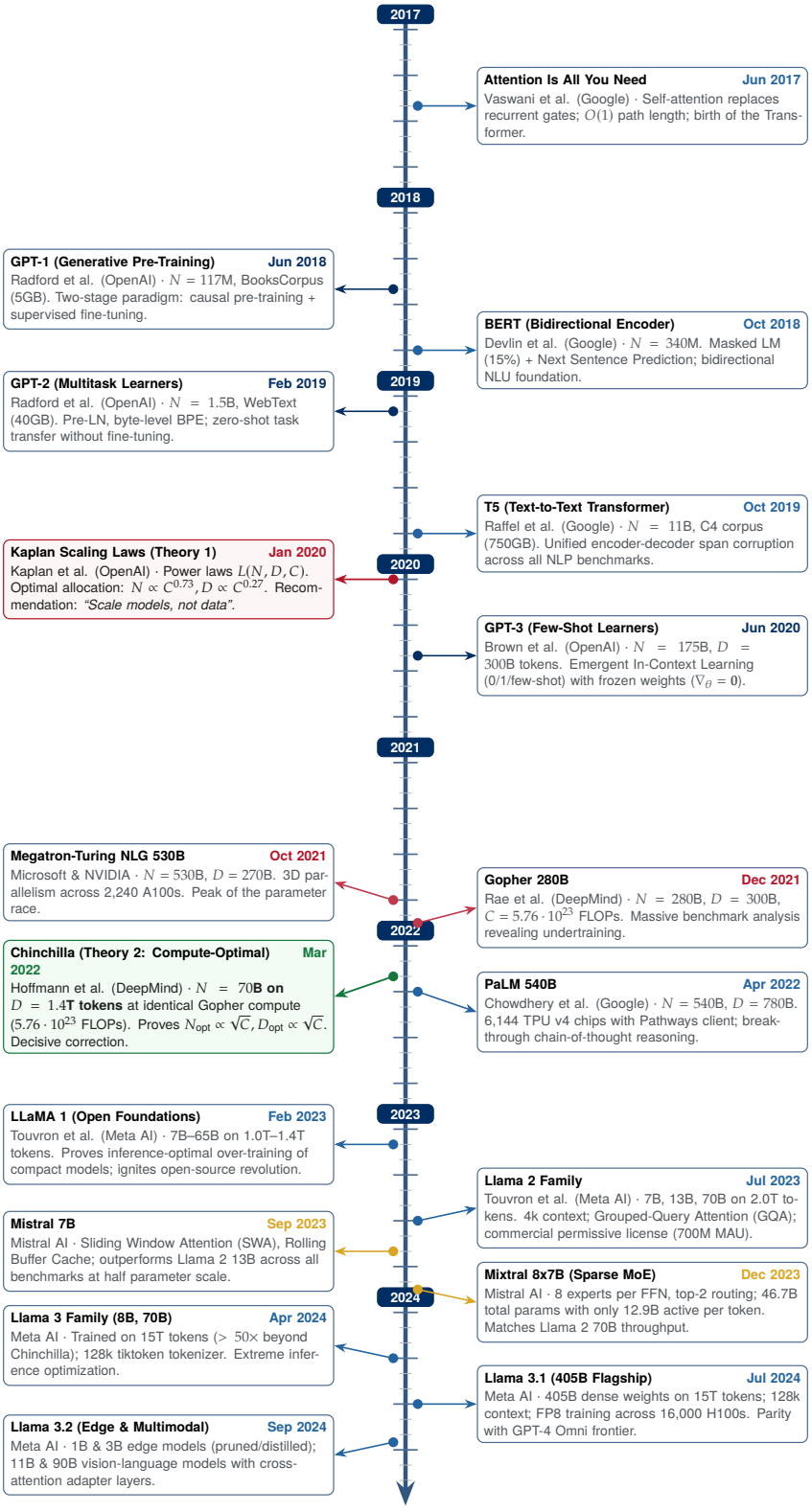



Figure 6.15: Chronological timeline of large language model milestones (2017–2024) with monthly granularity. Each year is partitioned into a 12-month axis where milestones are positioned according to their exact publication month (e.g., Kaplan scaling laws in January at 1/12 of 2020, Chinchilla in March at 3/12 of 2022). The historical trajectory charts the convergence on causal decoder-only models and the transition from parameter scale to compute optimality and inference-optimal open weights.

## 6.6 GPT-1

In 2018, Radford et al. [64] published "Improving Language Understanding by Generative Pre-Training", introducing the original **GPT-1**. The central thesis of GPT-1 was a two-stage semi-supervised framework:

1. **Stage 1: Unsupervised pre-training**: Train a high-capacity 12-layer decoder-only Transformer on a massive, unannotated corpus of contiguous text using causal next-token prediction to acquire general linguistic representations.
2. **Stage 2: Supervised fine-tuning**: Adapt the pre-trained model to specific downstream discriminative tasks by appending a lightweight task-specific linear classification head and fine-tuning all model weights on labeled data.

This canonical two-stage architecture and decoder-only block topology are illustrated in Figure 6.16.

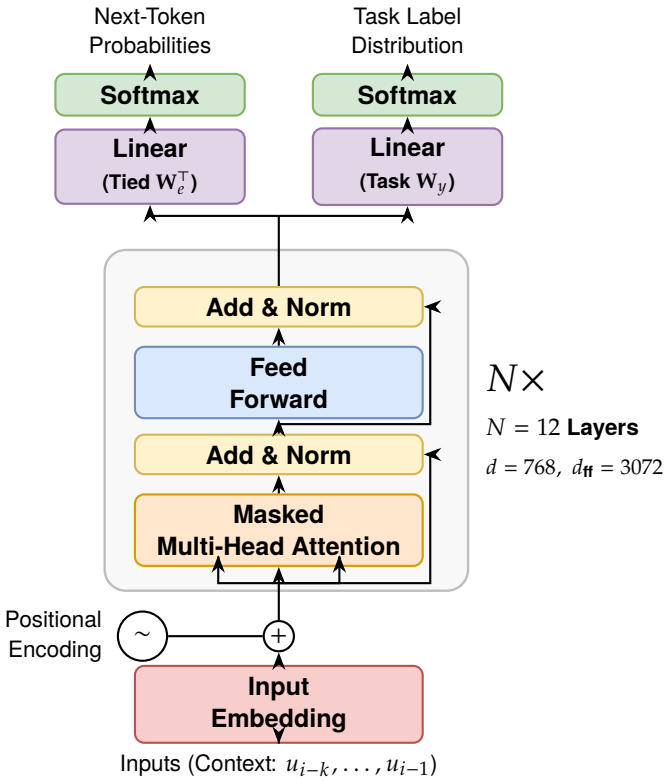



Figure 6.16: The GPT-1 decoder-only Transformer architecture and two-stage adaptation framework [64]. Input tokens and learned positional embeddings are summed and propagated through a stack of $N = 12$ Post-LN decoder blocks. Each block features causal Masked Multi-Head Attention and a position-wise Feed-Forward Network, coupled with identity residual shortcuts and post-layer normalization (`Add & Norm`). In Stage 1 (unsupervised pre-training), output activations pass through tied linear projection $\mathbf{W}_e^\top$ and softmax to predict next tokens. In Stage 2 (supervised fine-tuning), the final token activation passes through task-specific linear classifier $\mathbf{W}_y$ and softmax.

### 6.6.1 Unsupervised pre-training mechanics

Let $\mathcal{U} = \{u_1, u_2, \ldots, u_N\}$ denote an unlabeled corpus of discrete tokens. The unsupervised pre-training objective minimizes the negative log-likelihood over a sliding context window of

size $k$:

$$\mathcal{L}_1(\mathcal{U}) = -\sum_{i=1}^{N} \log \mathbb{P}(u_i \mid u_{i-k}, \ldots, u_{i-1}; \Theta), \tag{6.38}$$

where $\Theta$ represents the complete parameter set of the network.

*Mathematical formulation and layer pass.* Given an input sequence of token indices $\mathbf{u} = [u_{-k}, \ldots, u_{-1}]$, the initial activation matrix $\mathbf{H}_0 \in \mathbb{R}^{k \times d}$ is constructed by summing the token embedding matrix $\mathbf{W}_e \in \mathbb{R}^{|\mathcal{V}| \times d}$ and the learned positional embedding matrix $\mathbf{W}_p \in \mathbb{R}^{k \times d}$:

$$\mathbf{H}_0 = \mathbf{U}\mathbf{W}_e + \mathbf{W}_p, \tag{6.39}$$

where $\mathbf{U} \in \{0, 1\}^{k \times |\mathcal{V}|}$ is the one-hot indicator matrix of the input prefix.

The activation tensor is then sequentially propagated through a stack of $L$ identical Transformer decoder blocks:

$$\mathbf{H}_l = \text{TransformerBlock}(\mathbf{H}_{l-1}), \quad \forall l \in \{1, 2, \ldots, L\}. \tag{6.40}$$

Each GPT-1 block employs the original post-layer-normalization formulation:

$$\mathbf{H}'_l = \text{LayerNorm}\left(\mathbf{H}_{l-1} + \text{MultiHead}\left(\mathbf{H}_{l-1}, \mathbf{H}_{l-1}, \mathbf{H}_{l-1}\right)\right), \tag{6.41}$$

$$\mathbf{H}_l = \text{LayerNorm}\left(\mathbf{H}'_l + \text{FFN}(\mathbf{H}'_l)\right), \tag{6.42}$$

where the Multi-Head Attention (MultiHead $(\cdot)$) enforces causal masking $\mathbf{M}_{i,j} = -\infty$ for $j > i$, and the position-wise feed-forward network uses the Gaussian Error Linear Unit (GELU) activation:

$$\text{FFN}(\mathbf{X}) = \text{GELU}(\mathbf{X}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2, \quad \mathbf{W}_1 \in \mathbb{R}^{d \times d_{\text{ff}}}, \ \mathbf{W}_2 \in \mathbb{R}^{d_{\text{ff}} \times d}. \tag{6.43}$$

Finally, the categorical probability distribution over the vocabulary $\mathcal{V}$ for predicting the next token $u_i$ is computed by projecting the top hidden state $\mathbf{H}_L[k] \in \mathbb{R}^d$ through the transpose of the embedding matrix (weight tying):

$$\mathbb{P}(u_i \mid u_{i-k}, \ldots, u_{i-1}) = \text{softmax}\left(\mathbf{H}_L[k]\mathbf{W}_e^\top\right). \tag{6.44}$$

### 6.6.2 Supervised task adaptation and input formatting

In the second stage, the pre-trained Transformer is adapted to a labeled dataset $C$, where each instance consists of an input sequence of tokens $\mathbf{x} = [x_1, \ldots, x_m]$ and a categorical target label $y \in \mathcal{Y}$. The sequence $\mathbf{x}$ is passed through the pre-trained Transformer to obtain the final activation of the last token, $\mathbf{h}_L^m \in \mathbb{R}^d$. A linear projection matrix $\mathbf{W}_y \in \mathbb{R}^{d \times |\mathcal{Y}|}$ maps this hidden state to class logits:

$$\mathbb{P}(y \mid x_1, \ldots, x_m) = \text{softmax}\left(\mathbf{h}_L^m \mathbf{W}_y\right). \tag{6.45}$$

The supervised objective minimizes cross-entropy loss over $C$:

$$\mathcal{L}_2(C) = -\sum_{(\mathbf{x}, y) \in C} \log \mathbb{P}(y \mid x_1, \ldots, x_m). \tag{6.46}$$

Crucially, Radford et al. [64] discovered that including an **auxiliary language modeling objective** during fine-tuning substantially improves generalization and prevents catastrophic forgetting of pre-trained syntactic representations:

$$\mathcal{L}_3(C) = \mathcal{L}_2(C) + \lambda \mathcal{L}_1(C), \tag{6.47}$$

where $\lambda$ is a balancing hyperparameter (typically $\lambda = 0.5$).

A key architectural innovation of GPT-1 was the avoidance of custom architectural modifications for different downstream NLP tasks. Instead of designing specialized pooling or recurrent layers, all structured inputs were flattened into a single linear sequence using delimiter tokens (Start $\langle s \rangle$, Delimiter $\langle \text{delim} \rangle$, and Extract $\langle e \rangle$), as depicted in Figure 6.17:

- **Text Classification**: The input text is flanked by start and end delimiters: $[\langle s \rangle, \text{Sequence}, \langle e \rangle]$. The final state at $\langle e \rangle$ is fed into the classifier.
- **Natural Language Inference (NLI)**: The premise and hypothesis are concatenated with a delimiter token: $[\langle s \rangle, \text{Premise}, \langle \text{delim} \rangle, \text{Hypothesis}, \langle e \rangle]$.
- **Semantic Similarity / Paraphrasing**: Because similarity is symmetric, the input order should not matter. Two sequences are passed in parallel: $[\langle s \rangle, \text{Text 1}, \langle \text{delim} \rangle, \text{Text 2}, \langle e \rangle]$ and $[\langle s \rangle, \text{Text 2}, \langle \text{delim} \rangle, \text{Text 1}, \langle e \rangle]$. Their final extract representations are element-wise summed prior to classification.
- **Multiple-Choice Question Answering**: For context $C$, question $Q$, and candidate answers $\{A_1, \ldots, A_k\}$, $k$ separate sequences are formed: $[\langle s \rangle, C, \langle \text{delim} \rangle, Q, \langle \text{delim} \rangle, A_k, \langle e \rangle]$. Each sequence produces an extract representation projected to a scalar logit, normalized across all candidates via Softmax.

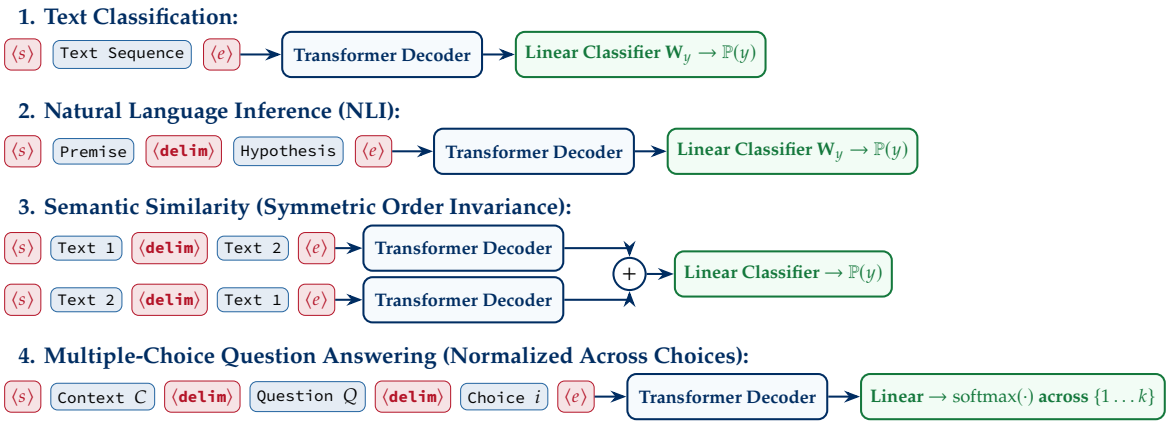



Figure 6.17: Downstream task input transformations with delimiter tokens in GPT-1 [64]. By linearizing structured inputs with special delimiter tokens ($\langle s \rangle, \langle \text{delim} \rangle, \langle e \rangle$), diverse NLP tasks—classification, entailment (NLI), symmetric semantic similarity, and multiple-choice question answering—are processed through the pre-trained Transformer decoder without any task-specific architectural modifications.

The physical configuration of GPT-1 was established as follows:

- **Model Dimensions**: $L = 12$ layers, $H = 12$ attention heads, hidden embedding dimension $d = 768$, and intermediate feed-forward dimension $d_{\text{ff}} = 3072$, totaling approximately $117 \times 10^6$ parameters (117M).
- **Context and Positional Embeddings**: Maximum context length $T = 512$ tokens, using learned absolute positional embeddings $\mathbf{W}_p \in \mathbb{R}^{512 \times 768}$.
- **Tokenization**: Byte-Pair Encoding (BPE) vocabulary with 40,000 merge operations.
- **Pre-training Corpus**: The **BooksCorpus** dataset, containing over 7,000 unpublished fiction books totaling approximately 5 GB of raw text. Crucially, BooksCorpus provided long, contiguous passages of prose, enabling the model to condition on extended historical context and learn long-range semantic dependencies.

While GPT-1 achieved major improvements over state-of-the-art recurrent models across 9 of 12 evaluated NLP benchmarks, it still required collecting task-specific annotated datasets and updating model parameters via gradient descent ($\nabla_\theta \neq \mathbf{0}$) for every single target task. This bottleneck prompted researchers to ask: *can a sufficiently large language model perform tasks directly from raw natural language instructions, without requiring any weight updates?* This question directly motivated GPT-2.

## 6.7 GPT-2

In 2019, Radford et al. [46] released GPT-2 with the landmark paper "Language Models are Unsupervised Multitask Learners". GPT-2 introduced a radical paradigm shift: **zero fine-tuning**. The authors hypothesized that sufficiently high-capacity language models trained on diverse, naturalistic text corpora naturally learn to perform multiple downstream tasks *without any parameter updates* ($\nabla_\theta = \mathbf{0}$).

Mathematically, supervised learning estimates the conditional distribution $\mathbb{P}(\text{target} \mid \text{input})$. Multitask learning conditions additionally on the task identifier: $\mathbb{P}(\text{target} \mid \text{input}, \text{task})$. Radford et al. posited that in natural language, both the input and the task description can be framed as a single unified sequence of tokens. For example:

- **Machine Translation**: $\mathbf{x}$ = "Translate English to French: The book is on the table =>"
- **Summarization**: $\mathbf{x}$ = "[Article text] TL;DR:"
- **Reading Comprehension**: $\mathbf{x}$ = "[Passage] Q: What color was the car? A:"

By simply continuing the sequence autoregressively, the language model solves the specified task without explicit fine-tuning.

This unified zero-shot generative decoder architecture is illustrated in Figure 6.18.

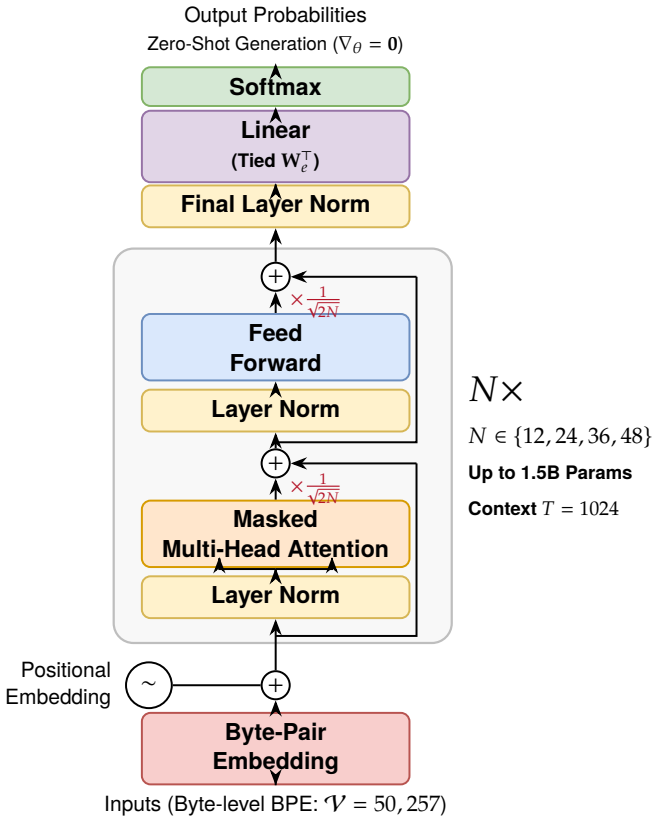



Figure 6.18: The GPT-2 zero-shot multitask decoder architecture [46]. Built upon the Pre-Layer Normalization (Pre-LN) paradigm, Layer Normalization precedes each sublayer while an uninterrupted identity residual shortcut preserves unperturbed gradient highways. Output projection layers are scaled at initialization by $\frac{1}{\sqrt{2N_{\text{layer}}}}$ to bound residual variance growth across deep stacks ($N \in \{12, 24, 36, 48\}$). An additional final LayerNorm stabilizes activations prior to the tied linear projection and softmax language modeling head.

### 6.7.1 Pre-training corpus curation: the WebText heuristic

Pre-training GPT-1 on BooksCorpus limited the model's exposure to diverse writing styles, non-fiction reasoning, code, and dialogue. To support generalist multitask emergence, GPT-2 required a radically larger and more diverse corpus. However, naive web scraping (such as raw Common Crawl) contains vast amounts of low-quality text, repetitive programmatic spam, duplicate marketing copy, and machine-translated gibberish. Radford et al. engineered an elegant, human-curated filtering heuristic:

- They scraped all outbound web links submitted to the social news platform **Reddit**.
- They filtered exclusively for links originating from posts that received at least **3 karma upvotes**.
- The karma threshold served as an automated human curation heuristic, selecting content that human users found interesting, informative, or well-written.
- The resulting dataset, named **WebText**, encompassed text from over 45 million links, totaling approximately 40 GB of high-quality text (an 8× expansion over BooksCorpus).
- Importantly, Wikipedia articles were explicitly filtered out of WebText to test whether the model could generalize to unseen encyclopedic domains during evaluation.

### 6.7.2 Architectural stabilization: Pre-LN and byte-level BPE

While preserving the core causal decoder-only Transformer backbone, GPT-2 introduced critical architectural refinements to stabilize training dynamics across deep networks and resolve out-of-vocabulary representation failures.

#### Pre-layer normalization (Pre-LN)

In GPT-1 (and the original Transformer [1]), **Post-LN** placed layer normalization on the residual path strictly after the addition:

$$\mathbf{x}_l = \text{LayerNorm}(\mathbf{x}_{l-1} + \text{SubLayer}(\mathbf{x}_{l-1})). \tag{6.48}$$

Under Post-LN, backpropagating gradients must traverse the derivative of the LayerNorm operator at every single layer. For deep architectures ($L \geq 24$), the gradient magnitude through the unnormalized skip connections decays exponentially or oscillates wildly, necessitating delicate learning-rate warmup schedules to avoid substantial early divergence.

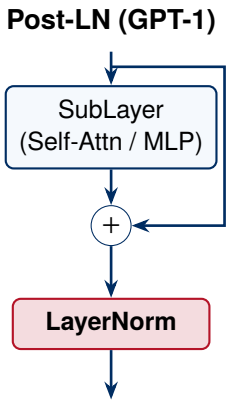
To resolve this instability, GPT-2 adopted **Pre-Layer Normalization (Pre-LN)**, applying normalization strictly to the input of each sub-layer:

$$\mathbf{x}'_l = \mathbf{x}_{l-1} + \text{SubLayer}(\text{LayerNorm}(\mathbf{x}_{l-1})). \tag{6.49}$$

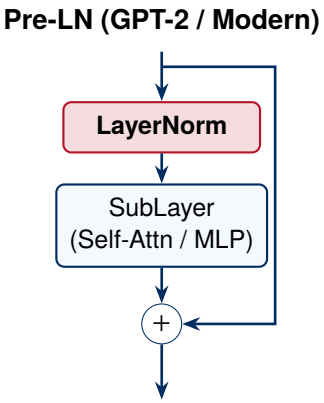
Because $\mathbf{x}_{l-1}$ enters the addition unmodified, the residual stream forms a continuous identity highway:

$$\mathbf{x}_L = \mathbf{x}_0 + \sum_{l=1}^{L} \text{SubLayer}(\text{LayerNorm}(\mathbf{x}_{l-1})). \tag{6.50}$$

Differentiating with respect to the input embedding $\mathbf{x}_0$ yields an uninterrupted identity matrix $\mathbf{I}$, allowing error gradients to propagate unattenuated throughout all layers. An additional final layer normalization $\text{LayerNorm}_{\text{final}}$ is appended after the final layer to normalize the accumulated residual sum prior to output projection.




(a) Post-LN (GPT-1): Normalization follows residual addition.




(b) Pre-LN (GPT-2): Normalization precedes sub-layer computation.

Figure 6.19: Structural comparison between Post-LN and Pre-LN Transformer blocks. Pre-LN maintains a clean identity residual highway, preventing vanishing and exploding gradients in deep networks.

### Residual projection scaling

While Pre-LN ensures clean gradient propagation, the unnormalized additive summation across depth causes the variance of the residual stream to grow monotonically with the number of layers:

$$\mathbb{V}\mathrm{ar}(\mathbf{x}_l) \approx \mathbb{V}\mathrm{ar}(\mathbf{x}_{l-1}) + \mathbb{V}\mathrm{ar}\big(\mathrm{SubLayer}(\mathrm{LayerNorm}(\mathbf{x}_{l-1}))\big). \tag{6.51}$$

If each sub-layer output has variance $\sigma^2 \approx \mathbb{V}\mathrm{ar}(\mathbf{x}_{l-1})$, after $M$ residual additions the variance at layer $L$ expands linearly: $\mathbb{V}\mathrm{ar}(\mathbf{x}_L) \approx \mathbb{V}\mathrm{ar}(\mathbf{x}_0) + M\sigma^2$. Because each Transformer decoder block contains *two* distinct residual branches—the causal Multi-Head Attention and the position-wise Feed-Forward Network—the total number of residual additions across the network is:

$$M = 2N_{\text{layer}}. \tag{6.52}$$

In deep models such as GPT-2 Extra Large ($N_{\text{layer}} = 48$), the cumulative variance would reach $96\sigma^2$, leading to extreme activation explosion and numerical instability in half-precision representations.

To bound the variance across the entire residual stream so that $\mathbb{V}\mathrm{ar}(\mathbf{x}_L) \approx O(1)$, Radford et al. [46] introduced **residual projection scaling**. At initialization, the weights of the final linear projection layer in each residual branch are scaled down by:

$$\gamma = \frac{1}{\sqrt{2N_{\text{layer}}}}. \tag{6.53}$$

*Exact structural location in the decoder*: As illustrated in Figure 6.20, this scaling factor is applied precisely to two weight matrices inside every decoder layer:

1. **Attention Output Projection $\mathbf{W}^O$**: The linear projection matrix mapping concatenated attention heads back to the hidden dimension $d$:

$$\mathbf{W}^O \leftarrow \frac{1}{\sqrt{2N_{\text{layer}}}}\mathbf{W}^O, \quad \mathbf{W}^O \in \mathbb{R}^{d\times d}. \tag{6.54}$$

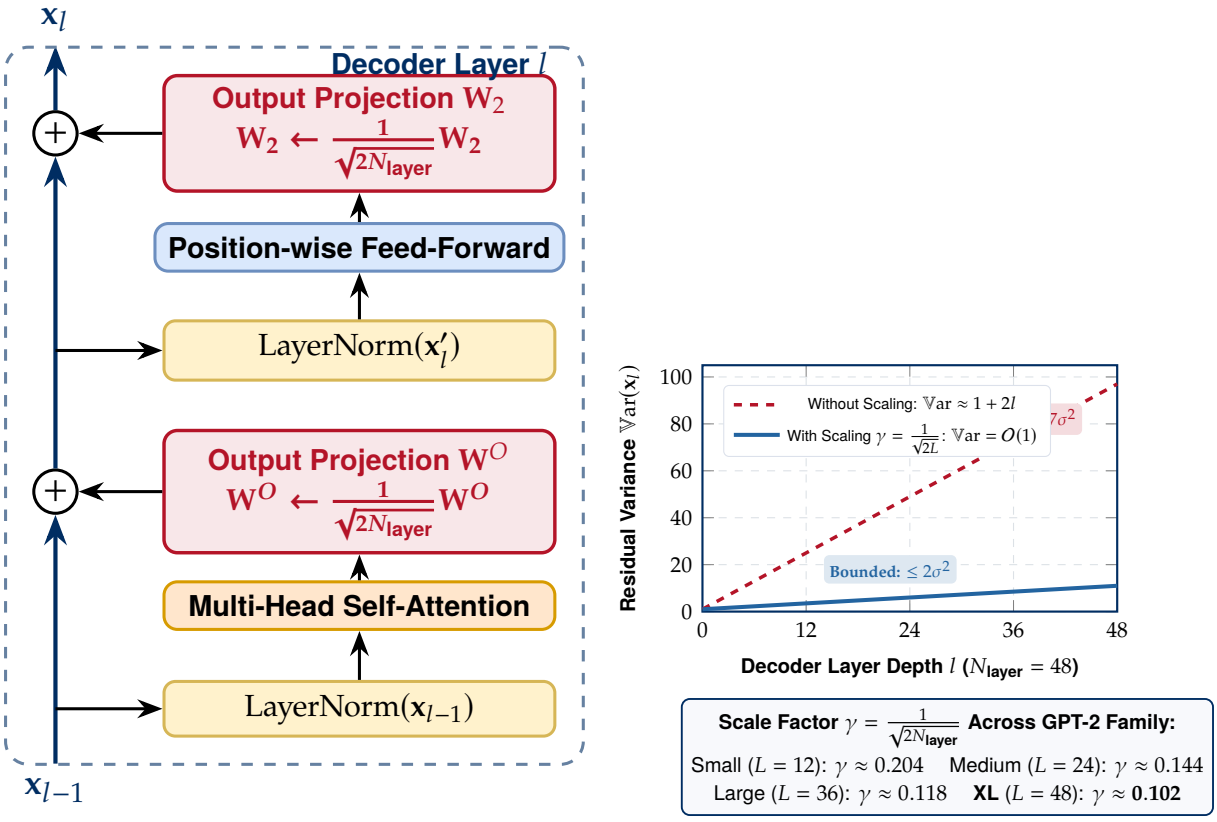
2. **Feed-Forward Output Projection $\mathbf{W}_2$**: The second linear projection matrix mapping the intermediate expanded representation back to dimension $d$:

$$\mathbf{W}_2 \leftarrow \frac{1}{\sqrt{2N_{\text{layer}}}}\mathbf{W}_2, \quad \mathbf{W}_2 \in \mathbb{R}^{d_{\text{ff}}\times d}. \tag{6.55}$$

Because output variance is quadratic in weight magnitude ($\mathbb{V}\text{ar}(\mathbf{W}\mathbf{x}) = \|\mathbf{W}\|_F^2\, \mathbb{V}\text{ar}(\mathbf{x})$), multiplying weights by $\frac{1}{\sqrt{2N_{\text{layer}}}}$ scales the variance of each residual branch by $\frac{1}{2N_{\text{layer}}}$. Summing across all $2N_{\text{layer}}$ residual branches yields:

$$\mathbb{V}\text{ar}(\mathbf{x}_L) \approx \mathbb{V}\text{ar}(\mathbf{x}_0) + \sum_{m=1}^{2N_{\text{layer}}} \frac{\sigma^2}{2N_{\text{layer}}} = \mathbb{V}\text{ar}(\mathbf{x}_0) + \sigma^2 \approx O(1), \tag{6.56}$$

maintaining stable variance across arbitrary network depths.




(a) Exact structural location within decoder block (b) Variance accumulation dynamics across depth

Figure 6.20: Residual projection scaling mechanism in GPT-2 [46]. (a) Exact structural location within the decoder block: scaling by $\frac{1}{\sqrt{2N_{\text{layer}}}}$ is applied strictly to the output linear projection matrices ($\mathbf{W}^O$ in Multi-Head Attention and $\mathbf{W}_2$ in the Feed-Forward Network) right before the residual summation nodes ($\bigoplus$). (b) Variance accumulation dynamics across network depth $l \in [0, 48]$: without scaling, residual addition causes activation variance to explode linearly ($\mathbb{V}\text{ar} \propto 2l$), reaching nearly 100× in GPT-2 XL. With residual projection scaling, cumulative variance remains strictly bounded ($O(1)$), ensuring numerical stability during training.

### Byte-level byte-pair encoding (BPE)

GPT-1 used character-level BPE with 40,000 merges, which frequently encountered out-of-vocabulary (⟨UNK⟩) tokens when processing foreign characters, unicode symbols, or source code tokens. GPT-2 introduced **byte-level BPE** operating over raw UTF-8 bytes with a vocabulary of 50,257 tokens (comprising 256 raw byte values, special tokens, and 50,000 learned merges). Because any text sequence decomposes into valid bytes, byte-level BPE guarantees zero out-of-vocabulary tokens on any arbitrary string. Additionally, regex rules prevented BPE from merging across distinct character classes (e.g., preventing punctuation from fusing into words).

**Expanded context and batch capacity**

GPT-2 doubled maximum context length from $T = 512$ to $T = 1024$ tokens, enabling the capture of broader contextual discourse. The training batch size was increased to 512 sequences (totaling 524,288 tokens per training step), stabilizing stochastic gradient descent across massive distributed clusters.

### 6.7.3 Model family scaling and staged release governance

GPT-2 was released across four distinct model scales, demonstrating continuous, predictable performance scaling as parameter count grew:

- **GPT-2 Small**: 117M parameters ($L = 12, d = 768, H = 12$) — identical in size to GPT-1.
- **GPT-2 Medium**: 345M parameters ($L = 24, d = 1024, H = 16$).
- **GPT-2 Large**: 762M parameters ($L = 36, d = 1280, H = 20$).
- **GPT-2 Extra Large (1.5B)**: 1542M parameters ($L = 48, d = 1600, H = 25$).

Empirically, Radford et al. [46] observed that *perplexity decreased log-linearly with model capacity across all evaluation benchmarks*. Furthermore, without any task-specific fine-tuning, on language modeling benchmarks (WikiText-103, LAMBADA), GPT-2 established new state-of-the-art perplexities in zero-shot settings, beating supervised models. On reading comprehension (CoQA), the 1.5B model achieved an F1 score of 55, matching or outperforming baseline supervised models. On French-to-English translation, prompting with simple natural language exemplars yielded 11.5 BLEU, outperforming several unsupervised translation baselines despite having almost no French in WebText.

In February 2019, OpenAI announced GPT-2 alongside a controversial decision: they declined to release the full 1.5B parameter weights, citing concerns over dual-use technology and potential malicious applications, such as the automated generation of deceptive political disinformation, spam, and synthetic abusive content. They initially published only the 117M model and technical paper. Over the subsequent nine months, OpenAI conducted staged incremental releases: the 345M model in May 2019, the 762M model in August 2019, and finally the complete 1.5B model in November 2019. While this staged release drew sharp initial criticism from open-source advocates, it marked the historical inception of AI safety discourse, model governance protocols, and modern alignment concerns surrounding high-capacity foundation models.

GPT-2 established that increasing parameter count from 117M to 1.5B dramatically improved language modeling loss and unlocked emergent zero-shot capabilities. However, scaling remained an intuitive, trial-and-error craft. Researchers had no mathematical formula to answer: *If we have a fixed budget of compute, how big should our model be? How many tokens should we collect? Does network depth matter more than width?* To answer these questions systematically, researchers turned to empirical physics-inspired scaling laws.

CHAPTER 7

# Pre-Training and Scaling Laws

Chapter 6 analyzed the structural paradigms of Transformer architectures and the decoding algorithms that govern sequence generation. While model topology dictates how representations interact, the predictive capability of foundation models is determined by the scale of self-supervised pre-training and the volume of compute allocated during optimization.

Understanding modern foundation models requires appreciating the long-term historical evolution of language modeling, from discrete Markov counting tables to continuous deep neural networks. As architectures scaled, empirical research uncovered quantitative power laws governing the relationship between model parameter count, training token volume, compute budgets, and downstream test loss.

This chapter formalizes the principles of large-scale pre-training and scaling laws. We begin with a historical survey tracing 75 years of language modeling milestones, examine the empirical power-law scaling formulations of Kaplan et al., and analyze the undertraining dilemma. We then derive the compute-optimal scaling frontier established by Chinchilla and evaluate hardware memory constraints and open foundation models.

## 7.1 Evolution of Language Models

Understanding modern large language models requires appreciating the 75-year evolutionary trajectory that progressively addressed the mathematical and computational barriers of discrete symbol counting. Figure 7.1 synthesizes the macro-historical progression of language modeling from Claude Shannon's foundational 1948 information theory to modern autonomous software engineering systems.

### 7.1.1 1948–1980s: Shannon's information theory and Markov n-grams

The conceptual origin of language modeling stems from Claude Shannon's landmark 1948 paper *"A Mathematical Theory of Communication"* [27]. In that work, written language was modeled as a discrete stochastic process generated by an underlying stationary Markov source over a finite alphabet.

Shannon introduced the *Shannon game*—a thought experiment in which an observer predicts the next letter in an unknown sequence given preceding characters. Shannon demonstrated that as the conditioning context length increases from zero-order (uniform random selection) to unigram (letter frequencies), bigram (character pairs), and trigram (triplets), generated text converges progressively toward natural English orthography and syntax.

In this framework, the conditional probability of token $w_t$ given context $w_1, \ldots, w_{t-1}$ is simplified via an order-$(n-1)$ Markov assumption, as illustrated in Figure 7.2 for programming language tokens.

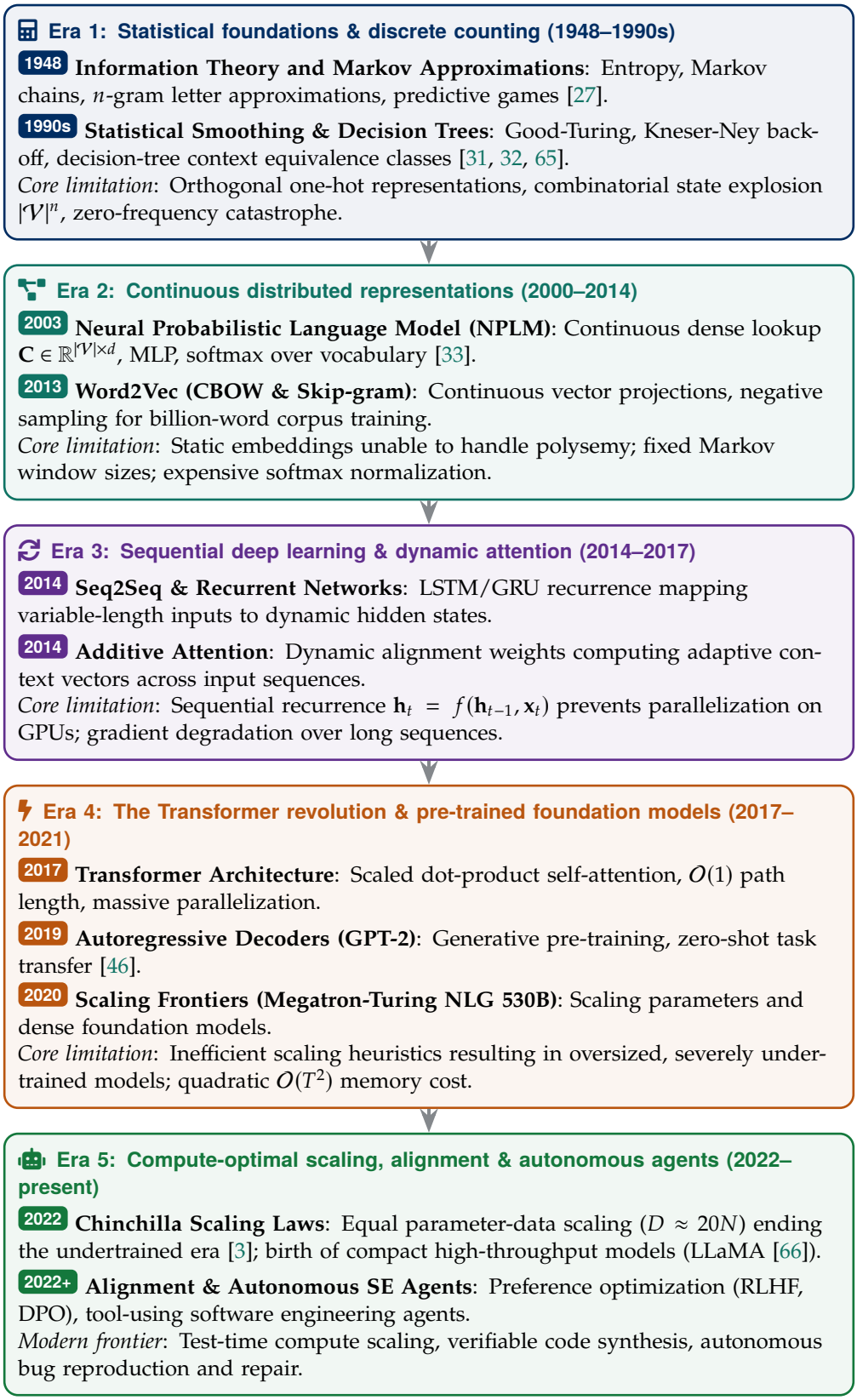



Figure 7.1: Chronological timeline of language model evolution from 1948 to the present.

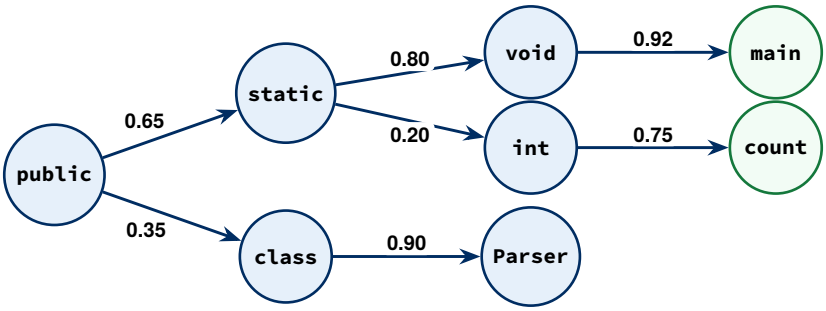



Figure 7.2: Markov chain state transition graph for software tokens.

### 7.1.2 1990s: statistical decision trees and context partitioning

Throughout the 1990s, as speech recognition and machine translation systems expanded, flat $n$-gram tables could not accommodate long histories without exponential parameter growth. To address this limitation, *decision-tree language models* were developed in [65]. Rather than conditioning on an exact sequence of $n-1$ tokens, a binary decision tree poses a hierarchy of lexical and syntactic queries regarding the history $\mathcal{H} = (w_{t-1}, w_{t-2}, \dots)$.

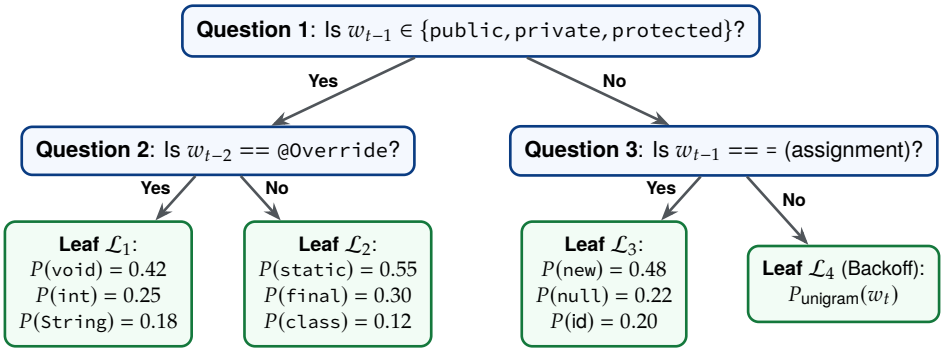



Figure 7.3: Decision tree context partitioning for conditional language modeling [65].

As shown in Figure 7.3, context histories that exhibit similar predictive patterns are pooled into the same leaf node $\mathcal{L}_k$, yielding distributions $\mathbb{P}(w_t \mid \mathcal{L}_k)$. While this reduced free parameters relative to full combinatorial tables, the architecture remained reliant on discrete categorical partitions without smooth semantic interpolation.

### 7.1.3 2000–2003: The neural probabilistic language model

In the seminal 2003 paper *"A Neural Probabilistic Language Model"* [33], Yoshua Bengio and colleagues introduced continuous distributed representations to alleviate the curse of dimensionality.

The neural probabilistic formulation in [33] replaced discrete count tables with a continuous parameterized architecture (Figure 7.4):

1. **Continuous Embedding Projection**: Each word $w \in \mathcal{V}$ is associated with a real-valued distributed feature vector $\mathbf{c}_w \in \mathbb{R}^d$ (where $d \ll |\mathcal{V}|$, e.g., $d = 100$). The vocabulary lookup matrix $\mathbf{C} \in \mathbb{R}^{|\mathcal{V}| \times d}$ is learned jointly via backpropagation.

2. **Semantic Smoothing via Continuous Space**: If *"cat"* and *"feline"* share similar embedding vectors ($\mathbf{c}_{\text{cat}} \approx \mathbf{c}_{\text{feline}}$), observing *"the cat chased the mouse"* automatically increases the model's confidence in *"the feline chased the mouse"*. Continuous geometry overcomes the orthogonal symbol limitation.

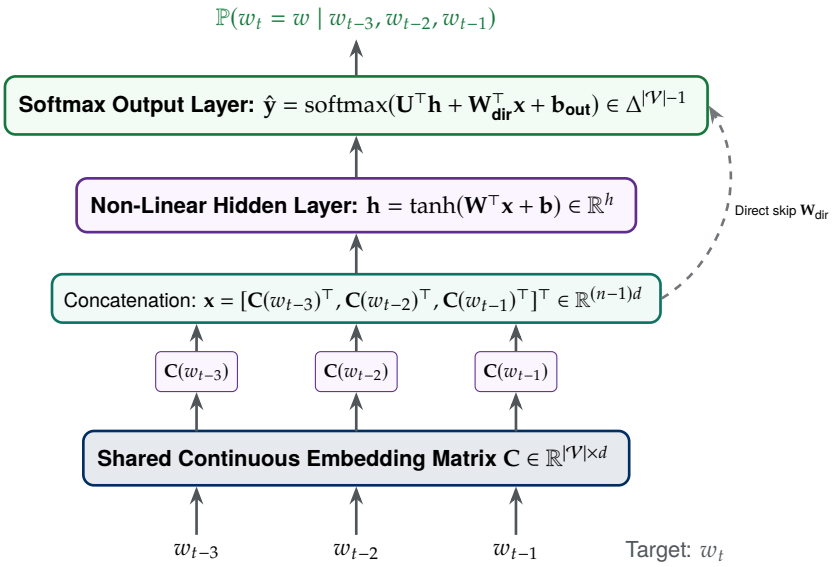



Figure 7.4: The Neural Probabilistic Language Model (NPLM) architecture [33]. Discrete context words are mapped into continuous dense feature vectors via shared matrix $\mathbf{C} \in \mathbb{R}^{|\mathcal{V}| \times d}$, concatenated, and processed through a non-linear hidden layer to predict next-token probabilities via softmax.

3. **Softmax Output Normalization**: The network projects hidden activations to $|\mathcal{V}|$ unnormalized logits and applies the softmax function to guarantee a valid probability distribution:

$$\mathbb{P}(w_t = i \mid w_{<t}) = \frac{\exp(z_i)}{\sum_{j=1}^{|\mathcal{V}|} \exp(z_j)}. \tag{7.1}$$

### 7.1.4 2018–2019: Generative pre-training and GPT-2

While continuous representations were established in [33], models throughout the 2000s and early 2010s remained constrained in parameter scale and context length. In 2018–2019, the GPT-2 architecture was introduced in [46] (*"Language Models are Unsupervised Multitask Learners"*).

GPT-2 demonstrated that autoregressive decoder-only models trained on hundreds of gigabytes of uncurated web text could perform diverse natural language and software engineering tasks without task-specific supervised fine-tuning. By predicting next tokens across context windows of 1,024 tokens, the model exhibited emergent zero-shot transfer across reading comprehension, machine translation, and Python code generation.

### 7.1.5 2020–2022: Megatron-Turing NLG 530B and compute-optimal scaling

Following GPT-2 and GPT-3, deep learning research prioritized aggressive model scaling, culminating in dense models such as Megatron-Turing NLG 530B [67]. However, early power-law scaling heuristics [68] recommended scaling parameters faster than training data. Consequently, models such as Megatron-Turing NLG 530B and GPT-3 (175B parameters trained on 300B tokens) were undertrained relative to their computational capacity.

In 2022, the study *"Training Compute-Optimal Large Language Models"* [3] established the **Chinchilla** scaling laws:

$$D \approx 20 \times N, \tag{7.2}$$

where $N$ denotes parameter count and $D$ represents training token volume. For compute optimality, model size and training data must scale in equal proportion. This insight initiated

the current era of compact, thoroughly trained open-weight models (e.g., LLaMA [66] trained on trillions of tokens), improving inference efficiency in production and software engineering environments.

### 7.1.6 From discrete counting tables to continuous parametric learners: bridge to deep learning

The formal trajectory examined throughout this chapter highlights two structural constraints inherent in discrete language modeling:

- **The Discrete Counting Limit**: Exact probability factorization via the chain rule of probability is mathematically unconstrained, yet empirical $n$-gram tables suffer from exponential state-space growth ($V^n$), rigid context truncation ($n-1$), and orthogonal symbol representation.
- **The Smoothing Ceiling**: While statistical smoothing methods prevent zero-frequency collapse ($\mathbb{P} = 0 \implies \text{PPL} = \infty$), they operate solely by reallocating probability mass across discrete table entries. Consequently, they cannot capture non-linear feature interactions or transfer statistical strength between semantically related tokens.

To overcome the discrete counting paradigm, natural language processing transitioned from frequency tables to continuous, parameterized neural networks:

$$f(\mathbf{x};\boldsymbol{\theta}) = \mathbf{W}^\top \mathbf{x} + \mathbf{b}. \tag{7.3}$$

Rather than counting token co-occurrences directly, modern language modeling parameterizes estimators as continuous differentiable functions whose weights $\boldsymbol{\theta}$ are optimized systematically through gradient descent and backpropagation, establishing the computational framework developed in subsequent chapters.

## 7.2 Kaplan Scaling Laws

Prior to 2020, scaling neural networks was largely an empirical art driven by intuition and trial-and-error. Researchers lacked quantitative formulas to answer fundamental engineering questions: *If compute budget increases by* 100×*, how much should we expand model parameters versus training dataset size? Does network depth matter more than width? Is training to full convergence optimal?*

In January 2020, Jared Kaplan and colleagues at OpenAI [68] published the landmark paper "Scaling Laws for Neural Language Models". By training dozens of autoregressive Transformer models spanning over four orders of magnitude in compute, three orders of magnitude in dataset size, and three orders of magnitude in parameter count, the authors demonstrated that **language model performance follows precise, predictable power laws** over immense dynamic ranges.

### 7.2.1 Empirical power-law formulations

When model performance is not bottlenecked by any single resource constraint (such as data exhaustion or inadequate parameter capacity), the test cross-entropy loss $\mathcal{L}$ adheres to power-law scaling relationships:

$$\mathcal{L}(N) \approx \left(\frac{N_c}{N}\right)^{\alpha_N}, \quad \text{for unlimited data } D \text{ and compute } C, \tag{7.4}$$

$$\mathcal{L}(D) \approx \left(\frac{D_c}{D}\right)^{\alpha_D}, \quad \text{for unlimited parameters } N \text{ and compute } C, \tag{7.5}$$

$$\mathcal{L}(C) \approx \left(\frac{C_c}{C}\right)^{\alpha_C}, \quad \text{for optimal allocation of } N \text{ and } D, \tag{7.6}$$

where:

- $N$ is the number of non-embedding parameters.
- $D$ is the number of training tokens.
- $C$ is the total training compute expressed in floating-point operations (FLOPs).
- $N_c, D_c, C_c$ are normalization constants, and $\alpha_N, \alpha_D, \alpha_C$ are scaling exponents.

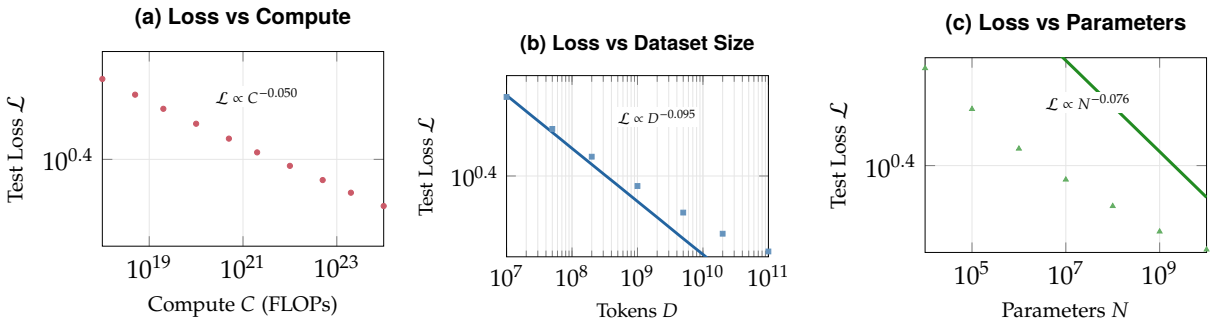
Kaplan et al. empirically estimated the power-law exponents for autoregressive language modeling as:

$$\alpha_N \approx 0.076, \quad \alpha_D \approx 0.095, \quad \alpha_C \approx 0.050. \tag{7.7}$$

A striking finding of Kaplan et al. was that performance depends **very weakly** on detailed architectural hyperparameters:

- **Depth versus width**: For a fixed parameter count $N$, varying the ratio between the number of layers $n_{\text{layer}}$ and the hidden embedding dimension $d_{\text{model}}$ over a wide range has negligible impact on cross-entropy loss.
- **Attention heads**: Varying the number of attention heads $H$ (or per-head dimension $d_k$) produces virtually identical loss trajectories.
- **Feed-forward dimension**: Altering the feed-forward multiplier $d_{\text{ff}}/d_{\text{model}}$ produces minimal deviation.

The fundamental implication is that **scale dominates shape**: macroscopic resource allocation $(N, D, C)$ dictates model performance, while microscopic architectural tuning yields only minor second-order effects.




(a) Compute scaling (b) Dataset size scaling (c) Parameter scaling

Figure 7.5: Empirical scaling laws for language model performance from Kaplan et al. [68]. Cross-entropy test loss demonstrates smooth, predictable power-law scaling spanning multiple orders of magnitude across compute $C$ (FLOPs), dataset size $D$ (tokens), and parameter count $N$ (non-embedding parameters).

### 7.2.2 Floating-point arithmetic and compute allocation

To analyze compute-optimal allocation, we must establish the physical relationship between floating-point operations, model parameters, and training tokens.

**Theorem 7.1:** *Transformer compute budget*

For a standard Transformer decoder consisting of $N$ non-embedding parameters trained on $D$ tokens, the total training compute $C$ (in FLOPs) is given by:

$$C \approx 6ND. \tag{7.8}$$

*Proof.* Consider a general linear matrix multiplication $\mathbf{Y} = \mathbf{X}\mathbf{W}$, where input activations $\mathbf{X} \in \mathbb{R}^{B \times d_{\text{in}}}$ and weight matrix $\mathbf{W} \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$. Computing each scalar entry of $\mathbf{Y}$ requires $d_{\text{in}}$ multiplications and $d_{\text{in}}$ additions, totaling $2d_{\text{in}}$ floating-point operations. The matrix has $d_{\text{in}} \times d_{\text{out}}$ parameters and produces $B \times d_{\text{out}}$ outputs. Thus, processing 1 token through $\mathbf{W}$ requires:

$$\text{FLOPs}_{\text{forward}} = 2 \times (\text{number of parameters}). \tag{7.9}$$

Summing across all linear projections in the multi-head attention blocks ($\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V, \mathbf{W}_O$) and feed-forward networks ($\mathbf{W}_1, \mathbf{W}_2$), the forward pass requires:

$$C_{\text{forward}} \approx 2N \text{ FLOPs per token.} \tag{7.10}$$

In the backward pass (backpropagation), gradients must be computed with respect to both the activations (to propagate error backward) and the weight parameters (to update weights):

- Gradient with respect to activations: $\frac{\partial \mathcal{L}}{\partial \mathbf{X}} = \frac{\partial \mathcal{L}}{\partial \mathbf{Y}} \mathbf{W}^\top \implies 2N$ FLOPs per token.
- Gradient with respect to weights: $\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \mathbf{X}^\top \frac{\partial \mathcal{L}}{\partial \mathbf{Y}} \implies 2N$ FLOPs per token.

Thus, the backward pass requires twice the compute of the forward pass:

$$C_{\text{backward}} \approx 4N \text{ FLOPs per token.} \tag{7.11}$$

Summing forward and backward passes yields the standard total:

$$C_{\text{total}} = C_{\text{forward}} + C_{\text{backward}} \approx 2ND + 4ND = 6ND. \tag{7.12}$$

□

*Derivation of the Kaplan allocation exponents.* Given a fixed compute budget $C \approx 6ND$, how should resources be divided between model size $N$ and token count $D$? Kaplan et al. derived the following empirical optimal scaling relationships:

$$N_{\text{opt}} \propto C^{0.73}, \quad B_{\text{opt}} \propto C^{0.24}, \quad S_{\text{opt}} \propto C^{0.03}, \tag{7.13}$$

where $B$ is the critical batch size and $S$ is the number of optimization steps. Since dataset size is the product of batch size and steps, $D = B \times S$:

$$D_{\text{opt}} = B_{\text{opt}} \times S_{\text{opt}} \propto C^{0.24+0.03} = C^{0.27}. \tag{7.14}$$

The exponents $N \propto C^{0.73}$ and $D \propto C^{0.27}$ deliver a startling engineering conclusion: If the computational budget $C$ is scaled by a factor of 10×:

- Model parameters should expand by: $10^{0.73} \approx 5.37\times$.
- Training tokens should expand by only: $10^{0.27} \approx 1.86\times$.
- Optimization steps increase by: $10^{0.03} \approx 1.07\times$.

Kaplan et al. explicitly advised the field: *"As computational budget C increases, it should be spent primarily on larger models, without dramatic increases in training time or dataset size."*

This paper became the gospel of the AI industry between 2020 and 2022. It directly catalyzed the global race to build massive models with hundreds of billions of parameters, while keeping training datasets relatively static at roughly 300 billion tokens. The immediate first fruit of this philosophy was GPT-3.

## 7.3 GPT-3 and In-Context Learning

Armed with Kaplan's scaling laws, OpenAI developed and released **GPT-3** in June 2020 with the monumental paper "Language Models are Few-Shot Learners" by Brown et al. [2]. GPT-3 scaled the autoregressive Transformer architecture by an order of magnitude:

- **Parameters**: 175 billion ($175 \times 10^9$) parameters—over 100× larger than GPT-2 Extra Large (1.5B) and nearly 1,500× larger than the original GPT-1 (117M).
- **Architecture**: $L = 96$ layers, $H = 96$ attention heads ($d_{\text{head}} = 128$), hidden state dimension $d = 12,288$, intermediate feed-forward dimension $d_{\text{ff}} = 49,152$, and context window $T = 2048$ tokens.
- **Alternating Dense and Locally Banded Sparse Attention**: Similar to the Sparse Transformer, GPT-3 alternated between standard dense multi-head attention and locally banded sparse attention patterns to accelerate long-sequence processing.

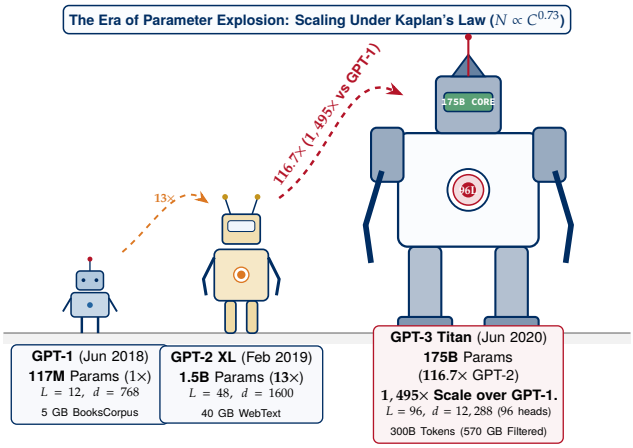



Figure 7.6: Visual parameter scale comparison across the first three generations of OpenAI's Generative Pre-trained Transformers (GPT-1, GPT-2 XL, and GPT-3). Driven by the Kaplan scaling law ($N \propto C^{0.73}$), AI research institutions poured virtually all available compute into expanding model parameters from 117 million to 175 billion ($\approx 1,500\times$ increase in two years), while training data scale remained comparatively constrained (5 GB to 300B tokens, barely 60×).

The physical layout and signal flow of the 175-billion-parameter GPT-3 architecture are illustrated in Figure 7.7.

To train a 175B model without substantial overfitting, OpenAI curated a pre-training corpus spanning hundreds of billions of tokens. Recognizing that raw web crawls contain overwhelming quantities of low-quality text, the authors built an aggressive multi-stage filtering pipeline:

1. **Quality Classifier Filtering**: A high-quality reference corpus (WebText) was used to train a logistic regression classifier with word and sentence-level features to distinguish high-quality prose from low-quality web garbage. Common Crawl was filtered through this classifier, discarding over 98% of raw pages (compressing 45 TB of raw text into 570 GB of filtered text).
2. **Fuzzy MinHash Deduplication**: MinHash LSH was applied to remove duplicate and near-duplicate web documents.
3. **High-Quality Dataset Blending**: The final pre-training mixture combined Filtered Common Crawl (410B tokens), WebText2 (19B tokens), Books1 (12B tokens), Books2 (55B tokens), and English Wikipedia (3B tokens), totaling approximately 300 billion training

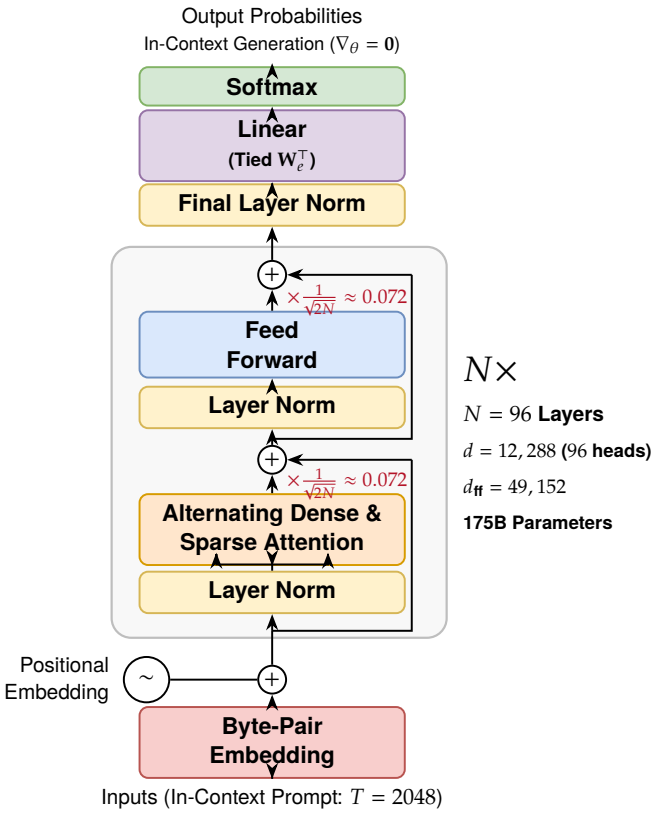



Figure 7.7: The GPT-3 autoregressive Transformer decoder architecture [2]. Scaling to 175 billion parameters ($N = 96$ layers, $d = 12,288$, 96 attention heads), GPT-3 preserves the Pre-LN residual backbone and output projection variance scaling ($\frac{1}{\sqrt{2N_{\text{layer}}}} \approx 0.072$) while alternating dense and locally banded sparse attention patterns across context windows of $T = 2048$ tokens. All downstream tasks are performed through frozen in-context generation ($\nabla_\theta = \mathbf{0}$) without parameter updates.

tokens. Sources were sampled according to perceived quality rather than raw volume (e.g., Wikipedia was sampled 3.4× per epoch, whereas Common Crawl was sampled 0.44×).

### In-context learning modalities

The central discovery of GPT-3 was the triumph of **In-Context Learning (ICL)**. Prior to GPT-3, applying a pre-trained model to a task required fine-tuning:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}_{\text{task}}(\theta_t), \tag{7.15}$$

updating millions of parameters via gradient descent on task-specific training data. In contrast, in-context learning leaves the model parameters **completely frozen** ($\nabla_\theta = \mathbf{0}$). The model is prompted with a natural language task description and zero, one, or several demonstration exemplars entirely within the input prompt context $\mathbf{x}_{\text{prompt}}$. Depending on the number of conditioning exemplars provided prior to the target query, in-context learning encompasses three distinct operational regimes:

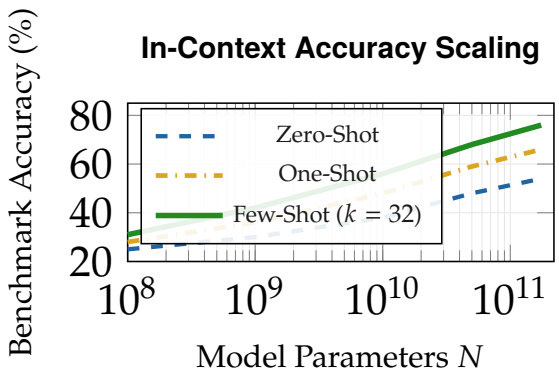
- **Zero-Shot**: Conditioning exclusively on the task instruction without demonstrations (e.g., `"Translate English to French: cheese => fromage"`). The model relies entirely on semantic priors acquired during pre-training.
- **One-Shot**: Providing exactly one worked input-output demonstration (e.g., `"sea otter => loutre de mer"`) to illustrate syntax and expected output structure before presenting the target query.
- **Few-Shot**: Providing $k$ demonstration pairs (typically $10 \leq k \leq 100$, bounded by the 2048-token context window). This conditioning activates in-context meta-learning, drastically resolving task ambiguities without executing any backward pass.

Whereas fine-tuning creates separate specialized model copies by altering network weights ($\Delta\theta \neq \mathbf{0}$), in-context learning employs a single universal frozen model ($\Delta\theta = \mathbf{0}$) that adapts dynamically through attention routing over prompt activations.

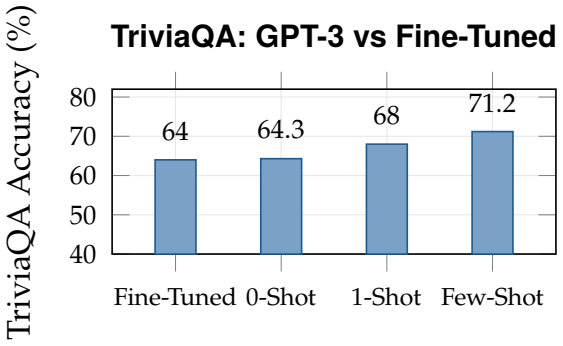
### Empirical capabilities and benchmark evaluations

The empirical findings of Brown et al. [2] yielded three profound takeaways that fundamentally transformed natural language processing:

1. **The power of few-shot prompting**: Across virtually every benchmark, the performance gap between zero-shot and one-shot was massive, with few-shot prompting ($k = 32$) achieving substantial further gains (Figure 7.8a). Providing even a single demonstration exemplar dramatically clarifies the expected output format and task semantics.
2. **Rivaling fine-tuned state of the art**: On demanding closed-book QA benchmarks such as TriviaQA (64.3% accuracy) and WebQuestions, GPT-3 175B in the few-shot setting **matched or exceeded state-of-the-art models that had been explicitly fine-tuned on the task training set** (e.g., T5-11B), as illustrated in Figure 7.8b. Scaling parameters eliminated the absolute necessity of task-specific training sets.
3. **Emergent reasoning abilities**: While smaller models (under 10B parameters) achieved near-zero accuracy on complex multi-digit arithmetic (e.g., 3-digit addition, subtraction), anagram unscrambling, and synthetic word manipulation, GPT-3 175B demonstrated sudden, qualitative leaps in accuracy.




(a) Accuracy scaling across prompt regimes.




(b) Few-shot GPT-3 surpasses fine-tuned SOTA without updates.

Figure 7.8: Empirical scaling and performance of in-context learning in GPT-3 [2]. (a) Scaling curves showing prompt effectiveness. (b) Performance on TriviaQA demonstrating that scaling eliminates task-specific fine-tuning.

## 7.4 The Undertraining Crisis

Following the release of GPT-3 and the widespread adoption of Kaplan's scaling laws ($N \propto C^{0.73}, D \propto C^{0.27}$), the global AI research community entered a period of frantic model expansion. Research institutions concluded that the primary path to superior intelligence was expanding parameter count while holding training tokens roughly constant at approximately 300 billion tokens.

Between 2020 and early 2022, several colossal models were trained:

- **Jurassic-1 (AI21 Labs, 2021) [69]**: A 178-billion-parameter autoregressive model featuring an expanded vocabulary of 250,000 subword tokens (including multi-word expressions) and trained on 300 billion tokens.
- **Gopher (DeepMind, Dec 2021) [70]**: A 280-billion-parameter model ($L = 80, d = 16,384, H = 128$) trained on the MassiveText corpus (300 billion tokens) using $5.76 \times 10^{23}$ FLOPs. The Gopher publication spanned an unprecedented 120 pages: pages 1–7 presented the architecture, while pages 8–120 documented an exhaustive analysis of capabilities, hallucinations, bias, and toxic text generation, establishing the modern format for foundation model technical reports.
- **Megatron-Turing NLG (Microsoft & NVIDIA, Oct 2021) [67]**: A massive 530-billion-parameter model ($L = 105, d = 20,480, H = 128$) trained on 270 billion tokens. Training MT-NLG pushed distributed hardware to its limits, requiring 3D parallelism: combining 8-way Megatron tensor parallelism, 35-way pipeline parallelism across DeepSpeed, and 8-way data parallelism across 2,240 NVIDIA A100 GPUs.
- **PaLM (Google Pathways, April 2022) [71]**: A 540-billion-parameter model ($L = 118, d = 18,432, H = 48$) trained on 780 billion tokens across 6,144 TPU v4 chips using the Pathways system. PaLM introduced SwiGLU activations, parallel attention-MLP layers, and Rotary Position Embeddings (RoPE), achieving breakthrough performance on mathematical reasoning (GSM8K) and code generation.

Table 7.1: The race to scale: Landmark dense foundation models trained between 2020 and 2022.

| Model | Organization | Release | Params ($N$) | Tokens ($D$) | Ratio ($N/D$) | Compute ($C$ FLOPs) |
|---|---|---|---|---|---|---|
| **GPT-3** | OpenAI | Jun 2020 | 175B | 300B | 0.58 | $3.15 \times 10^{23}$ |
| **Jurassic-1** | AI21 Labs | Sep 2021 | 178B | 300B | 0.59 | $3.20 \times 10^{23}$ |
| **Megatron-Turing** | NVIDIA / Microsoft | Oct 2021 | 530B | 270B | **1.96** | $8.58 \times 10^{23}$ |
| **Gopher** | DeepMind | Dec 2021 | 280B | 300B | 0.93 | $5.76 \times 10^{23}$ |
| **PaLM** | Google | Apr 2022 | 540B | 780B | 0.69 | $2.53 \times 10^{24}$ |

### 7.4.1 Pathologies of oversized models

Notice the profound structural imbalance revealed in Table 7.1. Between GPT-1 (117M) and PaLM (540B), model parameters had exploded by over $4,600\times$. Yet during this exact same period, training corpus volume expanded from 5 GB to barely 300–780 billion tokens—a factor of scarcely $60\times$.

Every major AI laboratory was strictly executing Kaplan's prescription: spend virtually all available compute scaling parameters ($N \propto C^{0.73}$), while treating training token volume as a secondary consideration ($D \propto C^{0.27}$). In Megatron-Turing NLG (530B), the parameter-to-data ratio reached an astounding:

$$\frac{N}{D} = \frac{530 \times 10^9}{270 \times 10^9} \approx 1.96 \text{ parameters per token.} \tag{7.16}$$

In other words, the model possessed nearly **two parameters for every single token seen during its entire training lifetime**. Each parameter received on average only $\approx 0.51$ tokens of gradient signal. This severe asymmetry induced multiple substantial operational and theoretical pathologies:

1. **Capacity under-utilization and generalization brittleness**: Statistical learning theory dictates that overparameterized models require rich empirical data distributions to regularize high-dimensional hypothesis spaces. When a 530B-parameter model is trained on only 270B tokens, vast sub-circuits and feed-forward weight blocks remain severely under-trained, having received minuscule gradient updates. While the model achieves high memorization of superficial syntactic patterns, its deeper semantic representations remain brittle. This manifested in practice as extreme prompt fragility (where substituting minor synonyms or altering punctuation drastically altered outputs), frequent multi-step reasoning breakdowns, and elevated rates of fluent hallucinations and toxic generations. In the 120-page Gopher technical report [70], DeepMind demonstrated that scaling parameter count from 44M to 280B at fixed $D = 300$B yielded diminishing returns on factual accuracy while providing no protection against hallucinatory confabulations.

2. **Inference memory-bandwidth saturation**: While pre-training compute is amortized once across the model's development cycle, inference costs scale linearly with every generated token across millions of daily user interactions. Crucially, autoregressive language model inference is fundamentally **memory-bandwidth bound**, rather than compute-bound. During token-by-token autoregressive decoding, each forward pass generates exactly one new token per sequence ($T = 1$). Computing that single token requires loading **every single weight of the model from High-Bandwidth Memory (HBM) into the GPU streaming multiprocessors (SMs)**, yielding an operational arithmetic intensity of approximately $\approx 1$ FLOP per byte of memory loaded.

   For a 530B-parameter model stored in 16-bit floating point precision (FP16 or BF16, at 2 bytes per parameter), the static weight footprint alone is:

   $$M_{\text{weights}} = 530 \times 10^9 \times 2 \text{ bytes} \approx 1,060 \text{ GB} = 1.06 \text{ TB}. \tag{7.17}$$

   On standard NVIDIA A100 GPUs (equipped with 80 GB HBM2e memory), storing merely the static weights requires a strict theoretical minimum of $\lceil 1060/80 \rceil = 14$ GPUs, realistically requiring a full server pod of 16 A100 GPUs. Furthermore, serving concurrent user requests requires allocating high-speed memory for the dynamic Key-Value (KV) cache across all layers and attention heads:

   $$M_{\text{KV}} = 2 \times B \times L \times d \times T_{\text{ctx}} \times 2 \text{ bytes}, \tag{7.18}$$

   where $B$ is batch size, $L = 105$ layers, $d = 20,480$, and $T_{\text{ctx}}$ is context length. For a modest batch size of $B = 16$ at $T_{\text{ctx}} = 2048$, the KV cache requires an additional $\approx 280$ GB of VRAM, pushing total memory consumption beyond 1.34 TB.

3. **Multi-node cluster communication latency and MFU collapse**: Distributing a 530B model across 16 GPUs requires combining Tensor Parallelism ($TP = 8$) across NVLink within a node and Pipeline Parallelism ($PP = 2$) across nodes via InfiniBand. At every single Transformer layer, tensor parallelism performs an `All-Reduce` collective communication primitive across the high-speed crossbar to synchronize activation vectors. Because the arithmetic execution for a single token completes in microseconds, the GPU compute cores spend the vast majority of execution time stalled waiting for inter-GPU communication and memory bus transfers. Consequently, Model FLOPs Utilization (MFU) during interactive serving collapses to under 10–15%, causing per-token generation latencies to exceed 200–500 milliseconds per token.

4. **The economic fallacy of "cheap pre-training, ruinous inference"**: Kaplan's scaling laws optimized exclusively for training compute efficiency ($C = 6ND$), implicitly treating inference costs as zero. In industrial software engineering and commercial software ecosystems, however, inference queries outnumber pre-training passes by many orders of magnitude. An oversized, undertrained model like MT-NLG (530B) or Gopher (280B) represents an economic disaster: it was relatively inexpensive to pre-train (because it saw only 270B–300B tokens), but substantially expensive to host and execute indefinitely. Conversely, a compute-optimal model (e.g., 70B parameters trained on 1.4 trillion tokens) achieves identical or superior validation loss while occupying $\frac{1}{7.5}$ of the memory footprint, fitting comfortably onto a single 8-GPU node (or a quantized 2-GPU setup), and delivering $\approx 7\times$ higher inference throughput at a fraction of the electricity and cloud operational expenditure (OpEx).

### 7.4.2 Methodological diagnosis of Kaplan scaling laws

How could a landmark empirical study conducted by OpenAI reach scaling laws that led the entire field into a multi-year undertraining cul-de-sac? When researchers at DeepMind rigorously audited the experimental protocol of Kaplan et al. [68], they uncovered four fundamental methodological flaws:

1. **Suboptimal learning rate schedules and absence of cosine cooldown**: In modern Transformer pre-training, the learning rate $\eta_t$ follows a cosine decay schedule with initial linear warmup:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})\left(1 + \cos\left(\frac{t}{T_{\text{total}}}\pi\right)\right), \tag{7.19}$$

   where $T_{\text{total}}$ represents the predetermined total step horizon, decaying to $\eta_{\min} = 0.1\eta_{\max}$. A well-documented phenomenon in deep optimization is that **a disproportionately large fraction of total loss reduction occurs during the final cosine cooldown phase** (the last 10–20% of steps where the learning rate anneals to its minimum). Kaplan et al. did not re-tune $T_{\text{total}}$ independently for every model run. Instead, they held the schedule length fixed to a constant step count or evaluated intermediate checkpoints along a single extended trajectory (e.g., evaluating a run at step 20,000 when $T_{\text{total}} = 100,000$). At step 20,000, the learning rate was still hovering near $\eta_{\max}$, meaning the model had never experienced the critical loss-reducing cooldown. Because smaller models reached lower compute budgets earlier in the trajectory, they were systematically evaluated in an under-converged, un-cooled state. This severely depressed the apparent sample efficiency of smaller models, creating the false mathematical illusion that increasing tokens yields diminishing returns compared to expanding parameters. When DeepMind properly trained every model with a cosine schedule matched exactly to its training duration, smaller models exhibited dramatically steeper loss drops.

2. **Exclusion of vocabulary and embedding parameters from compute accounting**: Kaplan et al. defined model size strictly as $N_{\text{non-emb}}$, explicitly discarding the parameters of the input embedding matrix $\mathbf{W}_e \in \mathbb{R}^{|\mathcal{V}|\times d}$ and output unembedding layer $\mathbf{W}_u \in \mathbb{R}^{d\times|\mathcal{V}|}$ from the compute formula $C \approx 6N_{\text{non-emb}}D$. Their justification was that embedding weights do not participate in token-to-token attention operations. However, for a standard Byte-Pair Encoding vocabulary of $|\mathcal{V}| = 50,257$, the embedding matrix comprises $50,257 \times d$ parameters. For smaller models with $d = 768$ (such as GPT-1), $\mathbf{W}_e$ accounts for $\approx 38.6$M parameters—nearly **33% of the entire network**. Excluding embeddings introduced a substantial non-linear distortion at small and medium model scales ($10^7 \leq N \leq 10^9$), corrupting the parameter-to-token trade-off and biasing the fitted regression toward expanding depth and width rather than training data.

3. **Evaluation on sequence tail tokens rather than whole-sequence loss**: Kaplan et al. evaluated validation cross-entropy loss exclusively on the final tokens of each sequence (the sequence tail, e.g., the last 10% of tokens in context $T = 1024$), rather than averaging loss across the entire context window. While intended to measure performance under maximal historical context, this evaluation protocol created a structural bias in favor of massive parameter counts: high-capacity models possess superior associative memory for long prefixes, but this masked the severe degradation that undertrained models suffer when predicting tokens across early and middle context positions. Averaging loss over the entire sequence reveals that well-trained smaller models generalize vastly better than undertrained giants.

4. **Restricted token dynamic range and regression multicollinearity**: In Kaplan et al.'s experimental runs, model parameters $N$ were varied across nearly six orders of magnitude ($10^3$ to $10^9$), but training tokens $D$ were varied across a significantly narrower dynamic range, often held constant across entire sub-experiments. When fitting a bivariate power-law loss surface:
$$\mathcal{L}(N, D) = \left(\frac{N_c}{N}\right)^{\alpha_N} + \left(\frac{D_c}{D}\right)^{\alpha_D} + \mathcal{L}_0, \tag{7.20}$$
the lack of an independently perturbed, balanced two-dimensional grid of $(N, D)$ runs induced severe **multicollinearity** in the log-linear regression. The optimization algorithm attributed the majority of compute-driven loss reduction to $N$ ($\alpha_N \approx 0.076 \implies a = 0.73$), while severely dampening the sensitivity to $D$ ($\alpha_D \approx 0.057 \implies b = 0.27$).

This critical realization proved that the industry was building models that were **oversized and significantly undertrained**. To discover the true compute-optimal allocation, DeepMind launched an exhaustive empirical investigation that resulted in the Chinchilla scaling laws.

## 7.5 Chinchilla Compute-Optimal Scaling

In March 2022, Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, and colleagues at DeepMind published one of the most influential papers in modern artificial intelligence: **"Training Compute-Optimal Large Language Models"** [3].

The central claim of the Chinchilla paper removed the prevailing industry consensus:

> *"We find that current large language models are significantly under-trained, a consequence of the recent focus on scaling language models whilst keeping the amount of training data constant. . . For every doubling of model size, the number of training tokens should also be doubled."*

DeepMind proved that the conclusions of Kaplan et al. [68] gave vastly too much weight to model size. While OpenAI advocated $N \propto C^{0.73}$ and $D \propto C^{0.27}$, DeepMind demonstrated that the true compute-optimal frontier requires **equal, balanced scaling**:

$$N_{\text{opt}} \propto \sqrt{C}, \quad D_{\text{opt}} \propto \sqrt{C}. \tag{7.21}$$

To establish this result incontrovertibly, DeepMind trained over 400 language models across varying scales and evaluated the data through **three independent analytical approaches**.

### 7.5.1 Approach 1: minimum loss envelopes

In the first approach, the authors trained a family of models with fixed parameter sizes $N \in \{70\text{M}, \ldots, 16\text{B}\}$ across varying training token durations.

For each individual model size $N$, training traces out a smooth curve of cross-entropy loss as a function of consumed compute $C = 6ND$:

1. For any specified compute budget $C_0$ (e.g., $C_0 = 10^{20}$ FLOPs), we inspect all model curves intersecting this budget.
2. We identify the specific model parameter count $N$ that achieves the **global minimum loss** $\min \mathcal{L}$ at $C_0$.
3. We record the pair $(N_{\text{opt}}, C_0)$ and the corresponding token count $D_{\text{opt}} = C_0/(6N_{\text{opt}})$.
4. Repeating this across all compute budgets from $10^{18}$ to $10^{22}$ FLOPs, we fit power-law relationships:
$$N_{\text{opt}} = GC^a, \quad D_{\text{opt}} = HC^b. \tag{7.22}$$

Empirically, Approach 1 yielded:

$$a = 0.50, \quad b = 0.50. \tag{7.23}$$

When projected to the computational budget of Gopher ($C = 5.76 \times 10^{23}$ FLOPs), Approach 1 indicated that the optimal model size was **67 billion parameters**, requiring **1.43 trillion tokens**—not 280 billion parameters trained on 300 billion tokens. Figure 7.9 displays the empirical envelope across model sizes.

### 7.5.2 Approach 2: isoflop profiles

In the second approach, DeepMind fixed the total compute budget $C \in \{10^{18}, 10^{19}, 10^{20}, 10^{21}, 10^{22}\}$ FLOPs and systematically varied the trade-off between parameter count $N$ and dataset size $D = C/(6N)$.

For each IsoFLOP slice, the final training loss was plotted against parameter count $N$ on a logarithmic scale. Empirically, the data forms an exact **parabola** (a convex bowl):

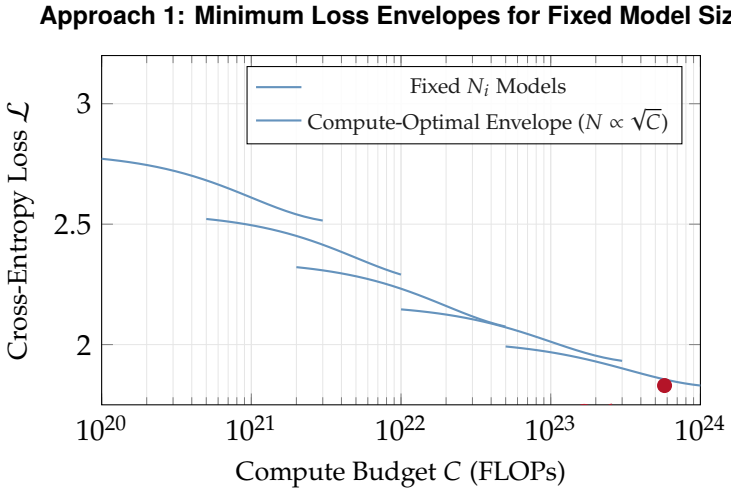



Figure 7.9: Chinchilla Approach 1 (fixed-size model envelopes) [3]. By evaluating training runs across varying parameter counts, the lower convex envelope identifies the compute-optimal parameter count for each compute budget ($a = 0.50, b = 0.50$), proving Gopher should have had 67B parameters on 1.43T tokens.

- If $N$ is too small (left side of parabola), the model has insufficient capacity to absorb the information in $D$ tokens, causing high loss due to under-parameterization.
- If $N$ is too large (right side of parabola), the model is severely undertrained because $D = C/(6N)$ is too small, causing high loss due to data starvation.

The loss within an IsoFLOP slice is modeled by fitting a quadratic polynomial in $\log N$:

$$\log \mathcal{L}(N) = c_0 + c_1 \log N + c_2 (\log N)^2, \quad \text{with } c_2 > 0. \tag{7.24}$$

The optimal model size corresponds precisely to the **vertex (minimum)** of the parabola:

$$\frac{\partial \log \mathcal{L}}{\partial \log N} = c_1 + 2c_2 \log N = 0 \implies \log N_{\text{opt}} = -\frac{c_1}{2c_2} \implies N_{\text{opt}} = \exp\left(-\frac{c_1}{2c_2}\right). \tag{7.25}$$

By extracting $N_{\text{opt}}$ for each IsoFLOP curve and fitting power-law relations $N_{\text{opt}} \propto C^a$ and $D_{\text{opt}} \propto C^b$, Approach 2 yielded:

$$a = 0.49, \quad b = 0.51. \tag{7.26}$$

For Gopher's compute budget, Approach 2 determined that the compute-optimal model size was **63 billion parameters** trained on **1.53 trillion tokens**. Figure 7.10 displays the parabolic loss profiles across IsoFLOP slices.

### 7.5.3 Approach 3: parametric loss surface

The third approach formulated an analytical parametric model of the loss surface $\mathcal{L}(N, D)$ over all training runs simultaneously.

> **Theorem 7.2:** *Parametric scaling law formulation*
>
> The expected cross-entropy loss $\mathcal{L}(N, D)$ for an autoregressive language model with $N$ parameters trained on $D$ tokens can be decomposed into an irreducible linguistic entropy

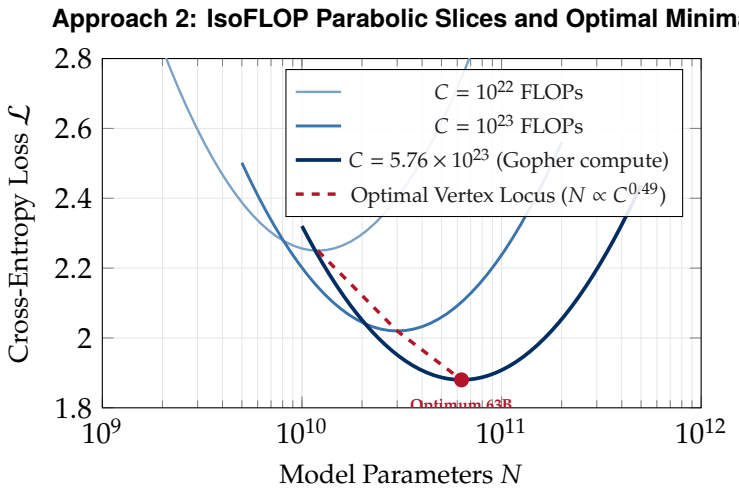



Figure 7.10: Chinchilla Approach 2 (IsoFLOP profiles) [3]. For any fixed compute budget $C$, the loss follows a parabolic curve whose vertex identifies the optimal parameter count ($a = 0.49, b = 0.51$), demonstrating that Gopher should have been 63B parameters.

and power-law penalties for finite capacity and finite data:

$$\hat{\mathcal{L}}(N, D) = E + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}}, \tag{7.27}$$

where:

- $E$ is the **irreducible entropy** of natural text (the theoretical floor of cross-entropy loss).
- $\frac{A}{N^{\alpha}}$ represents the risk penalty due to **finite model capacity**.
- $\frac{B}{D^{\beta}}$ represents the risk penalty due to **finite training dataset size**.
- $A, B, E, \alpha, \beta$ are positive constants.

Fitting this functional form over all 400+ experimental runs using Huber loss minimization yielded:

$$E \approx 1.69, \quad A \approx 406.4, \quad B \approx 410.7, \quad \alpha \approx 0.34, \quad \beta \approx 0.28. \tag{7.28}$$

We can now mathematically derive the compute-optimal allocation by solving a **constrained optimization problem**:

$$\min_{N,D} \hat{\mathcal{L}}(N, D) = E + AN^{-\alpha} + BD^{-\beta} \quad \text{subject to } 6ND = C. \tag{7.29}$$

*Proof.* Formulate the Lagrangian function with multiplier $\lambda$:

$$\mathcal{L}_{\text{Lagrange}}(N, D, \lambda) = E + AN^{-\alpha} + BD^{-\beta} + \lambda(6ND - C). \tag{7.30}$$

Setting the first-order partial derivatives with respect to $N$ and $D$ to zero:

$$\frac{\partial \mathcal{L}_{\text{Lagrange}}}{\partial N} = -\alpha AN^{-\alpha-1} + 6\lambda D = 0 \implies 6\lambda ND = \alpha AN^{-\alpha}, \tag{7.31}$$

$$\frac{\partial \mathcal{L}_{\text{Lagrange}}}{\partial D} = -\beta BD^{-\beta-1} + 6\lambda N = 0 \implies 6\lambda ND = \beta BD^{-\beta}. \tag{7.32}$$

Equating the two expressions for $6\lambda ND$:

$$\alpha A N^{-\alpha} = \beta B D^{-\beta}. \tag{7.33}$$

Equation (7.33) establishes the **fundamental economic optimality condition**: at the compute-optimal frontier, the marginal reduction in loss obtained by spending compute on model parameters must exactly balance the marginal reduction in loss obtained by spending compute on additional tokens.

To express $N_{\text{opt}}$ and $D_{\text{opt}}$ purely in terms of total compute $C$, substitute $D = \frac{C}{6N}$ into Equation (7.33):

$$\alpha A N^{-\alpha} = \beta B \left(\frac{C}{6N}\right)^{-\beta} = \beta B 6^{\beta} C^{-\beta} N^{\beta}. \tag{7.34}$$

Grouping powers of $N$:

$$N^{\alpha+\beta} = \frac{\alpha A}{\beta B 6^{\beta}} C^{\beta} \implies N_{\text{opt}}(C) = \left(\frac{\alpha A}{\beta B 6^{\beta}}\right)^{\frac{1}{\alpha+\beta}} C^{\frac{\beta}{\alpha+\beta}}. \tag{7.35}$$

By symmetry, substituting $N = \frac{C}{6D}$ yields the optimal dataset size:

$$D_{\text{opt}}(C) = \left(\frac{\beta B}{\alpha A 6^{\alpha}}\right)^{\frac{1}{\alpha+\beta}} C^{\frac{\alpha}{\alpha+\beta}}. \tag{7.36}$$

Substituting the empirically fitted exponents $\alpha \approx 0.34$ and $\beta \approx 0.28$:

$$a = \frac{\beta}{\alpha+\beta} = \frac{0.28}{0.34+0.28} = \frac{0.28}{0.62} \approx 0.45, \tag{7.37}$$

$$b = \frac{\alpha}{\alpha+\beta} = \frac{0.34}{0.34+0.28} = \frac{0.34}{0.62} \approx 0.55. \tag{7.38}$$

□

Figure 7.11a and Figure 7.11b display the contour geometry of this parametric loss surface and contrast the resulting compute-optimal trajectory against Kaplan's undertrained scaling regime.

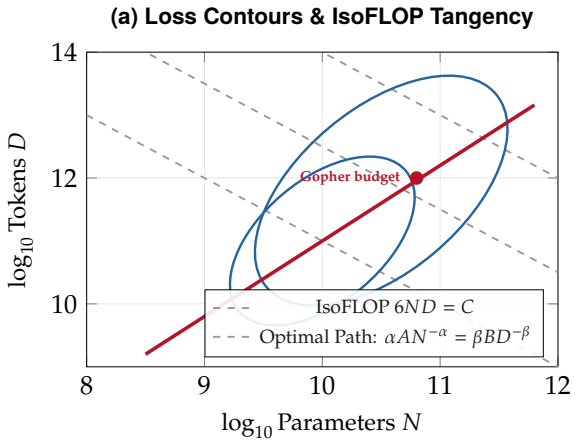
### 7.5.4 Empirical validation: Chinchilla vs. Gopher

To empirically validate their theoretical derivations, DeepMind trained **Chinchilla**:

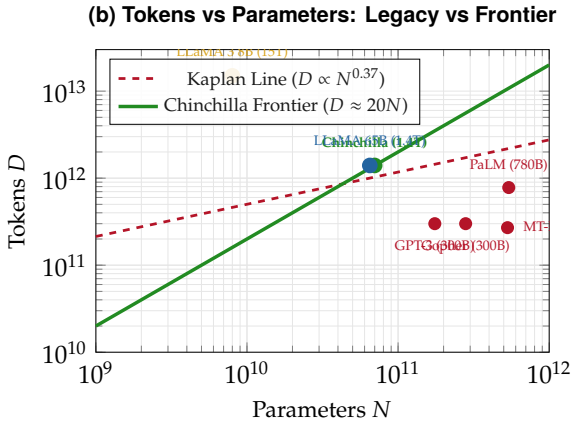
- **Identical Compute Budget**: Chinchilla was allocated exactly the same compute budget as Gopher ($5.76 \times 10^{23}$ FLOPs).
- **Parameters**: 70 billion parameters ($L = 80, d = 8192, H = 64$)—exactly **one-quarter the size of Gopher** (280B).
- **Training Tokens**: 1.4 trillion ($1.4 \times 10^{12}$) tokens—over **four times more data than Gopher** (300B).

The empirical evaluation results were decisive:

- Chinchilla uniformly **outperformed Gopher (280B), GPT-3 (175B), Jurassic-1 (178B), and Megatron-Turing NLG (530B)** across virtually every downstream evaluation benchmark.
- On the Massive Multitask Language Understanding (MMLU) benchmark, Chinchilla achieved an average score of 67.5%, outperforming Gopher (60.0%) by 7.5 percentage points.

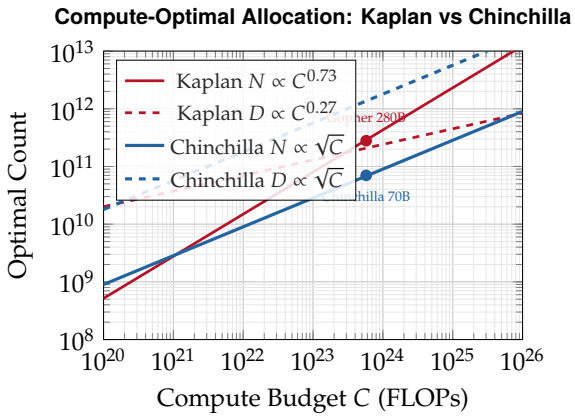



(a) Parametric loss contour surface: IsoFLOP slices and compute-optimal tangency.

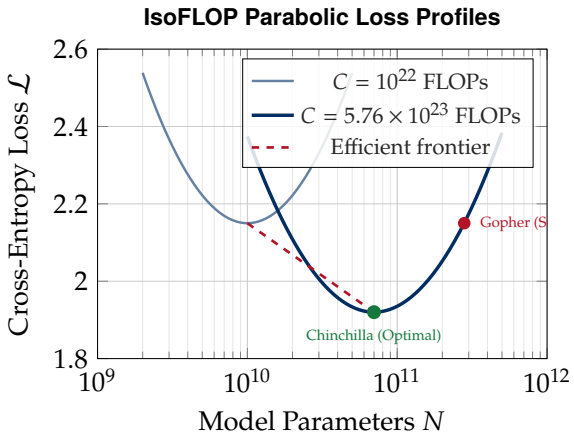



(b) Model comparison: legacy half-trillion giants were severely undertrained.

Figure 7.11: Chinchilla Approach 3 parametric loss surface modeling and empirical model comparison [3]. (a) Optimal allocation occurs at the tangency of IsoFLOP constraint lines and loss contours. (b) Dataset size versus parameter count, proving earlier giants were starved for tokens.




(a) Scaling exponents: Kaplan favors massive parameters ($a = 0.73$), while Chinchilla proves balanced scaling ($a \approx b \approx 0.50$).




(b) IsoFLOP profile at Gopher budget: Gopher sits on the undertrained right wing; Chinchilla sits at the parabolic minimum.

Figure 7.12: Analytical derivations of compute-optimal language modeling from DeepMind's Chinchilla study [3].

Table 7.2: Direct architectural and performance showdown: Gopher (280B) versus Chinchilla (70B) at identical compute.

| Attribute | Gopher (Kaplan Baseline) | Chinchilla (Compute-Optimal) |
|---|---|---|
| **Parameters ($N$)** | 280 Billion ($280 \times 10^9$) | **70 Billion** ($70 \times 10^9$) [$\frac{1}{4}\times$ size] |
| **Training Tokens ($D$)** | 300 Billion ($300 \times 10^9$) | **1.4 Trillion** ($1.4\times10^{12}$) [4.7× tokens] |
| **Compute Budget ($C$)** | $5.76 \times 10^{23}$ FLOPs | $5.76 \times 10^{23}$ FLOPs (**Identical**) |
| **MMLU Benchmark** | 60.0% average accuracy | **67.5%** average accuracy (+7.5%) |
| **BIG-bench Tasks** | Baseline | Outperforms Gopher on > 65% of tasks |
| **Inference Memory (FP16)** | ≈ 560 GB VRAM (min 8 A100s) | ≈ **140 GB VRAM** (fits on 2 A100s) |
| **Inference Latency** | High (∝ 280B) | **Cut by** ≈ 75% (∝ 70B) |

- On commonsense reasoning, reading comprehension, and math benchmarks (GSM8K), Chinchilla established comprehensive state-of-the-art results.
- Crucially, because Chinchilla had only 70B parameters, its **inference latency was dramatically lower and memory consumption was cut by 75%**, requiring only two NVIDIA A100 GPUs instead of eight.

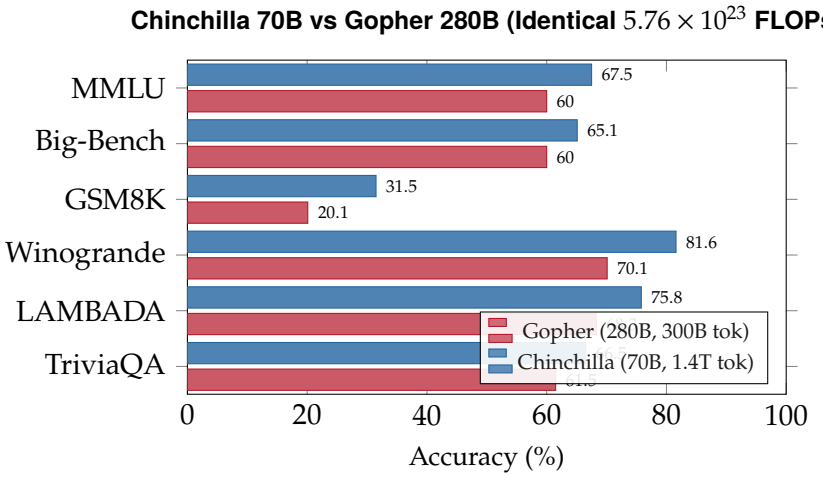



Figure 7.13: Empirical validation of compute-optimality: Chinchilla 70B versus Gopher 280B across major benchmarks [3]. With one-quarter the parameter count and four times more tokens at identical training compute ($5.76 \times 10^{23}$ FLOPs), Chinchilla comprehensively outperforms Gopher while drastically slashing inference serving overhead.

## 7.6 Hardware Realities and Open Models

The transition from the Kaplan scaling regime to Chinchilla compute-optimality fundamentally restructured the economics and accessibility of artificial intelligence. To appreciate why compute-optimal scaling catalyzed the explosive rise of **open-weights foundation models**, we must examine the stark contrast between the hardware barriers of the 2020–2022 parameter race and the deployment realities of compact, highly trained architectures.

### 7.6.1 The economic and hardware drivers of the open-weight revolution

Under Kaplan et al. [68], leading laboratories allocated over 70% of scaling compute to parameter expansion ($N \propto C^{0.73}$). This doctrine culminated in half-trillion-parameter monoliths: GPT-3 (175B), Gopher (280B), and Megatron-Turing NLG (530B). These mega-models introduced an prohibitive hardware barrier:

- **Colossal Memory Requirements**: Storing 175B to 530B parameters in 16-bit precision requires between 350 GB and 1,060 GB of VRAM strictly for model weights. Serving interactive generation requires allocating an additional 200 GB to 400 GB for the dynamic Key-Value (KV) cache.
- **Cluster-Scale Hardware Monopolies**: Serving a single instance of a 175B+ model necessitated high-end multi-GPU server nodes equipped with 8 to 16 NVIDIA A100 (80 GB) GPUs connected via high-speed NVLink and InfiniBand fabrics. The acquisition, electrical power, and cooling infrastructure required to run such clusters cost hundreds of thousands of dollars, confining practical deployment exclusively to trillion-dollar tech conglomerates.
- **Centralized API Enclosures**: Because individual researchers, universities, and small enterprises could neither run nor fine-tune these models locally, frontier AI was locked behind proprietary commercial web APIs (such as OpenAI's completions endpoint). Users were reduced to passive consumers, unable to audit model internals, inspect training data contamination, or fine-tune weights for privacy-sensitive corporate workflows.

The Chinchilla revelation (Hoffmann et al. [3]) exposed the fundamental flaw of this paradigm: monster models were severely under-trained, wasting immense parameter capacity on small data volumes. A compute-optimal 70B model trained on 1.4 trillion tokens achieved parity with a 280B model trained on 300 billion tokens while consuming identical pre-training compute.

Crucially, in industrial deployment and software engineering, pre-training compute is amortized once, whereas inference serving costs scale perpetually across billions of operational queries ($Q \gg 1$). The **lifecycle computational cost** of a model is given by:

$$C_{\text{lifecycle}} = C_{\text{train}} + Q \times C_{\text{inference}} \approx C_{\text{train}} + Q \times \left(2NL_{\text{gen}}\right), \tag{7.39}$$

where $L_{\text{gen}}$ is the average sequence length generated per query. Because per-token inference compute scales strictly linearly with parameter count $N$, serving a 175B model is approximately 25× more expensive per query than serving a 7B model.

This realization gave birth to **inference-optimal overtraining**: investing surplus compute during pre-training to train compact models ($N \in \{7\text{B}, 8\text{B}, 13\text{B}\}$) far past the Chinchilla point on multi-trillion-token corpora (1.0T to 15T tokens). While sub-optimal for pre-training FLOP efficiency alone, this strategy produces compact models with capabilities rivaling legacy mega-models, while drastically slashing lifetime serving costs.

#### The consumer hardware democratization threshold

The breakthrough for the open-source community was physical:

1. A 7-billion-parameter model in 16-bit precision requires only $\approx$ 14 GB of memory.
2. Applying state-of-the-art 4-bit Post-Training Quantization (PTQ) techniques (such as AWQ or GPTQ) compresses the entire model footprint to **under 4.5 GB of memory** with negligible perplexity degradation.

For the first time in history, a frontier-quality foundation model could be loaded into memory and executed locally on commodity consumer hardware: a standard mid-range gaming laptop (e.g., an NVIDIA RTX 3060 or 4060 with 6–8 GB VRAM), an Apple Silicon Mac with unified memory, or a single commodity cloud GPU.

This collapse of the hardware barrier removed corporate monopolies, sparking a decentralized explosion of open research. Practitioners could now execute private code generation in local IDEs without leaking intellectual property, perform parameter-efficient fine-tuning (LoRA, QLoRA) on single workstations, and rigorously audit safety, alignment, and algorithmic bias.

### 7.6.2 The LLaMA family and modern open architectures

Starting in February 2023, Meta AI introduced the **LLaMA** (Large Language Model Meta AI) family, establishing the architectural gold standard for open-weights models:

- **LLaMA 1 (Feb 2023) [66]**: Trained across four sizes: 7B, 13B, 33B, and 65B on 1.0T to 1.4T tokens. LLaMA demonstrated that training exclusively on publicly available datasets (Common Crawl, GitHub, Wikipedia, ArXiv, StackExchange) could match or exceed proprietary closed models like GPT-3. LLaMA modernized the Transformer architecture with three critical innovations:
  1. **RMSNorm (Root Mean Square Normalization)**: Replacing standard LayerNorm to eliminate mean-centering, saving 7% of memory bandwidth:
  $$\text{RMSNorm}(\mathbf{x}) = \frac{\mathbf{x}}{\sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2 + \epsilon}} \odot \gamma. \tag{7.40}$$
  2. **SwiGLU Activation Function**: Replacing standard ReLU or GELU with Swish-Gated Linear Units [17]:
  $$\text{SwiGLU}(\mathbf{x}) = (\mathbf{x}\mathbf{W} \odot \text{SiLU}(\mathbf{x}\mathbf{V}))\mathbf{W}_2, \quad \text{SiLU}(z) = z \cdot \sigma(z). \tag{7.41}$$
  3. **Rotary Position Embeddings (RoPE)** [54]: Injecting positional information by rotating query and key vectors in complex 2D subspaces, enabling natural length extrapolation.
- **Llama 2 (July 2023) [72]**: Scaled training tokens to 2.0 trillion tokens across 7B, 13B, and 70B sizes. Llama 2 doubled the context window to 4096 tokens and introduced **Grouped-Query Attention (GQA)** on the 70B model, sharing single key and value heads across groups of query heads to drastically compress KV-cache memory during autoregressive generation. Meta also released official instruction-tuned models (Llama-2-Chat) aligned with Reinforcement Learning from Human Feedback (RLHF).
- **Llama 3 (April 2024) and Llama 3.1 (July 2024) [73]**: Pushed inference-optimal overtraining to the absolute extreme:
  - The 8B and 70B models were trained on over **15 trillion tokens**—overtrained by more than 50× beyond the Chinchilla compute-optimal frontier.
  - Implemented a massive 128,256-token tiktoken tokenizer, boosting compression efficiency on code and multilingual text by 15%.

  - Llama 3.1 introduced the frontier **405B flagship model** trained on 16 trillion tokens across 16,000 NVIDIA H100 GPUs, supporting a native 128k context window and rivaling top proprietary models (GPT-4, Claude 3.5 Sonnet).

- **Llama 3.2 (September 2024)**: Expanded the family into edge deployment with lightweight 1B and 3B models optimized for on-device mobile hardware, alongside multimodal vision foundation models (11B and 90B).

- **Specialized Variants (e.g., Code Llama)**: Fine-tuned specifically on 500B tokens of source code, supporting the Fill-in-the-Middle (FIM) objective for real-time code infilling and completion in software engineering IDEs.

**Licensing and commercial barriers**

Meta released the LLaMA weights under a bespoke permissive license that catalyzed academic and industrial adoption while establishing a strategic defensive moat: usage is entirely free for research and commercial deployment, *provided the licensee's product serves fewer than 700 million monthly active users* (set at 100 million in earlier drafts). As highlighted in lecture, this threshold served as an explicit barrier against massive hyperscale competitors (such as Google and Amazon) utilizing LLaMA to power their commercial search ecosystems without reciprocal licensing, while ensuring universal free access for startups, universities, and independent practitioners.

### 7.6.3 Mistral AI and sparse mixture-of-experts

Founded in Paris in 2023, Mistral AI introduced architectural innovations focusing on maximum compute efficiency:

- **Mistral 7B (October 2023)** [74]: Outperformed Llama 2 13B across all benchmarks despite having half the parameter count. Key mechanisms included:

  1. **Sliding Window Attention (SWA)**: Each token attends only to the preceding $W = 4096$ tokens at each layer. Across $L$ layers, information propagates through an effective receptive field of $L \times W$, achieving linear $O(T \times W)$ computational and cache memory complexity.
  2. **Rolling Buffer Cache**: Storing keys and values in a fixed-size circular cache of capacity $W$, overwriting slots modulo $W$ to eliminate memory growth on infinite generation streams.
  3. **Grouped-Query Attention (GQA)**: Standard across all model sizes (8 query heads per KV head).

- **Mixtral 8x7B (December 2023)**: Pioneered open-weights **Sparse Mixture-of-Experts (MoE)**. While the model contains 46.7 billion total parameters, a learned router routes each token to only 2 out of 8 feed-forward experts per layer. Consequently, only **12.9 billion parameters are active per token**, delivering the inference latency and compute cost of a 13B model while matching the performance of a 70B dense model.

- **Pixtral**: A 12B multimodal model supporting native image and video understanding alongside text.

Beyond Meta and Mistral, other prominent open foundation families made decisive contributions:

- **EleutherAI (GPT-Neo, GPT-J, GPT-NeoX-20B)**: Early non-profit open-source pioneers that reproduced GPT-3-scale architectures, creating the Pile dataset (825 GB) and establishing open benchmarks to accelerate transparent AI alignment research.

- **Falcon Series (Technology Innovation Institute, UAE) [75]**: Released 7B, 40B, and 180B parameter models trained on **RefinedWeb**, demonstrating that meticulously filtered web text alone could produce state-of-the-art reasoning without proprietary curation. Crucially, Falcon addressed the acute tokenization penalty suffered by non-English languages (specifically Arabic). In standard English-centric BPE tokenizers, Arabic morphological structures fragment into excessive byte sequences (often 4 to 8 tokens per word), drastically compressing the effective context window and inflating generation costs. Falcon engineered a custom balanced tokenizer to overcome this bottleneck. Furthermore, the institute pioneered hybrid architectures combining **State Space Models (Mamba)** with attention layers, exploring sub-quadratic alternatives to standard Transformers.
- **GLM (General Language Model, Zhipu AI & Tsinghua University)**: An autoregressive blank infilling architecture trained on massive bilingual corpora, achieving top-tier proficiency across both English and Chinese.

The historical evolution from the original 2017 Transformer thus demonstrates a definitive convergence: while encoder-only and encoder-decoder models established specialized benchmarks, the autoregressive decoder-only architecture—fortified by compute-optimal pre-training and inference-optimal overtraining—has emerged as the universal foundation for modern artificial intelligence.

CHAPTER 8

# Evaluation and Benchmarks

Chapter 7 described how model size, training data, and compute affect next-token cross-entropy. This objective measures token prediction under teacher forcing, where the model conditions on the reference prefix. It does not directly measure whether generated text is coherent, factually accurate, appropriate to its context, or useful for a reasoning task. Evaluation therefore requires measures that reflect the intended use of the model.

Natural-language generation is difficult to evaluate because a meaning can be expressed by many valid strings. Classification and regression often use a bounded output space with a defined target, whereas a language or code prompt may admit many correct responses. A candidate can share most of its words with a reference while reversing its meaning by adding a negation. Human review can assess qualities that automatic measures miss, but it is costly and may vary across reviewers, so it does not by itself provide a scalable basis for comparing models or checkpoints.

Evaluation methods address different properties. Intrinsic measures such as cross-entropy and perplexity assess likelihood assignments on held-out text. Reference-based metrics such as BLEU and ROUGE compare generated candidates with reference strings using $n$-gram overlap and length penalties. Contextual measures such as BERTScore, model-based evaluators, and task benchmarks such as GLUE, MMLU, and HumanEval provide evidence about semantic similarity or performance on specified tasks.

This chapter formalizes the principles of language model evaluation. We begin by examining the core mathematical properties required for rigorous evaluation metrics, focusing on the distinction between additive and multiplicative averaging, before conducting an exhaustive investigation of intrinsic uncertainty estimation via perplexity and surface-string reference matching via BLEU.

## 8.1 Principles of Evaluation

Evaluation metrics in natural language processing can be partitioned into two distinct paradigms: intrinsic likelihood metrics and extrinsic generation metrics. Intrinsic metrics evaluate the model's mathematical representation of language by computing the probability distribution it assigns to unseen test corpora under teacher forcing. They require no sample generation, no decoding algorithms, and no reference texts; they assess the predictive confidence of the probability model itself. Extrinsic metrics, by contrast, operate on the discrete sequences emitted during autoregressive inference, comparing the generated hypothesis against one or more human gold-standard references.

### The geometric mean and multiplicative error penalization

Before formalizing specific metrics, we must establish the mathematical foundation of sequence-level score aggregation. When evaluating models across multiple tokens, sentences, or $n$-gram orders, individual scores must be aggregated into a single scalar summary. While the standard arithmetic mean is familiar, it possesses structural properties that render it unsuitable for probability spaces and multi-criteria quality evaluation.

Let $\mathbf{x} = (x_1, x_2, \ldots, x_n) \in \mathbb{R}^n_{>0}$ denote a collection of $n$ positive evaluation values (such as token probabilities or $n$-gram precision scores). The arithmetic mean (AM) and geometric mean (GM) are defined as:

$$\mathrm{AM}(\mathbf{x}) = \frac{1}{n}\sum_{i=1}^{n} x_i, \tag{8.1}$$

$$\mathrm{GM}(\mathbf{x}) = \left(\prod_{i=1}^{n} x_i\right)^{\frac{1}{n}}. \tag{8.2}$$

The critical distinction between these two operators lies in how they penalize low values. The arithmetic mean exerts an *additive effect*: a severely low score in one component can be easily masked by high scores in other components. For example, consider three evaluation scores $\mathbf{x} = (0.1, 0.9, 0.9)$. The arithmetic mean yields:

$$\mathrm{AM}(0.1, 0.9, 0.9) = \frac{0.1 + 0.9 + 0.9}{3} = \frac{1.9}{3} \approx 0.6333, \tag{8.3}$$

producing a deceptively favorable aggregate score despite substantial failure on the first criterion. In contrast, the geometric mean exerts a *multiplicative effect*:

$$\mathrm{GM}(0.1, 0.9, 0.9) = (0.1 \times 0.9 \times 0.9)^{\frac{1}{3}} = (0.081)^{\frac{1}{3}} \approx 0.4327. \tag{8.4}$$

Because the terms multiply, any degradation in an individual component disproportionately suppresses the entire product. Crucially, if any single term reaches zero ($x_i = 0$), the geometric mean collapses to zero:

$$\exists i \text{ such that } x_i = 0 \implies \mathrm{GM}(\mathbf{x}) = 0. \tag{8.5}$$

This property is indispensable in evaluation: a model must achieve reasonable competence across all evaluated dimensions to attain a high overall score. This behavior directly parallels the harmonic mean used in the $F_1$-score to balance precision and recall.

### Numerical stability via logarithmic formulation

In practical computation, evaluating long sequences or high-order $n$-grams involves multiplying dozens or hundreds of terms bounded in $(0, 1]$. In finite-precision floating-point arithmetic (IEEE

754 float32 or float64), direct repeated multiplication inevitably causes arithmetic underflow, rounding the running product to absolute zero.

To guarantee numerical stability, the geometric mean is computed in the logarithmic domain. Utilizing the monotonic identity $\ln(\prod x_i) = \sum \ln x_i$, the $n$-th root is transformed into an arithmetic average of logarithms:

$$\mathrm{GM}(\mathbf{x}) = \exp\left(\ln\left(\prod_{i=1}^{n} x_i\right)^{\frac{1}{n}}\right) = \exp\left(\frac{1}{n}\sum_{i=1}^{n} \ln x_i\right). \tag{8.6}$$

As emphasized in lecture, the base of the logarithm must strictly match the base of the exponentiation operator. Standard scientific implementations employ the natural logarithm ln (base $e$) and natural exponential $\exp(\cdot)$ so that $\exp(\ln(z)) = z$ is an exact inverse mapping. If base-2 logarithms ($\log_2$) are adopted, the outer transformation must evaluate as $2^{\frac{1}{n}\sum \log_2 x_i}$.

### 8.1.1 Perplexity

The primary intrinsic metric for evaluating autoregressive language models is **perplexity** (PPL). Rooted in Claude Shannon's mathematical theory of communication and information entropy [27], perplexity quantifies the degree of uncertainty a probabilistic model exhibits when predicting a sequence of tokens.

#### Conceptual intuition and the effective branching factor

Rather than assessing whether a model generates fluent answers, perplexity measures how surprised the model is when presented with human-written, grammatically correct text. If a model has acquired an accurate statistical representation of English, it will assign high probability to legitimate sentences from held-out corpora. Conversely, if the model is poorly trained or faces out-of-domain text, it will assign low probability to valid tokens, indicating substantial confusion.

Perplexity provides an intuitive geometric interpretation as the **effective branching factor** of the language model. Suppose that at a specific step in a sentence, the model assigns probability $P(w_i \mid w_{<i})$ to the ground-truth token. The reciprocal:

$$K_i = \frac{1}{P(w_i \mid w_{<i})} \tag{8.7}$$

represents the number of equally probable alternative words among which the model was undecided.

Consider a thought experiment where the model's vocabulary contains 50,000 discrete tokens:

- If the model assigns probability $P = 0.25$ to the correct token, $\frac{1}{0.25} = 4$. Even though the vocabulary contains 50,000 candidates, the model's uncertainty is equivalent to rolling a fair 4-sided die of equally likely tokens.
- If the model assigns probability $P = 0.90$, $\frac{1}{0.90} \approx 1.11$. The model is near-certain of the true token, undecided between approximately 1.1 alternatives.
- If the model assigns probability $P = 1.0$, $\frac{1}{1.0} = 1.0$. The model has zero uncertainty.
- If the model assigns probability $P = 0.01$, $\frac{1}{0.01} = 100$. The model is as confused as choosing uniformly among 100 distinct words.

Across an entire sequence of $N$ words, perplexity represents the geometric average of these individual branching factors. A perplexity of 50 indicates that, on average across each position in the sentence, the model was as uncertain as choosing uniformly among 50 equally probable words. Lower perplexity reflects greater predictive confidence and superior alignment with the underlying data distribution.

**Formal mathematical derivation and negative log-likelihood**

Let $W = (w_1, w_2, \ldots, w_N)$ denote a sequence of $N$ tokens. By the probability chain rule, the joint probability assigned to sequence $W$ by an autoregressive language model parameterized by $\theta$ is:

$$P(W) = \prod_{i=1}^{N} P(w_i \mid w_1, \ldots, w_{i-1}; \theta) = \prod_{i=1}^{N} P(w_i \mid w_{<i}). \tag{8.8}$$

The perplexity of the model over the sequence $W$ is formally defined as the reciprocal of the geometric mean of the token conditional probabilities:

$$\mathrm{PPL}(W) = P(W)^{-\frac{1}{N}} = \left( \prod_{i=1}^{N} P(w_i \mid w_{<i}) \right)^{-\frac{1}{N}} = \left( \prod_{i=1}^{N} \frac{1}{P(w_i \mid w_{<i})} \right)^{\frac{1}{N}}. \tag{8.9}$$

Applying the logarithmic transformation in Equation (8.6), we rewrite perplexity in terms of the sum of token negative log-likelihoods (NLL):

$$\ln \mathrm{PPL}(W) = -\frac{1}{N} \ln P(W) = -\frac{1}{N} \sum_{i=1}^{N} \ln P(w_i \mid w_{<i}), \tag{8.10}$$

$$\mathrm{PPL}(W) = \exp\left( -\frac{1}{N} \sum_{i=1}^{N} \ln P(w_i \mid w_{<i}) \right) = \exp\left( \frac{1}{N} \sum_{i=1}^{N} \mathrm{NLL}_i \right), \tag{8.11}$$

where $\mathrm{NLL}_i = -\ln P(w_i \mid w_{<i}) \geq 0$ is the negative log-likelihood of token $w_i$.

Equation (8.11) reveals the direct mathematical link between perplexity and the pre-training cross-entropy loss $\mathcal{L}_{\mathrm{CE}}$:

$$\mathcal{L}_{\mathrm{CE}} = \frac{1}{N} \sum_{i=1}^{N} \mathrm{NLL}_i \implies \mathrm{PPL}(W) = e^{\mathcal{L}_{\mathrm{CE}}}. \tag{8.12}$$

Perplexity is simply the exponentiated cross-entropy loss. Figure 8.1 illustrates the exponential growth of perplexity as average cross-entropy loss increases, underscoring the disproportionately severe penalty imposed on uncertain predictions.

*Mathematical justification for the geometric mean.* The selection of the geometric mean in Equation (8.9) is mathematically mandatory for three reasons:

1. **Consistency with probability factorization**: In probability theory, the likelihood of a sequence is formed by the product of conditional probabilities, not their sum. Normalizing a product of $N$ terms by sequence length requires taking the $N$-th root, so that $\mathrm{PPL}(W)^N = \frac{1}{P(W)}$. Taking an arithmetic mean of probabilities ($\frac{1}{N} \sum P_i$) violates the product rule and lacks theoretical interpretation.

2. **Information-theoretic duality**: Taking the logarithm of the geometric mean yields the arithmetic mean of negative log-likelihoods (cross-entropy), establishing an exact duality with Shannon entropy. An arithmetic mean of probabilities cannot be converted into cross-entropy.

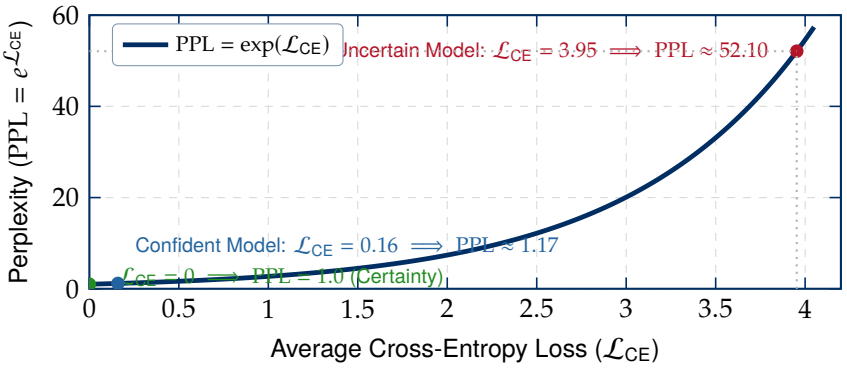



Figure 8.1: Perplexity (PPL) as an exponential function of the average cross-entropy loss $\mathcal{L}_{\text{CE}}$. The metric attains its theoretical minimum at PPL = 1.0 when $\mathcal{L}_{\text{CE}} = 0$. Due to the exponential mapping PPL = $e^{\mathcal{L}_{\text{CE}}}$, modest increments in cross-entropy loss trigger dramatic escalations in perplexity, effectively penalizing poor probability assignments.

3. **Zero-frequency penalty**: If a model assigns zero probability to a single valid token ($P(w_k \mid w_{<k}) = 0$), the geometric mean product collapses to 0, which inverts to infinite perplexity: $\text{PPL}(W) \to \infty$. This correctly identifies the model as broken, since it declared a real-world sentence impossible. An arithmetic mean would merely incur a negligible $\frac{1}{N}$ reduction.

The theoretical lower bound of perplexity is PPL = 1.0. This minimum occurs if and only if the model assigns probability $P(w_i \mid w_{<i}) = 1.0$ to every token in the sequence ($\text{NLL}_i = 0 \implies e^0 = 1$), representing absolute, deterministic certainty. In practice, PPL = 1.0 is observed only when a model severely overfits a training sequence.

**Worked examples: confident versus uncertain models**

To illustrate the mechanics of perplexity, we walk through the exact numerical evaluations presented in the lecture. Consider a held-out test sentence delimited by boundary markers:

$$W = \langle\text{bos}\rangle \text{ The dog chased the cat } \langle\text{eos}\rangle.$$

The beginning-of-sequence token ⟨bos⟩ serves as initial conditioning prefix ($P = 1.0$). The sequence contains $N = 6$ evaluated transitions ($w_1 = \text{The}, \ldots, w_6 = \langle\text{eos}\rangle$).

*Case 1: Confident, well-trained model.* Suppose a model assigns high probability to each correct token, reflecting strong linguistic familiarity. The conditional probabilities, negative log-likelihoods, and equivalent branching numbers are listed in Table 8.1.

Table 8.1: Step-by-step perplexity computation for a confident model on $W$ = ⟨bos⟩ The dog chased the cat ⟨eos⟩.

| $i$ | Token $w_i$ | Probability $P(w_i \mid w_{<i})$ | $\text{NLL}_i = -\ln P(w_i \mid w_{<i})$ | Branching $K_i = \frac{1}{P_i}$ |
|---|---|---|---|---|
| 1 | The | 0.90 | $-\ln(0.90) \approx 0.1054$ | 1.1111 |
| 2 | dog | 0.85 | $-\ln(0.85) \approx 0.1625$ | 1.1765 |
| 3 | chased | 0.88 | $-\ln(0.88) \approx 0.1278$ | 1.1364 |
| 4 | the | 0.75 | $-\ln(0.75) \approx 0.2877$ | 1.3333 |
| 5 | cat | 0.95 | $-\ln(0.95) \approx 0.0513$ | 1.0526 |
| 6 | ⟨eos⟩ | 0.80 | $-\ln(0.80) \approx 0.2231$ | 1.2500 |
| **Sum / Mean** | | — | $\sum \text{NLL}_i = 0.9578 \implies \overline{\text{NLL}} = 0.1596$ | $\prod K_i \approx 2.6074$ |

Evaluating the perplexity via the average negative log-likelihood:

$$\begin{aligned}\mathrm{PPL}(W) &= \exp\left(\frac{1}{6}\sum_{i=1}^{6}\mathrm{NLL}_i\right)\\ &= \exp\left(\frac{0.1054+0.1625+0.1278+0.2877+0.0513+0.2231}{6}\right)\\ &= e^{0.15963} \approx \mathbf{1.1731}.\end{aligned} \tag{8.13}$$

Equivalently, taking the 6-th root of the product of equivalent words:

$$\mathrm{PPL}(W) = (1.1111 \times 1.1765 \times 1.1364 \times 1.3333 \times 1.0526 \times 1.2500)^{\frac{1}{6}} \approx (2.6074)^{\frac{1}{6}} \approx \mathbf{1.1731}. \tag{8.14}$$

The model's perplexity is 1.1731, indicating that on average, the model was undecided over approximately 1.17 equivalent words per position. It possessed near-complete certainty throughout the sequence.

*Case 2: Uncertain, poorly trained model.* Now suppose an under-trained or out-of-domain model evaluates the identical sentence, assigning substantially lower probabilities to the correct tokens, as detailed in Table 8.2.

Table 8.2: Step-by-step perplexity computation for an uncertain model on $W$ = ⟨bos⟩ The dog chased the cat ⟨eos⟩.

| $i$ | Token $w_i$ | Probability $P(w_i \mid w_{<i})$ | $\mathrm{NLL}_i = -\ln P(w_i \mid w_{<i})$ | Branching $K_i = \frac{1}{P_i}$ |
|---|---|---|---|---|
| 1 | The | 0.050 | $-\ln(0.050) \approx 2.9957$ | 20 |
| 2 | dog | 0.100 | $-\ln(0.100) \approx 2.3026$ | 10 |
| 3 | chased | 0.200 | $-\ln(0.200) \approx 1.6094$ | 5 |
| 4 | the | 0.010 | $-\ln(0.010) \approx 4.6052$ | 100 |
| 5 | cat | 0.005 | $-\ln(0.005) \approx 5.2983$ | 200 |
| 6 | ⟨eos⟩ | 0.001 | $-\ln(0.001) \approx 6.9078$ | 1000 |
| **Sum / Product** | | — | $\sum \mathrm{NLL}_i = 23.7190 \implies \overline{\mathrm{NLL}} = 3.9532$ | $\prod K_i = 2 \times 10^{10}$ |

Computing the geometric mean directly from the branching factors:

$$\prod_{i=1}^{6} K_i = 20 \times 10 \times 5 \times 100 \times 200 \times 1000 = 2 \times 10^{10}, \tag{8.15}$$

$$\mathrm{PPL}(W) = \left(2 \times 10^{10}\right)^{\frac{1}{6}} \approx \mathbf{52.10}. \tag{8.16}$$

Verifying via the average negative log-likelihood:

$$\mathrm{PPL}(W) = \exp\left(\frac{23.7190}{6}\right) = \exp(3.95317) \approx \mathbf{52.10}. \tag{8.17}$$

Both formulations yield identical results: PPL = 52.10. On average, the uncertain model was confused across approximately 52 equally likely candidate tokens at every position.

As highlighted by the professor, perplexity serves as a diagnostic probe into the temporal and thematic distribution of a model's pre-training data. If an older model trained on corpora prior to 2016 is evaluated on the prompt "The President of the United States is [NAME]", evaluating "Barack Obama" yields low perplexity, whereas evaluating "Donald Trump" or "Joe Biden" produces high perplexity. The elevated perplexity does not imply that the model is architecturally defective; rather, it indicates statistical surprise relative to the pre-training data distribution.

#### Advantages and fundamental limitations of perplexity

Perplexity possesses distinct methodological advantages that have cemented its status as standard reporting metric in pre-training literature:

- **Unsupervised and task-agnostic**: Computing perplexity requires only raw, unannotated text. It eliminates the need for expensive human reference translations, task labels, or subjective grading rubrics.
- **Zero inference compute overhead**: Because evaluation occurs under teacher forcing, perplexity is computed in a single forward pass over batches of tokens simultaneously with cross-entropy training loss, avoiding autoregressive generation.
- **Continuous and sensitive**: Perplexity is a smooth, continuous scalar that detects subtle improvements in language modeling capability long before downstream task accuracies show measurable gains.

Despite these strengths, perplexity suffers from fundamental limitations that prevent it from serving as a universal measure of model quality:

- **Vocabulary and tokenizer incomparability**: Perplexity is strictly incomparable across models that use different vocabularies or tokenization algorithms. Consider a byte-level tokenizer ($|\mathcal{V}| = 256$) versus a subword tokenizer ($|\mathcal{V}| = 100,000$). Because a byte-level model predicts tiny, predictable character fragments, its per-token perplexity will appear dramatically lower than the subword model, even though the subword model generates superior semantic text. Valid cross-model comparisons require normalizing perplexity by the number of words rather than subword tokens:

$$\text{PPL}_{\text{word}} = \exp\left(\frac{N_{\text{tokens}}}{N_{\text{words}}}\mathcal{L}_{\text{CE}}\right). \tag{8.18}$$

- **Disconnection from reasoning and factual truth**: A model can achieve outstanding (low) perplexity by mastering local grammatical syntax, common idioms, and surface transitions while remaining completely incapable of logical deduction, arithmetic calculation, or factual consistency. Perplexity rewards familiarity, not correctness.

### 8.1.2 Bilingual Evaluation Understudy (BLEU)

While perplexity evaluates the model's internal probability distribution over given text, practical deployment requires evaluating the actual text generated by the model during autonomous inference. In 2002, Papineni, Roukos, Ward, and Zhu [76] introduced the **Bilingual Evaluation Understudy** (BLEU) metric at IBM Research. Originally developed for machine translation, BLEU became the foundational reference-based evaluation metric across natural language processing.

#### Conceptual intuition and reference-based lexical matching

The fundamental premise of BLEU is that the closer a machine-generated candidate translation resembles a human gold-standard translation, the higher its quality. Evaluating open-ended text against human references requires navigating two competing linguistic dimensions:

1. **Adequacy (Word Choice)**: Did the model translate the correct conceptual content? This is captured by measuring lexical overlap at the unigram level (1-grams).

2. **Fluency (Word Order)**: Did the model assemble the words in a natural, syntactically coherent sequence? This is captured by measuring contiguous overlap across higher-order $n$-grams (2-grams, 3-grams, 4-grams).

BLEU is an extrinsic metric: it requires running the model autoregressively to generate a hypothesis string $G$ and comparing $G$ against one or more reference strings $R = \{R_1, \ldots, R_m\}$. It is defined as a modified $n$-gram precision metric multiplied by an exponential brevity penalty.

### Mathematical formulation and brevity penalty

To compute BLEU, we first extract all contiguous $n$-grams of size $i \in \{1, 2, \ldots, n\}$ from both the generated candidate sequence $G$ and the reference sequence $R$.

*Modified clipped n-gram precision.* A naive precision metric would simply count the fraction of candidate $n$-grams that appear in the reference:

$$p_{\text{naive}} = \frac{\sum_{g \in G} \mathbb{I}(g \in R)}{|G|}.$$

However, naive precision can be exploited through pathological repetition. For example, if the reference is "The cat sat on the mat" and a defective model outputs the candidate "the the the the the the", every single generated unigram matches the reference word "the". Naive precision would reward this degenerative output with a perfect score: $p_{\text{naive}} = \frac{6}{6} = 1.0$.

To eliminate this vulnerability, Papineni et al. [76] introduced **clipped precision** ($p_i$). For each $i$-gram in the candidate sequence, the count is clipped so that it cannot exceed the maximum number of times that identical $i$-gram appears in any single reference sentence:

$$\text{Count}_{\text{clip}}(g_i) = \min\left(\text{Count}_G(g_i), \max_{R_j \in R} \text{Count}_{R_j}(g_i)\right). \tag{8.19}$$

The modified $i$-gram precision $p_i$ is then computed as the sum of clipped counts divided by the total number of candidate $i$-grams generated:

$$p_i = \frac{\sum_{g_i \in G} \text{Count}_{\text{clip}}(g_i)}{\sum_{g_i \in G} \text{Count}_G(g_i)}. \tag{8.20}$$

In the pathological repetition example, the reference contains the unigram "the" twice ($\max \text{Count}_R(\text{"the"}) = 2$). The clipped count for the candidate is $\min(6, 2) = 2$, reducing the unigram precision to $\frac{2}{6} = 0.3333$.

*Multi-order precision and the brevity penalty.* While clipped precision prevents repetitive over-generation, precision alone introduces an opposite failure mode: **under-generation**.

Suppose the reference sentence contains 20 words, and the candidate model generates only two words: "The dog". Both words appear in the reference, so $p_1 = \frac{2}{2} = 1.0$. The bigram "The dog" also appears in the reference, so $p_2 = \frac{1}{1} = 1.0$. Without length regulation, the truncated output would achieve a flawless score: BLEU = 1.0, despite discarding 90% of the intended content.

Standard classification metrics resolve under-generation by computing recall. However, in machine translation and open-ended generation, defining recall is complicated because human translations exhibit natural length variance, and multiple reference translations often use different phrasing. Instead of explicit recall, BLEU introduces an exponential multiplicative **Brevity Penalty** (BP).

Let $g$ denote the length of the generated candidate sequence, and let $r$ denote the effective length of the reference sequence. The brevity penalty is defined as:

$$\text{BP} = \begin{cases} 1.0 & \text{if } g > r, \\ \exp\left(1 - \frac{r}{g}\right) & \text{if } g \leq r. \end{cases} \tag{8.21}$$

Figure 8.2 illustrates the mathematical behavior of the brevity penalty as a function of the length ratio $\frac{g}{r}$.

- When $g \geq r$, the candidate length equals or exceeds the reference length. In this regime, BP = 1.0: no brevity penalty is applied. Overly long generations are naturally penalized by the denominator of the precision terms in Equation (8.20).
- When $g < r$, the exponent $1 - \frac{r}{g}$ is strictly negative, causing BP to decay exponentially below 1.0. As the candidate length approaches zero ($g \to 0$), the ratio $\frac{r}{g} \to \infty$, and BP $\to 0$, aggressively driving the overall score to zero.

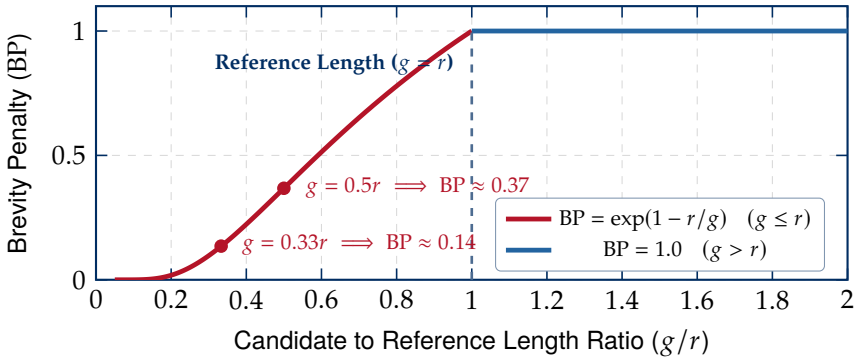



Figure 8.2: The BLEU Brevity Penalty (BP) as a function of the generated-to-reference length ratio $g/r$. For $g \geq r$, BP = 1.0 because over-generation is already penalized by precision denominators. For $g < r$, BP = $\exp(1 - r/g)$ decays exponentially toward zero, severely penalizing short or truncated generations to prevent precision gaming.

### Formal mathematical derivation of the BLEU metric

With the individual components defined, we now formally derive the composite BLEU formula from first principles, following the exact mathematical trajectory used for perplexity.

Let $\{p_1, p_2, \ldots, p_n\}$ denote the modified clipped precisions up to order $n$, with corresponding non-negative weights $w_i \geq 0$ satisfying $\sum_{i=1}^{n} w_i = 1$. The collective precision across all $n$-gram orders is combined using a weighted geometric mean:

$$\mathrm{GM}(p_1, \ldots, p_n) = \prod_{i=1}^{n} p_i^{w_i}. \tag{8.22}$$

Under standard uniform weighting across the $n$ orders, $w_i = \frac{1}{n}$, simplifying the product to:

$$\mathrm{GM}(p_1, \ldots, p_n) = \left( \prod_{i=1}^{n} p_i \right)^{\frac{1}{n}}. \tag{8.23}$$

In computational implementations, multiplying small probabilities or fractional precisions directly induces numerical underflow. To guarantee numerical stability, we map the geometric

mean into the log-domain:

$$\begin{aligned}\ln \mathrm{GM}(p_1, \ldots, p_n) &= \ln\left(\prod_{i=1}^{n} p_i^{w_i}\right) \\ &= \sum_{i=1}^{n} \ln\left(p_i^{w_i}\right) \\ &= \sum_{i=1}^{n} w_i \ln p_i. \end{aligned} \tag{8.24}$$

Exponentiating both sides restores the original geometric scale:

$$\mathrm{GM}(p_1, \ldots, p_n) = \exp\left(\sum_{i=1}^{n} w_i \ln p_i\right). \tag{8.25}$$

Notice the mathematical parallel with perplexity: just as perplexity maps the arithmetic mean of negative log-likelihoods back to probability space via $\exp(-\frac{1}{N}\sum \ln P_i)$, BLEU maps the weighted arithmetic mean of log-precisions back to precision space via $\exp(\sum w_i \ln p_i)$.

Finally, coupling this precision term multiplicatively with the brevity penalty BP yields the complete, definitive formulation of BLEU-$n$:

$$\text{BLEU-}n = \mathrm{BP} \cdot \mathrm{GM}(p_1, \ldots, p_n) = \mathrm{BP} \cdot \exp\left(\sum_{i=1}^{n} w_i \ln p_i\right) = \mathrm{BP} \cdot \left(\prod_{i=1}^{n} p_i\right)^{\frac{1}{n}}. \tag{8.26}$$

**Worked example: BLEU-2 calculation**

We now execute the step-by-step computation of BLEU-2 ($n = 2$, uniform weights $w_1 = w_2 = 0.5$) on the worked example from the lecture slides.

Let the reference and machine-generated candidate sequences be:

$$\begin{aligned} R &: \text{“The dog chased the cat”} \quad (r = 5 \text{ tokens}), \\ G &: \text{“The dog ran after the cat”} \quad (g = 6 \text{ tokens}). \end{aligned}$$

*Step 1: Unigram precision ($i = 1$).* Extract all unigrams from both sentences:

$$\begin{aligned} \text{Reference unigrams } R_1 &= \{\text{“The”, “dog”, “chased”, “the”, “cat”}\} \quad (|R_1| = 5), \\ \text{Generated unigrams } G_1 &= \{\text{“The”, “dog”, “ran”, “after”, “the”, “cat”}\} \quad (|G_1| = 6). \end{aligned}$$

We count matching unigrams between $G$ and $R$:

- “The”: appears in $R$ ($\mathrm{Count}_{\mathrm{clip}} = 1$)
- “dog”: appears in $R$ ($\mathrm{Count}_{\mathrm{clip}} = 1$)
- “ran”: absent in $R$ ($\mathrm{Count}_{\mathrm{clip}} = 0$)
- “after”: absent in $R$ ($\mathrm{Count}_{\mathrm{clip}} = 0$)
- “the”: appears in $R$ ($\mathrm{Count}_{\mathrm{clip}} = 1$)
- “cat”: appears in $R$ ($\mathrm{Count}_{\mathrm{clip}} = 1$)

Total matching unigrams = 4. The total generated unigrams = 6. The modified unigram precision is:

$$p_1 = \frac{4}{6} \approx \mathbf{0.6667}. \tag{8.27}$$

*Step 2: Bigram precision ($i = 2$).* Extract all contiguous pairs of adjacent tokens (bigrams):

Reference bigrams $R_2$ = {"The dog", "dog chased", "chased the", "the cat"} ($|R_2| = 4$),
Generated bigrams $G_2$ = {"The dog", "dog ran", "ran after", "after the", "the cat"} ($|G_2| = 5$).

We count matching bigrams:

- "The dog": matches $R_2$ ($\text{Count}_{\text{clip}} = 1$)
- "dog ran": absent in $R_2$ ($\text{Count}_{\text{clip}} = 0$)
- "ran after": absent in $R_2$ ($\text{Count}_{\text{clip}} = 0$)
- "after the": absent in $R_2$ ($\text{Count}_{\text{clip}} = 0$)
- "the cat": matches $R_2$ ($\text{Count}_{\text{clip}} = 1$)

Total matching bigrams = 2. The total generated bigrams = 5. The modified bigram precision is:

$$p_2 = \frac{2}{5} = \mathbf{0.4000}. \tag{8.28}$$

*Step 3: Geometric mean of precisions.* Combining $p_1$ and $p_2$ via the geometric mean:

$$\begin{aligned} \text{GM} &= \exp\left(\frac{1}{2}\left(\ln(0.6667) + \ln(0.4000)\right)\right) = \sqrt{p_1 \times p_2} \\ &= \sqrt{\frac{4}{6} \times \frac{2}{5}} = \sqrt{\frac{8}{30}} = \sqrt{\frac{4}{15}} \approx \mathbf{0.5165}. \end{aligned} \tag{8.29}$$

*Step 4: Brevity penalty calculation.* We compare the candidate length $g = 6$ against the reference length $r = 5$:

$$g = 6 > r = 5 \implies \text{BP} = 1.0. \tag{8.30}$$

Because the candidate sequence is not shorter than the reference, no brevity penalty is incurred (BP = 1.0).

*Step 5: Final BLEU-2 score.* Multiplying the brevity penalty by the precision geometric mean yields:

$$\text{BLEU-2} = \text{BP} \times \text{GM} = 1.0 \times 0.5165 = \mathbf{0.5165} \quad \text{(or 51.65\%)}. \tag{8.31}$$

This calculation exposes the defining characteristics and underlying limitations of $n$-gram lexical matching. The candidate phrase "ran after" is semantically equivalent to the reference verb "chased". A human evaluator would judge "The dog ran after the cat" as a completely correct, fluent, and faithful translation. However, because BLEU enforces exact surface string equality, the bigrams "dog ran", "ran after", and "after the" were scored as total errors (0 matches). Consequently, $p_2$ dropped from an expected near-perfect score to 0.4000, driving the overall BLEU-2 score down to 0.5165.

**Advantages and foundational limitations of BLEU**

BLEU has served as the universal benchmark in machine translation and conditional generation for over two decades due to several compelling advantages:

- **Automated and cost-effective**: BLEU replaces slow, expensive human judges with an instantaneous, deterministic algorithmic script that costs nothing to evaluate.
- **Language-independent**: BLEU operates strictly on string matching over whitespace-separated tokens. It requires no morphological analyzers, syntactic dependency parsers, or semantic ontologies.
- **Empirical human correlation**: Across large corpora containing thousands of sentences, aggregate BLEU scores correlate reasonably well with human assessments of translation quality, effectively discriminating between high-performing and defective models.
- **Enforcement of phrasing structure**: By incorporating higher-order $n$-grams ($n \geq 2$), BLEU aggressively penalizes "word salad" outputs that contain relevant keywords in scrambled or ungrammatical orders.

Despite these operational strengths, BLEU exhibits a single, overarching foundational limitation: **it relies exclusively on exact surface word matching**. Every recorded failure mode and empirical deficiency of BLEU is simply a direct branch of this core constraint:

- **Synonymy and paraphrasing blindness**: Because BLEU matches character sequences rather than semantics, semantically identical phrases receive zero credit. As demonstrated in our worked example, replacing the reference verb "chased" with "ran after" destroys bigram precision despite preserving perfect semantic fidelity. Similarly, active-to-passive voice transformations and morphological inflections are penalized as complete errors.
- **Negation and factual insensitivity**: Surface character counting does not evaluate truth value. Inserting a single negation operator (e.g., generating "The system is not secure" when the reference is "The system is secure") preserves nearly all unigrams and bigrams, earning a stellar BLEU score while stating the exact opposite of the truth.
- **Brittle reference dependency**: The metric rewards adherence to the specific lexical choices of the human reference annotator. If the reference author employed uncommon vocabulary or idiosyncratic phrasing, a model generating natural, fluent, and correct prose is penalized simply because its surface tokens diverge from the human author.
- **Divergence in open-ended generation**: In tasks where generation is divergent and creative (such as dialogue, essay writing, or storytelling), there exists no unique reference sentence. Demanding exact $n$-gram alignment against a single reference penalizes valid linguistic creativity.

### 8.1.3 Recall-oriented understudy for gisting evaluation (ROUGE)

While BLEU was conceived for machine translation—where generating spurious words incurs severe penalties and precision is paramount—other natural language generation tasks require a fundamentally different optimization target. In text summarization, the primary failure mode is not verbosity, but *omission*: a candidate summary that fails to convey critical facts, characters, or conclusions is defective, even if every word it does produce matches the reference with 100% precision.

To address this challenge, Lin [77] introduced **ROUGE** (*Recall-Oriented Understudy for Gisting Evaluation*). In contrast to BLEU's precision-centric design, ROUGE evaluates generative performance primarily through **recall**, measuring the fraction of reference $n$-grams and structural subsequences successfully captured in the machine-generated hypothesis.

### ROUGE-$N$: $N$-gram recall

Let $R = \{r_1, r_2, \ldots, r_m\}$ denote a set of reference summaries, and let $G$ denote the model-generated candidate summary. For a chosen $n$-gram length $n \geq 1$, ROUGE-$N$ computes the ratio of matched $n$-grams over the total $n$-grams appearing in the reference texts:

$$\text{ROUGE-}N = \frac{\sum_{r \in R} \sum_{g_n \in r} \text{Count}_{\text{match}}(g_n)}{\sum_{r \in R} \sum_{g_n \in r} \text{Count}(g_n)}, \tag{8.32}$$

where $\text{Count}_{\text{match}}(g_n)$ represents the maximum number of times the candidate $n$-gram $g_n$ co-occurs in the candidate text $G$ and the reference $r$.

- **ROUGE-1 ($n = 1$)**: Measures unigram recall, quantifying whether the candidate summary contains the key thematic entities, terminology, and content words present in the human gold standard.
- **ROUGE-2 ($n = 2$)**: Measures bigram recall, testing whether vocabulary is assembled with local syntactic cohesion and pairwise grammatical fluency.

### ROUGE-L: Longest common subsequence

A limitation of fixed $n$-gram matching is that it requires choosing an arbitrary context window $n$. To capture flexible, sentence-level syntactic structure without imposing fixed $n$-gram boundaries, Lin [77] introduced **ROUGE-L**, based on the **Longest Common Subsequence** (LCS).

Given two sequences $X = (x_1, \ldots, x_m)$ and $Y = (y_1, \ldots, y_n)$, a sequence $Z = (z_1, \ldots, z_k)$ is a common subsequence if $Z$ can be derived by deleting zero or more elements from both $X$ and $Y$ without altering the relative ordering of the remaining tokens. Unlike $n$-grams, the tokens in an LCS do not need to be contiguous.

Let $R$ denote a reference sequence of length $r = |R|$, and let $G$ denote a candidate sequence of length $g = |G|$. The LCS-based recall ($R_{\text{LCS}}$), precision ($P_{\text{LCS}}$), and $F$-measure ($F_{\text{LCS}}$) are defined as:

$$R_{\text{LCS}} = \frac{\text{LCS}(G, R)}{r}, \tag{8.33}$$

$$P_{\text{LCS}} = \frac{\text{LCS}(G, R)}{g}, \tag{8.34}$$

$$F_{\text{LCS}} = \frac{(1 + \beta^2) R_{\text{LCS}} P_{\text{LCS}}}{R_{\text{LCS}} + \beta^2 P_{\text{LCS}}}, \tag{8.35}$$

where $\beta$ controls the relative weighting between precision and recall. In summarization benchmarks, $\beta$ is typically set to a large value (such as $\beta = 1.2$ or $\beta \to \infty$) to ensure that recall dominates the evaluation.

*Structural advantage of ROUGE-L.* Because LCS requires tokens to appear in the same relative order, ROUGE-L naturally penalizes scrambled sentences without requiring predefined $n$-gram orders. If a candidate retains the subject-verb-object progression of the reference with intermediate modifiers deleted, LCS assigns a high score, reflecting accurate gist extraction.

### ROUGE-W: Weighted longest common subsequence

A structural blind spot of standard LCS is that it treats all common subsequences equally, regardless of whether their matching tokens are contiguous or widely separated across the text. For example, given the reference $R = (w_1, w_2, w_3, w_4, w_5)$, candidate $G_1 = (w_1, w_2, w_3, y_a, y_b)$ and candidate $G_2 = (w_1, y_a, w_2, y_b, w_3)$ both yield LCS = 3. However, $G_1$ possesses continuous phrasing, whereas $G_2$ is severely fragmented. **ROUGE-W** assigns higher weight to contiguous matching subsequences by applying a polynomial weight function $f(k) = k^2$ to consecutive matches.

**Task-aligned metric selection: Precision vs. recall**

The structural divergence between BLEU and ROUGE underscores a fundamental principle of generative evaluation: metric design must mirror the primary failure mode of the downstream task. In **machine translation**, the cardinal sin is introducing spurious, fabricated, or ungrounded words that distort the original semantic meaning of the source sentence; precision is paramount, and fluent syntax is strictly monitored by higher-order $n$-gram precision, making BLEU the natural operational choice. Conversely, in **text summarization**, the primary failure mode is information omission—generating a grammatically flawless snippet that fails to mention the main event, characters, or conclusions renders the summary useless; recall is paramount, making ROUGE the appropriate instrument. Nevertheless, both metrics remain fundamentally bound to surface lexical matching, remaining equally blind to semantic synonymy, paraphrasing, and negation inversion.

### 8.1.4 Metric for evaluation of translation with explicit ordering (METEOR)

To overcome the rigid surface-matching constraints of BLEU and ROUGE, Banerjee and Lavie [78] developed **METEOR** (*Metric for Evaluation of Translation with Explicit ORdering*). METEOR introduces two fundamental innovations: **multi-stage linguistic matching** (incorporating morphological stemming and semantic synonymy) and an **explicit fragmentation penalty** to measure structural coherence.

**The four-stage linguistic alignment cascade**

METEOR aligns candidate tokens against reference tokens by constructing a generalized bipartite matching where each token matches at most one token in the other sequence. The alignment is resolved through four successive linguistic modules:

1. **Exact match**: Pairs identical surface character strings (e.g., *chased* ↔ *chased*).
2. **Stem match**: Applies the Porter stemmer [78] to match words sharing identical morphological roots (e.g., *chased* ↔ *chasing*, *ran* ↔ *running*).
3. **Synonym match**: Queries WordNet synsets to match semantically equivalent terms (e.g., *chased* ↔ *pursued*, *cat* ↔ *feline*).
4. **Paraphrase match**: Matches multi-word phrase equivalents identified through statistical phrase tables.

**Harmonic score and fragmentation penalty**

Once the alignment is finalized, unigram precision ($P$) and unigram recall ($R$) are computed over the aligned matches. METEOR combines them via a parameterized harmonic mean $F_{\text{mean}}$ that explicitly prioritizes recall:

$$F_{\text{mean}} = \frac{10\,P\,R}{R + 9\,P} = \frac{P\,R}{\alpha\,P + (1-\alpha)\,R}, \quad \text{with } \alpha = 0.9. \tag{8.36}$$

To penalize scrambled syntax, METEOR groups the matched tokens into the minimum number of adjacent contiguous blocks, called **chunks** (ch). If candidate and reference share identical word order, all matched words form a single contiguous chunk (ch = 1). If the words are randomly permuted, each word constitutes an isolated chunk (ch = $m$, where $m$ is the total number of matched unigrams). The **fragmentation penalty** (Pen) is defined as:

$$\text{Pen} = \gamma \left(\frac{\text{ch}}{m}\right)^{\beta}, \quad \text{typically } \gamma = 0.5,\ \beta = 3. \tag{8.37}$$

The final METEOR score is computed as:

$$\text{METEOR} = F_{\text{mean}} \cdot (1 - \text{Pen}). \tag{8.38}$$

When syntax is preserved (ch = 1), the penalty Pen $\approx 0$ and METEOR $\approx F_{\text{mean}}$. When syntax is scrambled, Pen $\to 0.5$, cutting the score in half.

#### Advantages and limitations of METEOR

By incorporating stemming and synonymy, METEOR achieves substantially higher correlation with human judgments than BLEU on sentence-level translation benchmarks. However, it incurs significant trade-offs:

- **Resource dependency**: It requires curated external resources (Porter stemmers, WordNet synsets, paraphrase tables), making it difficult to port to low-resource languages.
- **Computational cost**: Constructing multi-stage graph matchings is computationally slower than computing raw $n$-gram intersections.

### 8.1.5 Contextual embedding similarity (BERTScore)

Despite the additions of stemming and WordNet lookups in METEOR, all string-matching metrics share a common Achilles' heel: **context insensitivity**. Surface strings possess polysemy; a word like "bank" refers to a financial institution or a river shore depending entirely on surrounding syntax. A static lexicon cannot distinguish context-dependent meaning.

In 2020, Zhang, Kishore, Wu, Weinberger, and Artzi [79] introduced **BERTScore**, transitioning evaluation from discrete character matching into continuous contextual vector spaces. Rather than matching strings, BERTScore evaluates generated text by computing pairwise cosine similarities between deep contextual embeddings generated by models such as BERT [52], RoBERTa, or DeBERTa.

#### Mathematical formulation and greedy bipartite matching

Let $R = (x_1, x_2, \ldots, x_k)$ denote a reference sequence of $k$ tokens, and let $G = (y_1, y_2, \ldots, y_m)$ denote a candidate generated sequence of $m$ tokens. Both sequences are processed through a pre-trained contextual transformer, producing sequences of contextual vectors:

$$\mathbf{x}_i = \text{ContextualEncoder}(R)_i \in \mathbb{R}^d, \quad \text{for } i \in \{1, \ldots, k\}, \tag{8.39}$$

$$\mathbf{y}_j = \text{ContextualEncoder}(G)_j \in \mathbb{R}^d, \quad \text{for } j \in \{1, \ldots, m\}. \tag{8.40}$$

Every vector is normalized onto the unit hypersphere: $\|\mathbf{x}_i\|_2 = \|\mathbf{y}_j\|_2 = 1$. The pairwise similarity between reference token $x_i$ and candidate token $y_j$ is evaluated via the inner product (cosine similarity):

$$\mathbf{S}_{i,j} = \mathbf{x}_i^\top \mathbf{y}_j = \cos(\mathbf{x}_i, \mathbf{y}_j) \in [-1, 1]. \tag{8.41}$$

Instead of solving an expensive global assignment problem, BERTScore performs **greedy bipartite matching**: each token in one sequence is matched to the single most similar token in the other sequence.

- **Recall ($R_{\text{BERT}}$)**: For each reference token $x_i$, find the maximum similarity across all generated tokens $y_j$, and average over reference length $k$:

$$R_{\text{BERT}} = \frac{1}{|R|} \sum_{x_i \in R} \max_{y_j \in G} \mathbf{x}_i^\top \mathbf{y}_j. \tag{8.42}$$

- **Precision ($P_{\text{BERT}}$)**: For each generated token $y_j$, find the maximum similarity across all reference tokens $x_i$, and average over candidate length $m$:

$$P_{\text{BERT}} = \frac{1}{|G|} \sum_{y_j \in G} \max_{x_i \in R} \mathbf{x}_i^\top \mathbf{y}_j. \tag{8.43}$$

- **$F_1$-measure ($F_{\text{BERT}}$)**: Synthesizes precision and recall via the harmonic mean:

$$F_{\text{BERT}} = 2 \cdot \frac{P_{\text{BERT}} \cdot R_{\text{BERT}}}{P_{\text{BERT}} + R_{\text{BERT}}}. \tag{8.44}$$

**Worked numerical calculation from lecture**

To illustrate how BERTScore resolves the semantic failure modes of BLEU, we reproduce the exact numerical evaluation delivered by Prof. Giobergia during lecture.

Consider the identical reference and candidate pair analyzed in Section 8.1.2:

$$R : \text{“the dog chased the cat”} \quad (|R| = 5 \text{ tokens}),$$
$$G : \text{“the dog ran after the cat”} \quad (|G| = 6 \text{ tokens}).$$

Under BLEU-2, this pair suffered a substantial drop to 0.5165 because “ran after” did not match “chased” at the surface level. Table 8.3 displays the complete $6 \times 5$ pairwise cosine similarity matrix $\mathbf{S}$ computed from contextual BERT embeddings.

Table 8.3: Pairwise cosine similarity matrix $\mathbf{S}_{j,i} = \mathbf{y}_j^\top \mathbf{x}_i$ between Generated tokens (rows $G$) and Reference tokens (columns $R$) from lecture slides 17–19. Row maxima indicate candidate precision matches; column maxima indicate reference recall matches.

| **Candidate** $G \setminus$ **Ref** $R$ | **the** ($x_1$) | **dog** ($x_2$) | **chased** ($x_3$) | **the** ($x_4$) | **cat** ($x_5$) | **Row Max** ($\max_i \mathbf{S}_{j,i}$) |
|---|---|---|---|---|---|---|
| **the** ($y_1$) | **0.9967** | 0.4120 | 0.3815 | 0.9921 | 0.3950 | **0.9967** (matches $x_1$) |
| **dog** ($y_2$) | 0.4110 | **0.9968** | 0.4520 | 0.4080 | 0.5820 | **0.9968** (matches $x_2$) |
| **ran** ($y_3$) | 0.3250 | 0.4610 | **0.9351** | 0.3180 | 0.4410 | **0.9351** (matches $x_3$) |
| **after** ($y_4$) | 0.3640 | 0.4100 | **0.8868** | 0.3720 | 0.3910 | **0.8868** (matches $x_3$) |
| **the** ($y_5$) | 0.9921 | 0.4080 | 0.3790 | **0.9921** | 0.3980 | **0.9921** (matches $x_4$) |
| **cat** ($y_6$) | 0.3920 | 0.5810 | 0.4430 | 0.3980 | **0.9960** | **0.9960** (matches $x_5$) |
| **Col Max** ($\max_j \mathbf{S}_{j,i}$) | **0.9967** | **0.9968** | **0.9351** | **0.9921** | **0.9960** | — |

*Step 1: Precision calculation ($P_{BERT}$).* We evaluate the maximum similarity across columns for each generated token row:

$$\begin{aligned}
\text{Row 1 (the)} &\rightarrow 0.9967 \\
\text{Row 2 (dog)} &\rightarrow 0.9968 \\
\text{Row 3 (ran)} &\rightarrow 0.9351 \quad \text{(semantically aligned with “chased”)} \\
\text{Row 4 (after)} &\rightarrow 0.8868 \quad \text{(semantically aligned with “chased”)} \\
\text{Row 5 (the)} &\rightarrow 0.9921 \\
\text{Row 6 (cat)} &\rightarrow 0.9960
\end{aligned}$$

$$\sum_{j=1}^{6} \max_i \mathbf{S}_{j,i} = 0.9967 + 0.9968 + 0.9351 + 0.8868 + 0.9921 + 0.9960 = 5.8035. \tag{8.45}$$

Averaging over the $|G| = 6$ candidate tokens yields the precision reported in lecture:

$$P_{\text{BERT}} = \frac{5.8035}{6} \approx \mathbf{0.96725}. \tag{8.46}$$

*Step 2: Recall calculation ($R_{BERT}$).* We evaluate the maximum similarity across rows for each reference token column:

$$\begin{aligned}
\text{Col 1 (the)} &\rightarrow 0.9967 \\
\text{Col 2 (dog)} &\rightarrow 0.9968 \\
\text{Col 3 (chased)} &\rightarrow 0.9351 \quad \text{(best matched by candidate “ran”)} \\
\text{Col 4 (the)} &\rightarrow 0.9921 \\
\text{Col 5 (cat)} &\rightarrow 0.9960
\end{aligned}$$

$$\sum_{i=1}^{5} \max_{j} \mathbf{S}_{j,i} = 0.9967 + 0.9968 + 0.9351 + 0.9921 + 0.9960 = 4.9167. \tag{8.47}$$

Averaging over the $|R| = 5$ reference tokens yields:

$$R_{\text{BERT}} = \frac{4.9167}{5} \approx \mathbf{0.98334}. \tag{8.48}$$

*Step 3: Harmonic $F_1$ score.*

$$F_{\text{BERT}} = 2 \cdot \frac{0.96725 \times 0.98334}{0.96725 + 0.98334} = \frac{1.90220}{1.95059} \approx \mathbf{0.97519}. \tag{8.49}$$

**Strengths and opaque semantic embedding bottlenecks**

Comparing BLEU-2 (51.65%) against BERTScore (97.52%) demonstrates the expressive power of contextual representations:

- **Paraphrase and synonym recognition**: The verbal predicate “ran after” matches “chased” with high similarity (cos $\approx$ 0.9351 and 0.8868). The model is not penalized for natural vocabulary variation.
- **Contextual disambiguation**: Because embeddings are produced by multi-head self-attention layers, each token vector encodes sentence syntax.

However, as emphasized by Prof. Giobergia, BERTScore introduces serious methodological challenges:

1. **Opacity and lack of interpretability (“BERT says so”)**: If BLEU produces a low score, an engineer can inspect the exact missing $n$-grams. If BERTScore produces a score of 0.84, there is no symbolic, explainable justification: the score exists purely because a multi-billion-parameter neural network generated latent vectors with a specific cosine angle.
2. **Computational overhead**: Evaluating BLEU requires simple hash-map lookups taking microseconds. BERTScore requires running two complete forward passes through a transformer encoder per sentence pair, making it computationally heavy for million-sample evaluations.
3. **Inherited model bias**: Any failure mode, linguistic blind spot, or domain shift present in the evaluator model directly corrupts the evaluation metric.

### 8.1.6 Exact match and information retrieval ranking metrics

In tasks where the target output space is strictly bounded, continuous overlap scores are neither necessary nor desirable. When evaluating closed-domain question answering, factual entity retrieval, or programmatic code execution, a prediction is either entirely correct or unusable.

### Exact match (EM)

The **Exact Match** (EM) metric evaluates binary equality between the generated hypothesis string $G$ and the gold-standard reference string $R$:

$$\mathrm{EM}(G, R) = \begin{cases} 1 & \text{if Normalize}(G) = \text{Normalize}(R), \\ 0 & \text{otherwise.} \end{cases} \tag{8.50}$$

To prevent trivial punctuation or casing discrepancies from registering as false errors, the Normalize$(\cdot)$ function standardizes strings by:

- Converting all text to lowercase.
- Stripping whitespace, line breaks, and punctuation marks.
- Removing leading/trailing stop words or English articles (*a*, *an*, *the*).

Exact Match serves as the standard evaluation standard on extractive QA benchmarks (such as SQuAD [80]) and mathematical datasets (such as GSM8K [81]).

### Information retrieval and probability ranking metrics

When a language model is evaluated on multiple-choice questions, cloze passage completion, or entity ranking, it produces a probability distribution over a set of candidates $C = \{c_1, c_2, \ldots, c_K\}$. We sort the candidates by descending model probability:

$$P(c_{(1)}) \geq P(c_{(2)}) \geq \cdots \geq P(c_{(K)}).$$

Let $\mathrm{rank}_i \in \{1, 2, \ldots, K\}$ denote the position at which the ground-truth target token appears. Several canonical ranking metrics quantify performance:

- **Mean Reciprocal Rank (MRR)**: Computes the average of the reciprocal ranks of the correct answer across a query test set $Q$:

$$\mathrm{MRR} = \frac{1}{|Q|} \sum_{q=1}^{|Q|} \frac{1}{\mathrm{rank}_q}. \tag{8.51}$$

  If the target item is consistently ranked first (rank = 1), MRR = 1.0. If it is ranked second, MRR = 0.5.

- **Precision at $k$ (P@$k$) and Recall at $k$ (R@$k$)**: Measures whether the correct answer appears within the top-$k$ highest-probability candidates:

$$\mathrm{Hit@}k = \mathbb{I}(\mathrm{rank} \leq k). \tag{8.52}$$

- **Normalized Discounted Cumulative Gain (**NDCG@$k$**)**: When answers have varying degrees of graded relevance $rel_i \in \mathbb{R}_{\geq 0}$, NDCG@$k$ penalizes relevant items that appear lower in the ranked list using a logarithmic position discount:

$$\mathrm{DCG@}k = \sum_{i=1}^{k} \frac{2^{rel_i} - 1}{\log_2(i+1)}, \quad \mathrm{NDCG@}k = \frac{\mathrm{DCG@}k}{\mathrm{IDCG@}k}, \tag{8.53}$$

  where IDCG@$k$ represents the Ideal DCG obtained under a perfect sorting of relevance values.

### 8.1.7 Task-specific classification metrics

Many modern language model evaluations frame generative tasks as discrete classification problems by prompting the model to output a single categorical label (such as positive/negative sentiment, or entailment/contradiction in inference tasks). In these configurations, classical machine learning evaluation metrics are directly applicable:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}, \tag{8.54}$$

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \tag{8.55}$$

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2\text{TP}}{2\text{TP} + \text{FP} + \text{FN}}, \tag{8.56}$$

where TP, TN, FP, FN denote True Positives, True Negatives, False Positives, and False Negatives, respectively.

*Multi-class aggregation.* For multi-class tasks (such as 3-way natural language inference), scores are aggregated using either **Macro-$F_1$** (unweighted average across class $F_1$ scores, which prevents dominant majority classes from concealing minority class failures) or **Micro-$F_1$** (global pooling of instances, identical to overall accuracy).

### 8.1.8 Human qualitative evaluation and model-based judges

Despite the sophistication of $n$-gram matching, LCS alignment, and contextual embeddings, automated metrics consistently fail to evaluate high-level cognitive properties of natural language:

- **Global coherence and argumentative structure**: Does an essay or multi-paragraph response follow a logical sequence of reasoning, or does it wander between disconnected points?
- **Factuality and hallucination detection**: A generated passage can exhibit high BLEU, ROUGE, and BERTScore overlap against a reference while introducing an unmentioned, completely false fact.
- **Creativity, tone, and stylistic appropriateness**: Measuring humor, empathy, or instruction-following fidelity cannot be reduced to cosine similarity or token overlap.

Consequently, human evaluation remains the gold standard for assessing text quality.

#### Human evaluation: Absolute scales vs. pairwise comparisons

Two primary paradigms govern human evaluation in research and industry:

1. **Absolute Likert rating scales**: Annotators are presented with a generated response and asked to assign a score on a discrete 1-to-5 scale across specific criteria (e.g., fluency, coherence, relevance, factual accuracy).
2. **Pairwise comparative evaluation (A/B testing)**: Annotators are presented with a prompt and two anonymized, randomized candidate outputs ($A$ and $B$) produced by competing models, and asked to answer: *"Which response is better, or are they tied?"*

#### Cognitive failure mode of absolute scoring

As emphasized in lecture, humans are notoriously poor at assigning absolute grades. If ten human judges are asked to grade an essay on a 1-to-5 scale, their scores exhibit severe variance due to:

- **Calibration drift**: Annotator standards shift across time due to fatigue, mood, or exposure to previous high- or low-quality samples.
- **Subjective anchor disparity**: One annotator's "4" may correspond to another annotator's "2".

In contrast, humans are remarkably consistent at **relative pairwise comparisons**: deciding whether response $A$ is more helpful, coherent, or factually sound than response $B$ is a significantly simpler cognitive task. Pairwise preferences are converted into global scalar rankings using the **Elo rating system** [82] (the foundation of modern public benchmarks like the LMSYS Chatbot Arena).

### Scalability bottleneck and LLM-as-a-judge

While human evaluation provides the highest qualitative fidelity, it is fundamentally bottlenecked by resource constraints:

- **Extreme financial cost**: Conducting statistically significant human trials across thousands of prompts requires hundreds of thousands of dollars in crowd-sourcing compensation.
- **High latency**: Collecting human annotations takes days or weeks, making it impossible to integrate into real-time CI/CD model training pipelines.
- **Non-reproducibility**: Human pool variance makes exact experimental replication difficult across research labs.

These constraints have catalyzed the widespread adoption of **LLM-as-a-judge** [82]. By supplying a state-of-the-art foundation model (such as GPT-4) with detailed evaluation rubrics, system instructions, and few-shot calibration examples, automated systems can evaluate model outputs pairwise with high correlation ($\rho > 0.8$) to human expert preferences, operating at fraction-of-a-cent costs and second-level latency.

While metrics provide the mathematical scoring instruments, they cannot exist in an operational vacuum. A metric like BLEU is meaningless without the task of Machine Translation; Exact Match is meaningless without Question Answering; Accuracy is meaningless without Natural Language Inference. To understand how foundational models are developed and validated, we must examine the formal problem formulations across which these metrics are deployed.

## 8.2 Evaluation Tasks and Problem Formulations

Evaluating foundation models requires probing a wide spectrum of linguistic, semantic, and reasoning capabilities. In early NLP literature, models were trained and evaluated on isolated, specialized tasks using task-specific heads. In modern foundation model engineering, diverse tasks are unified under the shared autoregressive interface: every problem is formulated as a text prompt conditioning the generation of target tokens.

This section systematically reviews the primary task paradigms covered in the course, analyzing their structural formulation, canonical datasets, and evaluation metrics.

### 8.2.1 Autoregressive text completion and story cloze tasks

The most direct evaluation of an autoregressive language model is **text completion**: given an initial prompt or narrative passage, the model must predict a plausible, grammatically coherent, and logically consistent continuation. This can be evaluated in two operational modes:

1. **Generative mode**: The model autoregressively generates text, which is scored via perplexity or human qualitative inspection.
2. **Discriminative choice mode**: The model is presented with a context prefix and a discrete set of competing candidate continuations, evaluating the conditional log-likelihood assigned to each continuation:

$$\hat{c} = \arg\max_{c \in C} \frac{1}{|c|} \sum_{i=1}^{|c|} \ln P(w_i^{(c)} \mid \text{Context}, w_{<i}^{(c)}). \tag{8.57}$$

**Canonical QA benchmark datasets**

- **LAMBADA (Language Modeling Broadened to Account for Discourse-Level Analysis)** [83]: A corpus of narrative passages specifically constructed such that human readers can guess the final target word accurately when reading the entire paragraph, but cannot guess it if exposed only to the final sentence. This benchmark directly measures the model's ability to retain long-range discourse context across dozens of tokens without succumbing to recency bias. Evaluated via Accuracy (Exact Match on the final target word) and Perplexity.
- **ROCStories and StoryCloze** [84]: A benchmark composed of 5-sentence commonsense stories capturing everyday events. The StoryCloze test presents the first four sentences as context and provides two candidate endings (one logical, one absurd or contradictory). Models are evaluated on their accuracy in assigning higher likelihood to the correct narrative resolution.
- **HellaSwag** [85]: A continuation benchmark designed to counter statistical shortcut learning. Early models easily achieved high scores on story completion by exploiting shallow surface heuristics (e.g., matching length or common idioms). Zellers et al. introduced **Adversarial Filtering (AF)**, iteratively generating negative endings from a generator model that trick baseline discriminators while remaining obviously incorrect to humans.

### 8.2.2 Question answering architectures: Open-book versus closed-book

Question Answering (QA) evaluates a model's ability to comprehend queries, retrieve knowledge, and synthesize truthful, accurate answers. As categorized in lecture, QA problem settings differ across two foundational axes: **access to external knowledge** and **answer formulation format**.

**Axis 1: Access to knowledge (Closed-book vs. open-book)**

- **Closed-Book QA**: The model is given only the question prompt in its context window ($x$ = "What year was the University of Turin founded?"). The model must retrieve the factual answer entirely from its internal **parametric memory**—the weights optimized during pre-training. This evaluates factual retention and internal world models.
- **Open-Book QA**: The model is provided with external reference documents in its context window alongside the question. In modern systems, this is realized via **Retrieval-Augmented Generation (RAG)**, where an external retriever queries a document store or vector database (e.g., Wikipedia) and injects relevant context chunks into the prompt. This evaluates reading comprehension and contextual synthesis.

**Axis 2: Answer format**

- **Extractive QA**: The answer is guaranteed to exist as a contiguous substring (span) within the provided context passage. The model predicts the starting and ending token indices.
- **Abstractive QA**: The model must generate a free-form natural language response, paraphrasing and synthesizing information across multiple sentences.
- **Multiple-Choice QA**: The model chooses among discrete candidate options (e.g., A, B, C, D).

**Overview of canonical QA benchmarks**

- **SQuAD (Stanford Question Answering Dataset)** [80]: Contains over 100,000 question-answer pairs curated by crowd-workers on Wikipedia articles. Answers are exact extractive text spans. SQuAD 2.0 [86] introduced over 50,000 *unanswerable questions* that look syntactically plausible but cannot be answered from the text, penalizing models that hallucinate answers instead of outputting an abstention signal. Evaluated via Exact Match (EM) and token-level $F_1$.
- **TriviaQA** [87]: Over 650,000 question-answer pairs compiled from trivia websites paired with independently retrieved web and Wikipedia evidence documents, introducing natural linguistic noise and distant supervision.
- **Natural Questions (NQ)** [88]: A dataset released by Google comprising real, uncurated user queries entered into the Google search engine, paired with full Wikipedia pages. Annotators identified both a "long answer" (an entire paragraph containing the explanation) and a "short answer" (a concise entity or span).
- **WebQuestions** [89]: Contains 6,642 question-answer pairs generated by querying the Google Suggest API and annotating answers via Amazon Mechanical Turk over Freebase knowledge graph entities.

### 8.2.3 Conditional sequence generation: Translation and summarization

Conditional sequence generation tasks require mapping an input source text into an output text with different surface syntax or different length, while preserving invariant core semantics.

#### Machine translation tasks

Machine translation maps a source sequence $\mathbf{x} = (x_1, \ldots, x_S)$ in language $\mathcal{L}_S$ to a target sequence $\mathbf{y} = (y_1, \ldots, y_T)$ in language $\mathcal{L}_T$. The premier competitive venue is the annual **WMT (Workshop on Machine Translation)** benchmarks, which provide standardized training corpora, validation splits, and human reference sets across dozens of high-resource and low-resource language pairs (e.g., English-German, English-Czech). Evaluation relies on BLEU, METEOR, and neural metrics like COMET.

#### Text summarization tasks

Text summarization condenses an input document $\mathbf{x}$ into a concise summary $\mathbf{y}$ ($|\mathbf{y}| \ll |\mathbf{x}|$). It is divided into:

- **Extractive summarization**: Ranking and selecting the most salient verbatim sentences from the source document.
- **Abstractive summarization**: Rewriting and paraphrasing key ideas into novel, fluid sentences. Foundation models excel at abstractive summarization.

Standard benchmarks include:

- **CNN/Daily Mail** [77]: Over 300,000 news articles paired with multi-sentence bullet-point story highlights.
- **PubMed Diabetes**: Long-form scientific and biomedical research articles paired with author abstracts, requiring technical jargon retention and factual rigor.

Evaluated via ROUGE-1, ROUGE-2, and ROUGE-L.

### 8.2.4 Natural language inference and grammatical acceptability

#### Natural language inference (NLI)

NLI (also termed *recognizing textual entailment*) is the foundational benchmark for formal semantic reasoning. Given two sentences—a **Premise** ($P$) and a **Hypothesis** ($H$)—the model must determine whether the hypothesis is:

$$\text{Relationship}(P, H) \in \{\textbf{Entailment}, \textbf{Contradiction}, \textbf{Neutral}\}. \tag{8.58}$$

- *Entailment*: If $P$ is true, $H$ is necessarily true ($P$ = "A dog is chasing a tennis ball", $H$ = "An animal is running").
- *Contradiction*: If $P$ is true, $H$ is definitely false ($P$ = "A man is sleeping on a couch", $H$ = "A man is running a marathon").
- *Neutral*: $H$ might or might not be true; $P$ provides insufficient evidence ($P$ = "Two people are eating pizza", "The pizza has mushrooms").

Canonical benchmarks include **SNLI (Stanford NLI)** [90] (570,000 sentence pairs derived from image captions) and **MultiNLI (Multi-Genre NLI)** [91] (433,000 sentence pairs spanning 10 diverse spoken and written genres).

**Grammatical acceptability and linguistic competence**

To assess whether a model has internalized syntax independent of semantic plausibility (exemplified by Noam Chomsky's famous formulation *"Colorless green ideas sleep furiously"*), researchers utilize the **Corpus of Linguistic Acceptability (CoLA)** [92]. CoLA contains 10,657 English sentences drawn from theoretical linguistics publications, annotated as grammatically acceptable (1) or unacceptable (0). Because grammatical errors are subtle (e.g., auxiliary inversion, island constraints), CoLA is evaluated using the **Matthews Correlation Coefficient (MCC)**, a balanced measure for binary classification robust to class imbalance.

### 8.2.5 Text and sentiment classification

Text classification evaluates a model's capacity to categorize documents into discrete semantic classes based on overall content, style, or author intent.

- **Sentiment analysis**:
  - **IMDb Movie Reviews** [93]: 50,000 long-form movie reviews labeled with binary polarity (positive versus negative).
  - **Yelp Review Dataset**: 7 million customer reviews annotated on a 1-to-5 star rating scale.
  - **SST-2 (Stanford Sentiment Treebank)** [94]: Binary sentiment sentences derived from movie reviews, equipped with compositional syntactic parse trees.
- **Topic classification**:
  - **AG News**: 120,000 news articles categorized into four topical classes: *World*, *Sports*, *Business*, and *Sci/Tech*.
  - **20 Newsgroups**: Classic benchmark of 20,000 Usenet discussion forum messages partitioned across 20 distinct technical and social newsgroups.

Evaluated via Accuracy and Macro-$F_1$.

### 8.2.6 Mathematical and formal reasoning

Autoregressive models trained exclusively on next-token prediction frequently struggle with rigorous multi-step deductive reasoning. Small models memorize common arithmetic tables (e.g., $2 + 2 = 4$) but fail when required to execute multi-step symbolic manipulations.

To benchmark quantitative reasoning, researchers established two premier datasets:

- **GSM8K (Grade School Math 8K)** [81]: 8,500 linguistically diverse grade-school math word problems requiring 2 to 8 steps of arithmetic operations. Each problem includes a detailed chain-of-thought solution with intermediate calculation annotations (delimited by «...»). Foundation models are prompted using few-shot Chain-of-Thought (CoT) [95], and evaluated on **Exact Match** of the final numerical answer.
- **MATH Dataset** [96]: 12,500 challenging problems gathered from high-school American Mathematics Competitions (AMC 10, AMC 12, AIME), formatted entirely in LaTeX. Spans seven mathematical subfields: *Prealgebra*, *Algebra*, *Number Theory*, *Counting & Probability*, *Geometry*, *Intermediate Algebra*, and *Precalculus*, stratified across 5 difficulty levels.

### 8.2.7 Commonsense and causal reasoning

Commonsense reasoning represents the broad repository of unstated physical, social, temporal, and causal knowledge that humans utilize to navigate daily interactions.

**Ambiguity resolution: The Winograd schema challenge**

Pronoun ambiguity resolution tests whether a model understands the physical and situational constraints of entities. The foundational benchmark is the **Winograd Schema Challenge (WSC)** [97], based on paired sentences called *twin sentences* that differ by only one or two words:

> **Sentence 1**: *The trophy didn't fit in the suitcase because **it** was too **big**.* $\Longrightarrow$ What was too big? $\Longrightarrow$ **The trophy**.
> **Sentence 2**: *The trophy didn't fit in the suitcase because **it** was too **small**.* $\Longrightarrow$ What was too small? $\Longrightarrow$ **The suitcase**.

A model cannot resolve "it" through grammatical parsing alone; it must understand that containers are constrained by volume and that objects must be smaller than containers to fit inside them.

**WinoGrande** [98] scaled this paradigm to 44,000 adversarially filtered problems, systematically eliminating lexical selection biases to expose models relying on associative correlations rather than physical reasoning.

**Causal reasoning: COPA and ATOMIC**

- **COPA (Choice of Plausible Alternatives)** [99]: 1,000 questions composed of a premise and two alternative hypotheses. The prompt explicitly asks for either the **cause** or the **effect**:

  *Premise*: The man broke his toe. What was the **cause**?
  *Alternative 1*: He dropped a hammer on his foot. [Correct]
  *Alternative 2*: He tied his shoelaces too tightly.

- **ATOMIC (An Atlas of Machine Commonsense for If-Then Reasoning)** [100]: A knowledge graph containing 877,000 everyday if-then relations organized into 9 semantic dimensions spanning agent intent (*xIntent*), emotional effect (*xReact*), and consequences for others (*oEffect*).

### 8.2.8 Synthetic probe tasks: Isolating capabilities from memorization

A persistent difficulty in evaluating foundation models on standard public datasets is determining whether a model solved a problem through general reasoning or simply reproduced text memorized during web-scale pre-training.

To address this, researchers design **synthetic probe tasks**: algorithmically generated problems that are easy to construct programmatically, yet non-trivial for autoregressive models to solve:

- **Arbitrary multi-digit arithmetic**: Generating random large-integer operations (e.g., $4123 + 9421 = 13544$) guarantees that the specific numerical pair was never observed in pre-training corpora.
- **String perturbation and character scrubbing**: Requesting a model to strip a specific punctuation symbol from an arbitrary sentence (e.g., *"Remove the $ from the sentence 'th$s is a$sentenc$e" '*). Because subword tokenizers merge characters into subword units, manipulating raw characters requires navigating token boundaries.
- **Anagram unscrambling**: Asking the model to unscramble randomly permuted words (e.g., *"aplpe"* $\rightarrow$ *"apple"*).

Synthetic tasks ensure that test data is guaranteed to be unseen, providing an untainted diagnostic probe into the model's computational capabilities.

*While evaluating models on individual tasks (such as SQuAD for QA, or CoLA for grammar) reveals localized strengths and weaknesses, reporting dozens of separate accuracies is unwieldy. The research community requires holistic, standardized evaluation suites that aggregate performance across diverse tasks into unified benchmarks.*

## 8.3 Standardized Benchmark Suites

Foundation models are designed as general-purpose reasoners. Evaluating a model on a single task provides an incomplete picture: a model could excel at text classification while failing substantially at arithmetic or causal reasoning. To establish reproducible comparisons between competing architectures and training paradigms, the NLP community introduced **standardized multi-task benchmark suites**.

However, the widespread reliance on fixed, public benchmarks has triggered an epistemological crisis in AI evaluation: **benchmark data contamination**. This section examines the canonical evaluation suites and formalizes the contamination phenomenon that threatens the validity of modern LLM leaderboards.

### 8.3.1 Holistic multi-task benchmarking: GLUE and SuperGLUE

#### The GLUE benchmark suite

In 2018, Wang et al. [101] introduced **GLUE** (*General Language Understanding Evaluation*), an integrated benchmark suite designed to evaluate general-purpose sentence representations. GLUE combined nine diverse natural language understanding tasks spanning three broad families:

1. **Single-sentence tasks**: Linguistic acceptability on CoLA [92] and binary sentiment analysis on SST-2 [94].
2. **Similarity and paraphrase tasks**: Sentence pair equivalence on MRPC (Microsoft Research Paraphrase Corpus), QQP (Quora Question Pairs), and semantic similarity scoring on STS-B (Semantic Textual Similarity Benchmark).
3. **Inference tasks**: Natural language inference on MNLI (MultiNLI) [91], QNLI (Question NLI, derived from SQuAD), RTE (Recognizing Textual Entailment), and WNLI (Winograd NLI).

GLUE established a public leaderboard evaluated on hidden private test sets, aggregating performance across all nine tasks into a **single summary score** (the unweighted average of task accuracies and correlation coefficients).

#### Benchmark saturation and the advent of SuperGLUE

When GLUE was introduced, state-of-the-art models scored substantially below human performance (87.1). However, with the advent of the Transformer pre-training revolution (BERT [52], RoBERTa, and T5 [59]), models rapidly saturated the benchmark, exceeding non-expert human performance within eighteen months. Analysis revealed that models were exploiting spurious statistical cues and shallow lexical overlaps in GLUE's test sets rather than exercising deep comprehension.

In response, Wang et al. [102] introduced **SuperGLUE** in 2019. Retaining GLUE's single-metric aggregate format, SuperGLUE replaced saturated tasks with substantially harder challenges requiring multi-sentence reasoning, coreference resolution, and open-ended QA:

- **BoolQ**: Boolean (yes/no) question answering over complex passages.
- **CB (CommitmentBank)**: Three-way discourse clause inference.
- **COPA**: Commonsense causal reasoning [99].
- **MultiRC**: Multi-sentence reading comprehension where questions have multiple correct answers.

- **ReCoRD**: Reading comprehension with commonsense reasoning over news articles.
- **RTE**: Recognizing textual entailment across diverse news datasets.
- **WiC (Words in Context)**: Identifying whether a polysemous word shares identical meaning across two different sentence contexts.
- **WSC**: The Winograd Schema Challenge pronoun coreference task [97].

### 8.3.2 Massive multitask language understanding (MMLU)

As foundation models scaled past tens of billions of parameters, evaluation expanded beyond basic linguistic understanding toward broad, multi-domain academic and professional knowledge.

In 2020, Hendrycks et al. [103] introduced **MMLU** (*Massive Multitask Language Understanding*), which has become the de facto gold standard for reporting foundation model capabilities (utilized to benchmark GPT-3, PaLM, LLaMA, GPT-4, and Claude).

#### Architecture and scope of MMLU

- **Scale and format**: Comprises 15,908 multiple-choice questions, each formulated with four options (A, B, C, D).
- **Breadth of domains**: Spans **57 diverse subjects** categorized into four overarching disciplines:
  1. **STEM**: Advanced Mathematics, Physics, Chemistry, Computer Science, Machine Learning, Electrical Engineering.
  2. **Humanities**: Formal Logic, Philosophy, World History, Prehistory, Jurisprudence.
  3. **Social Sciences**: Microeconomics, Macroeconomics, Psychology, Sociology, US Government and Politics.
  4. **Other / Professional**: Clinical Medicine, Professional Law (Bar Exam), Professional Accounting, Virology.
- **Difficulty stratification**: Covers elementary school facts, high school examinations, undergraduate coursework, and professional licensing standards.

#### MMLU evaluation protocol

Models are evaluated on MMLU under **zero-shot** or **5-shot in-context learning** conditions. The prompt provides the question, four formatted choices, and a prompt prefix (e.g., *"Answer:"*). The model's score is computed by comparing the conditional log-probabilities assigned to the four option tokens:

$$\hat{y} = \underset{k \in \{\text{'A'},\text{'B'},\text{'C'},\text{'D'}\}}{\arg\max} P(w_k \mid \text{Prompt}). \tag{8.59}$$

The overall MMLU score is the unweighted percentage of questions answered correctly across all 57 subjects. A random baseline achieves 25.0%; modern frontier models (such as GPT-4 and Gemini 1.5) exceed 85%, outperforming human domain experts on the majority of specialized subjects.

### Worked benchmark question examples

To illustrate the depth and format of MMLU across diverse disciplines, consider the representative questions highlighted in the lecture:

**MMLU Example 1: Conceptual Physics**

**Question**: When you drop a ball from rest it accelerates downward at $9.8$ m/s$^2$. If you instead throw it downward assuming no air resistance its acceleration immediately after leaving your hand is:

**(A)** $9.8$ m/s$^2$ *[Correct]*

**(B)** more than $9.8$ m/s$^2$

**(C)** less than $9.8$ m/s$^2$

**(D)** Cannot say unless the speed of throw is given

*Physics insight*: Once the ball leaves the hand, gravity is the only accelerating force acting on the mass; the initial throw velocity alters instantaneous speed, not downward acceleration ($g = 9.8$ m/s$^2$).

**MMLU Example 2: College Mathematics**

**Question**: In the complex $z$-plane, the set of points satisfying the equation $z^2 = |z|^2$ is a:

**(A)** pair of points

**(B)** circle

**(C)** half-line

**(D)** line *[Correct]*

*Mathematical derivation*: Let $z = x + iy$. Then $z^2 = x^2 - y^2 + 2ixy$ and $|z|^2 = x^2 + y^2$. Equating real and imaginary components yields $2xy = 0$ (so $x = 0$ or $y = 0$) and $x^2 - y^2 = x^2 + y^2 \implies 2y^2 = 0 \implies y = 0$. Since $y = 0$ and $x \in \mathbb{R}$ satisfies the equation for all $x$, the solution set is precisely the real axis, which constitutes a **line** in the complex plane.

## 8.3.3 The benchmark contamination crisis

The very large performance gains reported on public benchmarks have brought to light an existential vulnerability in modern AI evaluation: **data contamination**.

**Definition 8.1:** *Benchmark Data Contamination*

**Data contamination** refers to the inadvertent or deliberate inclusion of evaluation benchmark test data (questions, passages, or gold-standard target labels) in the training corpus of a language model.

As highlighted in the lecture, Foundation models are pre-trained on multi-terabyte web crawls (such as Common Crawl, GitHub, Wikipedia, and academic preprint servers) containing hundreds of billions of web pages. Because popular benchmarks (GLUE, SQuAD, MMLU, GSM8K) are hosted on public GitHub repositories, open-access papers, and web forums, their test sets are routinely ingested by automated web scrapers during pre-training.

### Taxonomy and levels of benchmark contamination

Contamination can occur across multiple distinct stages of the model lifecycle:

1. **Pre-training contamination**: Public benchmark test splits are included directly in the pre-training scrape. The model observes the verbatim test questions and answers during self-supervised next-token prediction, memorizing them as factual text.
2. **Supervised fine-tuning (SFT) contamination**: Benchmark questions are inadvertently used to generate synthetic instruction-tuning datasets (e.g., using one LLM to generate Q&A pairs from existing benchmarks to train another model).
3. **Post-deployment feedback contamination**: Real users paste benchmark questions into public chatbot interfaces, which are subsequently collected and added to subsequent retraining pipelines.

#### Consequences: Memorization vs. reasoning

Data contamination undermines the foundational assumption of statistical machine learning: that generalization performance on a held-out test set reflects genuine out-of-distribution capability.

In their seminal study *"NLP Evaluation in Trouble: On the Need to Measure LLM Data Contamination for Each Benchmark"*, Sainz et al. [104] demonstrated that models exhibiting near-perfect scores on popular reasoning benchmarks suffered substantial performance drops when evaluated on slightly perturbed or synthetically generated instances of the identical problems. A model that achieves 90% on GSM8K might simply be reciting a memorized solution path observed dozens of times during pre-training, rather than executing mathematical deduction.

#### The closed-model auditing barrier

As emphasized by Prof. Giobergia, data contamination introduces severe asymmetries between open-weight and proprietary closed-source models:

- For open-source models (such as LLaMA or Mistral), researchers can inspect public pre-training manifests, search for exact $n$-gram overlaps, and audit decontamination pipelines.
- For proprietary commercial models (such as GPT-4), the exact composition of the pre-training corpus is an undisclosed trade secret. Consequently, **contamination cannot be formally proven or disproven**. Continuous benchmark gains reported by proprietary models may stem from architectural innovations, or simply from aggressive pre-training exposure to the evaluation test sets.

#### Decontamination protocols and countermeasures

To safeguard evaluation integrity, modern foundation model research employs rigorous decontamination heuristics:

- **$N$-gram overlap filtering**: Scrutinizing the pre-training corpus for contiguous 13-gram or 15-gram matches against benchmark test splits and removing matching documents prior to training.
- **Perplexity anomaly inspection**: Measuring the model's perplexity on the benchmark test set compared to a fresh, private validation set. If perplexity on the public test set is anomalously lower than on independent text from the same domain, contamination is strongly indicated.
- **Dynamic and synthetic benchmarks**: Generating novel evaluation instances on-the-fly (e.g., dynamic arithmetic problems or continuous human evaluation arenas like Chatbot Arena) that can never appear in static training crawls.

## 8.4 Evaluation Synthesis

To provide a comprehensive reference for researchers and software engineers, Table 8.4 synthesizes the entire hierarchy of language model evaluation examined in this chapter, connecting evaluation paradigms, operational metrics, canonical benchmark datasets, and primary failure modes.

Language model evaluation has evolved from simple probabilistic uncertainty metrics (PPL) into multi-faceted empirical benchmarking across thousands of specialized tasks. As foundation models approach and surpass human performance across standard benchmarks, evaluation methodology itself becomes the central engineering frontier: moving from static public test sets toward dynamic, synthetic, and adversarial evaluation protocols capable of discriminating between superficial web memorization and genuine machine intelligence.

Table 8.4: Comprehensive synthesis of language model evaluation paradigms, tasks, canonical datasets, standard metrics, and structural failure modes.

| Paradigm | Core Task | Canonical Datasets | Standard Metrics | Primary Failure Modes |
|---|---|---|---|---|
| **Intrinsic Likelihood** | Next-token prediction | Raw text corpora | Perplexity (PPL), Cross-Entropy | Incomparable across tokenizers; rewards surface familiarity over truth. |
| **Lexical Surface Matching** | Machine translation, summarization | WMT, CNN/Daily Mail | BLEU-$n$, ROUGE-$N$, ROUGE-L, METEOR | Semantic blindness; penalizes valid paraphrasing; negation insensitivity. |
| **Semantic Embeddings** | Paraphrase generation, open dialogue | General NLG corpora | BERTScore ($P_{\text{BERT}}, R_{\text{BERT}}, F_{\text{BERT}}$) | High computational cost; opaque interpretability ("BERT says so"). |
| **Exact Match & Ranking** | Extractive QA, multiple choice | SQuAD, Natural Questions | Exact Match (EM), MRR, NDCG@$k$, Hit@$k$ | Zero partial credit; sensitive to superficial formatting discrepancies. |
| **Linguistic Inference** | NLI, grammatical acceptability | SNLI, MultiNLI, CoLA | Accuracy, Macro-$F_1$, MCC | Susceptible to lexical overlap heuristics; dataset artifact exploitation. |
| **Multi-Step Reasoning** | Grade-school & competition math | GSM8K, MATH | Exact Match on final answer, Pass@1 | Calculation brittleness; superficial step hallucination; CoT drift. |
| **Commonsense & Causality** | Ambiguity resolution, if-then causality | Winograd, Winogrande, COPA, ATOMIC | Accuracy, Choice Log-Likelihood | Memorized associative priors override true physical/temporal reasoning. |
| **Holistic Benchmarks** | Multi-task understanding, professional knowledge | GLUE, SuperGLUE, MMLU | Aggregate Score, Mean Macro-$F_1$ | Rapid benchmark saturation; severe vulnerability to data contamination. |
| **Qualitative & Human** | Open-ended chat, instruction following | Chatbot Arena, MT-Bench | Elo rating, Pairwise win rate, LLM-as-a-judge | Human judge fatigue and calibration drift; evaluator LLM verbosity and position bias. |

# Bibliography


[1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, pages 5998–6008, 2017.

[2] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 1877–1901, 2020.

[3] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 30016–30030, 2022.

[4] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4th edition, 2020.

[5] Warren S. McCulloch and Walter Pitts. A logical calculus of the ideas immanent in nervous activity. *The Bulletin of Mathematical Biophysics*, 5(4):115–133, 1943.

[6] Frank Rosenblatt. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408, 1958.

[7] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.

[8] Christopher M. Bishop and Hugh Bishop. *Deep Learning: Foundations and Concepts*. Springer, 2024. URL https://www.bishopbook.com/.

[9] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. URL https://www.deeplearningbook.org/.

[10] Simon J. D. Prince. *Understanding Deep Learning*. MIT Press, 2023. URL https://udlbook.github.io/udlbook/.

[11] Guido F. Montúfar, Razvan Pascanu, Kyunghyun Cho, and Yoshua Bengio. On the number of linear regions of deep neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 27, pages 2924–2932, 2014. URL https://proceedings.neurips.cc/paper/2014/hash/1a5b1e4daae265b790965a275b53ae50-Abstract.html.

[12] Marvin Minsky and Seymour Papert. *Perceptrons: An Introduction to Computational Geometry*. MIT Press, Cambridge, MA, 1969.

[13] George Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals, and Systems*, 2(4):303–314, 1989.

[14] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366, 1989.

[15] Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 249–256, 2010.

[16] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (GELUs). *arXiv preprint arXiv:1606.08415*, 2016.

[17] Noam Shazeer. GLU variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020. URL https://arxiv.org/abs/2002.05202.

[18] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.

[19] Jeffrey L. Elman. Finding structure in time. *Cognitive Science*, 14(2):179–211, 1990.

[20] Paul J. Werbos. Backpropagation through time: what it does and how to do it. *Proceedings of the IEEE*, 78(10):1550–1560, 1990.

[21] Yoshua Bengio, Patrice Simard, and Paolo Frasconi. Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5(2):157–166, 1994.

[22] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. On the difficulty of training recurrent neural networks. In *International Conference on Machine Learning (ICML)*, pages 1310–1318, 2013.

[23] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

[24] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734, 2014.

[25] Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. Sequence to sequence learning with neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 27, pages 3104–3112, 2014.

[26] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. In *International Conference on Learning Representations (ICLR)*, 2015.

[27] Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423, 1948.

[28] Christopher D. Manning and Hinrich Schütze. *Foundations of Statistical Natural Language Processing*. MIT Press, 1999.

[29] Daniel Jurafsky and James H. Martin. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition*. Prentice Hall / Online Draft, 3rd draft edition, 2024. URL https://web.stanford.edu/~jurafsky/slp3/.

[30] I. J. Good. The population frequencies of species and the estimation of population parameters. *Biometrika*, 40(3-4):237–264, 1953.

[31] Slava M. Katz. Estimation of probabilities from sparse data for the language model component of a speech recognizer. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 35(3):400–401, 1987.

[32] Reinhard Kneser and Hermann Ney. Improved backing-off for m-gram language modeling. In *IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, pages 181–184, 1995.

[33] Yoshua Bengio, Réjean Ducharme, Pascal Vincent, and Christian Jauvin. A neural probabilistic language model. *Journal of Machine Learning Research*, 3:1137–1155, 2003.

[34] Sebastian Raschka. *Build a Large Language Model (From Scratch)*. Manning Publications, 2024.

[35] Zellig S. Harris. Distributional structure. *Word*, 10(2-3):146–162, 1954.

[36] John R. Firth. A synopsis of linguistic theory 1930–1955. *Studies in Linguistic Analysis*, pages 1–32, 1957.

[37] Andrej Karpathy. Building makemore part 2: MLP language model. YouTube Video Tutorial and GitHub Repository, 2022. URL https://github.com/karpathy/makemore.

[38] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. In *International Conference on Learning Representations (ICLR) Workshop Track*, 2013.

[39] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S. Corrado, and Jeffrey Dean. Distributed representations of words and phrases and their compositionality. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 26, pages 3111–3119, 2013.

[40] Grant Sanderson. Visualizing attention, embeddings and neural network trajectories. 3Blue1Brown Educational Video Series, 2024. URL https://www.3blue1brown.com/lessons/gpt.

[41] Frederic Morin and Yoshua Bengio. Hierarchical probabilistic neural network language model. In *Proceedings of the Tenth International Workshop on Artificial Intelligence and Statistics (AISTATS)*, pages 246–252, 2005.

[42] Jeffrey Pennington, Richard Socher, and Christopher D. Manning. GloVe: Global vectors for word representation. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543, 2014.

[43] Piotr Bojanowski, Edouard Grave, Armand Joulin, and Tomas Mikolov. Enriching word vectors with subword information. *Transactions of the Association for Computational Linguistics (TACL)*, 5:135–146, 2017.

[44] Enkk. Cosa sono gli embeddings: Geometria semantica, distanza coseno e vector databases. YouTube Educational Lecture, 2023. URL https://www.youtube.com/watch?v=5sYkC8aVv1k.

[45] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1715–1725, 2016.

[46] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019.

[47] Taku Kudo and John Richardson. Sentencepiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 66–71, 2018.

[48] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. CodeBERT: A pre-trained model for programming and natural languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 1536–1547, 2020.

[49] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[50] Philip Gage. A new algorithm for data compression. *C Users Journal*, 12(2):23–38, 1994.

[51] Jonas Gehring, Michael Auli, David Grangier, Denis Yarats, and Yann N. Dauphin. Convolutional sequence to sequence learning. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 1243–1252, 2017. URL https://arxiv.org/abs/1705.03122.

[52] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 4171–4186, 2019.

[53] Peter Shaw, Jakob Uszkoreit, and Ashish Vaswani. Self-attention with relative position representations. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 464–468, 2018. URL https://arxiv.org/abs/1803.02155.

[54] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024. URL https://arxiv.org/abs/2104.09864.

[55] Ofir Press, Noah A. Smith, and Mike Lewis. Train short, test long: Attention with linear biases enables input length extrapolation. In *International Conference on Learning Representations (ICLR)*, 2022. URL https://arxiv.org/abs/2108.12409.

[56] Grant Sanderson. Attention in transformers, visually explained. 3Blue1Brown Video Series, 2024. URL https://www.youtube.com/watch?v=eMlx5fFNoYc.

[57] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.

[58] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016. URL https://arxiv.org/abs/1607.06450.

[59] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020. URL https://jmlr.org/papers/v21/19-460.html.

[60] Kevin Clark, Urvashi Khandelwal, Omer Levy, and Christopher D. Manning. What does BERT look at? an analysis of BERT's attention. In *Proceedings of the 2019 ACL Workshop*

*BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*, pages 276–286, 2019. URL https://arxiv.org/abs/1906.04341.

[61] Paul Michel, Omer Levy, and Graham Neubig. Are sixteen heads really better than one? In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, pages 14014–14024, 2019. URL https://arxiv.org/abs/1905.10650.

[62] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. In *International Conference on Learning Representations (ICLR)*, 2020. URL https://arxiv.org/abs/1904.09751.

[63] Angela Fan, Mike Lewis, and Yann Dauphin. Hierarchical neural story generation. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 889–898, 2018. URL https://arxiv.org/abs/1805.04833.

[64] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving language understanding by generative pre-training. Technical report, OpenAI, 2018. URL https://openai.com/research/language-unsupervised.

[65] Gerasimos Potamianos and Frederick Jelinek. A study of n-gram and decision tree letter language modeling methods. *Speech Communication*, 24(2):171–192, 1998.

[66] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[67] Shaden Smith, Mostofa Patwary, Brandon Norick, Patrick LeGresley, Samyam Rajbhandari, Jared Casper, Zhun Liu, Shrimai Prabhumoye, Yuxiong He, et al. Using DeepSpeed and Megatron to train Megatron-Turing NLG 530b, a large-scale generative language model. *arXiv preprint arXiv:2201.11990*, 2022. URL https://arxiv.org/abs/2201.11990.

[68] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

[69] Opher Lieber, Or Sharir, Barak Lenz, and Yoav Shoham. Jurassic-1: Technical details and evaluation. *AI21 Labs Whitepaper*, 2021. URL https://www.ai21.com/white-paper/jurassic-1.

[70] Jack W. Rae, Sebastian Borgeaud, Trevor Cai, Katie Millican, Jordan Hoffmann, Francis Song, John Aslanides, Sarah Henderson, Roman Ring, Susannah Young, et al. Scaling language models: Methods, analysis & insights from training Gopher. *arXiv preprint arXiv:2112.11446*, 2021. URL https://arxiv.org/abs/2112.11446.

[71] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. PaLM: Scaling language modeling with Pathways. *Journal of Machine Learning Research*, 24(240):1–113, 2023. URL https://jmlr.org/papers/v24/22-1144.html.

[72] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023. URL https://arxiv.org/abs/2307.09288.

[73] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. URL https://arxiv.org/abs/2407.21783.

[74] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023. URL https://arxiv.org/abs/2310.06825.

[75] Ebtesam Almazrouei, Hamza Alobeidli, Abdulaziz Alshamsi, Alessandro Cappelli, Ruxandra Cojocaru, Mérouane Debbah, Étienne Goffinet, Daniel Hesslow, Julien Launay, Quentin Malartic, et al. The Falcon series of open language models. *arXiv preprint arXiv:2311.16867*, 2023. URL https://arxiv.org/abs/2311.16867.

[76] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. BLEU: A method for automatic evaluation of machine translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 311–318, 2002. URL https://aclanthology.org/P02-1040/.

[77] Chin-Yew Lin. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out: Proceedings of the ACL-04 Workshop*, pages 74–81, 2004. URL https://aclanthology.org/W04-1013/.

[78] Satanjeev Banerjee and Alon Lavie. METEOR: An automatic metric for MT evaluation with improved correlation with human judgments. In *Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization*, pages 65–72, 2005. URL https://aclanthology.org/W05-0909/.

[79] Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q. Weinberger, and Yoav Artzi. BERTScore: Evaluating text generation with BERT. In *International Conference on Learning Representations (ICLR)*, 2020. URL https://openreview.net/forum?id=SkeHuCVFDr.

[80] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. SQuAD: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 2383–2392, 2016. URL https://aclanthology.org/D16-1264/.

[81] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021. URL https://arxiv.org/abs/2110.14168.

[82] Lianmin Zheng, Wei-Lin Chiang, Hao Zhang, Siyuan Zhuang, Minmin Chen, Sheng Shen, Stephanie Shen, Zheyuan Li, Zi Lin, Yonghao Zhuang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pages 46595–46623, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/91f18a1287b3e83161a0524458d92976-Abstract-Datasets_and_Benchmarks.html.

[83] Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Quan Ngoc Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernández. The LAMBADA dataset: Word prediction requiring a broad discourse context. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1525–1534, 2016. URL https://aclanthology.org/P16-1144/.

[84] Nasrin Mostafazadeh, Nathanael Chambers, Xiaodong He, Devi Parikh, Dhruv Batra, Lucy Vanderwende, Pushmeet Kohli, and James Allen. A corpus and cloze evaluation for deeper understanding of commonsense stories. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 839–849, 2016. URL https://aclanthology.org/N16-1098/.

[85] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. HellaSwag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 4791–4800, 2019. URL https://aclanthology.org/P19-1472/.

[86] Pranav Rajpurkar, Robin Jia, and Percy Liang. Know what you don't know: Unanswerable questions for SQuAD. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 784–789, 2018. URL https://aclanthology.org/P18-2124/.

[87] Mandar Joshi, Eunsol Choi, Daniel S. Weld, and Luke Zettlemoyer. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1601–1611, 2017. URL https://aclanthology.org/P17-1147/.

[88] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei Chang, Andrew M. Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics (TACL)*, 7: 452–466, 2019. URL https://aclanthology.org/Q19-1026/.

[89] Jonathan Berant, Andrew Chou, Roy Frostig, and Percy Liang. Semantic parsing on Freebase from question-answer pairs. In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1533–1544, 2013. URL https://aclanthology.org/D13-1160/.

[90] Samuel R. Bowman, Gabor Angeli, Christopher Potts, and Christopher D. Manning. A large annotated corpus for learning natural language inference. In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 632–642, 2015. URL https://aclanthology.org/D15-1075/.

[91] Adina Williams, Nikita Nangia, and Samuel R. Bowman. A broad-coverage challenge corpus for sentence understanding through inference. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 1112–1122, 2018. URL https://aclanthology.org/N18-1101/.

[92] Alex Warstadt, Amanpreet Singh, and Samuel R. Bowman. Neural network acceptability judgments. *Transactions of the Association for Computational Linguistics (TACL)*, 7:625–641, 2019. URL https://aclanthology.org/Q19-1040/.

[93] Andrew L. Maas, Raymond E. Daly, Peter T. Pham, Dan Huang, Andrew Y. Ng, and Christopher Potts. Learning word vectors for sentiment analysis. In *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies (ACL-HLT)*, pages 142–150, 2011. URL https://aclanthology.org/P11-1015/.

[94] Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D. Manning, Andrew Y. Ng, and Christopher Potts. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1631–1642, 2013. URL https://aclanthology.org/D13-1170/.

[95] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 24824–24837, 2022.

[96] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Advances in Neural Information Processing Systems (NeurIPS) Track on Datasets and Benchmarks*, volume 1, 2021. URL https://proceedings.neurips.cc/paper/2021/hash/be83ab3ecd0db773eb2dc1b0a17836a1-Abstract.html.

[97] Hector J. Levesque, Ernest Davis, and Leora Morgenstern. The Winograd schema challenge. In *Proceedings of the Thirteenth International Conference on Principles of Knowledge Representation and Reasoning (KR)*, pages 552–561, 2012. URL https://cdn.aaai.org/ocs/4492/4492-21843-1-PB.pdf.

[98] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. WINOGRANDE: An adversarial Winograd schema challenge at scale. In *Proceedings of the AAAI Conference on Human Computation and Crowdsourcing (AAAI)*, volume 34, pages 8732–8740, 2020. URL https://ojs.aaai.org/index.php/AAAI/article/view/6399.

[99] Melissa Roemmele, Cosmin Adrian Bejan, and Andrew S. Gordon. Choice of plausible alternatives: An evaluation of commonsense causal reasoning. In *AAAI Spring Symposium: Logical Formalizations of Commonsense Reasoning*, pages 90–95, 2011. URL https://cdn.aaai.org/ocs/2418/2418-10878-1-PB.pdf.

[100] Maarten Sap, Ronan Le Bras, Emily Allaway, Chandra Bhagavatula, Nicholas Lourie, Hannah Rashkin, Vered Shwartz, Noah A. Smith, and Yejin Choi. ATOMIC: An atlas of machine commonsense for if-then reasoning. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, volume 33, pages 3027–3035, 2019. URL https://doi.org/10.1609/aaai.v33i01.33013027.

[101] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. GLUE: A multi-task benchmark and analysis platform for natural language understanding. In *Proceedings of the Workshop: Analyzing and Interpreting Neural Networks for NLP (BlackboxNLP)*, pages 353–355, 2018. URL https://aclanthology.org/W18-5446/.

[102] Alex Wang, Yada Pruksachatkun, Nikita Nangia, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. SuperGLUE: A stickier benchmark for general-purpose language understanding systems. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019. URL https://proceedings.neurips.cc/paper/2019/hash/4496bf24afe7fab6f046bf4923da8de6-Abstract.html.

[103] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations (ICLR)*, 2021. URL https://openreview.net/forum?id=d7KBjmI3GmQ.

[104] Oscar Sainz, Jon Ander Campos, Iker García-Ferrero, Rachel Rudinger, and Eneko Agirre. NLP evaluation in trouble: On the need to measure LLM data contamination for each benchmark. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 10776–10787, 2023. URL https://aclanthology.org/2023.findings-emnlp.722/.